A2560Me

Hardware Reference

The CPU bus and memory map, GAVIN, the video (VICKY II, text, MemText, the GUI bitmap engine and the HDMI output), the memory system and the DMA engines (SDMA, MDMA, VDMA), the sound, the debug port, the Game Graphics Engine (GGE) and the PCI Express root port of the A2560Me 68LC060 computer

Revision	`1.8`
FPGA build	`#87 (66.67 MHz), RevB3A`
Date	`2026-10-16`
Device	`Kintex-7 xc7k160t`



FOENIX LABS

Foenix Labs. Generated from the FPGA sources of this build: the register tables follow the RTL.

Contents

1	System overview and the CPU bus
2	GAVIN: system controller and I/O
3	Video: VICKY II, text, MemText and the GUI bitmap engine
4	Memory, DMA and the debug port
5	BEATRIX: sound
6	Game Graphics Engine (GGE)
7	PCI Express root port
A	Appendix: register index
B	Appendix: notes, limits, revisions

1 System overview and the CPU bus

The A2560Me is a single-board computer built around a Motorola MC68LC060 CPU and a Xilinx Kintex-7 160T FPGA. The FPGA holds almost all of the system logic: the CPU bus interface (address decoder, transfer acknowledge, bus arbiter), the interrupt controller, the system controller GAVIN, the sound chip BEATRIX, the legacy video chip VICKY II (text mode, MemText, mouse pointer, gamma), the GUI bitmap engine, the Game Graphics Engine (GGE) with its DAC player, the DDR3 memory controller with a 64 KB L2 cache, three DMA engines: the SDMA (SRAM to SRAM), the MDMA (SRAM <-> DDR3) and the VDMA (DDR3 block mover and blitter), and, since build 74, a PCI Express root port.

Two memories sit directly on the CPU bus: 8 MB of fast 32-bit SRAM and the boot flash. The FPGA drives their chip selects and strobes. Everything else - DDR3 and all registers - is reached through the FPGA.

The CPU bus clock (BCLK) is 66.67 MHz; the 68LC060 core runs at BCLK (1:1). The bus is big-endian: data lines D31-D24 carry the byte at the lowest address of a longword.

This manual describes FPGA build 87 (BCLK 66.67 MHz). Since build 59 the 66.67 MHz design is the only released one. Build 87 has the build 39 local-bus hardening, the build 40 board-type switch (white / black board, DIP switch 7), the build 59 L2 write fix, the Game Graphics Engine (builds 60-69, with the DAC player's second output on the AK4432 since build 69) the BEATRIX mixer controls of build 68, the VS1053b decoder link and the reworked WizFi360 UART of build 70, the PCI Express root port and the L2 line burst read fix of build 74 and, new in builds 75-83, debug port protocol v2 (with DDR3 access while the CPU runs and flash programming), a seventh DDR3 arbiter port for the debug port, and boot block selection through GAVIN (BOOT_SEL, see Reset and boot sequence) and, new in build 84, a new I2C master at `$FEC0_0E00` that the CPU can use (the MAC address EEPROM and the other devices on the system I2C bus; see the GAVIN chapter). Builds 85 and 86 rework the debug port: memory access while the CPU runs, pause and step without a reset, 4 breakpoints and a monitor stop on interrupt level 7 (build 85), and protocol v2 as the only protocol (build 86; the old `$55` protocol is gone). Build 87 adds the Floating Math Block (a single-precision FPU at `$FEC0_4000`) and the FPGA temperature monitor at `$FEC0_0D00` (see the GAVIN chapter).

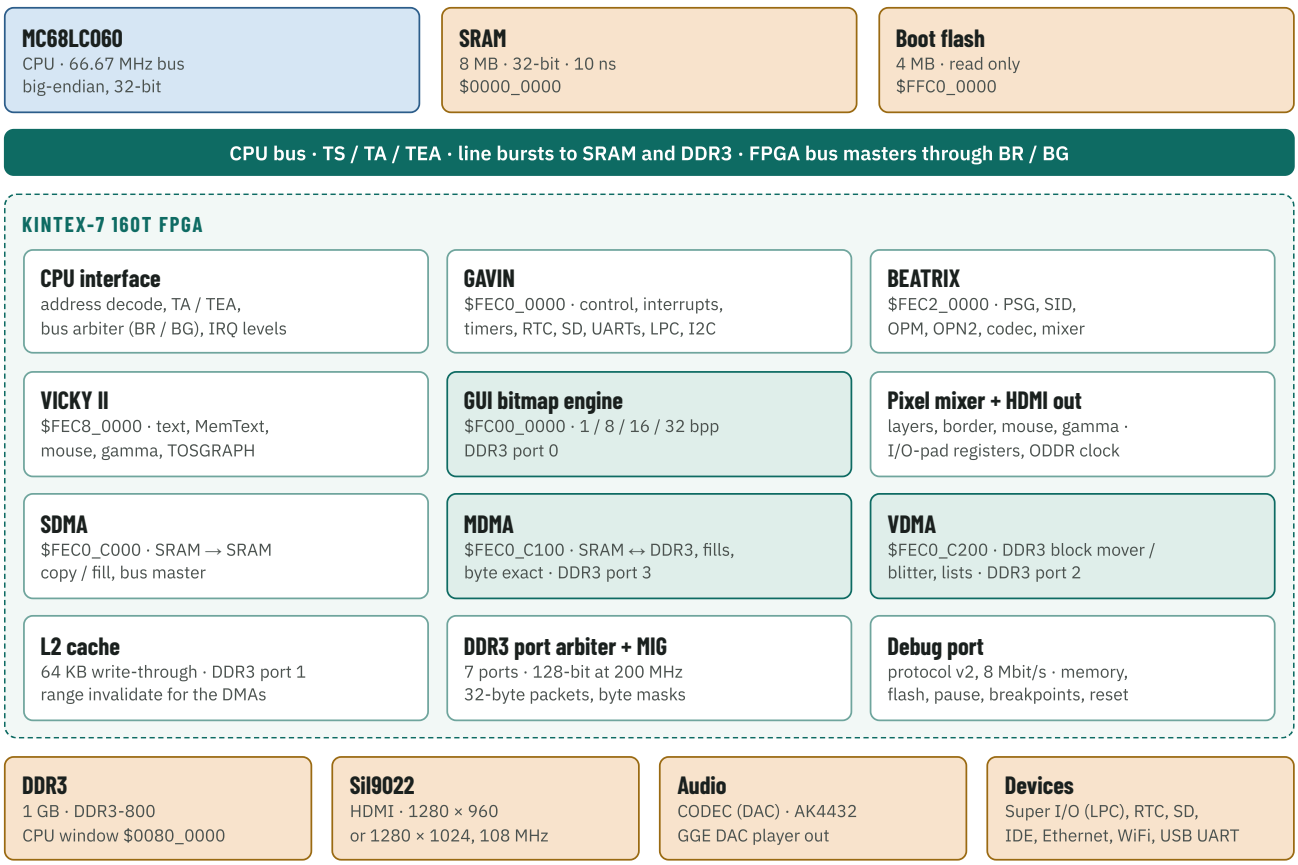


Figure 1. The A2560Me (FPGA build 87): the CPU, its memories, and the blocks inside the FPGA. Teal: blocks that read or write DDR3 on their own.

Build configuration (build 87)

Some parts of the design are switched on or off when the FPGA is built. The current settings decide what software finds in the memory map.

Build options

OPTION	BUILD 87	EFFECT
`CPU bus clock (BCLK)`	66.67 MHz	The 68LC060 runs at BCLK (1:1). BUILD_ID (`$FEC0_003C`) reads `$57` (87); the CPU_SPEED field of GAVIN_ID (bits 7-4) reads 6. SRAM reads have no wait state and writes 1; the SD0 / SD1 fast clock is 22.2 MHz; the local-bus clock is 33.33 MHz (see Clocks). The sources can still build a 50 MHz design (define CLK66 off in Config_A256x.vh); it is not released and not described in this manual.
`DDR3 L2 cache`	on	64 KB L2 cache between the CPU and DDR3.
`Line bursts`	on	68060 line transfers (16 bytes) to SRAM and DDR3 run as real 4-beat bursts.
`DDR3 port arbiter`	on	DDR3 is shared through a 7-port arbiter: port 0 = GUI bitmap engine or GGE line engine (one owner at a time, GGE_ENABLE selects), port 1 = CPU (through the L2), port 2 = VDMA, port 3 = MDMA, port 4 = GGE DAC player (build 66), port 5 = PCIe DMA (build 74), port 6 = debug port (build 80).
`Debug port`	on	Debug / programming UART on the bus clock (66.67 MHz), protocol v2 only since build 86 (`$5A` frames, 8 Mbit/s; the old `$55` protocol of FoenixMgr / the GenX Uploader was removed). DDR3 access build 81, flash programming build 82; build 85: SRAM / flash access while the CPU runs, pause / continue / bus step, 4 breakpoints, monitor stop on level 7. See the debug port sections of the memory chapter.

OPTION	BUILD 87	EFFECT
MDMA	on	SRAM <-> DDR3 DMA at $FEC0_C100 (build 26), interrupt $11.
VDMA	on	DDR3 block mover / blitter at $FEC0_C200 (build 27), interrupt $10.
Game Graphics Engine (GGE)	on	Built in since build 60 (a new design; the old VICKY IV game engine, GGE_ON, is retired). Registers, palettes and the DAC player at $FD00_0000 - $FD0F_FFFF. See the GGE chapter.
PCI Express	on	Root port since build 74: AXI Bridge for PCI Express (PG055), Gen2 x4. Configuration (ECAM) window $FA00_0000 - $FA7F_FFFF, memory window $4000_0000 - $7FFF_FFFF, card DMA into DDR3 through arbiter port 5, interrupt source $23. See the PCI Express chapter.
Boot block selection	on	Build 83: BOOT_SEL, BOOT_INFO, BOOT_BASE and BOOT_SIZE at $FEC0_0040 - $FEC0_004C. Software chooses the flash block that the boot copy uses on the next boot. See Reset and boot sequence.
I2C master	on	Build 84: A2560Me_I2C at $FEC0_0E00 (GAVIN). Transactions with a 256-byte FIFO and byte-level commands, 100 kHz / 400 kHz / 1 MHz, multi-master safe (the RP2040 shares the bus), the automatic SiI9022 set-up. Polled (no interrupt line). See the GAVIN chapter.
Floating Math Block	on	Build 87: A2560Me_FPU at $FEC0_4000 (GAVIN). Single-precision multiply, divide, add / subtract, square root, sine and cosine; float32, 20.12 fixed point and int32 conversions; a BUSY bit (no wait states). See the GAVIN chapter.
FPGA temperature monitor	on	Build 87: A2560Me_SysMon at $FEC0_0D00 (GAVIN). The die temperature from the DDR3 controller's XADC, in 1/16 °C, with MAX / MIN and a limit flag. See the GAVIN chapter.

CPU address map

The address decoder in the CPU interface splits the 4 GB address space into the regions below. Each region also checks the kind of access (the 68060 transfer type and transfer modifier): register areas only answer data accesses, the flash answers data and instruction accesses, and SRAM and DDR3 answer every access type. An access that matches no region ends with a bus error (TEA). Interrupt acknowledge and other CPU-space cycles are always answered. Since build 81 the interrupt-acknowledge (CPU space) decode is qualified by TIP like every other chip select; before, floating TT pins between cycles could turn on the FPGA data drivers.

Holes inside a decoded region are not bus errors: they get a normal acknowledge, and a read returns whatever that block puts on its data bus (often a fixed filler such as $DEAD_BEEF).

> Flash window: the flash output enable is asserted on every access to $FFC0_0000-$FFFF_FFFF, reads and writes alike, so a CPU write there makes the flash and the CPU drive the data bus at the same time. The write still gets TA and changes nothing. Treat the window as read-only. (From the RTL.) The register sub-block windows are taken from the decoders (build 87); register layouts are in each block's chapter.

Main regions

ADDRESS	SIZE	WHAT	ACCESS TYPES / NOTES
0000_0000-007F_FFFF	8 MB	Fast SRAM, 32-bit, directly on the CPU bus	All access types. 0 read and 1 write wait state. Line bursts. The first 64 KB are loaded from flash at power-up (reset vectors). The top 16.5 KB ($007F_BE00 - $007F_FFFF) are used by convention: debug mailbox, boot hand-over record, flash staging (see Fast SRAM in the memory chapter).
0080_0000-3FFF_FFFF	1016 MB	DDR3 (1 GB device), through the 64 KB L2 cache	All access types except CPU space (including cache pushes and MMU table walks). Line bursts. DDR3 address = CPU address, so the first 8 MB of DDR3 are hidden behind the SRAM for the CPU.

ADDRESS	SIZE	WHAT	ACCESS TYPES / NOTES
4000_0000-7FFF_FFFF	1 GB	PCIe memory window (build 74): PCIe memory space at the same addresses (CPU $4010_0000 = PCIe $4010_0000)	Uncached (TCI on every access). Byte, word and long accesses, aligned; no MOVE16. Address invariant byte order. Accesses are sent only after the root port's bridge enable is set. See the PCI Express chapter.
8000_0000-F9FF_FFFF	-	Unmapped	Bus error.
$FA00_0000-$FA7F_FFFF	8 MB	PCIe configuration window (ECAM, build 74): configuration space of buses 0-7 and the root port's core registers	Data accesses only. Uncached (TCI). Byte, word and long accesses. An access that the core does not complete within about 2 ms ends normally and reads $FFFF_FFFF (no bus error).
$FA80_0000-$FAFF_FFFF	8 MB	Mirror of the PCIe configuration window	As $FA00_0000 - $FA7F_FFFF.
$FB00_0000-$FBFF_FFFF	-	Unmapped	Bus error.
$FC00_0000-$FC00_0FFF	4 KB	GUI bitmap engine (chip select CS_GUI, build 60)	Data accesses only. Up to build 59 the whole $FC00_0000 - $FC0F_FFFF (VICKY IV window) answered, with the GUI registers repeated.
$FC00_1000-$FCFF_FFFF	-	Unmapped (since build 60)	Bus error.
$FD00_0000-$FD0F_FFFF	1 MB	Game Graphics Engine (chip select CS_GGE, build 60): registers, palettes, DAC player	Data accesses only. Internal register timing. Unused addresses inside read 0 (no bus error).
$FD10_0000-$FEBF_FFFF	-	Unmapped	Bus error.
$FEC0_0000-$FEC1_FFFF	128 KB	GAVIN (system controller)	Data accesses only.
$FEC2_0000-$FEC3_FFFF	128 KB	BEATRIX (sound)	Data accesses only.
$FEC4_0000-$FEC7_FFFF	256 KB	Legacy window (old VICKY III channel A). Nothing is there.	Any access gets TA at once (no bus error); writes are ignored and reads return an undriven (undefined) bus. Kept because older MCP versions still write here.
$FEC8_0000-$FECF_FFFF	512 KB	VICKY II (text mode, mouse pointer, gamma, fonts, MemText and GemTOS registers)	Data accesses only.
$FED0_0000-$FFBF_FFFF	-	Unmapped	Bus error.
$FFC0_0000-$FFFF_FFFF	4 MB	Boot flash	Data and instruction reads (no MMU table walks). 5 wait states. The CPU cannot program the flash: the flash write strobe is driven only by the debug port. Do not write to this window (see the note).

Sub-blocks of the register area (overview; registers are described in the GAVIN, BEATRIX and video chapters)

ADDRESS	BLOCK
$FEC0_0000-$FEC0_007F	GAVIN control registers (LEDs, IDs, DIP switches, memory-system control, boot block selection)
$FEC0_0080-$FEC0_00BF	Real-time clock (BQ4802, local bus)
$FEC0_0100-$FEC0_01FF	Interrupt controller

ADDRESS	BLOCK
$FEC0_0200-$FEC0_02FF	Timers
$FEC0_0300 / $FEC0_0340 / $FEC0_0380	SD card 0 (front) / SD card 1 (back) / SD card 2 (local bus)
$FEC0_0400-$FEC0_04FF	IDE / SATA bridge (local bus)
$FEC0_0500-$FEC0_05FF	Joystick / joypad (local bus)
$FEC0_0600-$FEC0_07FF	Ethernet LAN9221 (local bus)
$FEC0_0800-$FEC0_09FF	WizFi360 UART
$FEC0_0B00 / $FEC0_0C00	USB UART COM3 / COM4
$FEC0_0D00-$FEC0_0DFF	FPGA temperature monitor (build 87)
$FEC0_0E00-$FEC0_0EFF	I2C master (system I2C bus: HDMI transmitter, MAC EEPROM, fan / temperature, PLL)
$FEC0_2000-$FEC0_23FF	LPC bus / Super I/O (COM1, COM2, PS/2, parallel, floppy)
$FEC0_3000-$FEC0_301F	Maths unit (fixed point)
$FEC0_4000-$FEC0_41FF	Floating Math Block: single-precision FPU (build 87)
$FEC0_C000-$FEC0_C0FF	SDMA (SRAM to SRAM DMA)
$FEC0_C100-$FEC0_C1FF	MDMA (SRAM <-> DDR3 DMA)
$FEC0_C200-$FEC0_C2FF	VDMA (DDR3 block mover / blitter)
$FEC2_0000-$FEC2_35FF	BEATRIX: control, PSG, OPL3, OPN2, OPM, CODEC, SID, DAC FIFOs, VS1053 link, YM2149
$FEC8_0000-$FEC8_00FF	VICKY II control registers
$FEC8_0400-$FEC8_0CFF	Mouse pointer graphics and registers
$FEC8_4000-$FEC8_42FF	Gamma tables (blue, green, red)
$FEC8_8000-$FEC8_9FFF	Text-mode font memory
$FEC9_7000-$FEC9_7FFF	GemTOS bitmap registers
$FEC9_8000 / $FEC9_9000 / $FEC9_A000	MemText control / MemText LUTs / MemText font
$FECA_0000 / $FECA_8000	Text memory / colour memory
$FECA_C400 / $FECA_C440	Text foreground / background colour tables
$FC00_0000-$FC00_0FFF	GUI bitmap engine
$FD00_0000-$FD0F_FFFF	Game Graphics Engine: master control, layers, collision, DAC player, sprites, palettes
$FA00_0000-$FA7F_FFFF	PCI Express root port: configuration space (ECAM) and core registers

Bus errors (TEA)

Since build 18, an access that no block decodes ends with TEA (transfer error acknowledge) instead of TA. The 68060 then takes an access-fault (bus error) exception. Before build 18 such an access got TA and read garbage.

The bus error is given one clock after the cycle starts; the cycle is not acknowledged with TA.

- Unmapped means: not SRAM, not DDR3, not a PCIe window, not the GUI window `$FC00_0000` - `$FC00_0FFF`, not the GGE window `$FD00_0000` - `$FD0F_FFFF`, not GAVIN / BEATRIX / VICKY II, not the legacy window `$FEC4_0000` - `$FEC7_FFFF`, not the flash, and not a CPU-space cycle.

- A decoded region accessed with the wrong kind of access is also unmapped. Examples: an instruction fetch from register space (`$FA..`, `$FC00_0..`, `$FD0..`, `$FEC..`), or an MMU table walk into the flash or into register space.
- Since build 60, `$FC00_1000` - `$FC0F_FFFF` is unmapped: an access there ends with a bus error. Up to build 59 it repeated the GUI engine registers.
- Holes inside a register block are not bus errors.
- Interrupt acknowledge and other CPU-space cycles are always answered.
- MCP 107 and later has a test command, TEST BUSERR [addr], which reads `$F000_0000` by default and should end on the exception screen.

> If unexpected bus errors appear after a software change, check first whether the code touches an address outside the map above. The legacy window `$FEC4_0000-$FEC7_FFFF` deliberately does not give bus errors, so older MCP builds that still write there keep working.

Transfer sizes and byte lanes

The 68060 uses four transfer sizes: byte, word (16 bits), longword (32 bits) and line (16 bytes, four longwords). Misaligned operands are split into aligned cycles by the CPU itself.

The bus is big-endian. Within a longword, the byte at offset 0 travels on D31-D24 and the byte at offset 3 on D7-D0. A word at offset 0 uses D31-D16, a word at offset 2 uses D15-D0. Longword and line transfers use all four lanes.

- SRAM and DDR3 accept every size. The SRAM byte enables follow the 68060 byte-select lines.
- Register blocks see the size of the access. Many registers only react to one size; for example the GAVIN control registers only take 32-bit writes. Check each register's description.
- On every access to a register block (GAVIN, BEATRIX, VICKY II, GUI engine, GGE, PCIe windows) the FPGA asserts TCI together with TA, so the 68060 never loads register data into its caches, whatever the MMU page attributes say.
- DDR3 endian inversion: when GAVIN control register 0 bit 6 is set, byte and word accesses to DDR3 are mirrored inside the longword (byte offset n is stored at offset 3-n, word offset 0 at offset 2). Longword and line accesses are not changed. The bit is 0 after reset since build 23.

Byte lanes

SIZE	OFFSET (A1:A0)	DATA LINES
`Byte`	0	D31-D24
`Byte`	1	D23-D16
`Byte`	2	D15-D8
`Byte`	3	D7-D0
`Word`	0	D31-D16
`Word`	2	D15-D0
`Longword / line`	0	D31-D0

Line (burst) transfers `$FEC0_0000`

The 68060 uses line transfers for cache line fills, cache pushes (copyback) and MOVE16. A line is 16 bytes, moved as four longwords with one TA each. The order wraps around inside the line, starting with the longword addressed by A3:A2.

Line transfers to the SRAM and to DDR3 are served as real bursts. The FPGA generates the SRAM address bits A3 and A2 itself during a burst. An SRAM line read is 2-2-2-2 (no read wait state: one TA clock and one gap clock per longword) and an SRAM line write is 3-3-3-3. For DDR3 the burst starts when the L2 cache has the whole line ready.

A line transfer to any other target (flash, register blocks) is refused with TBI (transfer burst inhibit) on the first TA. The 68060 then repeats the line as four separate longword cycles. Instruction fetches from the flash therefore run as single longword reads.

Bursts can be switched off per memory type with the memory-system control register below. With bursts off, line transfers to that memory also get TBI, and MOVE16 uses the older single-cycle path.

This register lives in GAVIN (`$FEC0_0038`). It was added to locate memory-system problems; normal software leaves it at 0 and only uses L2_FLUSH.

Line timing (per 16-byte line, including the TS clock; 1 bus clock = 15 ns; SRAM from the wait-state settings, DDR3 from simulation)

TARGET	BUS CLOCKS	TIME
SRAM line read (9 clocks on builds 34-35)	8	120 ns
SRAM line write	12	180 ns
DDR3 read, L2 hit	13	195 ns
DDR3 read, L2 miss (cold)	about 25	about 375 ns (mostly DDR3 latency, which does not scale with the bus clock)
DDR3 line write	12	180 ns

$38 MEMSYS_CTRL (GAVIN control register 14)

RW · reset $4C00_0000 (read value)

Memory-system control: L2 cache and burst switches, and local-bus settings in bits 23-16 and 14-8 (bits 13-8 since build 34, bit 14 since build 38, bits 23-16 since build 39), which read back as written; bit 15 (read only, build 40) shows the board type. 32-bit writes only. All control bits are 0 after reset (normal operation: L2 on, bursts on).



BITS	FIELD	ACCESS	MEANING
31-24	**SIGNATURE**	R	Always reads `$4C`.
23-20	**IDE_REC_EXTRA**	RW	Build 39+: IDE write recovery + 2n bus clocks: the write data is held, and DIOW- stays high, that much longer after each data-register write (default 100 ns). Diagnostic / margin; 0 after reset.
19-16	**IDE_WR_EXTRA**	RW	Build 39+: IDE write strobe (DIOW-) + 2n bus clocks (default 340 ns; 2 = 420 ns). Diagnostic / margin; 0 after reset.
15	**WHITE_BOARD**	R	Build 40+: 1 = the FPGA latched white-board mode from DIP switch 7 at the last reset (F14 = IDE DMACK- held high, D14 = IDE DMARQ not driven); 0 = black board.
14	**LBCLK_PHASE**	RW	0 = local-bus clock edges half a BCLK after the BCLK edges (normal), 1 = on the BCLK edges (the build 37 phase; about half of the 10M02 writes are lost). 0 after reset.
13	**LBCLK_OFF**	RW	1 = the local-bus clock output (LOCAL_BUS_CLK) is held low. Diagnostic; 0 after reset (since build 34).
12	**LAN_FILTER_OFF**	RW	1 = the LAN9221 read majority filter is off. Diagnostic; 0 after reset = filter on (since build 34).
11-8	**LAN_RD_EXTRA**	RW	LAN9221 read strobe + n bus clocks. Diagnostic; 0 after reset (since build 34).
4	**SLOW_BEATS**	RW	1 = one extra gap clock per burst beat (3 clocks per longword). Debug setting.
3	**SRAM_BURST_OFF**	RW	1 = line transfers to the SRAM are refused with TBI (split into 4 longword cycles).
2	**DDR3_BURST_OFF**	RW	1 = line transfers to DDR3 are refused with TBI.
1	**L2_FLUSH**	W	Write 1 = invalidate the whole L2 cache (one-clock pulse). Reads 0.
0	**L2_OFF**	RW	1 = L2 cache disabled.

Memories: SRAM, DDR3 and the L2 cache

- SRAM: 8 MB, 32 bits wide (two ISSI IS61WV204816BLL-10, 2M x 16, 10 ns), on the CPU address and data pins. The FPGA makes the chip select, output enable, write strobe and byte enables. 0 read and 1 write wait state. The FPGA bus masters (MemText, GemTOS bitmap, SDMA, MDMA) also use it through the same pins (see Bus arbitration).
- DDR3: 1 GB, 32 bits wide (two 16-bit DDR3 chips), memory clock 400 MHz (DDR3-800), controller user clock 200 MHz. The CPU reaches it through the L2 cache and port 1 of the DDR3 port arbiter; the GUI bitmap engine or the GGE reads it through port 0, the GGE DAC player through port 4, PCIe cards (DMA) through port 5 (build 74), and the debug port through port 6 (build 80).
- L2 cache: 64 KB, 2-way set associative, 32-byte lines (one DDR3 packet), write-through with no write-allocate, 4-entry posted write buffer. A write gets TA at once unless the write buffer is full. A read miss waits until the write buffer has drained, so it never returns data older than a posted write.
- Coherence: DDR3 always holds the CPU's latest writes, so DMA engines and the video engines see them. The MDMA and VDMA invalidate the L2 lines they write in hardware before they report DONE, and since build 74 the PCIe DMA path invalidates them before a card's write completes, and since build 81 the debug port invalidates the L2 lines it wrote after each write frame; for any other master that writes DDR3, software must invalidate the L2 (L2_FLUSH) before the CPU reads that data.
- The 68060's own caches never see DMA writes. Push or invalidate the affected lines (CPUSHA / CINV) around a DMA transfer, or use cache-inhibited buffers.
- DDR3 timing figures from the design notes (simulation, single accesses): a read hit takes about 7 bus clocks inside the cache plus about 2.5 clocks of acknowledge pipeline.
- Build 59: the L2 takes CPU write data and byte enables into flip-flops before it writes its RAM and pushes the write buffer (one clock later). Builds 57 / 58 wrote them straight from the CPU pins into the block RAM (0.58 ns of timing margin at 66.67 MHz), and with the 68060 caches on a bit of the 4th longword of a burst write was sometimes stored wrong: NetBSD stopped at boot. TA timing is unchanged.
- Build 74: in a line burst read from DDR3 the L2 now moves its output to the next longword on the clock edge that ends each beat, so the next longword has two bus clocks before the next TA. Up to build 73 it moved on the same edge as the next TA, and TEST CACHE could fail on the slowest data bits (see the memory chapter).

Cycle termination and wait states

The FPGA ends every CPU cycle with TA (normal end), TEA (bus error) or, for an interrupt acknowledge at level 6 or 7, AVEC. The 68060 runs in the MC68040-compatible acknowledge mode (set at reset). TA, TEA, TBI, TCI, AVEC and BG come from rising-edge flip-flops in the I/O pads, and BCLK is sent out through an ODDR from a phase-advanced copy of the bus clock (about 6.5 ns early inside the FPGA), so BCLK reaches the CPU slightly before the FPGA's own clock edge.

The table gives the length of a single cycle in bus clocks, counted from the clock where the CPU asserts TS to the clock where it samples TA. The figures are derived from the acknowledge controller (wait counts are in bus clocks), not measured on the board.

> Wait counts are fixed in bus clocks. The flash wait is derived from an 80 ns rule (the flash is a 70 ns part), which gives 5 wait states at 66.67 MHz. SRAM reads have no wait state (since build 36): the SRAM chip select is held low while the CPU owns the bus and the CPU address bits A3:A2 pass straight to the SRAM, so the 10 ns SRAM has its full access time inside two clocks (routed A3:A2 path 6.95 ns against a 7 ns bound). Writes keep 1 wait state. Builds 34-35 used 1 read and 1 write wait state.

Acknowledge per region (single cycles; 1 clock = 15 ns)

REGION	TERMINATION	BUS CLOCKS
`SRAM`	TA. No wait state for reads, 1 for writes	Reads 2 (30 ns), writes 3 (45 ns)
`Flash`	TA after 5 wait states	7 (105 ns)
`GAVIN, BEATRIX, VICKY II, GUI engine, GGE registers`	TA after 2 wait states	4 (60 ns)
`GAVIN local-bus devices: RTC, SD card 2, IDE, joystick, Ethernet`	TA when the local-bus access has finished (handshake)	device dependent

REGION	TERMINATION	BUS CLOCKS
`LPC / Super I/O ($FEC0_2000-$FEC0_23FF)`	Writes: TA when the command is queued (posted). Reads: TA when the byte has come back from the LPC side.	variable; an LPC timeout returns `$FF`; a safety timeout of about 1.3 ms prevents a hang
`DDR3`	TA when the L2 cache is ready (handshake); writes are posted	variable (see Memories)
`PCIe memory window $4000_0000-$7FFF_FFFF (build 74)`	TA when the PCIe core has completed the access (handshake)	variable
`PCIe configuration window $FA00_0000-$FAFF_FFFF (build 74)`	TA when the PCIe core has completed the access; with no answer the access ends normally after about 2 ms and reads `$FFFF_FFFF`	variable
`Legacy window $FEC4_0000-$FEC7_FFFF`	TA at once, nothing driven	2
`Unmapped address`	TEA (bus error)	2
`Interrupt acknowledge`	TA with the vector, or AVEC for levels 6 and 7	2

Back-to-back cycles (register chip-select gating)

The 68060 keeps TIP (transfer in progress) asserted between two bus cycles that follow each other directly. This happens often with the instruction cache or the store buffer on: two register writes can run with no gap between them.

Most register blocks detect an access by the rising edge of their chip select. Before build 20 the chip selects were address decodes qualified by TIP, so two back-to-back cycles to the same block gave only one edge: the second write was lost, and a second local-bus access (RTC, IDE, SD card 2, joystick, Ethernet) never started, so its TA never came and the CPU hung.

Since build 20 a cycle gate (Reg_Cycle_Gate) makes the chip selects of the register blocks (GAVIN, BEATRIX, VICKY II, GUI engine, GGE, PCIe) active only from the clock where TS is sampled to the clock where TA or TEA ends the cycle. Every cycle therefore gives a fresh chip-select edge. Inside the FPGA an access starts one clock later, but every register block already has at least 2 wait clocks, so the cycle length does not change.

- Software does not need gaps or dummy accesses between register accesses.
- SRAM, DDR3 and flash are not affected by the gate (DDR3 starts on TS itself).
- LPC / Super I/O accesses are also detected per bus cycle (build 19 and 20 changes).

Bus arbitration (bus masters)

The CPU bus to the SRAM can be owned by the 68060 or by one FPGA bus master at a time. The FPGA masters only ever access the SRAM: while one of them owns the bus, the FPGA drives the SRAM address, data and strobes on the CPU pins. The arbiter runs on the bus clock (the whole SRAM side runs on the 66.67 MHz bus clock).

Bus parking: when no FPGA master is waiting, BG stays asserted to the CPU, so it starts its cycles without a request/grant round trip. BGR is always asserted, so the 68060 gives the bus up after its current cycle when BG is taken away, even inside a locked sequence (no FPGA master does read-modify-write).

Safe hand-over (build 18): the arbiter first negates BG, waits 8 clocks (120 ns: one BCLK edge with BG negated plus the synchronizers), and grants the FPGA master only when BB shows that the CPU has no bus tenure. If the CPU started a cycle meanwhile, the arbiter waits for the end of that tenure.

Fairness: after an FPGA master has finished, the CPU keeps the bus for at least 32 arbiter clocks (480 ns) if it wants it, so DMA bursts cannot starve the CPU. While an FPGA master is waiting, BG is taken away from the CPU even if it keeps requesting.

- The channel code tells the memory-management block which master drives the SRAM: bit 0 = direct DMA path, bit 1 = MDMA (0 = SDMA). MemText and GemTOS use code 00.
- The SDMA releases the bus after 32 transfer units or at the end of each 2D line, the MDMA after 32 longwords; both then stay off the bus for 8 clocks before asking again.
- Simulation (design notes): an FPGA master waited at most about 0.9 us for the bus with a busy CPU; MemText's worst wait was 3.5 us against a limit of about 33 us.
- Debug port: with the CPU held (command `$30` ATTR 0) it holds the CPU in reset and drives the SRAM and flash directly; since build 77 no FPGA bus master (display fetch, DMA) is granted meanwhile, and a debug job waits for a running master tenure to end. Since build 85 it is also a bus master of the arbiter, with the highest priority: with the CPU running or paused, a memory job requests the bus, drives the pins only while granted and gives the bus back every 256 clocks (between two bytes). Its DDR3 accesses go through DDR3 arbiter port 6.
- Pause and bus step (build 85): on a debug-port pause the arbiter takes the bus from the CPU with the safe hand-over and keeps BG negated; the CPU is not reset, it waits. FPGA masters are still granted, and their tenures return to the pause. A bus step gives BG back until the CPU's next tenure starts, then pauses again after that tenure.
- BB, TS and BTT have FPGA pull-ups (build 18b): TS is three-stated when the CPU does not own the bus, so the arbiter only uses BB to see a CPU tenure.
- TIP (build 44+): the 68060 three-states TIP when it does not own the bus (MC68060 UM 2.4.2), and every CPU chip select (SRAM, DDR3, flash, GAVIN, VICKY, BEATRIX, PCIe; since build 81 the interrupt-acknowledge decode too) is qualified by TIP. Up to build 43 TIP had no pull-up, so while an FPGA master owned the bus (display fetch, SDMA, MDMA) these decodes looked at a floating pin; build 43 showed CPU-side DDR3 corruption and hangs with the EmuTOS display on. Since build 44 TIP has an FPGA pull-up and the chip selects also need the arbiter's CPU_Cycle_Allowed flag (registered; on in IDLE and while the CPU finishes its cycle after BG is negated, off while an FPGA master owns the bus). GAVIN register 2 bits 31-24 count TIP-asserted events during FPGA tenures (diagnostic, should stay 0).
- Build 81: the arbiter's paths to the SRAM control pins and to the CPU address and data pins have timing bounds. In build 80 their placement corrupted CPU SRAM accesses while the TOSGraph (GemTOS bitmap) master shared the bus.

FPGA bus masters (highest priority first)

MASTER	WHAT IT DOES	SRAM CHANNEL CODE	BUILD 86
`Debug port (build 85)`	Host reads of SRAM / flash and writes of SRAM while the CPU runs or is paused; gives the bus back every 256 clocks	- (own debug path)	active on a host request
`GemTOS bitmap`	Reads the GemTOS/EmuTOS bitmap screen from SRAM for display	00	active when enabled
`MemText`	Reads text rows from SRAM for the MemText display	00	active when enabled
`SDMA`	SRAM to SRAM copy / fill engine (`$FEC0_C000`)	01	active
`MDMA`	SRAM side of SRAM <-> DDR3 transfers (`$FEC0_C100`)	11	active

Interrupts as seen by the CPU

All interrupt sources go to the interrupt controller in GAVIN (`$FEC0_0100` , described in the GAVIN chapter). It has 48 sources in 6 groups of 8. Each group drives one CPU interrupt level on IPL2-IPL0. When several groups are active, the highest level is presented. The IPL lines are registered on the bus clock, so they never glitch (one clock of extra latency).

When the 68060 takes an interrupt it runs an interrupt acknowledge cycle (transfer type 11, address `$FFFF_FFFF` , acknowledged level on TM2-TM0). The FPGA answers with the vector of the level that the CPU acknowledges - not of whatever is highest at that moment. Level 6 is answered with AVEC (autovector), and so is level 7, the debug port's monitor stop (build 85). Levels 5 to 1 return a vector number from the group: base + the number of the highest pending, enabled source in that group (source 7 of a group wins over source 0).

If the acknowledged level has nothing pending any more (for example because software cleared the pending bit while the CPU was already acknowledging), the FPGA returns the spurious interrupt vector `$18` (24).

- Install a handler on vector `$18` (spurious interrupt) that just returns with RTE. It can happen legitimately.
- Level 6 (group 0) uses the 68060 autovector, so all group 0 sources share vector `$1E`; the handler must read the controller's pending register to find the source.
- Level 7 (build 85+): the debug port raises it to stop the CPU in a monitor stub (see the debug port sections of the memory chapter). Software without a stub should install a handler on vector 31 that only does RTE. The MCP (172 and later) has the stub.
- Several sources are not connected in build 87 and never fire, among them: the OPL3, OPN2, OPM and PWM interrupts, the IDE/SATA interrupt and the 44.1 kHz and 48 kHz DAC interrupts (tied inactive; the 44.1 kHz FIFO was retired in build 69), and DMA channels `$12` - `$14` and `$16`. Since builds 61 / 66, SOF and SOL come from the GGE (see the GGE chapter) and `$15` is the GGE DAC player; since build 70 `$29` is the VS1053 link; since build 74 `$23` is the PCI Express root port.
- Pending, mask, polarity and edge registers, and the source list, are described in the GAVIN chapter.

> The source names in the table come from the interrupt controller's source list; check the GAVIN chapter for which inputs are wired in the current build.

Interrupt levels

IPL LEVEL	GROUP (SOURCES)	SOURCES (CONTROLLER NUMBERING)	VECTORS (VECTOR TABLE ADDRESS)
7	- (debug port)	Build 85+: the debug port's monitor stop only (command `$32`, or a breakpoint with the NMI flag). Not maskable (the 68060 NMI). Pending until the CPU acknowledges it. Never generated by the interrupt controller.	Autovector: vector `$1F` (31), address `$0000_007C`
6	Group 0 (sources `$00` - `$07`)	`$00` SOF (start of frame: the 108 MHz video, or the GGE while GGE_ENABLE is set), `$01` SOL (GGE line interrupt, LINE_IRQ), `$02` - `$03` reserved, `$04` keyboard, `$05` PS/2 keyboard, `$06` PS/2 mouse, `$07` USB UART COM3	Autovector: vector `$1E` (30), address `$0000_0078`
5	Group 1 (sources `$08` - `$0F`)	Super I/O: `$08` PS/2 keyboard, `$09` PS/2 mouse, `$0A` floppy, `$0B` COM2, `$0C` COM1, `$0D` LPT1, `$0E` MPU-401; `$0F` RTC (BQ4802)	`$40` - `$47` (address `$0000_0100` - `$0000_011C`)
4	Group 2 (sources `$10` - `$17`)	`$10` VDMA (DDR3 blitter), `$11` MDMA (SRAM <-> DDR3), `$12` - `$14` and `$16` DMA channels (not connected), `$15` GGE DAC player, `$17` SDMA	`$48` - `$4F` (`$0000_0120` - `$0000_013C`)
3	Group 3 (sources `$18` - `$1F`)	`$18` SD card insert, `$19` - `$1B` timers 0-2, `$1C` timer 3 (start of frame), `$1D` VIA0, `$1E` VIA1, `$1F` WizFi360 receive FIFO	`$50` - `$57` (`$0000_0140` - `$0000_015C`)
2	Group 4 (sources `$20` - `$27`)	`$20` Ethernet LAN9221, `$21` WizFi360, `$22` cartridge, `$23` PCI Express, `$24` USB UART COM4, `$25` IDE/SATA, `$26` north-bridge joystick, `$27` reserved	`$58` - `$5F` (`$0000_0160` - `$0000_017C`)
1	Group 5 (sources `$28` - `$2F`)	`$28` SAM2695 MIDI receive, `$29` VS1053 link (data FIFO low), `$2A` OPL3 timer, `$2B` OPN2 timer, `$2C` OPM timer, `$2D` PWM FIFO, `$2E` DAC 44.1 kHz FIFO (retired), `$2F` DAC 48 kHz FIFO	`$60` - `$67` (`$0000_0180` - `$0000_019C`)
any	-	Acknowledged level with nothing pending	Spurious interrupt, vector `$18` (24), address `$0000_0060`

Clocks

The board provides several crystal oscillators to the FPGA. Three clock managers (MMCMs) in the FPGA make the internal clocks; the DDR3 controller makes its own user clock.

The CPU gets BCLK from the FPGA: 66.67 MHz. The local-bus clock (LOCAL_BUS_CLK, used only by the 10M02) is BCLK / 2 = 33.33 MHz: the 10M02 is designed for a 33 MHz clock, and at 66.67 MHz its register writes and its SD card shifter fail. Its rising edges come half a BCLK period (7.5 ns) after the BCLK edges, half way between the local-bus strobe changes (build 38); MEMSYS_CTRL bit 14 = 1 moves them back onto the BCLK edges (the build 37 phase, about half of the 10M02 register writes were lost). The 68060 core and

bus run at the same frequency (1:1, clock enable held constant). BCLK is sent out through an ODDR from a phase-advanced copy of the bus clock (205 degrees, about 6.5 ns early inside the FPGA), so it reaches the CPU slightly before the FPGA's own edge.

The SRAM-side logic (bus arbiter, MMU, MemText and EmuTOS fetch, SDMA, the SRAM side of the MDMA, the SD0 / SD1 SPI engine) runs on the bus clock. The video side of VICKY II and EmuTOS stays on 100 MHz. The GGE runs its registers on the bus clock, its line engine and DAC player on the DDR3 user clock (200 MHz) and its timing on 25 MHz.

Software that times itself by bus clocks can read the CPU speed field (GAVIN_ID bits 7-4: 6 = 66 MHz). MCP scales its busy-wait loops (dly_us, the Super I/O reset, the PS/2 delay) by this field (since MCP 124). Use MCP 160 or later.

> The RTL drives the 68060 CLKEN pin with a constant level: the CPU runs at the bus clock (1:1). The pin polarity on the board was not checked for this manual. Signal names in the RTL still say 50.25, 100.5, 201 and 402 MHz from an older clock setup. The MMCM (VCO 1200 MHz from 80 MHz) gives exactly 66.67, 100, 200, 400 and 25 MHz. GAVIN reports CPU speed ID 6 (66 MHz).

Clocks in build 87

CLOCK	SOURCE	FREQUENCY	USED BY
`80 MHz`	Oscillator input	80.000 MHz	Reference of the CPU clock MMCM (CPU_Clock_MMCM66) and the DDR3/peripheral clock MMCM
`BCLK / bus clock`	CPU clock MMCM (VCO 80 MHz x 15 = 1200 MHz, /18)	66.67 MHz	CPU interface and acknowledge controller, all register blocks (GAVIN, BEATRIX, VICKY II, GUI engine and GGE bus side), interrupt controller, debug port, I2C master, and the SRAM side (bus arbiter, MMU, MemText and EmuTOS fetch, SDMA, MDMA SRAM side, SD0 / SD1 SPI engine).
`BCLK output clock`	CPU clock MMCM (/18, phase 205 degrees)	66.67 MHz	ODDRs of the 68060 BCLK pin and of the local-bus clock pin (divided by 2: 33.33 MHz), about 6.5 ns ahead of the bus clock
`100 MHz`	CPU clock MMCM (/12, 180 degrees)	100 MHz	VICKY II and EmuTOS video side, 2x domains of GAVIN/BEATRIX/VICKY II, video engine clock.
`200 MHz`	CPU clock MMCM (/6)	200 MHz	Reset controller, 4x domains of GAVIN/BEATRIX/VICKY II, video timing
`DDR3 memory clock`	CPU clock MMCM (/3)	400 MHz	DDR3 controller (DDR3-800, 2:1 mode)
`DDR3 user clock`	DDR3 controller	200 MHz	DDR3 port arbiter, L2 cache DDR3 side, GUI engine DDR3 side, GGE line engine and DAC player
`25 MHz`	CPU clock MMCM (/48)	25.000 MHz	GGE dot clock: GGE timing and output (and the HDMI pixel clock while GGE_ENABLE is set); debug-port flash programmer
`IODELAY reference`	DDR3/peripheral MMCM (80 MHz x 15 / 6)	200 MHz	DDR3 input delay calibration
`Serial clock`	DDR3/peripheral MMCM (/50)	24 MHz	Serial ports (UART domain)
`SAM2695 clock`	DDR3/peripheral MMCM (/100)	12 MHz	Output pin to the SAM2695 MIDI synthesiser
`VS1053 clock`	24.576 MHz / 2 (ODDR)	12.288 MHz	Output pin (D19) to the VS1053b decoder, its CLKI (build 70)
`LED clock`	DDR3/peripheral MMCM (/30)	40 MHz	RGB LED drivers
`Video (dot) clock`	Video MMCM (74.25 MHz x 14 / 9.625)	108 MHz	Video timing, text mode, MemText display, pixel mixer, GUI engine display, HDMI pixel clock while GGE_ENABLE is 0 (1280x960 and 1280x1024 at 60 Hz)
`74.25 MHz`	Oscillator input	74.25 MHz	Reference of the video MMCM

CLOCK	SOURCE	FREQUENCY	USED BY
`14.318 MHz`	Oscillator input	14.318 MHz	Sound chips, 32 kHz Super I/O clock, power-on / cold-reset timer
`22.5792 MHz`	Oscillator input	22.5792 MHz	AK4432 DAC (also sent out as its master clock, through an ODDR) and the 44.1 kHz tick of the GGE DAC player
`24.576 MHz`	Oscillator input	24.576 MHz	48 kHz CODEC (also sent out as the CODEC master clock, through an ODDR) and the 48 kHz tick of the GGE DAC player; divided by 2 for the VS1053 (build 70)
`33.333 MHz LPC`	Oscillator input	33.333 MHz	LPC bus (also forwarded to the LPC clock pin)
`33.333 MHz SYS`	Oscillator input	33.333 MHz	Not used

Reset and boot sequence

Reset is built around the board net COLD_RESETn. The FPGA can pull it low itself, and it also receives it as the external reset input. While COLD_RESETn is low, or while any clock manager is unlocked, every clock domain is held in reset.

Release happens in this order:

- 1. Cold reset timer: once the CPU clock MMCM is locked, a counter on the 14.318 MHz clock holds COLD_RESETn low for 2^24 clocks (about 1.17 s).
- 2. Reset controller: after COLD_RESETn goes high and all clocks are locked, a 20 ms filter runs, then the reset of each clock domain is released in step with its own clock.
- 3. HDMI encoder set-up: the I2C master holds the SiI9022 HDMI transmitter in reset for about 200 ms and the rest of the system waits for that; then it programs the SiI9022 over the I2C bus (at 100 kHz) while the system starts. Since build 84 this is the new I2C master at `$FEC0_0E00` (see the GAVIN chapter).
- 4. Boot copy: the FPGA copies the first 64 KB of the chosen flash block (block 0 = `$FFC0_0000`, unless BOOT_SEL chose another) into SRAM `$0000_0000` - `$0000_FFFF`. Since build 83 it reads each longword from the flash into an FPGA register and then writes it to the SRAM: the flash and the SRAM share the address bus, so a block other than 0 cannot be copied with one address. 64 KB take about 4.2 ms. GAVIN, BEATRIX, VICKY II and the 68060 stay in reset during the copy.
- 5. CPU start: the FPGA presents the reset configuration (IPL2-IPL0 negated: MC68040 acknowledge protocol, no extra write-data hold, acknowledge-ignore state off; the data bus is driven to `$FFFF_0000`), releases the CPU reset and grants the bus to the CPU. The 68060 reads its initial stack pointer and program counter from SRAM `$0000_0000` and `$0000_0004` (the reset vectors copied from the block). An image stored in block N runs at that block's address and must be linked for it.
- A new cold reset (the full sequence above, about 1.2 s) is started by: a software reset through GAVIN control register 0 (bit 15 with `$DEAD` in bits 31-16, see the GAVIN chapter), the 68060 RESET instruction (the CPU's reset output), the debug port's system reset (command `$3F`), or a BOOT_SEL write with its key (build 83).
- Debug-port command `$30` with ATTR 0 / 1 holds and releases the CPU in reset without a cold reset; while held, the debug port owns the SRAM and flash bus. Since build 85, `$30` ATTR 2 / 3 pause and continue the CPU without any reset.
- Boot block selection (build 83): a longword write to BOOT_SEL (`$FEC0_0040`) with the key `$B0` in bits 31-24 stores a flash layout (bits 5-4: 1 x 4 MB, 2 x 2 MB, 4 x 1 MB or 8 x 512 KB) and a block number (bits 2-0) and starts a board reset (as the reset button, about 1.2 s); the boot copy then uses that block. A write without the key is ignored. If the chosen block is blank (its first longword is `$FFFF_FFFF`) or does not exist in that layout, the copy falls back to block 0. BOOT_INFO, BOOT_BASE and BOOT_SIZE (`$FEC0_0044` - `$FEC0_004C`) tell software what was booted. The choice survives only the reboot it requested: power-on, the reset button (or the GAVIN soft reset), the debug port's system reset and the 68060 RESET instruction (an operating system reboot) all clear it, so the board boots block 0 again. The registers are in the GAVIN chapter; MCP 166 has the command BOOTSEL [<block> <512K|1M|2M|4M>].
- Boot loader: block 0 holds a boot loader that reads /sd2/boot.cfg, counts down and boots the chosen block through BOOT_SEL, after writing a hand-over record to SRAM `$007F_BF00` (see Boot loader and /sd2/boot.cfg in the memory chapter).
- The FPGA itself is not reloaded by any of these resets.

Executing the 68060 RESET instruction resets the whole board, not only the external devices: the FPGA treats the CPU reset output as a cold-reset request. Software that only wants to reset peripherals must not use RESET. (Derived from the RTL; not tested on hardware for this manual.)

Board identity and DIP switches `$FEC0_0000`

The board's DIP switches and identity values are read through GAVIN control registers. They are listed here for orientation; the GAVIN chapter has the full registers.

The DIP switches are read as raw pin levels; the switch-to-level polarity is set by the board, not by the FPGA. DIP_BOOT: since build 83 the FPGA, and since MCP 166 the MCP, no longer use these four switches for booting (the boot block is chosen by software through BOOT_SEL, see Reset and boot sequence). They are free for software (user switches); a switch that is OFF reads 1. DIP_GAMMA goes to VICKY II (gamma correction default).

Board types (build 40+). Two batches of the board are in use. The white board (first batch) has the JMicron JM20330 IDE bridge directly on the local bus; FPGA pin F14 is the IDE DMACK- and pin D14 the IDE DMARQ (a JM20330 output). The black board (second batch) has 3.3 V / 5 V translators between the local bus and the JM20330; F14 is the translator direction (DIR) and D14 its output enable (OEn). One bitstream serves both: DIP switch 7 as marked on the part (positions 1-8; number 6 when counting from 0; pin V14, DIP_USER bit 2) selects the board, OFF (reads 1) = black, ON (reads 0) = white. The FPGA reads the switch 13 clocks after every system reset and enables the two pins 3 clocks later; until then both are high-impedance with pull-ups (translator disabled on the black board, DMACK- inactive on the white board). White mode: F14 is driven high all the time and D14 is never driven. MEMSYS_CTRL bit 15 shows the mode that was latched.

`$00` GAVIN_CTRL0 (status part)

R · reset -

Upper bits of GAVIN control register 0 report board inputs. Bits 16-0 are control bits (LEDs, endian inversion, LPC reset, ...) described in the GAVIN chapter.



BITS	FIELD	ACCESS	MEANING
31	**SD1_WP**	R	SD card 1 (back) write-protect input.
30	**SD1_CD**	R	SD card 1 (back) card-detect input.
29	**SD0_WP**	R	SD card 0 (front) write-protect input.
28	**SD0_CD**	R	SD card 0 (front) card-detect input.
27	**IN_ENCLOSURE**	R	Enclosure jumper input.
26-24	**DIP_USER**	R	User DIP switches 2-0. Bit 2 = switch 7 as marked on the part (pin V14) selects the board type since build 40: 1 (OFF) = black board, 0 (ON) = white board.
23-20	**DIP_BOOT**	R	Former boot DIP switches 3-0. Not used for booting since build 83 / MCP 166: free for software (user switches). OFF reads 1.
19	**PCIE_X8**	R	1 = PCIe x8 present pin asserted.
18	**PCIE_X4**	R	1 = PCIe x4 present pin asserted.
17	**PCIE_X1**	R	1 = PCIe x1 present pin asserted.

$0C GAVIN_ID R · reset $0009_B06C

Machine, CPU and clock identification (constant).

31–16	15–12	11–8	7–4	3–0
CHIP_SUBVERSION	CPU_ID	–	CPU_SPEED	MACHINE_ID

BITS	FIELD	ACCESS	MEANING
31-16	**CHIP_SUBVERSION**	R	`$0009`.
15-12	**CPU_ID**	R	`$B` = MC68LC060.
7-4	**CPU_SPEED**	R	6 = 66 MHz.
3-0	**MACHINE_ID**	R	`$C` = A2560Me.

$10 GAVIN_CHIP R · reset $7338_0011

Chip number and version (constant).

31–16	15–0
CHIP_NUMBER	CHIP_VERSION

BITS	FIELD	ACCESS	MEANING
31-16	**CHIP_NUMBER**	R	`$7338`.
15-0	**CHIP_VERSION**	R	`$0011`.

$18 GAVIN_HW_REV R · reset $4233_4100

PCB revision as ASCII: 'B', '3', 'A' in bits 31-8.

68060 control pins driven by the FPGA

- CDIS and MDIS are negated: the 68060 caches and MMU are available to software.
- BGR is always asserted (the CPU gives up the bus after its current cycle when BG goes away).
- TCI is asserted with TA on every register-block access (data not cached).
- TBI is asserted on the first TA of a line transfer to a target without burst support.
- AVEC is asserted for a level 6 or level 7 interrupt acknowledge (level 7: the debug port's monitor stop, build 85).
- CLA is not used (negated).
- IPL2-IPL0 are registered; during the reset configuration they are negated.

2 GAVIN: system controller and I/O

GAVIN is the system controller inside the A2560Me FPGA. It holds the board control registers (LEDs, buzzer, identification, memory-system control), the interrupt controller, the timers, two SD card SPI controllers, three UARTs (two to the USB-C bridge, one to the WiFi module), a fixed-point maths unit, the Floating Math Block (a single-precision FPU, build 87), an FPGA temperature monitor (build 87), and the bridges to the slower chips: the LPC bus to the Super I/O, and a 16-bit local bus to the real-time clock, the IDE/SATA bridge, the Ethernet controller and the 10M02 FPGA (joysticks, internal micro SD).

GAVIN answers at `$FEC0_0000` - `$FEC1_FFFF`, for data accesses in user or supervisor mode, never cached. It runs on the CPU bus clock (66.67 MHz); internal registers answer in a few clocks, local-bus and LPC devices answer with a handshake when their cycle is done.

Every block accepts only certain write sizes (listed per block). Reads of any size work unless stated. This chapter describes build 87 of the FPGA; where the MCP headers disagree with the hardware, the hardware (RTL) is given and the difference is noted.

GAVIN address map `$FEC0_0000`

GAVIN answers at `$FEC0_0000` - `$FEC1_FFFF` (128 KB). The CPU interface selects GAVIN for data accesses only (user or supervisor data space); instruction fetches from this range end with a bus error. Every access to the range gets TCI (cache inhibit), so reads are never cached.

Inside the range GAVIN decodes address bits A16..A0. Every block needs A16 = 0, so `$FEC1_0000` - `$FEC1_FFFF` holds no device. Holes inside the range are still acknowledged (no bus error): reads return `$B00B_FEED` and writes are ignored.

Offsets in this chapter are relative to `$FEC0_0000` unless a section gives its own base. Byte lanes follow the 68060 big-endian convention: the byte at offset 0 of a longword is on D31-D24.

Register blocks detect an access on the rising edge of their chip select. Since build 20 the chip selects are only active from TS to the end of the cycle, so back-to-back accesses (instruction cache and store buffer on) each start a new access.

> Write sizes matter. Most GAVIN blocks only accept the write size given above: the CPU interface presents byte, word and longword write data on separate internal buses, and a block that takes its data from the byte bus sees `$00` on a word or longword write (and the reverse). A write of the wrong size is either ignored or writes zero. Reads of any size work unless a section says otherwise.

GAVIN windows

ADDRESS	SIZE	BLOCK	WRITE SIZES ACCEPTED	NOTES
`$FEC0_0000-$FEC0_007F`	128 B	GAVIN control registers (16 x 32 bit, and since build 83 the boot selection registers at `$40` - `$4C`)	32-bit only	System control, IDs, LEDs, buzzer, memory-system control, boot block selection; `$50` - `$7F` read 0
`$FEC0_0080-$FEC0_00FF`	128 B	Real-time clock BQ4802 (local bus)	8-bit only	16 registers at `$FEC0_0080` - `$FEC0_008F`
`$FEC0_0100-$FEC0_01FF`	256 B	Interrupt controller	16-bit only	32-byte block, mirrored 8 times
`$FEC0_0200-$FEC0_02FF`	256 B	Timers	32-bit only	64-byte block, mirrored 4 times
`$FEC0_0300-$FEC0_033F`	64 B	SD card SPI controller 0 (front slot)	8-bit only	4 registers, mirrored every 4 bytes
`$FEC0_0340-$FEC0_037F`	64 B	SD card SPI controller 1 (back slot)	8-bit only	4 registers, mirrored every 4 bytes

ADDRESS	SIZE	BLOCK	WRITE SIZES ACCEPTED	NOTES
$FEC0_0380-$FEC0_03FF	128 B	SD card 2 (internal micro SD, in the 10M02 on the local bus)	8-bit only	Local address = A1..A0
$FEC0_0400-$FEC0_04FF	256 B	JM20330 IDE (PATA) to SATA bridge (local bus)	16-bit data register, 8-bit others	Task file, mirrored every 32 bytes
$FEC0_0500-$FEC0_05FF	256 B	TRINITY: 10M02 joystick / SNES pad controller (local bus)	8-bit only	Registers are inside the 10M02
$FEC0_0600-$FEC0_07FF	512 B	LAN9221 Ethernet controller (local bus)	16 or 32-bit (byte writes ignored)	A longword access makes two LAN9221 cycles (build 25; FIFO-safe since build 29)
$FEC0_0800-$FEC0_09FF	512 B	WizFi360 WiFi UART	8-bit only	8 bytes, mirrored; COM3 / COM4 layout since build 70
$FEC0_0A00-$FEC0_0AFF	256 B	-	-	Hole
$FEC0_0B00-$FEC0_0BFF	256 B	USB-C UART COM3	8-bit only	8 bytes, mirrored
$FEC0_0C00-$FEC0_0CFF	256 B	USB-C UART COM4	8-bit only	8 bytes, mirrored
$FEC0_0D00-$FEC0_0DFF	256 B	FPGA temperature monitor (build 87): the die temperature from the XADC	8, 16 or 32-bit	64-byte register set, mirrored 4 times; unused offsets read 0
$FEC0_0E00-$FEC0_0EFF	256 B	I2C master (build 84): the system I2C bus (SiI9022 HDMI transmitter, MAC address EEPROM, fan controller, PLL)	8, 16 or 32-bit	64-byte register set, mirrored 4 times; big-endian byte registers
$FEC0_0F00-$FEC0_1FFF	4.25 KB	-	-	Hole
$FEC0_2000-$FEC0_23FF	1 KB	LPC bus: Super I/O (LPC47M107) I/O ports $000 - $3FF	8-bit only	LPC I/O address = offset from $FEC0_2000
$FEC0_2400-$FEC0_2FFF	3 KB	-	-	Hole
$FEC0_3000-$FEC0_307F	128 B	Fixed-point maths unit	32-bit only	16 registers, mirrored at + $40
$FEC0_3080-$FEC0_3FFF	-	-	-	Hole
$FEC0_4000-$FEC0_41FF	512 B	Floating Math Block (build 87): single-precision FPU	8, 16 or 32-bit	32-byte register set, mirrored 16 times
$FEC0_4200-$FEC0_BFFF	-	-	-	Hole
$FEC0_C000-$FEC0_C0FF	256 B	SDMA (SRAM to SRAM DMA) registers	32-bit only	8 registers, mirrored every 32 bytes
$FEC0_C100-$FEC0_C1FF	256 B	MDMA (SRAM <-> DDR3 DMA) registers	32-bit only	Build 26. See the memory and DMA chapter
$FEC0_C200-$FEC0_C2FF	256 B	VDMA (DDR3 blitter) registers	32-bit only	Build 27. See the memory and DMA chapter
$FEC0_C300-$FEC0_CFFF	-	-	-	Hole

ADDRESS	SIZE	BLOCK	WRITE SIZES ACCEPTED	NOTES
`$FEC0_D000-$FEC1_FFFF`	-	-	-	Hole

GAVIN control registers `$FEC0_0000`

Sixteen 32-bit registers at `$FEC0_0000` - `$FEC0_003F` (register n at offset 4 x n) and, since build 83, the four boot selection registers at `$FEC0_0040` - `$FEC0_004C`. The chip select of the control registers now covers `$FEC0_0000` - `$FEC0_007F` (it was `$00` - `$3F`); `$FEC0_0050` - `$FEC0_007F` read 0. Writes must be 32-bit (MOVE.L): byte and word writes are ignored. Reads of any size work.

Every register stores the full longword written to it. For the identification registers (`$0C` - `$1C`) the read value is fixed and the stored value is never visible.

The RGB LEDs are two strings of serial RGB LEDs, refreshed once per video frame: the board string (power, SD0, SD1, mSATA, network) and the SD-card sub-board string (network, mSATA, SD1, SD0, power). A LED whose enable bit in register 0 is 0 is sent black. The colour registers hold `$00` RRGGBB; the hardware reorders the bytes for the LEDs.

> Endian inversion (register 0 bit 6) swaps byte lanes of byte and word accesses to DDR3 only: with it on, a byte access to DDR3 is mirrored within its longword (offset 0 <-> 3, 1 <-> 2) and a word access swaps the two halves of the longword; longword and line (cache) accesses are not changed. Data written with one access size then reads back in a different place with another size, and the video engines read DDR3 as the CPU writes it with the bit off. Since build 23 it is 0 after reset (before: 1). Leave it 0. Soft reset: the `$DEAD` key plus bit 15 starts the same board-wide cold reset as power-on (the FPGA drives the COLD_RESET line low for 2^24 cycles of 14.318 MHz, about 1.17 s). The 68060 RESET instruction (RSTO) and the debug port's system reset (command `$3F`) start the same reset. BOOT_SEL (build 83): The choice survives only the reboot it requested: power-on, the reset button (or the GAVIN soft reset), the debug port's system reset and the 68060 RESET instruction (an operating system reboot) all clear it, so the board boots block 0 again.

$00 GAVIN_CTRL

RW (bits 31-17 R) · reset $0000_0001 (bits 16-0)

Main control register: board LEDs, buzzers, endian inversion, HDMI pixel-clock polarity, Super I/O reset, soft reset. Bits 16-0 read back what was written; bits 31-17 read live board inputs (DIP switches, SD card detect, jumpers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SD…	SD…	SD…	SD…	IN…	DIP_USER			DIP_BOOT				PC…	PC…	PC…	-	SO…	-					HD…	LP…	SP…	EN…	BU…	LE…	LE…	LE…	LE…	LE…

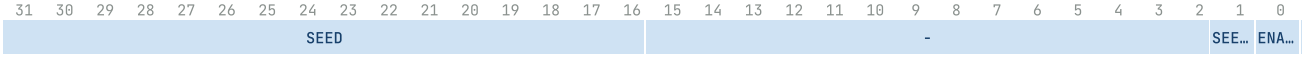
BITS	FIELD	ACCESS	MEANING
31	**SD1_WP**	R	Back SD slot write-protect switch, raw pin level. MCP: 0 = write-protected.
30	**SD1_CD**	R	Back SD slot card-detect, raw pin level. MCP: 0 = card present.
29	**SD0_WP**	R	Front SD slot write-protect switch, raw pin level. MCP: 0 = write-protected.
28	**SD0_CD**	R	Front SD slot card-detect, raw pin level. MCP: 0 = card present. The same pin is interrupt source `$18`.
27	**IN_ENCLOSURE**	R	IN_ENCLOSURE jumper, raw pin level.
26-24	**DIP_USER**	R	User DIP switches 2..0, raw pin levels (OFF = 1). Bit 2 = switch 7 (as marked on the part) selects the board type since build 40: 1 = black board, 0 = white board.
23-20	**DIP_BOOT**	R	Former boot DIP switches 3..0, raw pin levels (OFF = 1). Since build 83 the FPGA, and since MCP 166 the MCP, no longer use them for booting (up to MCP 165: bits 21-20 = boot mode (which flash block), bits 23-22 = boot size). They are free for software (user switches).
19	**PCIE_X8**	R	1 = a card is present in the PCIe slot (PRSNT8 pin low).
18	**PCIE_X4**	R	1 = PRSNT4 pin low.
17	**PCIE_X1**	R	1 = PRSNT1 pin low.
16	**-**	RW	Stored, no function.
15	**SOFT_RESET_EN**	RW	Soft reset: writing a longword with bit 15 = 1 and bits 31-16 = `$DEAD` (for example `$DEAD_8000`) resets the whole board (see the note).
14-10	**-**	RW	Stored, no function. (MCP names bit 14 TRIGGER_MAN_RST; the hardware does not use it: bit 15 with the `$DEAD` key is enough.)
9	**HDMI_IDCK_INV**	RW	HDMI pixel clock polarity (build 28), 0 after reset. 0 = IDCK rises in the middle of each pixel, where the SiI9022 latches; 1 = IDCK turned over. Leave it 0. Resynchronised to the dot clock.
8	**LPC_RSTn**	RW	Super I/O (LPC) reset: 0 = the Super I/O is held in reset (value after reset), 1 = running. Resynchronized to the 33 MHz LPC clock. MCP sets it when it initializes the Super I/O.
7	**SPEAKER_EN**	RW	1 = square wave on the ATX header speaker output; frequency from register 13 bits 31-16.
6	**ENDIAN_INV**	RW	DDR3 byte-lane inversion, 0 after reset (off). Keep it 0, see the note.
5	**BUZZER_EN**	RW	1 = square wave on the on-board buzzer; frequency from register 13 bits 15-0.
4	**LED_NETWORK**	RW	1 = network RGB LED on, with the colour of register 12.
3	**LED_MSATA**	RW	1 = mSATA/IDE RGB LED on, with the colour of register 11.
2	**LED_SD1**	RW	1 = SD1 RGB LED on (colour of register 10). Also drives the media LED output (OR of bits 1 and 2).
1	**LED_SD0**	RW	1 = SD0 RGB LED on (colour of register 9). Also drives the media LED output (OR of bits 1 and 2).

BITS	FIELD	ACCESS	MEANING
0	**LED_POWER**	RW	1 = power LED output on and the power RGB LED lit with the colour of register 8. 1 after reset.

$04 LFSR_CTRL

RW · reset - (not reset)

Control of the 16-bit pseudo-random generator (LFSR, XNOR feedback, taps 16, 15, 13, 4). It advances once per bus clock while enabled.

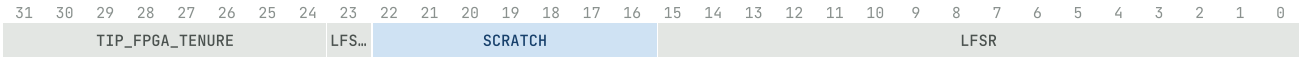


BITS	FIELD	ACCESS	MEANING
31-16	**SEED**	RW	Seed value.
15-2	**-**	RW	Stored, no function.
1	**SEED_LOAD**	RW	While 1 (and ENABLE = 1) the LFSR is loaded with SEED on every clock. Write 1, then 0.
0	**ENABLE**	RW	1 = the LFSR runs.

$08 LFSR_DATA

R (bits 22-16 RW) · reset -

LFSR output. A write stores the longword; only bits 6-0 of the written value read back (in bits 22-16), with no function. Bits 31-24 (build 44+): bus diagnostic counter.



BITS	FIELD	ACCESS	MEANING
31-24	**TIP_FPGA_TENURE**	R	Build 44+: number of times the CPU's TIP was seen asserted while an FPGA bus master (display fetch, SDMA, MDMA) owned the bus; saturates at 255, cleared by reset. Should stay 0. Build 49+: when MEMSYS_CTRL bits 26:24 are not 0, this byte shows byte (n - 1) of the L2 write recorder instead (diagnostic, used by MCP 141+ TEST MDMA).
23	**LFSR_DONE**	R	1 while the LFSR value equals SEED (once per 65 535-step cycle).
22-16	**SCRATCH**	RW	Bits 6-0 of the last value written to this register. No function.
15-0	**LFSR**	R	Current LFSR value.

$0C ID_CPU

R · reset $0009_B06C

Identification: chip subversion, CPU type, CPU speed, machine ID. Writes are stored internally but do not change what is read.

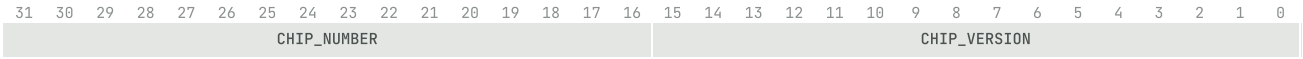


BITS	FIELD	ACCESS	MEANING
31-16	**CHIP_SUBVERSION**	R	`$0009`.
15-12	**CPU_ID**	R	`$B` = MC68LC060.
11-8	**-**	R	0.
7-4	**CPU_SPEED**	R	6 = 66 MHz (SYSCLK_66MHZ). MCP reads this field (GABE_SUBVER_ID) to scale its delay loops.
3-0	**MACHINE_ID**	R	`$C` = 12, A2560M family (A2560Me).

$10 ID_CHIP

R · reset $7338_0011

Chip number and version.



BITS	FIELD	ACCESS	MEANING
31-16	**CHIP_NUMBER**	R	`$7338`.
15-0	**CHIP_VERSION**	R	`$0011`.

$14 FPGA_DATE

R · reset $0017_0826

Date of the FPGA design, BCD.

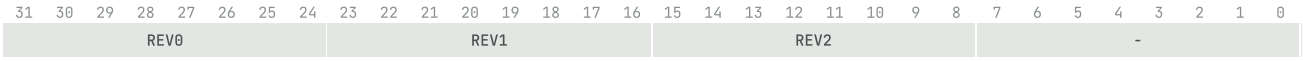


BITS	FIELD	ACCESS	MEANING
31-24	**-**	R	0.
23-16	**DAY**	R	`$17` (BCD).
15-8	**MONTH**	R	`$08` (BCD).
7-0	**YEAR**	R	`$26` = 2026 (BCD).

$18 HW_REVISION

R · reset $4233_4100

Board revision as ASCII text: "B3A".



BITS	FIELD	ACCESS	MEANING
31-24	**REV0**	R	`$42` = 'B'.
23-16	**REV1**	R	`$33` = '3'.
15-8	**REV2**	R	`$41` = 'A'.
7-0	**-**	R	0.

$1C PCB_DATE

R · reset $0017_0826

Date of the PCB, BCD, same layout as FPGA_DATE (day `$17`, month `$08`, year `$26`).

$20 RGB_POWER

RW · reset $0000_4000

Colour of the power RGB LED, `$00`RRGGBB (8 bits per colour). Shown while register 0 bit 0 = 1. Reset value: dim green.

$24 RGB_SD0

RW · reset $0080_0080

Colour of the SD0 RGB LED, `$00`RRGGBB. Shown while register 0 bit 1 = 1. Reset value: purple.

$28 RGB_SD1

RW · reset $0000_0080

Colour of the SD1 RGB LED, `$00`RRGGBB. Shown while register 0 bit 2 = 1. Reset value: blue.

$2C RGB_MSATA

RW · reset $0080_8000

Colour of the mSATA/IDE RGB LED, `$00`RRGGBB. Shown while register 0 bit 3 = 1. Reset value: yellow.

$30 RGB_NETWORK

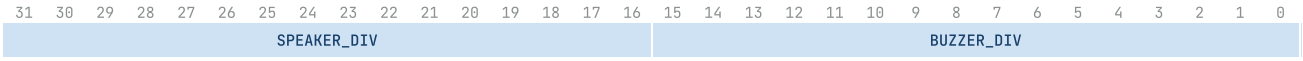
RW · reset $0000_8080

Colour of the network RGB LED, `$00`RRGGBB. Shown while register 0 bit 4 = 1. Reset value: cyan.

$34 BUZZER_DIV

RW · reset $1890_1890

Square-wave dividers. The output toggles every (divider + 1) bus clocks: f = bus clock / (2 x (divider + 1)). The reset value `$1890` gives about 5.30 kHz. A new value is taken at the next toggle.



BITS	FIELD	ACCESS	MEANING
31-16	**SPEAKER_DIV**	RW	Divider for the ATX header speaker (enabled by register 0 bit 7).
15-0	**BUZZER_DIV**	RW	Divider for the on-board buzzer (enabled by register 0 bit 5).

$38 MEMSYS_CTRL

RW · reset $4C00_0000 (read value)

Memory-system control (register 14). All bits 0 after reset = normal operation. Bits 1-4 are mainly for debugging the memory system; bits 23-16 and 14-8 are local-bus settings (bits 13-8 since build 34, bit 14 since build 38, bits 23-16 since build 39) and read back as written; bit 15 (read only, build 40) shows the board type. Writing 0 to them gives the normal behaviour. Longword accesses only.



BITS	FIELD	ACCESS	MEANING
31-28	**-**	R	Read as the `$4C` signature (high nibble); writes stored, no function.
27-24	**L2_REC**	W	Diagnostic (reads as the signature): bit 27 arms the L2 anomaly recorder. While armed it counts writes into the L2 data RAM (CPU write hits and line writes; refills excluded) to sets outside the last DMA range invalidate, and latches the first one (state, address); builds 57 / 58 check each write one clock later. Bits 26:24 select what GAVIN register 2 bits 31:24 show (0 = TIP counter, 1-7 = recorder bytes 0-6). Used by MCP 141+ TEST MDMA. Leave 0.
23-20	**IDE_REC_EXTRA**	RW	Build 39+: IDE write recovery + 2n bus clocks: the write data is held, and DIOW- stays high, that much longer after each data-register write (default 100 ns). Diagnostic / margin; 0 after reset.
19-16	**IDE_WR_EXTRA**	RW	Build 39+: IDE write strobe (DIOW-) + 2n bus clocks (default 340 ns; 2 = 420 ns). Diagnostic / margin; 0 after reset.
15	**WHITE_BOARD**	R	Build 40+: 1 = the FPGA latched white-board mode from DIP switch 7 at the last reset (F14 = IDE DMACK- held high, D14 = IDE DMARQ not driven); 0 = black board.
14	**LBCLK_PHASE**	RW	0 = local-bus clock edges half a BCLK after the BCLK edges (normal), 1 = on the BCLK edges (the build 37 phase; about half of the 10M02 writes are lost). 0 after reset.
13	**LBCLK_OFF**	RW	1 = the local-bus clock output (LOCAL_BUS_CLK) is held low. Diagnostic; 0 after reset (since build 34).
12	**LAN_FILTER_OFF**	RW	1 = the LAN9221 read majority filter is off. Diagnostic; 0 after reset = filter on (since build 34).
11-8	**LAN_RD_EXTRA**	RW	LAN9221 read strobe + n bus clocks. Diagnostic; 0 after reset (since build 34).
7-5	**-**	R	Read 0 (written values are stored but not shown).
4	**SLOW_BEATS**	RW	1 = one extra bus clock between the beats of a line burst (debug).
3	**SRAM_BURST_OFF**	RW	1 = line transfers (cache line fills/pushes, MOVE16) to the SRAM are refused with TBI; the CPU then makes 4 single longword cycles.
2	**DDR3_BURST_OFF**	RW	1 = the same for DDR3.
1	**L2_FLUSH**	W	Writing 1 invalidates the whole DDR3 L2 cache (1-clock pulse). Reads 0. DDR3 itself is always up to date, so use it after another master (DMA, debug port) has written DDR3.
0	**L2_OFF**	RW	1 = the 64 KB DDR3 L2 cache is disabled.

$3C BUILD_ID

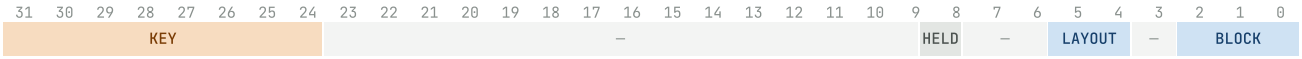
R · reset $0000_0057 (build 87); earlier: $56 = 86, $55 = 85, $54 = 84, $53 = 83, $4A = 74, $46 = 70, $45 = 69, $43 = 67, $42 = 66, $3B = 59, $3A = 58, $39 = 57, $32 = 50, $2D = 45, $28 = 40, $23 = 35

Register 15: the FPGA build number. Since build 25 it reads the build number itself (`$19` = build 25, `$23` = 35, `$42` = 66, `$43` = 67, `$45` = 69, `$46` = 70, `$4A` = 74, `$53` = 83, `$54` = 84, `$55` = 85, `$56` = 86, `$57` = 87). A write is stored but the read value is always the build number.

$40 BOOT_SEL

RW (keyed write) · reset $0000_0000 after a cold boot

Boot block selection (build 83). Writes must be longwords. A write with the key `$B0` in bits 31-24 stores LAYOUT and BLOCK and starts a board reset (like the reset button, about 1.2 s); the boot copy then copies the first 64 KB of that block into SRAM. A write without the key is ignored. A read returns the choice held for this boot.

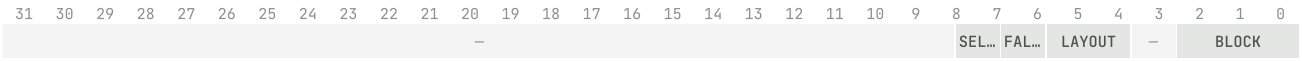


BITS	FIELD	ACCESS	MEANING
31-24	**KEY**	W	Must be `$B0`, or the write is ignored. Reads 0.
8	**HELD**	R	1 = a choice is held for this boot (this boot was requested through BOOT_SEL).
5-4	**LAYOUT**	RW	Flash layout: 0 = 1 x 4 MB, 1 = 2 x 2 MB, 2 = 4 x 1 MB, 3 = 8 x 512 KB.
2-0	**BLOCK**	RW	Block number within the layout. A block that does not exist in the layout (for example block 2 with layout 1) makes the boot copy fall back to block 0.

$44 BOOT_INFO

R · reset $0000_0000 after a cold boot

What the boot copy used for this boot (build 83). 0 = a normal cold boot of block 0.



BITS	FIELD	ACCESS	MEANING
7	**SEL_USED**	R	1 = BOOT_SEL was used for this boot.
6	**FALLBACK**	R	1 = the copy fell back to block 0: the chosen block was blank (its first longword is `$FFFF_FFFF`) or the block number does not exist in that layout.
5-4	**LAYOUT**	R	Layout used (as BOOT_SEL).
2-0	**BLOCK**	R	Block used.

$48 BOOT_BASE

R · reset $FFC0_0000 after a cold boot

CPU address of the booted block (build 83): `$FFC0_0000` + block offset, for example `$FFD0_0000` for block 2 of layout 3 (8 x 512 KB).

$4C BOOT_SIZE

R · reset $0040_0000 after a cold boot

Size of the booted block in bytes (build 83): 4 MB >> layout, so `$0040_0000`, `$0020_0000`, `$0010_0000` or `$0008_0000`.

Interrupt controller $FEC0_0100

48 interrupt inputs (`$00`-`$2F`) in six groups of eight. Each group drives one 68060 interrupt level. Each input has a pending, a polarity, an edge and a mask bit; the registers hold 16 bits, so each kind of bit takes three registers.

Write the registers with 16-bit accesses only (MOVE.W); byte and longword writes are ignored. A read returns the 16-bit register on both halves of the data bus, so a longword read at `$FEC0_0100` returns PENDING_0 twice, not PENDING_0 and PENDING_1. The 32-byte register block is mirrored over `$FEC0_0100`-`$FEC0_01FF`.

After reset every input is edge-triggered, active low and masked, and nothing is pending.

Inputs are sampled on the bus clock and taken once they have the same value on two clocks in a row (each input separately). An event that arrives on the same clock as a CPU write to a pending register is not lost.

Unused inputs are tied to their inactive level (high). Do not give them active-high polarity in level mode: they would then be pending all the time.

- An interrupt reaches the CPU when its pending bit is 1 and its mask bit is 0. The interrupt level is that of the highest group with such an interrupt (group 0 = level 6 is the highest). Level 7 never comes from the controller: since build 85 only the debug port's monitor stop uses it (autovector 31; install an RTE handler there if no monitor stub is present).
- Vectors for groups 1-5: vector = `$38` + interrupt number (interrupt `$08` -> vector `$40`, interrupt `$2F` -> vector `$67`). Inside a group, the vector is that of the highest-numbered enabled pending interrupt.
- Group 0 (level 6) uses the autovector (vector 30); its handler must read PENDING_0 to find the source.
- There are no vector registers: the vectors are fixed. During the acknowledge cycle the hardware takes the acknowledged level from the 68060 TM2-TM0 pins and returns the vector of that level's group, so a higher interrupt arriving between IPL and acknowledge does not give the wrong handler.
- Install a handler that only does RTE on vector `$18` (spurious interrupt). It is used when software clears a pending bit while the CPU is already acknowledging that level.
- The handler clears its pending bit by writing 1 to it. In level mode the bit comes back at once while the source is still active: clear the source first.
- The IPL lines are registered on the bus clock (one clock of latency, no glitches).

RTL wins over the MCP header interrupt.h for the Super I/O group: the RTL has `$0A` = COM1, `$0B` = COM2, `$0C` = LPT1, `$0D` = floppy, `$0E` = MPU-401, while interrupt.h names `$0A` floppy, `$0C` COM1 and `$0D` LPT1 (only `$0B` COM2 matches). interrupt.h also names `$20` W6100 (A2560Me: LAN9221), `$24` USB OTG (A2560Me: COM4), `$25`/`$26` reserved (A2560Me: JM20330 / TRINITY joystick).

Levels and vectors

CPU LEVEL	GROUP	INTERRUPTS	VECTORS	VECTOR TABLE ADDRESSES
7	-	debug port monitor stop only (build 85+)	Autovector 31 (`$1F`)	`$07C`
6	0	`$00` - `$07`	Autovector 30 (`$1E`)	`$078`
5	1	`$08` - `$0F`	`$40` - `$47`	`$100` - `$11C`
4	2	`$10` - `$17`	`$48` - `$4F`	`$120` - `$13C`
3	3	`$18` - `$1F`	`$50` - `$57`	`$140` - `$15C`
2	4	`$20` - `$27`	`$58` - `$5F`	`$160` - `$17C`
1	5	`$28` - `$2F`	`$60` - `$67`	`$180` - `$19C`
any	-	acknowledged level with nothing pending	Spurious interrupt `$18`	`$060`

Interrupt sources

NUMBER	GROUP	SOURCE	STATE IN BUILD 87	NOTES
`$00`	0	SOF: start of frame	Active	Pulse at each frame start: the 108 MHz video timing (start of the vertical blanking), or, while GGE_ENABLE is set (build 61), the GGE (start of line 0).
`$01`	0	SOL: GGE line interrupt	Active (build 61)	Pulse at the start of the GGE line set in LINE_IRQ (`$FD00_001C`) while its bit 31 is set. The GGE timing always runs.
`$02`	0	Video (reserved)	Tied inactive	
`$03`	0	Video (reserved)	Tied inactive	
`$04`	0	Foenix keyboard (MAU)	Tied inactive	Not on the A2560Me.
`$05`	0	PS/2 keyboard (non-LPC)	Tied inactive	Use the Super I/O keyboard, `$08`.
`$06`	0	PS/2 mouse (non-LPC)	Tied inactive	Use the Super I/O mouse, `$09`.
`$07`	0	USB-C UART COM3: receive FIFO not empty	Active	Input = RX FIFO empty flag (falling edge = first byte arrives).

NUMBER	GROUP	SOURCE	STATE IN BUILD 87	NOTES
$08	1	Super I/O keyboard (serial IRQ 1)	Active	Short active-low pulse on each rising edge of IRQ1.
$09	1	Super I/O mouse (serial IRQ 2)	Active	Pulse on each rising edge of IRQ2.
$0A	1	Super I/O COM1 (serial IRQ 4)	Active	Pulse on each rising edge of IRQ4.
$0B	1	Super I/O COM2 (serial IRQ 3)	Active	Pulse on each rising edge of IRQ3.
$0C	1	Super I/O LPT1 (serial IRQ 7)	Active	Pulse on each rising edge of IRQ7.
$0D	1	Super I/O floppy controller (serial IRQ 6)	Active	Pulse on each rising edge of IRQ6.
$0E	1	Super I/O MPU-401 (serial IRQ 5)	Active	Pulse on each rising edge of IRQ5.
$0F	1	RTC BQ4802 interrupt pin	Active	Active low.
$10	2	VDMA (DDR3 blitter, `$FEC0_C200`)	Active (build 27)	4-clock pulse per event: after a descriptor with CTRL bit 18, and at the end of a list with CONTROL bit 3.
$11	2	MDMA (SRAM <-> DDR3, `$FEC0_C100`)	Active (build 26)	4-clock pulse at the end of a job with CTRL bit 9.
$12-$14	2	DMA channels 2-4	Tied inactive	Reserved for future DMA channels.
$15	2	GGE DAC player	Active (build 66)	Active low while (EVENT AND EVENT_IRQ_EN) at `$FD00_400C` / `$FD00_4010` is not 0 (a level; in edge mode clear EVENT first). Was the unused 'DMA -> DAC' channel.
$16	2	DMA channel 6	Tied inactive	Reserved.
$17	2	SDMA end of transfer	Active	When SDMA control bit 3 is set.
$18	3	Front SD card detect (SD0 CD pin)	Active	Raw pin level, also readable in GAVIN_CTRL bit 28.
$19	3	Timer 0	Active	Active-high pulse, see Timers.
$1A	3	Timer 1	Active	
$1B	3	Timer 2	Active	
$1C	3	Timer 3 (frame counter)	Active	
$1D	3	VIA0	Tied inactive	Not on the A2560Me.
$1E	3	VIA1	Tied inactive	Not on the A2560Me.
$1F	3	WizFi360 UART: received byte waiting	Active	Input = RX_EMPTY (build 70: no byte in the receive head register).
$20	4	LAN9221 Ethernet IRQ pin	Active	
$21	4	WizFi360 module IRQ pin	Active	
$22	4	Cartridge	Tied inactive	Not on the A2560Me.

NUMBER	GROUP	SOURCE	STATE IN BUILD 87	NOTES
$23	4	PCI Express root port	Active (build 74)	Core interrupt output (INTx / MSI, as enabled in the core's Interrupt Mask register `$13C`), a level, active low. Set polarity active low, preferably level mode. See the PCI Express chapter.
$24	4	USB-C UART COM4: receive FIFO not empty	Active	Input = RX FIFO empty flag.
$25	4	JM20330 IDE/SATA bridge	Tied inactive	The bridge interrupt is not connected in this build: poll the status register.
$26	4	TRINITY (10M02) joystick interrupt pin	Active	
$27	4	Reserved	Tied inactive	
$28	5	SAM2695 MIDI receive FIFO	Tied inactive	
$29	5	VS1053b link: data FIFO low (BEATRIX, `$FEC2_2200`)	Active (build 70)	Active low while VS_STATUS LOW and VS_CTRL LOW_IRQ_EN are both 1 (a level: clear LOW in the handler). Was the unused 'VS1053 MIDI receive FIFO' input.
$2A	5	OPL3 timer	Tied inactive	
$2B	5	OPN2 timer	Tied inactive	
$2C	5	OPM timer	Tied inactive	
$2D	5	PWM FIFO almost empty	Tied inactive	
$2E	5	DAC 44.1 kHz FIFO (BEATRIX, retired)	Driven inactive (since build 69)	The CPU-fed AK4432 FIFO was retired in build 69: the AK4432 plays the GGE DAC player's second output (interrupt `$15`).
$2F	5	DAC 48 kHz FIFO (BEATRIX)	Driven inactive	No source yet.

$00 PENDING_0

`R/W1C · reset $0000`

Pending bits of interrupts `$00` - `$0F` (bit n = interrupt n). Write 1 to clear a bit; 0 bits are not changed.

$02 PENDING_1

`R/W1C · reset $0000`

Pending bits of interrupts `$10` - `$1F` (bit n = interrupt `$10` + n).

$04 PENDING_2

`R/W1C · reset $0000`

Pending bits of interrupts `$20` - `$2F` (bit n = interrupt `$20` + n).

$06 SCRATCH_0

`RW · reset -`

16-bit storage, no function.

$08 POLARITY_0

`RW · reset $0000`

Polarity of interrupts `$00` - `$0F`. 0 = the input is active low (falling edge or low level), 1 = active high (rising edge or high level).

$0A **POLARITY_1** RW · reset $0000

Polarity of interrupts $10 - $1F.

$0C **POLARITY_2** RW · reset $0000

Polarity of interrupts $20 - $2F.

$0E **SCRATCH_1** RW · reset -

16-bit storage, no function.

$10 **EDGE_0** RW · reset $FFFF

Trigger mode of interrupts $00 - $0F. 1 = edge (one event per active edge), 0 = level (the pending bit is set again on every clock while the input is active).

$12 **EDGE_1** RW · reset $FFFF

Trigger mode of interrupts $10 - $1F.

$14 **EDGE_2** RW · reset $FFFF

Trigger mode of interrupts $20 - $2F.

$16 **SCRATCH_2** RW · reset -

16-bit storage, no function.

$18 **MASK_0** RW · reset $FFFF

Mask of interrupts $00 - $0F. 1 = masked (no CPU interrupt), 0 = enabled. The pending bit is set even when masked.

$1A **MASK_1** RW · reset $FFFF

Mask of interrupts $10 - $1F.

$1C **MASK_2** RW · reset $FFFF

Mask of interrupts $20 - $2F.

$1E **SCRATCH_3** RW · reset -

16-bit storage, no function.

Timers $FEC0_0200

Four 32-bit up/down counters. Timers 0, 1 and 2 count CPU bus clocks (66.67 MHz, 15 ns per step). Timer 3 counts video frames (start-of-frame pulses: 60 per second in 1280 x 960 at 60 Hz; the GGE frames, 60 or 70 per second, while GGE_ENABLE is set): it is a frame counter, not a fast timer.

Write the registers with 32-bit accesses only; other sizes are ignored. The 64-byte block is mirrored over $FEC0_0200 - $FEC0_02FF. Control registers are cleared by reset; value and compare registers are not.

CLEAR and LOAD are levels, not pulses: while the bit is 1 the counter is held cleared or reloaded on every clock (even when not enabled). Write the bit to 1, then back to 0.

Each timer compares its count with its compare register on every clock. When they are equal: with CLEAR_ON_CMP the counter goes to 0 on the next step, with RELOAD_ON_CMP it is loaded with the VALUE register, and the timer's interrupt line gives an active-high pulse of 4 bus clocks (it stays high while the count stays equal to the compare value) if its IRQ_EN bit is set. The interrupt controller sources are `$19` - `$1C`.

Periodic interrupt example: VALUE = N, COMPARE = 0, count down, RELOAD_ON_CMP = 1, IRQ_EN = 1, ENABLE = 1. The period is N + 1 bus clocks (in general |COMPARE - VALUE| + 1 steps).

> RTL wins over the MCP header timers_a2560m.h: there the compare-equal status bits are at TCR1 bits 27-30 (RTL: bits 11-14), the timer 3 interrupt enable is TCR1 bit 7 (RTL: TCR0 bit 31), and a timer 4 is defined at `$FEC0_0228` (not in the RTL). Compare registers cannot be read back. Timers 0-2 count bus clocks of 15 ns.

$00 TIMER_CTRL0

RW · reset $0000_0000

Control of timers 0, 1 and 2 (one byte each) and the interrupt enable of timer 3.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IR…	-						IR…	-	RE…	CL…	UP2	LO…	CL…	EN…	IR…	-	RE…	CL…	UP1	LO…	CL…	EN…	IR…	-	RE…	CL…	UP0	LO…	CL…	EN…		

BITS	FIELD	ACCESS	MEANING
31	**IRQ_EN3**	RW	Interrupt enable of timer 3 (the frame counter). Note: this bit, not TIMER_CTRL1 bit 7.
30-24	**-**	RW	Stored, no function.
23	**IRQ_EN2**	RW	Timer 2 interrupt enable.
22	**-**	RW	Stored, no function.
21	**RELOAD_ON_CMP2**	RW	Timer 2: load TIMER2_VALUE when the count equals TIMER2_COMPARE.
20	**CLEAR_ON_CMP2**	RW	Timer 2: clear to 0 when the count equals TIMER2_COMPARE.
19	**UP2**	RW	Timer 2: 1 = count up, 0 = count down.
18	**LOAD2**	RW	Timer 2: while 1, the counter is loaded with TIMER2_VALUE on every clock.
17	**CLEAR2**	RW	Timer 2: while 1, the counter is held at 0.
16	**ENABLE2**	RW	Timer 2: 1 = count.
15	**IRQ_EN1**	RW	Timer 1 interrupt enable.
14	**-**	RW	Stored, no function.
13	**RELOAD_ON_CMP1**	RW	Timer 1, as bit 21.
12	**CLEAR_ON_CMP1**	RW	Timer 1, as bit 20.
11	**UP1**	RW	Timer 1, as bit 19.
10	**LOAD1**	RW	Timer 1, as bit 18.
9	**CLEAR1**	RW	Timer 1, as bit 17.
8	**ENABLE1**	RW	Timer 1, as bit 16.
7	**IRQ_EN0**	RW	Timer 0 interrupt enable.
6	**-**	RW	Stored, no function.
5	**RELOAD_ON_CMP0**	RW	Timer 0, as bit 21.
4	**CLEAR_ON_CMP0**	RW	Timer 0, as bit 20.
3	**UP0**	RW	Timer 0, as bit 19.
2	**LOAD0**	RW	Timer 0, as bit 18.
1	**CLEAR0**	RW	Timer 0, as bit 17.
0	**ENABLE0**	RW	Timer 0, as bit 16.

$04 TIMER_CTRL1 RW · reset $0000_0000

Control of timer 3 (frame counter) and live compare status of all timers.



BITS	FIELD	ACCESS	MEANING
31-15	**-**	RW	Stored, no function.
14	**EQ3**	R	1 while timer 3 equals its compare value (live, not sticky).
13	**EQ2**	R	1 while timer 2 equals its compare value.
12	**EQ1**	R	1 while timer 1 equals its compare value.
11	**EQ0**	R	1 while timer 0 equals its compare value.
10-8	**-**	R	Read 0.
7-6	**-**	RW	Stored, no function (bit 7 is not the timer 3 interrupt enable, see TIMER_CTRL0 bit 31).
5	**RELOAD_ON_CMP3**	RW	Timer 3: load TIMER3_VALUE when the count equals TIMER3_COMPARE.
4	**CLEAR_ON_CMP3**	RW	Timer 3: clear when the count equals TIMER3_COMPARE.
3	**UP3**	RW	Timer 3: 1 = count up, 0 = count down.
2	**LOAD3**	RW	Timer 3: load TIMER3_VALUE (applied at the next frame start).
1	**CLEAR3**	RW	Timer 3: clear (applied at the next frame start).
0	**ENABLE3**	RW	Timer 3: 1 = count one step per video frame.

$08 TIMER0_VALUE RW · reset -

Write: load value of timer 0. Read: current count of timer 0.

$0C TIMER0_COMPARE W · reset -

Compare value of timer 0. Reads `$0000_0000` (not the register).

$10 TIMER1_VALUE RW · reset -

Write: load value of timer 1. Read: current count.

$14 TIMER1_COMPARE W · reset -

Compare value of timer 1. Reads `$0101_0101`.

$18 TIMER2_VALUE RW · reset -

Write: load value of timer 2. Read: current count.

$1C TIMER2_COMPARE W · reset -

Compare value of timer 2. Reads `$0202_0202`.

$20 TIMER3_VALUE RW · reset -

Write: load value of timer 3 (frame counter). Read: current count.

$24 **TIMER3_COMPARE** W · reset -

Compare value of timer 3. Reads `$0303_0303`.

$28 - R · reset $0000_0000

No timer 4. Reads `$0000_0000`, writes ignored.

$2C - R · reset $0404_0404

No timer 4. Reads `$0404_0404`, writes ignored. `$30` - `$3C` read `$DEAD_BEEF`.

32 kHz clock

GAVIN makes the 32.768 kHz clock of the Super I/O (LPC 32 kHz input) from the 14.318 MHz oscillator with a fractional divider: the average frequency is exactly 32 768 Hz, and each edge can move by one 14.318 MHz period (70 ns).

It has no register. The 33 MHz LPC clock is passed through the FPGA to the Super I/O unchanged.

> GAVIN_NOTES.md (build 19) describes a fixed divide-by-438 (32 689 Hz). The current RTL (CLK32KhzCreation.v) uses the fractional divider described above.

Real-time clock (BQ4802) `$FEC0_0080`

The TI BQ4802 real-time clock is on the 16-bit local bus. GAVIN decodes `$FEC0_0080` - `$FEC0_00FF` and drives CPU address bits A8-A0 unchanged on the local bus; MCP uses the 16 registers at `$FEC0_0080` - `$FEC0_008F` (one register per byte address). See the BQ4802 datasheet for the register contents (BCD time, alarms, rates, enables, flags, control).

Writes must be byte writes (MOVE.B); a word or longword write sends `$00`. Reads of any size work: the chip's data byte (local D7-D0) is returned on all four byte lanes.

Each access is a full local-bus cycle with a handshake to the CPU (no fixed wait): a short front porch, a read strobe of 11 bus clocks (165 ns) or a write strobe of 8 bus clocks (120 ns), then a short back porch (see Local-bus cycles). The write data is held until the end of the cycle.

The RTC interrupt pin is interrupt source `$0F`.

> The RTL comment gives the window as `$FEC0_0080-$FEC0_00BF`, but the decode covers `$FEC0_0080-$FEC0_00FF`. Which local address lines reach the BQ4802 (and so the mirroring above `$FEC0_008F`) is set by the board wiring, not by the FPGA.

BQ4802 registers (names from MCP rtc_reg.h)

ADDRESS	REGISTER
`$FEC0_0080`	Seconds
`$FEC0_0081`	Alarm seconds
`$FEC0_0082`	Minutes
`$FEC0_0083`	Alarm minutes
`$FEC0_0084`	Hours
`$FEC0_0085`	Alarm hours
`$FEC0_0086`	Date (day of month)
`$FEC0_0087`	Alarm date
`$FEC0_0088`	Day of week

ADDRESS	REGISTER
$FEC0_0089	Month
$FEC0_008A	Year
$FEC0_008B	Rates
$FEC0_008C	Enables
$FEC0_008D	Flags
$FEC0_008E	Control
$FEC0_008F	Century

SD card controllers 0 and 1 (front and back slots) $FEC0_0300

Two identical SPI-mode SD card controllers (SimpleSPI4SDCard2x) inside the FPGA: controller 0 at $FEC0_0300 for the front SD slot, controller 1 at $FEC0_0340 for the back SD slot. Each has 4 byte registers, mirrored every 4 bytes over its 64-byte window.

Writes must be byte writes (MOVE.B); other sizes are ignored. Reads of any size work (the byte is returned on all lanes).

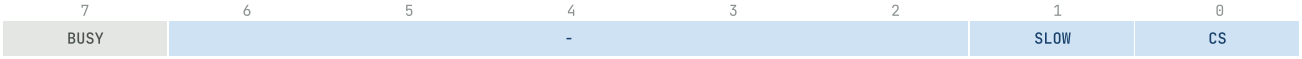
SPI mode 0: the clock idles low, MOSI (card CMD pin) changes after the falling edge, MISO (card DAT0 pin) is sampled while the clock is high. One byte = 8 clocks. The SPI engine runs on the 66.67 MHz bus clock. Fast mode: SCLK = engine clock / 3 = 22.2 MHz (since build 36; 16.7 MHz on builds 34 / 35). The divider is chosen when the FPGA is built: / 3 whenever that stays at or below 25 MHz.

Card-detect and write-protect of both slots are read in GAVIN_CTRL bits 28-31. The front slot's card detect is also interrupt source $18. The SD0 / SD1 activity LEDs are GAVIN_CTRL bits 1 and 2 (software controlled).

> SCLK in slow mode = engine clock / (4 x (delay + 2)). The reset value is derived from the engine clock: 53 at 66.67 MHz gives about 303 kHz. Delay 40 gives about 397 kHz. Build 25 made the register 8 bits wide (it was 1 bit wide in earlier builds, so slow mode ran at 8-12 MHz).

+$0 SD_CTRL RW (bit 7 R) · reset $00

Control and status.



BITS	FIELD	ACCESS	MEANING
7	**BUSY**	R	1 = a byte transfer is in progress. Set at the moment the CPU writes SD_DATA, cleared when the 8 bits have been exchanged and the received byte is valid.
6-2	**-**	RW	Stored, no function. (An old definition names bit 3 AUTOTX; it is not implemented.)
1	**SLOW**	RW	1 = slow SPI clock (card initialization), 0 = fast clock (22.2 MHz). See the note.
0	**CS**	RW	1 = card selected (the card's CS/DAT3 pin is driven low).

+$1 SD_DATA RW · reset -

Write: send this byte to the card (MSB first) and receive one byte at the same time; BUSY goes to 1. Read: the last byte received. Wait for BUSY = 0 before reading the received byte or writing the next one.

+$2 SD_SLOW_DIV RW · reset 53

Slow-clock delay. Each of the 4 states of an SPI bit lasts (delay + 2) clocks of the SPI engine clock (66.67 MHz) in slow mode. + $3 is the same register.

SD card 2 (internal micro SD, via the 10M02) `$FEC0_0380`

The internal micro SD card under the board is driven by the 10M02 FPGA (TRINITY) on the local bus. GAVIN only passes accesses through: the window `$FEC0_0380` - `$FEC0_03FF` puts CPU A1-A0 on the local address bus (4 registers, mirrored every 4 bytes). The registers are inside the 10M02; MCP drives them like SD controllers 0 and 1 (SD2_CTRL at `$FEC0_0380`, SD2_DATA at `$FEC0_0381`). The current 10M02 image also has the slow-mode divider N at `$FEC0_0382` (and `$0383`). Older 10M02 images do not decode A1: there `$0382` is SD2_CTRL again and `$0383` is SD2_DATA, so a write meant for the divider changes SD2_CTRL. Software should not depend on the divider (MCP 128 does not write it).

Writes must be byte writes; reads of any size return the byte on all lanes.

SPI clock: the 10M02 SPI runs on the local-bus clock, 33.33 MHz (build 37 and later). Slow SCLK = local-bus clock / (4 x (N + 2)); N resets to 40: about 198 kHz, below the 400 kHz card-initialisation limit. Fast SCLK = local-bus clock / 4 = 8.3 MHz. Builds 34 and 36 sent the 10M02 a 66.67 MHz clock: SD card 2 does not work on them. MCP 125-127 wrote N (54 or 26); MCP 128 and later leave N at its reset value.

Local-bus timing (handshake, no fixed wait): read strobe 8 bus clocks (120 ns), write strobe 4 bus clocks (60 ns). The 10M02 takes a write on the first local-bus clock edge that sees chip select and write strobe low; a strobe edge that coincides with its clock edge can be missed, which is why build 38 shifts the local-bus clock (see Clocks). For writes GAVIN drives the data from the start of the cycle, because the 10M02 does not drive the bus outside its read strobe.

IDE / SATA bridge (JM20330) `$FEC0_0400`

The JMicron JM20330 PATA-to-SATA bridge is connected as a PATA (IDE) device on the local bus: through a 3.3 V / 5 V transceiver on the black board, directly on the white board (see Board types below). GAVIN maps the ATA task file: CPU A3-A1 select the register (local A2-A0), A4 selects the chip select: A4 = 0 -> CS0 (command block), A4 = 1 -> CS1 (control block). The 32-byte block is mirrored over `$FEC0_0400` - `$FEC0_04FF`.

Board types (build 40+). Two batches of the board are in use. The white board (first batch) has the JMicron JM20330 IDE bridge directly on the local bus; FPGA pin F14 is the IDE DMACK- and pin D14 the IDE DMARQ (a JM20330 output). The black board (second batch) has 3.3 V / 5 V translators between the local bus and the JM20330; F14 is the translator direction (DIR) and D14 its output enable (OEn). One bitstream serves both: DIP switch 7 as marked on the part (positions 1-8; number 6 when counting from 0; pin V14, DIP_USER bit 2) selects the board, OFF (reads 1) = black, ON (reads 0) = white. The FPGA reads the switch 13 clocks after every system reset and enables the two pins 3 clocks later; until then both are high-impedance with pull-ups (translator disabled on the black board, DMACK- inactive on the white board). White mode: F14 is driven high all the time and D14 is never driven. MEMSYS_CTRL bit 15 shows the mode that was latched.

Why the board type matters (white board with builds up to 39): those builds drove F14 as the translator direction, 0 most of the time, so on a white board DMACK- was asserted. An ATA device answers the read strobe without its chip select while DMACK- is asserted, and the read strobe is shared by all local-bus devices: after any IDE read data phase the JM20330 drove its data onto the bus during every LAN9221, RTC and TRINITY read, until the next IDE write data phase (measured: LAN9221 BYTE_TEST read `$2167_65E7` instead of `$2143_6587`, 2000 of 2000 reads). DMACK- also toggled at every IDE write, and on build 35 nearly every sector written by TEST ATAWRITE had duplicated words. With build 40 and the switch ON, all of this is gone (white board: 1,550 ATAWRITE passes and 20,000 LAN reads after IDE reads, no errors). The FPGA also drove D14 against the JM20330 DMARQ output on those builds.

Data register (`$FEC0_0400`): use 16-bit accesses (MOVE.W). GAVIN swaps the two bytes, so the first byte of the sector (ATA D7-D0) is at the lower address: sector data come out in memory order. Numeric words (for example in the IDENTIFY data) therefore read byte-reversed.

Other registers: use byte accesses at the even addresses below (the byte is on ATA D7-D0). A word or longword write to them sends `$00`.

Timing (PIO, handshake): about 6 bus clocks (90 ns) from chip select to strobe; read strobe (DIOR-) 27 clocks, 405 ns (the data are taken as the majority of three samples at its end; 360-420 ns on earlier builds); write strobe (DIOW-) 23 clocks, 345 ns; the write data is on the bus about 100 ns before DIOW- falls and held about 120 ns after it rises (builds up to 38: data switched with the DIOW- falling edge, about 60 ns hold). Back-to-back data-register writes take about 735 ns each (simulation; ATA PIO mode 0 needs 600 ns). MEMSYS_CTRL bits 19-16 lengthen DIOW- and bits 23-20 the recovery, 2 bus clocks per step. The chip selects, transceiver controls, strobes and data are registered in the FPGA I/O cells.

Data reads use the same 3-sample majority filter as the LAN9221 (since build 34): the data bus is sampled three times, one clock apart, at the end of the read strobe, and the bitwise majority is taken.

The prototype boards have no IORDY and no INTRQ connection (both are planned for the final board), so use polled PIO only; source `$25` is tied inactive. Poll the status register. The mSATA/IDE activity LED is GAVIN_CTRL bit 3 (software controlled). See the JM20330 datasheet and the ATA/ATAPI specification for the registers.

MCP 124+ test: TEST ATAREAD [lba] [passes] [delay] reads one sector many times and compares the copies (read only). MCP 129: TEST ATAWRITE lba|END passes WRITE [delay|FAST] writes one sector with a pattern in which every word carries its own index, reads it back (FLUSH CACHE in between) and reports duplicated or lost words; before the last word it checks that DRQ is still set (DRQ already clear = the drive counted extra write strobes). The sector is saved first and restored at the end; END = the drive's last sector - 4096 (from IDENTIFY). FAST writes the 256 words from a MOVE.W / DBRA loop, close to back-to-back bus cycles.

> The CS1 address `$FEC0_041C` follows from the decode (A4 = 1, A3-A1 = 6); MCP's pata_a2560m.h only defines the CS0 registers.

ATA registers

ADDRESS	SIZE	READ	WRITE
`$FEC0_0400`	16-bit	Data	Data
`$FEC0_0402`	8-bit	Error	Features
`$FEC0_0404`	8-bit	Sector count	Sector count
`$FEC0_0406`	8-bit	Sector number / LBA 7-0	Sector number / LBA 7-0
`$FEC0_0408`	8-bit	Cylinder low / LBA 15-8	Cylinder low / LBA 15-8
`$FEC0_040A`	8-bit	Cylinder high / LBA 23-16	Cylinder high / LBA 23-16
`$FEC0_040C`	8-bit	Drive / head	Drive / head
`$FEC0_040E`	8-bit	Status	Command
`$FEC0_041C`	8-bit	Alternate status (CS1, register 6)	Device control (CS1, register 6)

TRINITY (10M02: joysticks and SNES pads) `$FEC0_0500`

The 10M02 FPGA on the local bus (TRINITY) handles the DB9 joystick ports, the internal and external SNES (4N4S) pads, and the internal micro SD card (see SD card 2). It runs on the local-bus clock, 33.33 MHz. Its register map is inside the 10M02 and is not part of this design; MCP's test commands use `$FEC0_0500` (joystick state), `$FEC0_0504` (game control) and `$FEC0_0508` - `$FEC0_0514` (pad data).

GAVIN drives CPU A8-A0 unchanged on the local bus. Writes must be byte writes (a longword write sends `$00`); reads of any size return the 8-bit value on all four lanes.

Timing (handshake): read strobe 9 bus clocks (135 ns), write strobe 7 bus clocks (105 ns), as in the Local-bus cycles table.

The 10M02 interrupt pin is interrupt source `$26`.

Ethernet (LAN9221) `$FEC0_0600`

The SMSC/Microchip LAN9221 Ethernet controller is on the 16-bit local bus, window `$FEC0_0600` - `$FEC0_07FF` (512 bytes). GAVIN drives CPU A8-A0 on the local bus (A1 is generated for the second half of a 32-bit access).

16-bit accesses (MOVE.W): one LAN9221 cycle. GAVIN swaps the bytes: the byte at the even address is LAN9221 D7-D0 and the byte at the odd address is D15-D8, so the little-endian chip's bytes appear in address order.

32-bit accesses (MOVE.L, and line bursts) are valid since build 25 and FIFO-safe since build 29: GAVIN makes two LAN9221 cycles under one CPU cycle, A1 = 0 first, then A1 = 1. The bytes come out in memory order: CPU D31-D24 = LAN9221 byte 0 ... D7-D0 = byte 3. Numeric register values therefore need a 32-bit byte swap (bswap32): BYTE_TEST reads `$2143_6587` with MOVE.L, and `$2143` / `$6587` with word reads at +0 / +2; ID_REV (`$FEC0_0650`) reads `$0000_2192` for the chip's `$9221_0000`. Byte writes are ignored (the cycle still ends normally).

Build 29 fix: A1 is held for one clock after the read strobe of the low half. Before, the LAN9221 could pop a FIFO DWORD early on longword reads.

Read majority filter (since build 34): every LAN9221 read samples the data bus three times, one clock apart, at the end of the strobe, and takes the bitwise majority. Reason: on the prototype boards the FPGA reads LAN9221 data line D6 as 1 instead of 0 about once in 1,000 reads (short glitches; the strobe length makes no difference). With the filter, 0 errors were seen in 300,000 reads. The filter is on by default; MEMSYS_CTRL (`$FEC0_0038`) bit 12 turns it off and bits 11-8 lengthen the read strobe (diagnostics only).

Timing (from the LAN9221 datasheet, in ns, converted to bus clocks of 15 ns): read strobe 5 bus clocks (75 ns) without the read filter; with the filter (default since build 34) 6 clocks (90 ns); write strobe 3 clocks (45 ns) with the data on the bus one clock before it. The FPGA enforces the datasheet gaps itself: 22 clocks (330 ns) after a write before reading a register at offset `$50` or above, and 12 clocks (180 ns) after reading a FIFO port or RX_DROP. Measured before the build 39 hardening (which made every read strobe one clock longer): 16-bit read 195 ns, 32-bit read 300 ns, back-to-back MOVE.L reads of the RX data FIFO 13.3 MB/s; the current figures were not measured.

The LAN9221 interrupt (FPGA pin A14, connected on all prototype boards) is interrupt source `$20` (group 4 bit 0, vector `$58`, CPU level 2); the network LED is GAVIN_CTRL bit 4 (software controlled). See the LAN9221 datasheet (16-bit bus mode) for the registers.

MCP 124+ tests: TEST LAN [frames] [16|32] [maxlen] (FIFO test in PHY loopback) and TEST LANBUS [count] (bus integrity: BYTE_TEST reads, FIFO_INT write and read-back). TEST LAN needs the network cable unplugged; MCP 134+ warns when the PHY reports a link.

> TEST LAN with a network cable connected: frames are cut after about 136 bytes and the TX status reports loss of carrier ($8002), on every build. This is a limit of the MCP loopback test only: normal transmission and reception with a cable are not affected (NetBSD networking works over the cable). Before MCP 134 the test then overwrote its own stack and MCP stopped answering; MCP 134 fixes that and warns about the link.

USB-C UARTs COM3 and COM4 `$FEC0_0B00`

Two simple UARTs in the FPGA, connected to two channels of the FT4232 USB-C bridge: COM3 at `$FEC0_0B00` and COM4 at `$FEC0_0C00`. Each has 8 bytes of registers, mirrored over its 256-byte window.

Format 8N1, LSB first, from a 24 MHz clock. Each direction has a 2048-byte FIFO. There is no hardware flow control: the CTS output is the RTS input looped back.

Use byte accesses for the control, status and data registers. A word read at + `$0` returns the head of the receive FIFO in bits 15-8 and the status in bits 7-0 without removing the byte; only a byte read at + `$1` removes a byte. Longword reads return `$DEAD_BEEF`.

Interrupts: the receive-FIFO-empty flag is the interrupt input, so with the default setting (edge, active low) an interrupt is raised when the first byte arrives in an empty FIFO. COM3 = interrupt `$07`, COM4 = interrupt `$24`.

> Fast mode in the RTL is 25 clocks of 24 MHz per bit, that is 960 000 baud in both directions (the source comments say 921 600, a 4 % difference). 115 200 mode is 207 clocks per bit (115 942 baud, +0.6 %).

+$0 UART_CTRL W · reset $00

Control (byte write).

7	6	5	4	3	2	1	0
-				RESET	-		FAST

BITS	FIELD	ACCESS	MEANING
7-4	-	W	Stored, no function.
3	**RESET**	W	1 = transmitter, receiver and both FIFOs held in reset (FIFOs emptied). Write 1, then 0.
2-1	-	W	Stored, no function.
0	**FAST**	W	0 = 115 200 baud, 1 = fast mode (nominally 921 600 baud, see the note).

+$0 UART_STATUS R · reset $06 after reset

Status (byte read).

7	6	5	4	3	2	1	0
-				RESET	TX_EMPTY	RX_EMPTY	FAST

BITS	FIELD	ACCESS	MEANING
7-4	-	R	0.
3	**RESET**	R	Control bit 3 as seen by the 24 MHz side.
2	**TX_EMPTY**	R	1 = transmit FIFO empty.
1	**RX_EMPTY**	R	1 = receive FIFO empty.
0	**FAST**	R	Control bit 0.

+$1 UART_DATA RW · reset -

Write: put one byte in the 2048-byte transmit FIFO. Read (byte): take one byte from the 2048-byte receive FIFO. Check RX_EMPTY first: reading an empty FIFO returns the previous byte.

+$2 RX_COUNT R · reset $0000

Number of bytes in the receive FIFO, 11 bits (word read at + `$2`, or bytes: + `$2` = bits 10-8, + `$3` = bits 7-0).

+$4 TX_COUNT R · reset $0000

Number of bytes in the transmit FIFO, 11 bits (word read at + `$4`, or bytes: + `$4` = bits 10-8, + `$5` = bits 7-0).

+$6 - R · reset -

Reads `$55` (byte) or `$DEAD` (word).

WiFi UART (WizFi360) $FEC0_0800

A UART connected to the WizFi360 Wi-Fi module (AT-command serial interface). Since build 70 it has the register layout of the USB-C UARTs COM3 / COM4. 8 bytes of registers, mirrored over `$FEC0_0800` - `$FEC0_09FF`. Format 8N1, LSB first, from the 24 MHz serial clock; a 2048-byte FIFO in each direction. FAST = 0 gives 115 200 baud, the module's factory rate; FAST = 1 gives 921 600 baud (26 clocks per bit, 923 kbaud).

Accesses: control, status and data are byte accesses; only byte writes are taken (control at + `$0`, transmit data at + `$1`). A word read at + `$0` returns the status in bits 15-8 and the receive head byte in bits 7-0 without taking it; word reads at + `$2` / + `$4` return the counts. Longword reads return `$DEAD_BEEF`.

Receive (build 70): the next received byte is fetched ahead from the FIFO into a head register. RX_EMPTY is 0 while the head holds a byte; a byte read of + `$1` returns that byte and takes it, and the next byte is fetched a few clocks after the read. The take is recognised on every new read of + `$1`, so tight status / data loops work. RX_COUNT includes the head byte; RX_EMPTY can read 1 for a few clocks after a data read although RX_COUNT is not 0.

Module reset: control bit 4 drives the module's RST pin (B16, with an RC on the board); 1 = hold the module in reset. It is 0 after a system reset, so the module runs. To reset it, write `$10`, wait a few ms (MCP: 20 ms), write `$00`: the module prints its start-up text and 'ready' after about 1-2 s and then takes AT commands.

Interrupts: received byte waiting = interrupt `$1F` (the input is RX_EMPTY, so with the default edge setting an interrupt is raised when a byte arrives while none was waiting); the module's own interrupt pin (K15) = interrupt `$21`. Pins: K16 = RxD into the FPGA (the module's TxD), K17 = TxD out (verified on the board: the module answers OK).

- MCP 160: TEST WIZFI [NORESET | LOOP | RAW cmd] resets the module (RST 20 ms), waits up to 6 s for its 'ready' line, sends AT (must answer OK) and AT+GMR (prints the firmware version), at 115 200 baud with a time-out on every wait. NORESET skips the reset; RAW sends one command and prints the answer in hex for 5 s; LOOP holds the module in reset and checks whether transmitted bytes come back on the board. Board-tested on build 70: AT -> OK, AT+GMR -> AT version 1.1.2.0.

Fixed in build 70 (build 69 and earlier): (1) a byte read of +$1 took a byte from the FIFO but returned the previous one, and the byte itself appeared at +$2 / +$3, so every byte arrived one read late; the head-byte prefetch replaces this. (2) The layout now matches COM3 / COM4: counts at +$2 and +$4 (were +$4 and +$6). (3) In fast mode the transmitter used 29 clocks per bit (about 828 000 baud) against 26 in the receiver; both now use 26 (923 kbaud). (4) The module's RST pin was not driven, so only a power cycle reset the module; control bit 4 now drives it. MCP 160's wifi_reg.h matches this layout ($FEC0_0800).

+$0 WIFI_CTRL W · reset $00

Control (byte write).

7	6	5	4	3	2	1	0
-			RESET	FIFO_RESET	-		FAST

BITS	FIELD	ACCESS	MEANING
7-5	-	W	Stored, no function.
4	RESET	W	1 = hold the WizFi360 in reset (its RST pin low). 0 after a system reset (build 70).
3	FIFO_RESET	W	1 = transmitter, receiver and both FIFOs held in reset (FIFOs emptied). Write 1, then 0.
2-1	-	W	Stored, no function.
0	FAST	W	0 = 115 200 baud, 1 = 921 600 baud.

+$0 WIFI_STATUS R · reset $06 after reset

Status (byte read).

7	6	5	4	3	2	1	0
-			RESET	FIFO_RESET	TX_EMPTY	RX_EMPTY	FAST

BITS	FIELD	ACCESS	MEANING
7-5	-	R	0.
4	RESET	R	Control bit 4.
3	FIFO_RESET	R	Control bit 3.
2	TX_EMPTY	R	1 = transmit FIFO empty.
1	RX_EMPTY	R	1 = no received byte in the head register (nothing to read).
0	FAST	R	Control bit 0.

+$1 WIFI_DATA RW · reset -

Write (byte): put one byte in the 2048-byte transmit FIFO (no full flag: check TX_COUNT). Read (byte): returns the received head byte and takes it. Check RX_EMPTY first: a read while it is 1 returns the last byte again and takes nothing.

+$2 RX_COUNT R · reset $0000

Bytes received and not read yet (the head byte included), 11 bits: word at + $2 , or bytes + $2 = bits 10-8, + $3 = bits 7-0.

+$4 TX_COUNT R · reset $0000

Bytes in the transmit FIFO waiting to be sent, 11 bits: word at + $4 , or bytes + $4 = bits 10-8, + $5 = bits 7-0. + $6 / + $7 read 0.

LPC bus and Super I/O (LPC47M107) $FEC0_2000

GAVIN is an LPC host. The window $FEC0_2000 - $FEC0_23FF maps the LPC I/O space $000 - $3FF : LPC I/O address = CPU address - $FEC0_2000 (for example COM1 at I/O $3F8 = $FEC0_23F8). Only LPC I/O cycles are made (no LPC memory, DMA or bus-master cycles), one byte per CPU access.

Behind it is an SMSC LPC47M107 Super I/O: PS/2 keyboard and mouse (8042-style controller), two 16550-type serial ports COM1/COM2, the parallel port LPT1, the floppy disk controller, the game port, an MPU-401 MIDI port, runtime/GPIO registers and fan control. Its devices, base addresses and IRQ numbers are set through its configuration ports ($2E / $2F = $FEC0_202E / $FEC0_202F). See the LPC47M107 datasheet.

Writes must be byte writes (MOVE.B); a word or longword write sends $00 . Use byte reads too: a read of any size makes one LPC byte read and returns the byte on all four lanes.

Writes are posted: the CPU gets its acknowledge as soon as the command is in GAVIN's 512-entry command FIFO. A read waits until its own data byte has come back from the 33 MHz LPC side. An LPC cycle that gets no answer within 64 LPC clocks (about 1.9 us) is ended; a read then returns $FF . A safety timeout of 65 535 bus clocks (about 1.3 ms) keeps the CPU from hanging if the LPC side does not run (for example while the Super I/O is held in reset).

The Super I/O is held in reset after power-up: set GAVIN_CTRL bit 8 (LPC_RSTn) before using it.

Interrupts come over the LPC serial IRQ line (quiet mode). GAVIN uses serial IRQ frames 1-7 and turns each rising edge into an interrupt pulse: IRQ1 keyboard -> $08 , IRQ2 mouse -> $09 , IRQ3 COM2 -> $0B , IRQ4 COM1 -> $0A , IRQ5 MPU-401 -> $0E , IRQ6 floppy -> $0D , IRQ7 LPT1 -> $0C . Configure the Super I/O devices to use these IRQ numbers (MCP does: keyboard 1, mouse 2, COM2 3, COM1 4, floppy 6, LPT1 7). Other serial IRQ numbers are ignored.

> Floppy: GAVIN does not run LPC DMA cycles, so the floppy controller must be used in non-DMA (PIO) mode.

Super I/O addresses as set up by MCP

DEVICE	LPC I/O ADDRESS	CPU ADDRESS	SERIAL IRQ	INTERRUPT
Configuration index / data	$2E / $2F	$FEC0_202E / $FEC0_202F	-	-
PS/2 keyboard data / status-command	$60 / $64	$FEC0_2060 / $FEC0_2064	1	$08
PS/2 mouse (same controller)	$60 / $64	$FEC0_2060 / $FEC0_2064	2	$09
Runtime registers (PME, SMI, GPIO, fan)	$100	$FEC0_2100	-	-
Game port	$200	$FEC0_2200	-	-
COM2	$2F8	$FEC0_22F8	3	$0B
LPT1	$378	$FEC0_2378	7	$0C
Floppy controller	$3F0	$FEC0_23F0	6	$0D
COM1	$3F8	$FEC0_23F8	4	$0A
MPU-401	$330 (set by MCP)	$FEC0_2330 - $FEC0_2331	5	$0E

I2C master $FEC0_0E00

Build 84 has a new I2C master, A2560Me_I2C, written for the A2560Me. It replaces the old block, which the CPU could not write on builds 38-83: up to build 83 only the automatic HDMI set-up worked. The automatic SiI9022 HDMI set-up is unchanged (same register writes, after reset and on every video mode change, see the video chapter).

One I2C bus on the board, shared with the RP2040 supervisor, which is also a master. The FPGA pins are on a 1.8 V bank; the SiI9022 HDMI transmitter is reached through a PCA9306 level shifter. The default speed is 100 kHz; 400 kHz and 1 MHz can be chosen, and devices may stretch the clock.

The block answers at $FEC0_0E00 - $FEC0_0EFF : a 64-byte register set, repeated 4 times. All registers are bytes. Any access size works (8, 16 or 32-bit) and the order is big-endian: the byte at + $00 is D31-D24, and a 16-bit value has its high byte first. Unused offsets

read 0.

There are two ways to use the bus. Transactions (XCTRL, + `$10` to + `$1C`): set the device, the register address and a byte count, and press GO; the hardware does START, address, register address, repeated START, up to 256 data bytes through a FIFO with ACK / NACK, and STOP. Byte level (CMD, + `$02`): one step at a time (START with the address, one byte, ACK / NACK, STOP), for devices that need an unusual sequence.

Three users share one byte engine, in this order of priority: the automatic HDMI set-up, then transactions, then byte-level commands. A request made while another one runs waits; nothing is interleaved. While a transaction runs (XBUSY), GO, CMD and FIFO accesses are ignored.

- Writing a device: push the bytes into the FIFO (+ `$18`), set ADDR, REG and COUNT, then XCTRL = GO + REG_SIZE (for example `$05` for an 8-bit register address). For the EEPROM's free half (`$00` - `$7F`) also set PAGE = 8 and EE_POLL: the controller splits the write at 8-byte pages and waits out each write cycle (it retries the address until the EEPROM answers, up to TIMEOUT).
- Bad request (ERR 5): COUNT above 256, a read with COUNT = 0, a write with COUNT larger than LEVEL, or ENABLE = 0. A write with COUNT = 0 sends only the register address. A write that fails keeps its unsent bytes in the FIFO. A read transaction empties the FIFO first.
- Byte level: with ADDR_AUTO = 1 (reset), a START command also sends the address byte (ADDR and the command's READ bit). Order: START, address, data byte, STOP. If the address is not acknowledged, NACK is set, the data byte is skipped, and a STOP is still sent if asked. A word write at + `$02` sets CMD and DATA together. RECOVER sends up to 9 SCL pulses until a stuck device releases SDA, then a STOP.
- Multi-master (the RP2040): the controller sees every START and STOP on the bus. It never starts while another master owns the bus (STATUS BUS_BUSY); a bus idle for 50 us counts as free (a missed STOP). If the other master wins the arbitration, the controller releases both lines at once and the command or transaction ends with ARB_LOST / ERR 3: just try again.
- The SiI9022 set-up after reset and on every video mode change has priority over CPU transactions; it always runs at 100 kHz. Software can ask for it again by writing 1 to bit 1 of I2C_HDMI (+ `$08`).
- Bus timing (SCL low / high): 100 kHz 5.0 / 4.5 us, 400 kHz 1.4 / 0.9 us, 1 MHz 0.56 / 0.38 us. The high time counts from the moment SCL is seen high, so a slow rise or clock stretching only makes the period longer. The inputs are filtered (2 flip-flops and a 4-clock glitch filter).
- Interrupt: CTRL bit 3 (IRQ_EN) exists, but in builds 84-87 the block's interrupt output is not connected to the interrupt controller. Poll STATUS / XSTATUS.
- MCP 171 has the I2C command: I2C (status), I2C SCAN, I2C MAC, I2C READ <dev> <reg> [n], I2C WRITE <dev> <reg> <bytes...>, I2C HDMI (program the SiI9022 again), I2C SPEED 100|400|1000. Hex numbers take 0x (for example I2C READ 0x39 0x1B 3). On older builds the command reports that it needs build 84.

> Builds 38-83: the old I2C block (I2C_CTRL / I2C_SLAVE / I2C_REG / I2C_DATA / I2C_COUNT byte registers) could not be written by the CPU, so only the automatic HDMI set-up worked. Software can read I2C_ID (`$FEC0_0E0C`) and compare with `$4932_4D01` before it uses the registers above. The LAN9221 does not load the MAC address by itself (it has no EEPROM of its own): a driver reads it from the 24AA02E48T and writes it into ADDRH / ADDRL at attach. Tested on the board (black board, build 84, MCP 171): MAC read at 100 and 400 kHz, SiI9022 ID registers B0 02 03, the HDMI picture through the new set-up and after a re-program from the CPU.

Devices on the I2C bus

7-BIT ADDRESS	8-BIT FORM	DEVICE	NOTES
`$39`	`$72`	SiI9022 HDMI transmitter (SII9022ACNU)	Through a PCA9306 level shifter. The FPGA programs it by itself after reset and on every video mode change.
`$4C`	`$98`	EMC2101 (EMC2101-R-ACZL-TR)	Fan controller and temperature: an internal sensor and an external diode input. It drives the 4-pin PWM fan. At power-on it is in standby / DAC mode (CONFIG `$03` bit 6 or 4 set): the PWM line stays high and the fan runs at full speed. MCP 174 FAN sets PWM mode at 25.7 kHz (PWM_F `$4D` = `$07`, PWM divide `$4E` = `$01`, FAN CONFIG `$4A` = `$27`, then CONFIG `$03` = `$84`; `$4A` = `$07` for the temperature table).
`$50`	`$A0`	24AA02E48T EEPROM (24AA02E48T-I/OT)	The board's MAC address at `$FA` - `$FF` (read only). `$00` - `$7F` is free EEPROM (8-byte pages).

7-BIT ADDRESS	8-BIT FORM	DEVICE	NOTES
$51-$57	$A2 - $AE	The same 24AA02E48T	The part ignores its address pins and answers on all eight addresses. Use $50.
$60	$C0	Si5351A (Si5351A-B-GMR)	The clock generator (PLL).
-	-	RP2040 supervisor	A second master on the bus. It does not answer as a slave in a scan.

Example: read the MAC address (6 bytes from the EEPROM at $FA)

STEP	ACCESS	VALUE	WHAT IT DOES
1	byte write + $07 (ADDR)	$50	Device: the 24AA02E48T.
2	word write + $14 (REG)	$00FA	Register address: the MAC.
3	word write + $12 (COUNT)	$0006	Six bytes.
4	byte write + $10 (XCTRL)	$07	GO + READ + 8-bit register address (REG_SIZE 1).
5	byte read + $11 (XSTATUS)	wait for bit 1	XDONE; about 1 ms at 100 kHz. Use a time limit in software.
6	byte read + $11 (XSTATUS)	bits 7-4	ERR: 0 = OK; 1 no device, 3 bus lost (try again), 4 timeout.
7	6 x byte read + $18 (FIFO)	MAC[0]..MAC[5]	Each read takes one byte from the FIFO. Tested: 74:C9:0F:07:0C:17 (black board), at 100 and 400 kHz.

+$00 I2C_CTRL RW · reset $11

Control.

7	6	5	4	3	2	1	0
-			ADDR_AUTO	IRQ_EN	SPEED		ENABLE

BITS	FIELD	ACCESS	MEANING
7-5	-	R	Read 0.
4	**ADDR_AUTO**	RW	1 (reset) = a byte-level START also sends the address byte: ADDR and the READ bit of the command. 0 = send the address yourself with a WRITE command.
3	**IRQ_EN**	RW	Interrupt enable (on DONE or XDONE). The interrupt output is not connected in builds 84-87: poll instead.
2-1	**SPEED**	RW	0 = 100 kHz (reset), 1 = 400 kHz, 2 = 1 MHz, 3 = the I2C_DIVIDER value.
0	**ENABLE**	RW	1 (reset) = the controller runs. With 0, commands are ignored and a transaction ends with ERR 5.

+$01 I2C_STATUS R / W1C · reset $C0 (idle bus)

Status of the byte-level engine and of the bus. A new CMD clears bits 1, 2, 3 and 5 too.

7	6	5	4	3	2	1	0
SDA	SCL	TIMEOUT	BUS_BUSY	ARB_LOST	NACK	DONE	BUSY

BITS	FIELD	ACCESS	MEANING
7	**SDA**	R	The SDA line, as seen after the input filter.
6	**SCL**	R	The SCL line, as seen after the input filter.
5	**TIMEOUT**	W1C	A byte-level command waited longer than I2C_TIMEOUT (clock stretching or a busy bus).
4	**BUS_BUSY**	R	Some master (this one or the RP2040) owns the bus: set by any START, cleared by its STOP or after 50 us of idle bus.
3	**ARB_LOST**	W1C	Another master won the arbitration: both lines were released at once.
2	**NACK**	W1C	The address or the byte was not acknowledged.
1	**DONE**	W1C	The last byte-level command has finished.
0	**BUSY**	R	A byte-level command is running.

+$02 I2C_CMD W (reads 0) · reset $00

Byte-level command. Several bits may be set in one write; they run in this order: START, address (ADDR_AUTO), WRITE or READ, STOP. Ignored while BUSY, while a transaction runs, or with ENABLE = 0.

7	6	5	4	3	2	1	0
RECOVER	-		STOP	NACK	READ	WRITE	START

BITS	FIELD	ACCESS	MEANING
7	**RECOVER**	W	Bus recovery: up to 9 SCL pulses until SDA is released, then a STOP. The other bits are ignored.
6-5	**-**	W	Ignored.
4	**STOP**	W	Send a STOP at the end and release the bus.
3	**NACK**	W	The reply to READ: 1 = NACK (the last byte), 0 = ACK.
2	**READ**	W	Read one byte into I2C_DATA. Also the R/W bit of the automatic address byte.
1	**WRITE**	W	Send the byte in I2C_DATA.
0	**START**	W	Send a START (a repeated START if this controller already owns the bus).

+$03 I2C_DATA RW · reset $00

Write: the byte for a byte-level WRITE (ignored while BUSY). Read: the last byte sent or received.

+$04 I2C_DIVIDER RW · reset $014D (333)

SPEED 3 only: the SCL low time and the high time, each in bus clocks (half a period). Minimum 20. 333 at 66.67 MHz gives about 100 kHz.

+$06 I2C_TIMEOUT RW · reset $19 (25)

Time limit in ms for clock stretching, a busy bus and EEPROM polling (0 = never). Used by byte-level commands and transactions.

+$07 I2C_ADDR RW · reset $50

The 7-bit device address (bits 6-0) for byte-level commands and transactions. The reset value is the MAC EEPROM.

+$08 I2C_HDMI RW · reset $01

The automatic SiI9022 set-up.

7	6	5	4	3	2	1	0
RUNS				-		ERROR / RERUN	RUNNING

BITS	FIELD	ACCESS	MEANING
7-4	**RUNS**	R	Number of set-ups done (counts modulo 16).
3-2	**-**	R	Read 0.
1	**ERROR / RERUN**	RW	Read: 1 = the last set-up had an error (NACK, bus lost or timeout). Write 1: program the SiI9022 again for the current video mode.
0	**RUNNING**	R	1 = a set-up is running or waiting to run.

+$0C I2C_ID R · reset $4932_4D01

Identification: "I2", "M", version 1. Software can read it to find the build 84 controller (older builds do not return this value).

+$10 I2C_XCTRL RW · reset $04

Transaction control. Ignored while XBUSY.

7	6	5	4	3	2	1	0
CLR	-		EE_POLL	REG_SIZE		RW	GO

BITS	FIELD	ACCESS	MEANING
7	**CLR**	W	Write 1: empty the FIFO. Reads 0.
6-5	**-**	R	Read 0.
4	**EE_POLL**	RW	Write cycle polling: if the address is not acknowledged, STOP and try again until I2C_TIMEOUT (for EEPROM writes).
3-2	**REG_SIZE**	RW	Register address size: 0 = none, 1 = 8-bit (reset), 2 = 16-bit (high byte first), 3 = 24LCxx (REG bits 10-8 go into the device address).
1	**RW**	RW	1 = read, 0 = write.
0	**GO**	W	Write 1: start the transaction (clears XDONE and ERR). Reads 0.

+$11 I2C_XSTATUS R / W1C · reset $00

Transaction status.

7	6	5	4	3	2	1	0
ERR				-		XDONE	XBUSY

BITS	FIELD	ACCESS	MEANING
7-4	**ERR**	R	0 = OK, 1 = no device (address not acknowledged), 2 = data refused (NACK), 3 = bus lost (arbitration), 4 = timeout, 5 = bad request.
3-2	**-**	R	Read 0.
1	**XDONE**	W1C	The transaction has finished. GO clears it too.
0	**XBUSY**	R	A transaction is running or waiting for the bus.

+$12 **I2C_COUNT** RW · reset $0001

Number of data bytes to transfer, 0-256.

+$14 **I2C_REG** RW · reset $0000

The register or memory address inside the device (sent as 0, 1 or 2 bytes, as REG_SIZE says).

+$16 **I2C_PAGE** RW · reset $00

Writes are split at PAGE-byte boundaries of REG, so an EEPROM page write never wraps (0 = no split). 8 for the 24AA02E48T.

+$18 **I2C_FIFO** RW · reset -

The 256-byte data FIFO (called DATA in the developer notes). Write: push a byte to send. Read: take a received byte. Only an access that covers + `$18` moves the FIFO; ignored while XBUSY.

+$1A **I2C_LEVEL** R · reset $0000

Number of bytes in the FIFO (0-256).

+$1C **I2C_XFER** R · reset $0000

Number of data bytes moved by the last transaction.

FPGA temperature monitor (System Monitor) `$FEC0_0D00`

Build 87 reports the temperature of the FPGA die. The Kintex-7 has one XADC (its on-chip analogue monitor), and the DDR3 memory controller (the AMD/Xilinx MIG) owns it: the MIG reads the die temperature for its DDR3 timing compensation and passes the 12-bit XADC code on, updating it by itself. This block takes that code into the CPU bus clock, converts it to 1/16 °C, keeps the highest and lowest values and compares them with a limit.

Conversion: T (°C) = code x 503.975 / 4096 - 273.15. The block computes code x 8064 / 4096 - 4370 in 1/16 °C (error below 0.1 °C). Example: code `$9F9` gives TEMP `$0000_0290` = 656 / 16 = 41.0 °C.

Clock crossing: the code comes from the MIG's clock. Every bit passes two flip-flops, and a new code is used only after it has stayed the same for 8 bus clocks, so a code caught while it changes is never used. A code of 0 means that the MIG has not started yet and is ignored.

The block answers at `$FEC0_0D00` - `$FEC0_0DFF`: a 64-byte register set, repeated 4 times. Any access size works (8, 16 or 32-bit), big-endian; unused offsets read 0. The STATUS write bits are in its lowest byte (+ `$1B`): use a long write to + `$18` or a byte write to + `$1B`. There is no interrupt: poll STATUS.

- Voltages (VCCINT, VCCAUX, VCCBRAM) are not available: the XADC belongs to the MIG, and only the temperature is passed on.
- MCP 176: TEMP shows the temperature, the XADC code, MAX, MIN and the limit; TEMP CLEAR writes `$14` to STATUS (restarts MAX / MIN and clears OVER_SEEN).
- The board's other temperature sensors are on the EMC2101 fan controller (I2C `$4C`, see I2C master): MCP FAN shows them.

+$00 **SYSMON_ID** R · reset $5359_4D31

Identification: `$5359_4D31` ('SYM1'). Software can read it to check that the block is there (build 87 and later).

+$04 **SYSMON_TEMP** R · reset -

The die temperature, signed, in 1/16 °C (`$0000_0290` = 41.0 °C). Valid only when STATUS bit 0 (VALID) is 1: before the first reading it reads `$FFFF_EEEE` (-4370, the formula for code 0).

+$08 SYSMON_RAW R · reset $0000_0000

The XADC code used for TEMP.



BITS	FIELD	ACCESS	MEANING
31-12	-	R	Read 0.
11-0	CODE	R	The 12-bit XADC temperature code. 0 until the first reading.

+$0C SYSMON_MAX R · reset $8000_0000

The highest TEMP since reset or the last restart (STATUS bit 4), signed, 1/16 °C. Reads `$8000_0000` until the first reading.

+$10 SYSMON_MIN R · reset $7FFF_FFFF

The lowest TEMP since reset or the last restart, signed, 1/16 °C. Reads `$7FFF_FFFF` until the first reading.

+$14 SYSMON_LIMIT RW · reset $0000_0550

Temperature limit, signed, 1/16 °C. Reset 1360 = 85 °C. STATUS bits 1 and 2 compare TEMP with it.

+$18 SYSMON_STATUS R / W · reset $0000_0000

Status. Writes act only through the lowest byte (+ `$1B`); bits 0 and 1 cannot be written.



BITS	FIELD	ACCESS	MEANING
31-5	-	R	Read 0.
4	MM_RESTART	W	Write 1: MAX and MIN restart from the current TEMP. Reads 0.
3	-	R	Read 0.
2	OVER_SEEN	R / W1C	Sticky: TEMP has reached LIMIT since reset or the last clear. Write 1 to clear it.
1	OVER	R	TEMP >= LIMIT now (only while VALID).
0	VALID	R	1 = a reading has arrived from the MIG. 0 after reset until the DDR3 controller has started.

Fixed-point maths unit $FEC0_3000

32 x 32 -> 64-bit multipliers and 32 / 32-bit dividers, signed and unsigned, computing continuously from their operand registers. Write operands with 32-bit accesses only. The 64-byte block is mirrored at `$FEC0_3040`. MCP does not use it.

There is no ready flag. The multipliers take 1 clock and the dividers 8 clocks: read a result at least about 12 bus clocks (180 ns) after the last operand write, or you get the previous result.

Floating point: see the Floating Math Block at `$FEC0_4000` (next section, build 87).

$00 UMUL_A RW · reset -

Unsigned multiply operand A.

$04 **UMUL_B** RW · reset -

Unsigned multiply operand B.

$08 **UMUL_LO** R · reset -

A x B, bits 31-0 (unsigned).

$0C **UMUL_HI** R · reset -

A x B, bits 63-32 (unsigned).

$10 **SMUL_A** RW · reset -

Signed multiply operand A.

$14 **SMUL_B** RW · reset -

Signed multiply operand B.

$18 **SMUL_LO** R · reset -

A x B, bits 31-0 (signed).

$1C **SMUL_HI** R · reset -

A x B, bits 63-32 (signed).

$20 **UDIV_DIVISOR** RW · reset -

Unsigned divide: divisor (denominator).

$24 **UDIV_DIVIDEND** RW · reset -

Unsigned divide: dividend (numerator).

$28 **UDIV_QUOTIENT** R · reset -

Unsigned quotient.

$2C **UDIV_REMAINDER** R · reset -

Unsigned remainder.

$30 **SDIV_DIVISOR** RW · reset -

Signed divide: divisor.

$34 **SDIV_DIVIDEND** RW · reset -

Signed divide: dividend.

$38 **SDIV_QUOTIENT** R · reset -

Signed quotient.

`$3C` **SDIV_REMAINDER** `R · reset -`

Signed remainder.

Floating Math Block (FPU) `$FEC0_4000`

Build 87 has a Floating Math Block: a single-precision (IEEE-754, 32-bit) maths unit with multiply, divide, add / subtract, square root, sine and cosine, and conversions from and to signed 20.12 fixed point and 32-bit integers. It is built on the AMD/Xilinx Floating-Point (7.1) and CORDIC (6.0) IP cores and replaces the old floating-point window, which was only a placeholder (it read `$BBBB_CCCC`). The 68LC060 has no FPU of its own.

Every unit computes all the time from the current operands. Software writes CTRL, IN0 and IN1, waits until STATUS bit 0 (BUSY) is 0, and reads the result. The registers answer at normal GAVIN speed: the CPU never waits for a result on the bus (no wait states); it polls BUSY or does other work meanwhile. A result read while BUSY is 1 may still be the old one.

IN0 and IN1 are each converted to float32 first (their format is set in CTRL). The multiplier and the divider take IN0 x IN1 and IN0 / IN1. The adder takes two inputs chosen in CTRL (IN0, IN1, the multiplier result or the divider result), so A x B + C / D or A x B - A needs one pass. Square root, sine and cosine take IN0. CTRL bits 11-9 choose which result appears in RESULT; RESULT_FIXED and RESULT_INT show the same value converted.

The block answers at `$FEC0_4000` - `$FEC0_41FF`: a 32-byte register set, repeated 16 times. Any access size works (8, 16 or 32-bit), big-endian; a byte or word write changes only the bytes written. CTRL, IN0 and IN1 are 0 after reset.

- BUSY times are counted from the clock after the write is stored; BUSY falls in the clock in which RESULT, RESULT_FIXED and RESULT_INT are final. Any write to offsets `$00` - `$0F` starts BUSY again for the path that CTRL selects, so write CTRL and both operands, then poll once. A 68060 read through GAVIN takes several bus clocks, so a STATUS poll or two is usually all the waiting there is.
- Formats: float32 is IEEE-754 single precision. Fixed point 20.12 is signed: 20 integer bits (including the sign) and 12 fraction bits, range -524288 .. +524287.99976 in steps of 1/4096. int32 is a signed 32-bit integer. Conversions to fixed point and int32 round to nearest.
- Limits: SIN and COS take an angle in radians within -pi .. +pi; software reduces larger angles first. STATUS bit 26 flags an angle outside the range. The sine / cosine core works with 22 fraction bits, so SIN and COS are good to about 6-7 decimal digits (a few steps of the last float32 bit), not to full float32 precision. MUL, DIV, ADD and SQRT are correctly rounded (round to nearest).
- Very small numbers (denormals) are treated as zero by the floating-point cores; NaN and infinity follow IEEE-754 and are flagged in STATUS.
- No interrupt: poll STATUS bit 0.
- Verified in simulation against the AMD/Xilinx IP simulation models: every RESULT path, every format, the flags, and that BUSY falls exactly when the outputs are final. MCP 176 FPU shows CTRL and STATUS; FPU TEST checks 10 vectors (3 x 2.5, 1 / 3, 1.5 + 2.25, 1.5 - 2.25, sqrt 2, 3 x 4 + 3 / 4, int 7 x 0.5, fixed 1.25 x 2, sin 0.5, cos 1.0) and the divide-by-zero and square-root invalid flags.

BUSY time (66.67 MHz bus clock, 15 ns)

RESULT SELECT	PATH	BUSY CLOCKS	TIME
`6 (IN0 as float32)`	input conversion	5	75 ns
`0 (MUL)`	input conversion, multiply	8	120 ns
`1 (DIV), 3 (SQRT)`	input conversion, divide or square root	13	195 ns
`2 (ADD) of IN0 / IN1`	input conversion, add	9	135 ns
`2 (ADD) with the MUL result as an input`	multiply, then add	12	180 ns
`2 (ADD) with the DIV result as an input`	divide, then add	17	255 ns
`4 (SIN), 5 (COS)`	angle conversion, CORDIC, conversion back to float32	36	540 ns

Example: 3 x 2.5, then sin(0.5)

STEP	ACCESS	VALUE	WHAT IT DOES
1	long write + `$00` (CTRL)	`$0000_0000`	RESULT = MUL, IN0 and IN1 float32.
2	long write + `$08` (IN0)	`$4040_0000`	3.0
3	long write + `$0C` (IN1)	`$4020_0000`	2.5
4	long read + `$04` (STATUS)	until bit 0 = 0	BUSY: 8 clocks after the last write, usually one or two polls.
5	long read + `$10` (RESULT)	`$40F0_0000`	7.5. RESULT_FIXED (+ `$14`) reads `$0000_7800` (7.5 in 20.12).
6	long write + `$00` (CTRL)	`$0000_0800`	RESULT = SIN, IN0 float32.
7	long write + `$08` (IN0)	`$3F00_0000`	0.5 (radians).
8	long read + `$04` (STATUS)	until bit 0 = 0	BUSY: 36 clocks. Check bit 26 (ANGLE) = 0.
9	long read + `$10` (RESULT)	about `$3EF5_7744`	0.4794255, within a few steps of the last bit (see Limits).

More CTRL values

CTRL	COMPUTES
`$0000_0200`	IN0 / IN1
`$0000_0400`	IN0 + IN1
`$0000_0410`	IN0 - IN1
`$0000_05C0`	IN0 x IN1 + IN0 / IN1 (ADD, ADD_A = MUL, ADD_B = DIV); 3 and 4 give 12.75
`$0000_0600`	square root of IN0
`$0000_0A00`	cosine of IN0
`$0000_0C02`	IN0 (int32) converted to float32 in RESULT
`$0000_0008`	IN0 (float32) x IN1 (int32)
`$0000_0001`	IN0 (20.12) x IN1 (float32)

+$00 FPU_CTRL

RW · reset $0000_0000

Formats, adder set-up and result select. A write starts BUSY.



BITS	FIELD	ACCESS	MEANING
31-12	**-**	RW	Stored and read back, no effect. Write 0.
11-9	**RESULT**	RW	What RESULT shows: 0 = MUL (IN0 x IN1), 1 = DIV (IN0 / IN1), 2 = ADD (the adder), 3 = SQRT (square root of IN0), 4 = SIN (sine of IN0), 5 = COS (cosine of IN0), 6 = IN0 as float32 (the input conversion alone, for example int32 to float). 7 acts as 6.
8-7	**ADD_B**	RW	Adder input B: 0 = IN1, 1 = IN0, 2 = the MUL result, 3 = the DIV result.
6-5	**ADD_A**	RW	Adder input A: 0 = IN0, 1 = IN1, 2 = the MUL result, 3 = the DIV result. With ADD_A = ADD_B = 0 the adder computes IN0 + IN1 (or IN0 - IN1).
4	**ADD_SUB**	RW	0 = A + B, 1 = A - B.
3-2	**IN1_FMT**	RW	Format of IN1: 0 = float32, 1 = signed fixed point 20.12, 2 = signed 32-bit integer. 3 acts as 0.
1-0	**IN0_FMT**	RW	Format of IN0, as IN1_FMT.

+$04 FPU_STATUS R · reset -

BUSY and the flags. The flags are not sticky: they describe the current operands and settings, and they are final when BUSY is 0. The unit flags (bits 8-20) are those of each unit, whatever RESULT selects. A write to this offset is ignored, but it starts BUSY like any write to offsets `$00`-`$0F`.



BITS	FIELD	ACCESS	MEANING
31-27	**-**	R	Read 0.
26	**ANGLE**	R	SIN / COS: the angle (IN0) is outside -pi .. +pi. The SIN / COS result is not valid.
25	**INT_OVF**	R	RESULT does not fit RESULT_INT (int32), or is not a number.
24	**FIX_OVF**	R	RESULT does not fit RESULT_FIXED (20.12), or is not a number.
23-21	**-**	R	Read 0.
20	**SQRT_INV**	R	Square root invalid (IN0 < 0).
19	**-**	R	Read 0.
18	**ADD_INV**	R	Adder invalid operation (for example infinity - infinity).
17	**ADD_UNF**	R	Adder underflow.
16	**ADD_OVF**	R	Adder overflow.
15	**DIV_ZERO**	R	Divide by zero (IN1 = 0).
14	**DIV_INV**	R	Divider invalid operation (0 / 0, infinity / infinity).
13	**DIV_UNF**	R	Divider underflow.
12	**DIV_OVF**	R	Divider overflow.
11	**-**	R	Read 0.
10	**MUL_INV**	R	Multiplier invalid operation (0 x infinity).
9	**MUL_UNF**	R	Multiplier underflow.
8	**MUL_OVF**	R	Multiplier overflow.
7-4	**-**	R	Read 0.
3	**NEG**	R	RESULT is negative (its sign bit; also set for -0).
2	**ZERO**	R	RESULT is +0 or -0.
1	**NAN**	R	RESULT is not a number (NaN).
0	**BUSY**	R	1 = a result is not final yet. Set by every write to offsets `$00`-`$0F`; falls in the clock in which RESULT, RESULT_FIXED and RESULT_INT are final (see the BUSY table).

+$08 FPU_IN0 RW · reset $0000_0000

Operand 0, in the format set by CTRL IN0_FMT. A write starts BUSY.

+$0C FPU_IN1 RW · reset $0000_0000

Operand 1, in the format set by CTRL IN1_FMT. A write starts BUSY.

+$10 **FPU_RESULT** R · reset -

The result chosen by CTRL RESULT, float32.

+$14 **FPU_RESULT_FIXED** R · reset -

RESULT as signed fixed point 20.12 (bits 31-12 integer part, bits 11-0 fraction; 1.0 = `$0000_1000`), rounded to nearest. STATUS bit 24 flags a value that does not fit.

+$18 **FPU_RESULT_INT** R · reset -

RESULT as a signed 32-bit integer, rounded to nearest. STATUS bit 25 flags a value that does not fit.

+$1C **FPU_ID** R · reset $4650_5531

Identification: `$4650_5531` ('FPU1'). Software can read it to check that the block is there (build 87 and later).

SDMA registers (pointer) `$FEC0_C000`

The SRAM-to-SRAM DMA engine (SDMA) is decoded by GAVIN at `$FEC0_C000` - `$FEC0_C0FF` (256 bytes since build 26; the 8 registers repeat every 32 bytes). It is a bus master: it requests the bus from the arbiter and moves 8, 16 or 32-bit units inside the 8 MB SRAM (`$0000_0000` - `$007F_FFFF`), 1D or 2D, copy or fill. Its end-of-transfer interrupt is source `$17`. Registers (32-bit writes only) are described in the memory and DMA chapter.

The MDMA (SRAM <-> DDR3, `$FEC0_C100`, interrupt `$11`) and the VDMA (DDR3 blitter, `$FEC0_C200`, interrupt `$10`) are decoded next to it; they are described in the same chapter.

SDMA register summary

ADDRESS	REGISTER
`$FEC0_C000`	Control / status (bit 31 = busy)
`$FEC0_C004`	Source address
`$FEC0_C008`	Destination address
`$FEC0_C00C`	1D size (bytes)
`$FEC0_C010`	2D size (X bytes per line, Y lines)
`$FEC0_C014`	2D strides (source, destination)
`$FEC0_C018`	Fill data

Local bus timing summary

The RTC, SD card 2, IDE, TRINITY and Ethernet share GAVIN's 16-bit local bus (address A8-A0, data D15-D0, RD and WR strobes, one chip select per device). Each device has its own access state machine; the CPU cycle ends with a handshake when the local cycle is finished, so these accesses take longer than internal registers. Only one local-bus access runs at a time.

The figures below are in bus clocks of 15 ns. They are derived from the access-module parameters (front porch, strobe, back porch, given in ns in the RTL and rounded up to whole clocks); they are approximate (within one clock). Build 39: the write data is on the bus from the start of the cycle, before the write strobe falls, and held until the end of the cycle (back porch); read strobes are one clock longer than before because the pins are registered.

Since build 34: LAN9221 and ATA data reads take the bitwise majority of three samples of the data bus, one clock apart, at the end of the strobe. The local-bus clock output (LOCAL_BUS_CLK, used only by the 10M02) is BCLK / 2 = 33.33 MHz (build 37+), with its edges half a BCLK after the strobe changes (build 38; MEMSYS_CTRL bit 14 = 1 gives the build 37 phase). MEMSYS_CTRL bit 13 holds it low (diagnostic).

Build 39 local-bus hardening. The local bus runs to devices far from the FPGA (long lines, many loads). Under NetBSD on build 35 the mSATA bridge stored some 16-bit writes twice (runs of 8 to 11 consecutive duplicated words; the drive reached 256 words before the CPU had written 255): a write strobe counted twice. Build 39 removes every path inside the FPGA that could cause that: (1) one device select per CPU bus cycle - the chip-select decode comes from the CPU address and TIP pins, so the local bus latches the selected device in the first clock of the cycle and holds it until the cycle ends (TS ... TA / TEA), and a glitch on those pins can no longer cut a strobe or start a second transfer; (2) every local-bus output (address, RD, WR, all chip selects, data and data enable) leaves through an I/O-cell flip-flop, after the multiplexers; (3) the data pins use SLOW slew (were FAST); (4) on every device the write data is on the bus before the write strobe falls (it used to switch on the same edge) and stays until after it rises; (5) longer IDE write recovery, with MEMSYS_CTRL bits 23-16 to lengthen the IDE write strobe and recovery for tests. Because the pins are registered, every read strobe is one clock longer, so the time from the strobe at the pin to the data capture is unchanged. Every build since 39 has these changes.

Board type (build 40): see IDE / SATA bridge. On a white board with DIP switch 7 OFF, or with builds up to 39, the JM20330 can drive the shared data bus during the other devices' reads.

> Builds up to 38: read strobes one clock shorter, and the write data switched on the same clock edge as the write strobe.

Local-bus cycles

DEVICE	WINDOW	DATA WIDTH / CPU ACCESS	READ STROBE	WRITE STROBE
BQ4802 RTC	`$FEC0_0080`	8-bit, byte writes	11 clocks (165 ns)	8 clocks (120 ns)
SD card 2 (10M02)	`$FEC0_0380`	8-bit, byte writes	8 clocks (120 ns)	4 clocks (60 ns)
JM20330 IDE	`$FEC0_0400`	16-bit data register (bytes swapped), 8-bit others	27 clocks (405 ns)	23 clocks (345 ns); data ~100 ns before and ~120 ns after; ~735 ns per write
TRINITY (10M02)	`$FEC0_0500`	8-bit, byte writes	9 clocks (135 ns)	7 clocks (105 ns)
LAN9221	`$FEC0_0600`	16-bit word accesses (bytes swapped); 32-bit = two cycles	6 clocks (90 ns) with the read filter, 5 without	3 clocks (45 ns), data on the bus 1 clock before

Other GAVIN functions without CPU registers

- Debug port: the USB-C debug UART (protocol v2 at 8 Mbit/s, the only protocol since build 86; memory access also while the CPU runs, flash programming, pause, breakpoints, the level-7 monitor stop and system reset by the host) runs inside the FPGA and has no CPU registers; see the debug port sections of the memory chapter. The boot block selection registers are GAVIN control registers (`$FEC0_0040` - `$FEC0_004C`).
- 32.768 kHz clock and 33 MHz LPC clock for the Super I/O (see 32 kHz clock).
- Power LED and media / SD-card LED outputs follow GAVIN_CTRL bits 0-2.
- SRAM memory-management block: shares the SRAM between MemText, the EmuTOS bitmap, the SDMA, the MDMA and the CPU; it runs on the bus clock (no registers in the GAVIN window; MemText and EmuTOS registers are in the VICKY II window).
- The PCIe slot present pins are readable in GAVIN_CTRL bits 17-19. The PCI Express root port itself (build 74) has its own chapter; its interrupt is source `$23` .

3 Video: VICKY II, text, MemText and the GUI bitmap engine

The A2560Me video system is built into the FPGA and drives an HDMI output through a SiI9022 transmitter. It runs at 1280 x 960 or 1280 x 1024, 60 Hz, from a 108 MHz dot clock, except while the Game Graphics Engine is on (GGE_ENABLE, see the GGE chapter): then the GGE drives the output at 640 x 480 or 640 x 400 from 25 MHz.

It has these parts: VICKY II (master control, border, the 160-column block text mode, mouse pointer, gamma), MemText (a larger text mode whose buffers are in SRAM), TOSGRAPH (an Atari-style bitplane bitmap in SRAM for EmuTOS), and the GUI bitmap engine (a full-screen chunky bitmap in DDR3 with 1 to 32 bits per pixel). A pixel mixer combines them.

This chapter describes FPGA build 87. The game graphics of the A2560Me are in the Game Graphics Engine (GGE, its own chapter), switched on by its own register GGE_CTRL at `$FD00_0000`. The old VICKY IV game-engine bits of the master control register (bits 3, 4, 5, 7, 11, 16) have no effect; master bit 2 (graphics) only selects the GUI bitmap layer.

Video address map and access sizes

The video blocks of this chapter are in two CPU windows: the VICKY II window `$FEC8_0000` - `$FECF_FFFF` (control registers, block text, mouse pointer, gamma, MemText, TOSGRAPH) and the GUI window `$FC00_0000` - `$FC00_0FFF` (GUI bitmap engine, chip select CS_GUI since build 60; the rest of `$FC0` x ends with a bus error). The GGE has its own window at `$FD00_0000` (GGE chapter).

The blocks do not decode every address bit, so registers repeat (alias) inside their window. Reads from an address inside `$FEC8_0000` - `$FECF_FFFF` that belongs to no block return `$DEAD_BEEF`.

Each block accepts only some access sizes. A wrong size is not an error on the bus: the write is silently ignored, or worse (see the note). The table lists the sizes that work.

> VICKY II control registers (`$FEC8_0000-$FEC8_003F`): since build 25 byte, word and longword writes all work and change only the bytes written (earlier builds ignored word writes, and a byte write cleared 16 bits). Write sizes that are not listed for a block are ignored, except for the MemText LUTs, where a byte or word write stores 0.

Video address map

ADDRESS	SIZE	CONTENTS
`$FEC8_0000-$FEC8_003F`	64 B	VICKY II control registers (repeat every 64 bytes up to `$FEC8_00FF`)
`$FEC8_0400-$FEC8_07FF`	1 KB	Mouse pointer image 0 (16 x 16 x 32-bit)
`$FEC8_0800-$FEC8_0BFF`	1 KB	Mouse pointer image 1
`$FEC8_0C00-$FEC8_0C0F`	16 B	Mouse pointer registers (repeat up to `$FEC8_0CFF`)
`$FEC8_4000-$FEC8_40FF`	256 B	Gamma table, blue
`$FEC8_4100-$FEC8_41FF`	256 B	Gamma table, green
`$FEC8_4200-$FEC8_42FF`	256 B	Gamma table, red
`$FEC8_8000-$FEC8_9FFF`	8 KB	Block text font memory window (build 25: `$FEC8_8000` = the displayed 8 x 16 font, `$FEC8_9000` = the two 8 x 8 fonts)
`$FEC9_7000-$FEC9_707F`	128 B	TOSGRAPH registers (repeat up to `$FEC9_7FFF`)
`$FEC9_8000-$FEC9_803F`	64 B	MemText registers (repeat up to `$FEC9_8FFF`)

ADDRESS	SIZE	CONTENTS
$FEC9_9000-$FEC9_97FF	2 KB	MemText foreground LUT (512 x 32-bit)
$FEC9_9800-$FEC9_9FFF	2 KB	MemText background LUT (512 x 32-bit)
$FEC9_A000-$FEC9_BFFF	8 KB	MemText font memory
$FECA_0000-$FECA_3FFF	16 KB	Block text: character memory
$FECA_8000-$FECA_BFFF	16 KB	Block text: colour memory
$FECA_C400-$FECA_C43F	64 B	Block text foreground LUT (16 x 32-bit)
$FECA_C440-$FECA_C47F	64 B	Block text background LUT (16 x 32-bit)
$FC00_0000-$FC00_003F	64 B	GUI bitmap engine registers
$FC00_0400-$FC00_07FF	1 KB	GUI bitmap engine palette (256 x 32-bit)
$FD00_0000-$FD0F_FFFF	1 MB	Game Graphics Engine (see its chapter)
$FEC0_0E00-$FEC0_0EFF	256 B	I2C master (GAVIN area, any access size): the SiI9022 set-up and the system I2C bus; see the GAVIN chapter

Access sizes that work

BLOCK	WRITE	READ
VICKY II control registers	8, 16 or 32-bit (build 25)	any size
Block text character / colour memory	8-bit only	any size; the byte is repeated in all 4 byte lanes, so use 8-bit reads
Block text font window	8, 16 or 32-bit, any offset (build 44)	any size, each byte on its own lane (build 45)
Block text FG / BG LUT	32-bit only	not readable (`$DEAD_BEEF`)
Gamma tables	8-bit only	8-bit (byte repeated in all lanes)
Mouse pointer registers	16-bit only	16-bit or 32-bit
Mouse pointer images	32-bit only	not readable (`$DEAD_BEEF`)
MemText registers	32-bit only	any size
MemText LUTs	32-bit only	not readable (always `$AAAA_5555`)
MemText font	8, 16 or 32-bit, any offset (build 44)	any size, each byte on its own lane (build 45)
TOSGRAPH registers	32-bit only	not reliable (see TOSGRAPH)
GUI engine registers and palette	32-bit only	any size
GGE registers, sprite attributes, palettes	32-bit only	32-bit

Video timing and HDMI output

While GGE_ENABLE is 0 the video output runs from the 108 MHz dot clock. Two modes are available, chosen with bit 8 of the master control register: 1280 x 960 at 60 Hz (bit 8 = 0, the reset default) and 1280 x 1024 at 60 Hz (bit 8 = 1). The new timing is loaded at the next start of frame. While GGE_ENABLE is set, the GGE drives the output instead (25 MHz, see the GGE chapter); the 108 MHz system keeps running unseen.

Every layer is generated at 1280 pixels per line. The lower resolutions (640 x 480 / 640 x 512 in the GUI engine and TOSGRAPH) are made by showing each pixel and each line twice; the monitor always receives 1280 x 960 or 1280 x 1024.

Each frame starts with the vertical blanking; the visible lines come last. Each line starts with the horizontal blanking; the 1280 visible pixels come last. A start-of-frame (SOF) pulse is generated once per frame at the start of the vertical blanking and goes to the interrupt controller (source `$00`). While GGE_ENABLE is set, source `$00` takes the GGE's frame start instead. The start-of-line interrupt (source `$01`) comes from the GGE line interrupt (LINE_IRQ), not from this timing.

The pixel mixer drives the SiI9022 HDMI transmitter with 24-bit RGB, DE, HSYNC and VSYNC. Both syncs are positive. Since build 28 the last register of each of these 27 signals sits in its I/O pad, and the pixel clock (IDCK) comes from an ODDR on the same clock, so the data-to-clock timing at the pins is the same in every build (figure 2). IDCK rises in the middle of each pixel, where the SiI9022 latches the bus; GAVIN register 0 bit 9 turns it over (leave it 0). This fixed the noisy text and grey levels of build 27, whose unconstrained output timing had moved by about 2.6 ns.

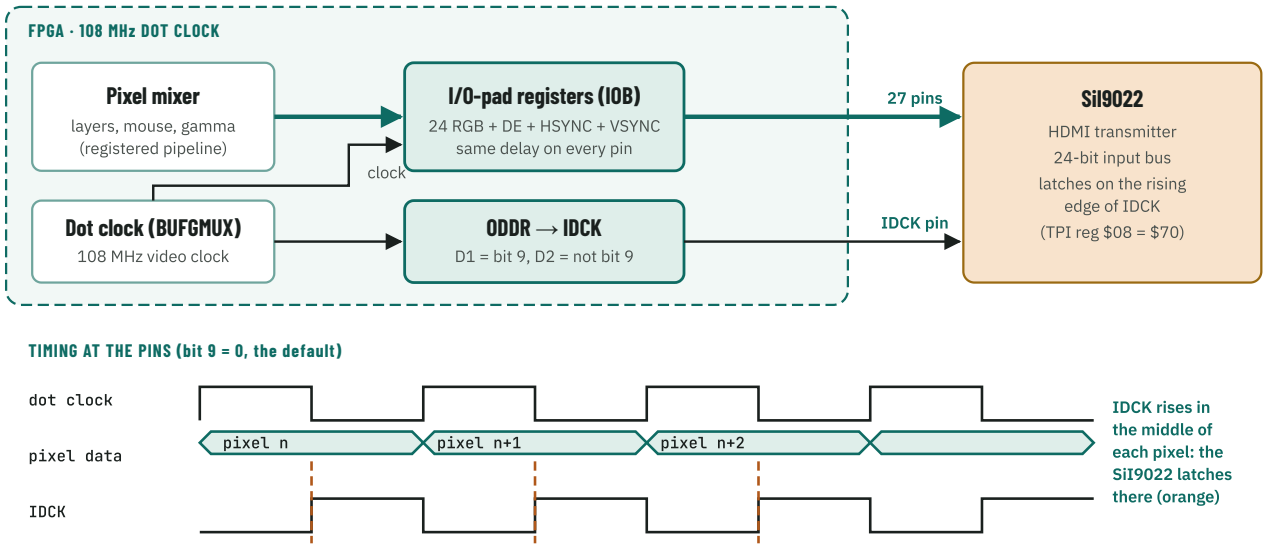


Figure 2. HDMI output path (build 28). The data and the pixel clock leave the FPGA from the I/O pads with fixed timing; GAVIN register 0 bit 9 turns IDCK over.

- HiRes DIP switch: the master control register shows a HiRes switch in bit 30, and MCP reads it at boot to choose 1280 x 1024. In build 87 this input is tied to 0 inside the FPGA, so bit 30 always reads 0 and the system starts in 1280 x 960. Software selects the mode with bit 8.
- Monitor sleep: master control bit 12 = 1 forces HSYNC, VSYNC and DE inactive.

> The horizontal sync of the RTL is 111 dot clocks wide (counter values 95..205 for 960 lines, 47..157 for 1024 lines); the comments and the transmitter set-up assume 112. Monitors accept this.

Video modes (from the timing generator)

PARAMETER	1280 X 960 (BIT 8 = 0)	1280 X 1024 (BIT 8 = 1)
Dot clock	108 MHz	108 MHz
Total pixels per line	1800	1688
Visible pixels	1280	1280
H front porch / sync / back porch	96 / 112 / 312	48 / 112 / 248
Horizontal blanking (first pixel shown)	520 (pixel 520)	408 (pixel 408)
Line frequency	60.0 kHz	63.98 kHz
Total lines per frame	1000	1066
Visible lines	960	1024

PARAMETER	1280 X 960 (BIT 8 = 0)	1280 X 1024 (BIT 8 = 1)
V front porch / sync / back porch (lines)	1 / 3 / 36	1 / 3 / 38
Frame rate	60.00 Hz	60.02 Hz
Sync polarity	H+ V+	H+ V+
Block text / MemText 8x16	160 x 60 characters	160 x 64 characters

Sil9022 HDMI transmitter set-up `$FEC0_0E00`

The SiI9022 HDMI transmitter is configured by hardware: the I2C master in the FPGA (GAVIN, `$FEC0_0E00`, new in build 84; the SiI9022 is at 7-bit address `$39`, `$72` in 8-bit form) programs it after reset and again whenever master control bits [9:8], GGE_ENABLE or the GGE MODE_400 bit change. With GGE_ENABLE set it uses the 640 x 480 or 640 x 400 table (see the GGE chapter). Software does not need to touch it.

After the FPGA starts, it holds the transmitter in reset (HDMI_RSTn low) for about 200 ms (a count derived from the bus clock: 13 333 400 cycles at 66.67 MHz), and keeps its own system reset active until the transmitter reset is released. Then the I2C master writes (at 100 kHz, the same writes as before build 84): TPI register `$82` = `$25` (twice), `$7C` = `$14`, `$C7` = `$00` (enter TPI mode), followed by 24 register writes from the table of the current mode.

The set-up has priority over CPU transfers on the I2C bus. Software can run it again (for the current video mode) by writing 1 to bit 1 of I2C_HDMI (`$FEC0_0E08`); reading I2C_HDMI shows whether a set-up is running, whether the last one had an error, and how many have run. MCP 171: I2C HDMI. The registers of the I2C master (`$FEC0_0E00` - `$FEC0_0EFF`, any access size) and the other devices on the bus are described in the GAVIN chapter (I2C master).

> Build 24 replaced the 1280 x 1024 table (in build 23 it was a copy of the 640 x 400 one). Both 1280 x 960 and 1280 x 1024 are verified on hardware, pixel for pixel, with the GUI test pattern. TPI register `$08` = `$70` makes the SiI9022 latch its input bus on the rising edge of IDCK; see the HDMI output paragraph of the video timing section.

Mode table written for 1280 x 960 (in this order)

TPI REG	VALUE	MEANING (FROM THE RTL COMMENTS)
`$00` / `$01`	`$30` / `$2A`	Pixel clock / 10 kHz = `$2A30` = 10 800 (108 MHz)
`$1A`	`$11`	TMDS output off, DVI mode, during set-up
`$1E`	`$00`	Power state D0
`$02`	`$3C`	Vertical frequency 60 Hz (low byte)
`$04` / `$05`	`$08` / `$07`	Total pixels per line `$0708` = 1800
`$06` / `$07`	`$E8` / `$03`	Total lines `$03E8` = 1000
`$08`	`$70`	Input bus format: full bus
`$09` / `$0A`	`$00` / `$00`	Input and output format: RGB
`$60` / `$61`	`$00` / `$00`	Sync generation control / polarity (read only)
`$62` / `$63`	`$A8` / `$41`	DE generator on, DE delay `$1A8` = 424 (sync + back porch)
`$64`	`$27`	DE top = 39 lines (sync + back porch)
`$66` / `$67`	`$00` / `$05`	DE width 1280
`$68` / `$69`	`$C0` / `$03`	DE height 960
`$1A`	`$00`	TMDS output on, DVI mode
`$3C`	`$00`	Transmitter interrupts off
`$1A`	`$00`	TMDS output on (repeated)

Mode table written for 1280 x 1024 (build 24; the entries that differ from 1280 x 960)

TPI REG	VALUE	MEANING
$00 / $01	$30 / $2A	Pixel clock 108 MHz (unchanged)
$04 / $05	$98 / $06	Total pixels per line $0698 = 1688
$06 / $07	$2A / $04	Total lines $042A = 1066
$62 / $63	$68 / $41	DE generator on, DE delay $168 = 360
$64	$29	DE top = 41 lines
$68 / $69	$00 / $04	DE height 1024

VICKY II master control register $FEC8_0000

One 32-bit register turns the layers on and off and selects the video mode. Bits 31..29 are read-only status; bits 28..0 read back what was written.

The game graphics are the GGE (its own chapter), which is switched on by GGE_CTRL at $FD00_0000 only; no master control bit affects the dot clock, the HDMI path or the SiI9022 mode. Bit 2 (graphics) only selects the GUI bitmap layer in the mixer, and the old game-engine bits 3, 4, 5, 7, 11 and 16 have no effect. The GUI engine itself is switched on in its own CTRL register at $FC00_0000; master bit 3 is not needed.

> Bits 0, 1, 6, 9:8, 10, 12, 13, 14, 17 and the border/cursor registers cross into the 108 MHz video clock through synchronisers: a change shows a few clock cycles later. Since build 25 byte and word writes change only the bytes written (see the access-size note).

$00 MSTR_CTRL RW · reset $0000_0001

Layer enables, video mode and status. Reads `$8000_0001` after reset (plus bit 29 when the gamma DIP switch is on). Write it only with 32-bit accesses.



BITS	FIELD	ACCESS	MEANING
31	**ID_ONE**	R	Always 1. (MCP calls it CLK40; it does not indicate a 40 MHz clock on this board.)
30	**HIRES_DIP**	R	HiRes DIP switch. Tied to 0 in build 87: always reads 0.
29	**GAMMA_DIP**	R	State of the gamma DIP switch input (1 = on).
28:23	**-**	RW	Stored, no function.
22:21	**TOS_MODE**	RW	No effect in this build. The TOSGRAPH depth is set in the TOSGRAPH CTRL register (`$FEC9_7000` bits 3:2).
20	**TOSGRAPH**	RW	1 = TOSGRAPH (EmuTOS bitmap) layer on. Only active while bit 14 = 0; setting both bits turns both off.
19:18	**-**	RW	Stored, no function. (MCP's header calls bit 18 SLEEP; the real sleep bit is 12.)
17	**GAMMA_SRC**	RW	Gamma control source: 0 = the gamma DIP switch turns gamma correction on/off, 1 = bit 6 does.
16	**GAME_GUI**	RW	0 = GUI, 1 = Game. No effect (the GGE is switched on with GGE_CTRL at `$FD00_0000`).
15	**MEMTEXT_SHOW_BG**	RW	No effect in this build (the mixer uses bit 13 for MemText too).
14	**MEMTEXT**	RW	1 = MemText layer on (replaces the block text in the mixer). Only active while bit 20 = 0. Bit 0 must also be 1 to show MemText, and MemText CTRL bit 0 must be 1.
13	**OVERLAY_SHOW_BG**	RW	With overlay (bit 1): 0 = only text foreground pixels are drawn over the graphics; 1 = text foreground and background colours are drawn, except where the cell's background colour index is 0 (there the graphics show through).
12	**SYNC_OFF**	RW	1 = HSYNC, VSYNC and DE forced inactive (monitor goes to sleep). 0 = normal video.
11	**PIXDIV_GGE**	RW	Pixel division for the old game engine. No effect.
10	**PIXDIV_GUI**	RW	1 = line doubling in the block text and mouse pointer logic. It does not affect the GUI engine (which has its own 640 modes), MemText or TOSGRAPH. Not hardware-verified; keep 0.
9:8	**VIDEO_MODE**	RW	Bit 8: 0 = 1280 x 960 at 60 Hz, 1 = 1280 x 1024 at 60 Hz. Bit 9 has no effect on the timing; keep it 0. Any change of bits 9:8 makes the hardware re-program the SiI9022. The timing changes at the next start of frame.
7	**DISABLE_VIDEO**	RW	Old game-engine fetch disable. No effect.
6	**GAMMA**	RW	1 = gamma correction on (used only when bit 17 = 1).
5	**SPRITE**	RW	Old game-engine sprites. No effect (see the GGE chapter).
4	**TILEMAP**	RW	Old game-engine tile maps. No effect (see the GGE chapter).
3	**BITMAP**	RW	Old game-engine bitmaps. No effect; not needed for the GUI engine.
2	**GRAPHICS**	RW	1 = show the graphics layer (GUI bitmap engine) in the mixer. It only switches the mixer input.
1	**OVERLAY**	RW	1 = text (block text or MemText) is drawn over the graphics layer. With bit 2 = 1 and bit 1 = 0, the text is hidden and only the graphics are shown.

BITS	FIELD	ACCESS	MEANING
0	**TEXT**	RW	1 = block text on. Also needed to show MemText, and to overlay block text on TOSGRAPH. Reset: 1.

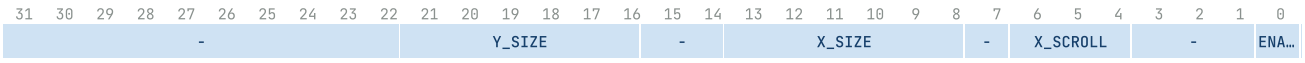
VICKY II control registers $04-$3C $FEC8_0000

The other control registers follow the master control register. All are 32-bit; since build 25 they also accept byte and word writes, and repeat every 64 bytes up to `$FEC8_00FF`. Border size, scroll, cursor and text registers are taken by the video logic at the start of each frame.

Several registers of the original VICKY II layout are still present as plain storage but drive nothing in this build: background colour, line interrupts, font size/count, the text start offset and the cursor colour.

$04 BORDER_CTRL RW · reset $0010_1001

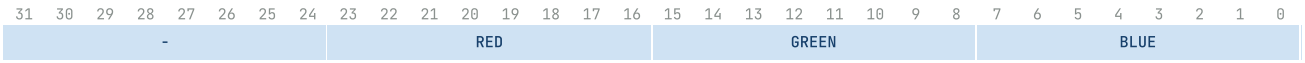
Border on/off, size and fine scroll. The border is drawn in the border colour around the block text area. It is never drawn while MemText is on.



BITS	FIELD	ACCESS	MEANING
31:22	**-**	RW	Stored, no function.
21:16	**Y_SIZE**	RW	Border height at top and bottom, in lines (0-63). Reset 16.
15:14	**-**	RW	Stored, no function.
13:8	**X_SIZE**	RW	Border width at left and right, in pixels (0-63). Reset 16.
7	**-**	RW	Stored, no function.
6:4	**X_SCROLL**	RW	Moves the start of the text fetch by 0-7 pixels (horizontal fine scroll of the block text). Not hardware-verified.
3:1	**-**	RW	Stored, no function.
0	**ENABLE**	RW	1 = border on. With the border on, the text window starts inside the border; characters under the border are hidden (rows stay 160 characters long). Reset: 1.

$08 BORDER_COLOR RW · reset $0020_3060

Border colour.



BITS	FIELD	ACCESS	MEANING
31:24	**-**	RW	Stored, no function.
23:16	**RED**	RW	Red, 0-255.
15:8	**GREEN**	RW	Green, 0-255.
7:0	**BLUE**	RW	Blue, 0-255.

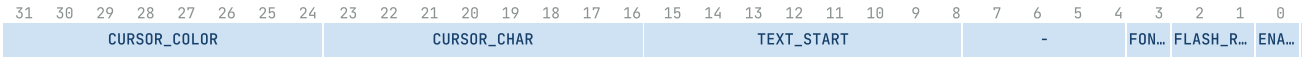
$0C BACKGROUND_COLOR RW · reset $0000_0000

Storage only. There is no background-colour layer in this build: with no layer on, the screen is black inside the border.

$10 CURSOR_CTRL

RW · reset $0000_0000

Block text cursor. When the cursor is visible, the character at the cursor position is replaced by the cursor character, drawn in that cell's colours.

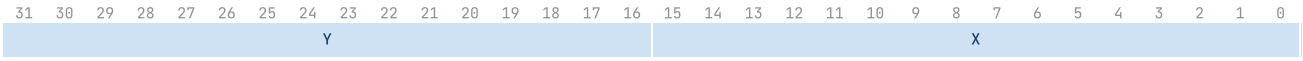


BITS	FIELD	ACCESS	MEANING
31:24	**CURSOR_COLOR**	RW	Stored. Not used by the hardware in this build (the cursor uses the cell colours).
23:16	**CURSOR_CHAR**	RW	Character code shown as the cursor.
15:8	**TEXT_START**	RW	Text start offset (scroll). Not used by the hardware in this build.
7:4	**-**	RW	Stored, no function.
3	**FONT_BANK**	RW	Build 45+: block text font bank shown. 0 = bank 0 (`$FEC8_8000`, reset), 1 = bank 1 (`$FEC8_9000`). Before build 45: stored, no function.
2:1	**FLASH_RATE**	RW	Flash rate, frames per on/off phase: 00 = 60 (1 s), 01 = 30 (0.5 s), 10 = 15 (0.25 s), 11 = 12 (0.2 s).
0	**ENABLE**	RW	1 = cursor on.

$14 CURSOR_POS

RW · reset $0000_0000

Cursor position in character cells.



BITS	FIELD	ACCESS	MEANING
31:16	**Y**	RW	Row. Only bits 23:16 (8 bits) are compared.
15:0	**X**	RW	Column (0-159). Only bits 7:0 are compared.

$18 LINE_INT_01

RW · reset $0000_0000

Line interrupt compare values 0 and 1 (bits 15:0 and 31:16). Storage only: VICKY II has no line interrupt. The line interrupt of build 61 and later is the GGE's LINE_IRQ (`$FD00_001C`, interrupt source `$01`).

$1C LINE_INT_23

RW · reset $0000_0000

Line interrupt compare values 2 and 3. Storage only.

$20 FONT_SIZE

RW · reset $1008_1008

Font container and font size of the original design. Storage only: the block text font is fixed at 8 x 16.

$24 FONT_COUNT

RW · reset $0000_1E50

Characters per line / lines of the original design. Storage only.

$28 SPARE0

RW · reset $0000_0000

Storage only (`$28`, `$2C`, `$30`, `$34` behave the same).

$38 VERSION R · reset $0000_0001

Chip version (bits 31:16) and sub-version (bits 15:0). A write is stored internally but cannot be read back.

$3C CHIP_ID R · reset $0009_7335

Chip number. A write is stored internally but cannot be read back.

Block text mode $FECA_0000

The block text mode shows a 160-column screen of 8 x 16 characters held in FPGA block RAM: 60 rows in 1280 x 960, 64 rows in 1280 x 1024. It is on at reset (master bit 0) and is what MCP uses for its console.

Each cell has one byte in the character memory and one byte in the colour memory, at the same offset: offset = row x 160 + column. Rows are 160 bytes long whatever the border size; 160 x 64 = 10 240 bytes of each 16 KB memory are used.

The colour byte holds two 4-bit indexes into two 16-entry colour tables: bits 7:4 select the foreground colour, bits 3:0 the background colour. A background index of 0 is also the 'transparent' marker for the overlay mode (master bit 13).

- Glyphs: 8 x 16, one byte per pixel line, bit 7 = leftmost pixel. Glyph line L of character C is at `$FEC8_8000` + C x 16 + L (bank 0) or `$FEC8_9000` + C x 16 + L (bank 1).
- The font RAM (8 KB) is preloaded with the FPGA image 'A2560M_2x 8x8 1x 8x16 Sets': the 8 x 16 font in bank 0 (`$FEC8_8000`) and two 8 x 8 fonts in bank 1 (`$FEC8_9000`; shown as 8 x 16 they look scrambled). Writing a glyph changes the screen at once. Up to build 44 the font RAM accepted byte writes only: word and long writes (a memcpy) were ignored and reads returned the addressed byte in all four lanes. Since build 45 any access size works (MCP 133 TEST FONT checks it).
- With master bit 2 (graphics) on, the text is shown only with bit 1 (overlay); see the mixer section.
- Cursor: see CURSOR_CTRL (`$FEC8_0010`) and CURSOR_POS (`$FEC8_0014`). The cursor and the border settings are taken at the start of each frame.

Block text memories

ADDRESS	CONTENTS	ACCESS
$FECA_0000-$FECA_3FFF	Character codes, one byte per cell	Byte writes only; reads return the byte in all 4 lanes
$FECA_8000-$FECA_BFFF	Colour bytes: [7:4] foreground index, [3:0] background index	Byte writes only; reads as above
$FECA_C400-$FECA_C43F	Foreground LUT: 16 x `$00` RRGGBB (bits 31:24 ignored)	Long writes only; not readable
$FECA_C440-$FECA_C47F	Background LUT: 16 x `$00` RRGGBB	Long writes only; not readable
$FEC8_8000-$FEC8_9FFF	Font RAM (8 KB) = two 8 x 16 banks: `$FEC8_8000` - `$8FFF` = bank 0 (shown after reset), `$FEC8_9000` - `$9FFF` = bank 1 (preloaded with two 8 x 8 fonts); CURSOR_CTRL bit 3 selects the bank shown	Byte, word and long writes and reads (build 45+; before: byte writes only)

Default LUT contents (both LUTs, loaded with the FPGA)

INDEX	COLOUR	INDEX	COLOUR
0	`$000000` black	8	`$FF4500`
1	`$800000`	9	`$8B4513`
2	`$008000`	10	`$200000`
3	`$000080`	11	`$002000`
4	`$808000`	12	`$000020`

INDEX	COLOUR	INDEX	COLOUR
5	`$008080`	13	`$202020`
6	`$800080`	14	`$0080FF`
7	`$808080`	15	`$FFFFFF` white

Mouse pointer `$FEC8_0C00`

A 16 x 16 hardware pointer with two images is drawn on top of every other layer. Its position is updated by the hardware from PS/2 mouse packets that software copies into three registers; there is no register to set the position directly in this build.

Pointer images: image 0 at `$FEC8_0400` - `$FEC8_07FF`, image 1 at `$FEC8_0800` - `$FEC8_0BFF`. Each is 16 rows x 16 pixels x 32 bits, row by row, 64 bytes per row, pixel (0,0) first. A pixel value of `$0000_0000` is transparent; any other value shows its RGB part (bits 23:0) - bits 31:24 only make the value non-zero. Write the images with 32-bit accesses; they cannot be read back.

The pointer position is the screen position of the image's top-left pixel. It is clamped to 0..1279 and 0..959 (or 0..1023); near the right and bottom edges the image is cut.

- Read-back of the packet registers is swapped relative to the write offsets: longword `$08` returns {byte 0 register, `$0000`}, longword `$0C` returns {byte 2 register, byte 1 register}.
- With master bit 10 (PIXDIV_GUI) = 1 the pointer is drawn with doubled lines and an X limit of 640.

`$00` MOUSE_CTRL

`RW · reset $0000`

Pointer enable and image select. 16-bit register, written with 16-bit accesses only.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-														IMAGE	ENABLE

BITS	FIELD	ACCESS	MEANING
15:2	-	RW	Stored, no function.
1	**IMAGE**	RW	0 = image 0 (`$FEC8_0400`), 1 = image 1 (`$FEC8_0800`). Taken at the next vertical sync.
0	**ENABLE**	RW	1 = pointer shown.

`$02` MOUSE_X_SET

`W · reset $0000`

Stored, no effect in this build (MCP names it MousePtr_A_X_Pos). Reading `$02` as a word returns MOUSE_CTRL.

`$04` MOUSE_POS

`R · reset $0000_0000`

Current pointer position, read as a longword: bits 31:16 = Y, bits 15:0 = X. As words: `$04` = Y, `$06` = X. A word write to `$04` is stored but has no effect.

`$0A` PS2_BYTE0

`W · reset $0000`

First byte of a PS/2 mouse packet in bits 7:0 (16-bit write). Bit 4 = X sign, bit 5 = Y sign.

`$0C` PS2_BYTE1

`W · reset $0000`

Second packet byte: X movement (bits 7:0, two's complement with the sign from byte 0).

`$0E` PS2_BYTE2

`W · reset $0000`

Third packet byte: Y movement (bits 7:0). Writing `$0E` after `$0A` and `$0C` (in this order) applies the movement: X += dx, Y -= dy (PS/2 Y is positive upwards).

Gamma correction `$FEC8_4000`

The last stage of the mixer passes each colour channel of the final pixel (after the mouse pointer) through a 256-entry table: output = table[input]. There is one table per channel.

Gamma correction is on when master bit 17 = 1 and bit 6 = 1, or, with bit 17 = 0, when the gamma DIP switch is on (its state reads in master bit 29). MCP sets bit 17 at boot.

- The tables are preloaded with the FPGA image with a brightening curve (`$00` , `$0B` , `$11` , `$15` , `$19` , ...). Write a straight ramp (table[i] = i) to get no correction while gamma is enabled.
- Up to build 54 a CPU write to the gamma tables had no effect: the write was made after the CPU cycle had ended. Builds 57 / 58 take the write from the VICKY II bus pipeline like the other VICKY II memories. MCP 143+ TEST LUTRB writes and reads back all three tables and the GUI palette, and restores them.

Gamma tables

ADDRESS	TABLE	ACCESS
`$FEC8_4000-$FEC8_40FF`	Blue	Byte writes only (since build 55 / 57-58: before, CPU writes were lost); byte reads (value repeated in all lanes)
`$FEC8_4100-$FEC8_41FF`	Green	same
`$FEC8_4200-$FEC8_42FF`	Red	same

Pixel mixer: layers and priority

The mixer combines the layers for every pixel at 108 MHz. From top to bottom the order is: mouse pointer, border, text (block text or MemText), graphics (GUI bitmap engine) or TOSGRAPH. Gamma correction is applied to the result.

Only one 'text' source and one 'picture' source are shown at a time. TOSGRAPH (master bit 20) and MemText (bit 14) exclude each other; with TOSGRAPH on, the GUI layer and MemText are not shown.

Overlay rules (master bit 1 = 1): with bit 13 = 0, text foreground pixels are drawn and background pixels show the picture below. With bit 13 = 1, both text colours are drawn except in cells whose background colour index is 0 (block text: colour bits 3:0 = 0; MemText: the low 4 bits of the background index = 0), where the picture shows through the background pixels.

- Border: when BORDER_CTRL bit 0 = 1 and MemText is off, the border area is drawn in BORDER_COLOR over everything except the mouse pointer.
- Mouse pointer: any non-zero pointer pixel replaces the mixed pixel.
- With master bit 2 = 1 but the GUI engine off (its CTRL bit 0 = 0), the graphics layer is black.
- Example settings: MCP console = `$0000_0001` (block text). GUI only = bit 2 + GUI CTRL bit 0. GUI with text console on top = bits 0, 1, 2 (`$0000_0007`). MemText alone = `$0000_4001` (MCP uses `$0000_4003`).
- While GGE_ENABLE (`$FD00_0000` bit 0) is set, the output of this mixer is not shown: the HDMI output carries the GGE picture.

What the mixer shows (inside the border)

MASTER BITS	BIT 2 (GRAPHICS)	BIT 0 (TEXT)	RESULT
`bit 14 = 0, bit 20 = 0`	0	0	Black
`bit 14 = 0, bit 20 = 0`	0	1	Block text (foreground and background colours)
`bit 14 = 0, bit 20 = 0`	1	0	GUI bitmap layer
`bit 14 = 0, bit 20 = 0`	1	1	Bit 1 = 0: GUI layer only. Bit 1 = 1: block text over the GUI layer (overlay rules)
`MemText (bit 14 = 1, bit 20 = 0)`	0	0	Border colour over the whole screen
`MemText (bit 14 = 1, bit 20 = 0)`	0	1	MemText (foreground, background and cursor colours)
`MemText (bit 14 = 1, bit 20 = 0)`	1	0	GUI bitmap layer

MASTER BITS	BIT 2 (GRAPHICS)	BIT 0 (TEXT)	RESULT
`MemText (bit 14 = 1, bit 20 = 0)`	1	1	Bit 1 = 0: GUI layer only. Bit 1 = 1: MemText over the GUI layer (overlay rules)
`TOSGRAPH (bit 20 = 1, bit 14 = 0)`	any	0	TOSGRAPH bitmap
`TOSGRAPH (bit 20 = 1, bit 14 = 0)`	any	1	Bit 1 = 0: TOSGRAPH only. Bit 1 = 1: block text over TOSGRAPH (overlay rules)

MemText: memory-backed text mode `$FEC9_8000`

MemText is a larger text mode whose character cells are in the 8 MB SRAM instead of FPGA block RAM. It shows 160 columns of 8 x 16 characters (60 rows at 1280 x 960, 64 rows at 1280 x 1024) or 8 x 8 characters (120 / 128 rows), with 256-colour foreground and background tables, two font banks (8 x 16) or four (8 x 8), reverse video, blinking and a programmable cursor shape. It uses the whole screen: no border is drawn.

Turn it on with master bits 14 = 1, 20 = 0 and 0 = 1, plus MemText CTRL bit 0 = 1. For 8 x 16 characters set both CTRL bit 1 (16-line rows) and bit 14 (8 x 16 glyph addressing), for example CTRL = `$0000_4003`.

During the two lines before each character row is displayed, the MemText fetcher becomes a bus master (BR/BG), reads the row's 160 cells (640 bytes) from SRAM and stores them in a double buffer. The CPU is held off the bus while this happens. SRAM stream rate: one longword per bus clock since build 47 (the data are taken on the falling clock edge, 22.5 ns after the address), two bus clocks per longword before. Up to build 46 the row was read as 320 bytes of characters and 320 bytes of colours from two arrays.

- Example (MCP 135+ TEST MEMTEXTROWS): cell = `$FF30_0241` = foreground 255 on background `$30`, attribute `$02` (8 x 16 font bank 1 at `$1000`), character 'A'; MOVE.L # `$FF300241`,(A0). On the 68060 (big-endian) the character is the last byte of the cell.
- Write mask example: MT_WR_MASK = %1110 and MOVE.L writes over a row change only the characters; %0111 changes only the foreground colours. Set the mask back to 0 afterwards. MCP 135+ TEST MTMASK checks every mask value with long, word and byte writes.
- Software written for builds up to 46 (two arrays of 16-bit words) must be updated for the cell layout.
- The LUTs and the font are preloaded with the FPGA image: LUT entries 0-15 hold the same 16 colours as the block text LUTs, the rest are 0; the font holds two 8 x 8 sets (`$0000` - `$0FFF`) and one 8 x 16 set (`$1000` - `$1FFF`).
- The MemText registers repeat every 64 bytes in `$FEC9_8000` - `$FEC9_8FFF` and are written with 32-bit accesses only.
- Cursor pixels are part of the MemText colour output, but the overlay decision uses only the glyph: in overlay mode with master bit 13 = 0, cursor pixels that fall on glyph background pixels show the picture below.
- Builds up to 16 lost LUT writes (the LUT used the control-register chip select). Build 23 has the fix: write the LUTs at their own addresses.

> MemText font access: since build 45 the font RAM takes byte, word and long writes at any offset and reads return all four bytes (MCP 133 TEST FONT, hardware-checked). Up to build 44 only byte writes at offsets 2 and 3 of each longword were taken, and CPU reads of the MemText blocks did not return the stored data (the read-data port into VICKY II was declared the wrong way); build 45 fixes both.

MemText cells in SRAM (build 49 and later): one 32-bit longword per character, 160 per row, rows follow each other

CELL BITS	BYTE (68060)	CONTENTS
`31:24`	+0	Foreground index (0-255)
`23:16`	+1	Background index (0-255)
`15:8`	+2	Attribute byte (see below)
`7:0`	+3	Character code

Attribute byte

BITS	MEANING
`7:6`	Not used

BITS	MEANING
5	Blink: the glyph is inverted every BLINK_RATE frames
4	Reverse: glyph inverted
3	Background LUT bank: selects entries 0-255 or 256-511 of the background LUT
2	Foreground LUT bank: selects entries 0-255 or 256-511 of the foreground LUT
1:0	Font bank. 8 x 8: bits 1:0 select one of four 2 KB banks. 8 x 16: bit 1 selects one of two 4 KB banks, bit 0 is ignored

MemText LUTs and font

ADDRESS	CONTENTS	ACCESS
$FEC9_9000-$FEC9_97FF	Foreground LUT, 512 x `$00` RRGGBB; entry = attribute bit 2 x 256 + foreground index	32-bit writes only; reads always `$AAAA_5555`
$FEC9_9800-$FEC9_9FFF	Background LUT, 512 x `$00` RRGGBB; entry = attribute bit 3 x 256 + background index	32-bit writes only; reads always `$AAAA_5555`
$FEC9_A000-$FEC9_BFFF	Font, 8 KB. 8 x 8: byte = bank x `$800` + code x 8 + line. 8 x 16: byte = bank x `$1000` + code x 16 + line. Bit 7 = leftmost pixel	Byte, word and long writes and reads (build 45+); see note

$00 MT_CTRL

RW · reset $0000_0000

MemText enable, row height and cursor control.

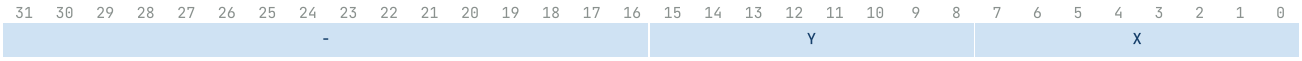


BITS	FIELD	ACCESS	MEANING
31:16	-	RW	Stored, no function.
15	-	RW	Stored, no function.
14	GLYPH_8X16	RW	Glyph addressing: 1 = 8 x 16 font, 0 = 8 x 8 font. Must match bit 1.
13:12	BLINK_RATE	RW	Blink rate for characters with attribute bit 5: frames per phase 00 = 60, 01 = 30, 10 = 15, 11 = 12.
11	CURSOR_SOLID	RW	1 = cursor does not flash.
10:9	CURSOR_RATE	RW	Cursor flash rate, frames per phase: 00 = 60, 01 = 30, 10 = 15, 11 = 12.
8	CURSOR_ENABLE	RW	1 = cursor on.
7:2	-	RW	Stored, no function.
1	ROW_16	RW	Row height: 1 = 16 lines (60 / 64 rows), 0 = 8 lines (120 / 128 rows).
0	ENABLE	RW	1 = MemText on (with master bit 14).

$04 MT_CURSOR_POS

RW · reset $0000_0000

Cursor position.



BITS	FIELD	ACCESS	MEANING
31:16	-	RW	Stored, no function.
15:8	Y	RW	Cursor row.
7:0	X	RW	Cursor column (0-159).

$08 MT_TEXT_ADDR

RW · reset $0001_0000

SRAM byte address of the cell array (row 0, column 0). Must be longword aligned and inside `$0000_0000` - `$007F_FFFF`. Bits 22:16 also select the 64 KB SRAM block where the cell write mask (MT_WR_MASK) applies: keep the whole array (160 x 64 cells = 40 KB) inside one 64 KB block.

$0C MT_COLOR_ADDR

RW · reset $0002_0000

Not used since build 49 (the cells hold the colours). Up to build 46: SRAM byte address of the colour buffer.

$10 MT_CURSOR_COLOR

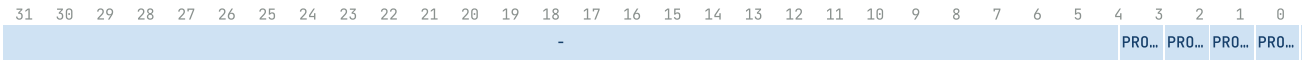
RW · reset $0000_2020

Cursor colour: bits 23:16 red, 15:8 green, 7:0 blue. Bits 31:24 ignored.

$14 MT_WR_MASK

RW · reset $0000_0000

Build 49+: cell write mask. CPU writes into the 64 KB SRAM block of MT_TEXT_ADDR keep the bytes whose bit is 1, so one field of many cells can be changed without a read-modify-write. Reads, DMA writes and writes outside the block are not masked. Reset 0. (`$18` , `$1C` , `$30` - `$3C` : storage only.)



BITS	FIELD	ACCESS	MEANING
31:4	-	RW	Stored, no function.
3	PROTECT_FG	RW	1 = keep the foreground index (cell bits 31:24, byte +0).
2	PROTECT_BG	RW	1 = keep the background index (cell bits 23:16, byte +1).
1	PROTECT_ATTR	RW	1 = keep the attributes (cell bits 15:8, byte +2).
0	PROTECT_CHAR	RW	1 = keep the character (cell bits 7:0, byte +3).

$20 MT_CURSOR_SHAPE0

RW · reset $55AA_55AA

Cursor shape lines 0-3: bits 31:24 = line 0 ... bits 7:0 = line 3. Bit 7 of each byte = leftmost pixel. A 1 pixel is drawn in the cursor colour over the character.

$24 MT_CURSOR_SHAPE1

RW · reset $55AA_55AA

Cursor shape lines 4-7.

$28 MT_CURSOR_SHAPE2

RW · reset $0000_0000

Cursor shape lines 8-11 (8 x 16 only).

$2C **MT_CURSOR_SHAPE3** RW · reset $0000_0000

Cursor shape lines 12-15 (8 x 16 only).

TOSGRAPH: EmuTOS bitmap $FEC9_7000

TOSGRAPH is a simple full-screen bitmap for EmuTOS, stored in the 8 MB SRAM in the Atari ST interleaved-bitplane format with 1, 2 or 4 bits per pixel and a 16-entry palette. It shows 1280 pixels per line, or 640 with pixel doubling.

Turn it on with master bit 20 = 1 (bit 14 = 0) and TOSGRAPH CTRL bit 0 = 1. While TOSGRAPH is on, the GUI layer and MemText are not shown; block text can be overlaid (master bits 0 and 1).

The TOSGRAPH fetcher works one line ahead of the screen (build 43+). In the horizontal blanking of screen line N it becomes a bus master (BR/BG) and reads line N+1 from SRAM (the same SRAM stream rate as MemText: one longword per bus clock) into the free half of a two-line raw memory; line 0 is fetched in the last blanking line before the picture. The fetch therefore has a whole line (about 15.6-16.7 us) instead of the blanking (about 4.8 us): a DMA burst holding the bus no longer shows the previous line on screen, and a fetch that is later than a full line is caught up. The video side turns the bitplanes into colour indexes and palette colours at 108 MHz on the fly, so every depth works at 1280 pixels, including 4 bpp at 1280 x 1024. The start address is reloaded between frames. Up to build 42 the line was fetched in its own blanking and converted by a 100 MHz pass that was too slow for 4 bpp at 1280 pixels.

- The registers are written with 32-bit accesses only. They repeat every 128 bytes in `$FEC9_7000` - `$FEC9_7FFF`.
- Master control bits 22:21 (named TOS mode) are not used; the depth is TOS_CTRL bits 3:2.

> TOSGRAPH registers cannot be read back reliably: a read returns the last value written to any offset with the same address bits 3:2 (for example, reading `$00` returns the last value written to `$00`, `$10`, `$20`, `$30` or `$40`). Keep a copy in RAM.

Palette reset values

ENTRY (OFFSET)	RESET	ENTRY (OFFSET)	RESET
0 ($10)	$FFFF_FFFF	8 ($30)	$FF40_4040
1 ($14)	$FF80_0000	9 ($34)	$FF80_4000
2 ($18)	$FF00_8000	10 ($38)	$FF20_8020
3 ($1C)	$FF80_8000	11 ($3C)	$FF80_8020
4 ($20)	$FF00_0080	12 ($40)	$FF40_2080
5 ($24)	$FF80_0080	13 ($44)	$FF80_2040
6 ($28)	$FF00_8080	14 ($48)	$FF20_8080
7 ($2C)	$FF80_8080	15 ($4C)	$FF00_0000

Line format (lines follow each other, no stride register)

DEPTH	BYTES PER LINE (1280 / 640 DOUBLED)	LAYOUT (16 PIXELS PER GROUP, BIT 15 OF EACH WORD = LEFTMOST PIXEL)
1 bpp	160 / 80	Plain bit stream: 1 = palette 1, 0 = palette 0
2 bpp	320 / 160	Word 0 = plane 0, word 1 = plane 1 for the same 16 pixels; colour = plane1 x 2 + plane0
4 bpp	640 / 320	Words 0-3 = planes 0-3 for the same 16 pixels; colour = plane3 x 8 + plane2 x 4 + plane1 x 2 + plane0

$00 TOS_CTRL W · reset $0000_0000

Enable, pixel doubling and depth. Depth and doubling are taken at the start of a frame.



BITS	FIELD	ACCESS	MEANING
31:4	-	W	No function.
3:2	**DEPTH**	W	00 = 1 bpp (2 colours), 01 = 2 bpp (4 colours), 10 or 11 = 4 bpp (16 colours).
1	**DOUBLE**	W	1 = 640 pixels per line, each pixel and each line shown twice (640 x 480 / 640 x 512).
0	**ENABLE**	W	1 = TOSGRAPH on (with master bit 20).

$04 TOS_ADDR W · reset $0000_0000

SRAM byte address of the first line (longword aligned, inside `$0000_0000` - `$007F_FFFF`). Taken during vertical blanking.

$08 TOS_SPARE W · reset $0000_0000

Storage only (`$08` and `$0C`).

$10 TOS_PAL0 W · reset $FFFF_FFFF

Palette entry 0, $xxRRGGBB (bits 31:24 ignored). Entries 1-15 follow at `$14` - `$4C`. 1 bpp uses entries 0-1, 2 bpp 0-3, 4 bpp 0-15.

GUI bitmap engine $FC00_0000

The GUI bitmap engine shows one full-screen bitmap stored in DDR3. It is the graphics layer of the mixer (master bit 2). It has 8 modes: 640 or 1280 pixels per line, each with 1, 8, 16 or 32 bits per pixel. The number of lines follows the video mode: 960 or 1024 in the 1280 modes, 480 or 512 in the 640 modes, where every pixel and every line is shown twice.

Pixels are stored packed ('chunky'), no bitplanes, in the same byte order the CPU writes (big-endian). Line N starts at ADDR + N x STRIDE.

The engine reads DDR3 on arbiter port 0 (highest priority), one line ahead of the display, into two line buffers. If a line fetch takes too long, it raises URGENT so that other DDR3 users are cut at their next packet. The worst case (1280 x 1024, 32 bpp) needs about 0.33 GB/s, about 10 % of the port. Since build 60 the GGE shares port 0: while GGE_ENABLE is set the GGE owns the port and the GUI engine gets no data (it is not shown then, and its underrun count may grow).

Tear-free: CTRL (enable and mode), ADDR, STRIDE, MONO and MONO_BG are taken once per frame, two lines before the first visible line. Software can draw into a second buffer and write ADDR to flip; the new buffer is shown from the next frame. The palette is not latched: palette writes take effect at once.

The engine is hardware-verified in all 8 modes at 1280 x 960 (MCP command TEST GUIPAT). The 1024-line modes are only simulated (see the HDMI note on 1280 x 1024).

- Turning it on: write ADDR, STRIDE (0), MONO / MONO_BG or the palette, then CTRL = 1 + mode x 2, then set master control bit 2 (and bits 0 + 1 to keep the text console on top). Master bits 3 and 16 are not needed. MCP TEST GUIPAT writes master bits 3:0 = `$F` and CTRL = 1 | mode << 1.
- DDR3 address space: ADDR is a DDR3 byte address, and DDR3 byte addresses equal CPU addresses in `$0000_0000` - `$3FFF_FFFF`. The CPU cannot reach DDR3 below `$0080_0000` (that range is the SRAM), so put frame buffers at `$0080_0000` or above. MCP uses `$3000_0000`.
- Byte order: GAVIN control bit 6 (endian inversion, off at reset since build 23) must be 0 while byte or word writes build the bitmap, otherwise 8-bit and 16-bit pixels land in swapped lanes.
- Caches: the L2 cache is write-through, so CPU writes reach DDR3. If the 68060 data cache runs in copyback mode, push the frame buffer (CPUSHL / CPUSHA) or map it write-through or cache-inhibited.

- The engine answers at `$FC00_0000` - `$FC00_0FFF` (CS_GUI, build 60). The registers repeat every 64 bytes in `$FC00_0000` - `$FC00_03FF`, and the 2 KB block (registers + palette) repeats once at `$FC00_0800`. Offsets `$18` - `$3C` read 0. `$FC00_1000` - `$FC0F_FFFF` ends with a bus error (up to build 59 the 2 KB block repeated through `$FC0F_FFFF`).
- Underrun: if a line is not in the buffer when it must be shown, that line is black and STATUS bits 31:16 count it; the next line is fetched normally.
- The engine runs on the 108 MHz video clock and the DDR3 clock only; it does not depend on the CPU bus clock. With GGE_ENABLE set it runs on but is not shown, and it gets no DDR3 data (port 0 belongs to the GGE).

Modes (CTRL bits 3:1)

CTRL[3:1]	PIXELS X LINES	DEPTH	BYTES PER LINE (NATURAL STRIDE)
000	640 x 480 / 512	1 bpp	80
001	640 x 480 / 512	8 bpp	640
010	640 x 480 / 512	16 bpp	1280
011	640 x 480 / 512	32 bpp	2560
100	1280 x 960 / 1024	1 bpp	160
101	1280 x 960 / 1024	8 bpp	1280
110	1280 x 960 / 1024	16 bpp	2560
111	1280 x 960 / 1024	32 bpp	5120

Pixel layout in memory (big-endian; the leftmost pixel is at the lowest address)

DEPTH	FIRST PIXEL OF A LONGWORD	FORMAT
32 bpp	the whole longword	`$AA` RRGGBB; alpha (bits 31:24) ignored
16 bpp	bits 31..16 (the word at the lower address)	RGB565: red = bits 15:11, green = 10:5, blue = 4:0; expanded to 8 bits by repeating the top bits
8 bpp	bits 31..24 (the byte at the lowest address)	Index into the palette
1 bpp	bit 31 (bit 7 of the byte at the lowest address)	1 = MONO colour, 0 = MONO_BG colour

$00 CTRL

RW · reset $0000_0000

Enable and mode. Bits 31:4 are stored and read back but have no function.



BITS	FIELD	ACCESS	MEANING
31:4	-	RW	Stored, no function.
3	**WIDTH_1280**	RW	0 = 640 pixels per line (pixels and lines doubled), 1 = 1280 pixels per line.
2:1	**DEPTH**	RW	00 = 1 bpp (MONO / MONO_BG), 01 = 8 bpp (palette), 10 = 16 bpp (RGB565), 11 = 32 bpp (`$AA` RRGGBB).
0	**ENABLE**	RW	1 = engine on. 0 = the graphics layer is black and no DDR3 reads are made.

$04 ADDR RW · reset $0000_0000

DDR3 byte address of the top-left pixel. Bits 29:5 are used: the address must be 32-byte aligned (bits 4:0 are ignored) and bits 31:30 are ignored.



BITS	FIELD	ACCESS	MEANING
31:30	-	RW	Ignored.
29:5	ADDR	RW	Frame buffer address in DDR3, in 32-byte units.
4:0	-	RW	Ignored (treated as 0).

$08 MONO RW · reset $00FF_FFFF

Colour of a 1 pixel in 1 bpp mode, `$00`RRGGBB (bits 31:24 ignored). Reset: white.

$0C STRIDE RW · reset $0000_0000

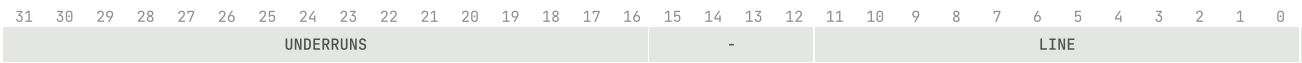
Bytes from the start of one line to the start of the next. Bits 12:0 are used (0-8191). 0 = natural width (pixels per line x bpp / 8). A larger value lets the bitmap be wider than the screen. Keep it a multiple of the pixel size (2 for 16 bpp, 4 for 32 bpp).

$10 MONO_BG RW · reset $0000_0000

Colour of a 0 pixel in 1 bpp mode, `$00`RRGGBB. Reset: black.

$14 STATUS R · reset $0000_0000

Engine status. Writes are ignored.



BITS	FIELD	ACCESS	MEANING
31:16	UNDERRUNS	R	Number of lines that were not fetched in time and were shown black (counts source lines; wraps at 65535; cleared only by reset). Expected to stay 0; a debug aid.
15:12	-	R	Read as 0.
11:0	LINE	R	Display line now being shown (0 = first visible line). Holds the last line number during vertical blanking.

$400 PALETTE RW · reset -

256 entries x 32 bits at `$FC00_0400` - `$FC00_07FF` (entry n at `$400` + 4n): `$00`RRGGBB colour of index n in 8 bpp mode. Bits 31:24 are stored but ignored. Write with 32-bit accesses. Not latched per frame.

4 Memory, DMA and the debug port

The A2560Me has three memories: 8 MB of fast SRAM and 4 MB of flash, both directly on the 68LC060 bus, and 1 GB of DDR3 behind the FPGA. The CPU reaches DDR3 through a 64 KB second-level (L2) cache in the FPGA, and every DDR3 client (the CPU, the GUI engine or the GGE, the VDMA, the MDMA, the GGE DAC player, DMA from PCI Express cards since build 74 and the debug port since build 80) shares the memory through a 7-port arbiter.

This chapter covers the memory map of the three memories, the L2 cache and its control register, the cache and coherency rules, the DDR3 port arbiter, the three DMA engines (SDMA: SRAM to SRAM; MDMA: SRAM <-> DDR3; VDMA: DDR3 block mover and blitter) the boot loader in flash block 0, and the serial debug port.

Everything here describes FPGA build 87. Clocks: the CPU bus (BCLK) and the SRAM-side logic (arbiter, SDMA, MDMA SRAM side) run at 66.67 MHz; the DDR3 user interface runs at 200 MHz.

Memory map

The FPGA decodes the 68060 address bus as shown below. An access to an address that no block decodes ends with a bus error (TEA). Holes inside a register block still answer normally.

DDR3 shares the bottom of the address space with the SRAM. The DDR3 byte address is CPU address bits A29..A0, so the first 8 MB of DDR3 (DDR3 bytes `$0000_0000` – `$007F_FFFF`) are hidden behind the SRAM and cannot be reached by the CPU. The CPU sees 1016 MB of DDR3.

> PCIe (build 74): the memory window `$4000_0000-$7FFF_FFFF` and the configuration window `$FA00_0000-$FA7F_FFFF` (mirrored up to `$FAFF_FFFF`) reach the PCI Express root port; see the PCI Express chapter. The FPGA asserts TCI on them, so the 68060 never caches them. Cards reach DDR3 by DMA at the CPU addresses `$0080_0000-$3FFF_FFFF`; the SRAM is out of their reach.

Memory windows

RANGE	SIZE	MEMORY	WIDTH	CACHED BY THE 68060	NOTES
`$0000_0000-$007F_FFFF`	8 MB	Fast SRAM	32 bit	Allowed	0 read / 1 write wait state; 4-beat line bursts. Holds the first 64 KB copied from the boot flash block at reset.
`$0080_0000-$3FFF_FFFF`	1016 MB	DDR3 (through the L2 cache)	32 bit to the CPU	Allowed	Line bursts. Wait states depend on L2 hit or miss.

RANGE	SIZE	MEMORY	WIDTH	CACHED BY THE 68060	NOTES
$FFC0_0000–$FFFF_FFFF	4 MB	Boot / system flash	32 bit	Allowed	Read-only for the CPU, 5 wait states, no bursts.
$FEC0_0000–$FECF_FFFF, $FC00_0000–$FC00_0FFF, $FD00_0000–$FD0F_FFFF	-	Register blocks (GAVIN, BEATRIX, VICKY II, GUI engine, GGE)	-	Never	The FPGA asserts TCI on these cycles, so the 68060 does not cache them. Data accesses only; an instruction fetch here is a bus error.

Fast SRAM $0000_0000

The fast SRAM is 8 MB made of two ISSI IS61WV204816BLL-10 parts (2M × 16, 10 ns access time). Together they form a 32-bit memory at $0000_0000 – $007F_FFFF. The SRAM data pins connect straight to the CPU data bus; the FPGA drives only the chip select, output enable, write enable, the four byte enables and the burst address bits A3/A2.

Single reads take 2 bus clocks (30 ns, no wait state) and single writes 3 (45 ns, 1 wait state); a line read is 2-2-2-2 (8 bus clocks, 120 ns) and a line write 3-3-3-3 (12 bus clocks, 180 ns). Builds 34-35 had 1 read wait state as well (line read 3-2-2-2). Line transfers (cache line fills, cache pushes and MOVE16) are real 4-beat bursts, and A3:A2 wrap around from the first longword as the 68060 expects.

Other bus masters also use the SRAM: the SDMA engine, the MemText and GemTOS display fetchers, and the debug port (with the CPU held in reset; since build 85 also as a bus master while the CPU runs or is paused). The bus arbiter takes the bus from the CPU between CPU bus cycles. Among the FPGA masters the priority is GemTOS, then MemText, then SDMA, then MDMA. After an FPGA master finishes, the CPU keeps the bus for at least 32 arbiter clocks (480 ns) if it wants it. The arbiter runs on the bus clock.

- Byte lanes are big-endian: the byte at address offset 0 is D31..D24.
- Line bursts to SRAM can be switched off with bit 3 of the memory-system control register ($FEC0_0038). Line transfers then end with TBI and the 68060 splits them into single cycles.
- At reset the FPGA copies the first 64 KB of the boot flash block (block 0 unless BOOT_SEL chose another, see Boot / system flash) to SRAM $0000_0000 – $0000_FFFF before it releases the CPU, so the CPU's reset vectors come from SRAM.
- Zero-wait reads (since build 36): the SRAM chip select is held low while the CPU owns the bus (output enable and write enable select the cycle), and CPU A3:A2 go straight to the SRAM instead of through a register. The routed A3:A2 path is 6.95 ns against a 7 ns bound; with the ISSI datasheet (tAA 10 ns, tDOE 6 ns) this leaves about 1 ns of address-access margin. Bus masters other than the CPU still use the registered path. Tested on the board: MCP TEST SRAM, 5 passes over 7.3 MB including MOVE16 line bursts, no errors.
- Memory test: MCP TEST SRAM [passes] (MCP 126+) checks $0001_0000 - $003C_FFFF and $0040_0000 - $0077_FFFF without touching MCP (address-in-address longwords and their complement, byte and word writes, MOVE16 line copies; a mismatch is read again to tell read errors from write errors). TEST MEM SYS writes over MCP's own data at $003D_0000 and the RAM kernel at $0078_0000, so it hangs while MCP runs from SRAM; it is not an SRAM fault.
- Top of SRAM: the areas in the table below are used by convention. Only the flash staging area is fixed in the FPGA. A program that uses the top 16.5 KB of SRAM for itself works, but then the monitor stub, the boot hand-over or a flash program frame may

overwrite its data (or it overwrites theirs).

SRAM areas used by convention

RANGE	SIZE	USED BY	NOTES
`$0000_0000-$0000_FFFF`	64 KB	Boot copy	Filled from the first 64 KB of the booted flash block at every reset (vector table at `$0000_0000`).
`$007F_BE00-$007F_BEFF`	256 B	Monitor stub mailbox (MCP 172+)	The MCP clears it at start-up and writes MAGIC 'DMON' (`$444D_4F4E`) at `$007F_BE00`. The stub saves the CPU registers here; the host reads and writes it through the debug port. See Debug port: pause, breakpoints and the monitor stop.
`$007F_BF00-$007F_BF63`	100 B	Boot hand-over record (boot loader)	Written by the boot loader just before the BOOT_SEL reboot; read once by the started program. See Boot loader and /sd2/boot.cfg.
`$007F_C000-$007F_FFFF`	16 KB	Debug port flash staging area	Fixed in the FPGA. Every flash program frame (command `$11`) first writes its payload here, so the old contents are lost.

Boot / system flash `$FFC0_0000`

The flash is a 4 MB, 32-bit wide memory (two Microchip SST39VF1601-70 16-bit parts, 70 ns) on the CPU bus, mapped at `$FFC0_0000` – `$FFFF_FFFF`. The CPU can read it in user or supervisor mode, as code or data.

Reads take 5 wait states (7 bus clocks, 105 ns): the wait is derived from an 80 ns rule. Line transfers to the flash are not bursted: the FPGA answers with TBI, and the 68060 completes the line as single longword reads.

The CPU cannot write to the flash. The flash write-enable pin is driven only by the debug port (commands `$11` program and `$12` erase, build 82+, with the CPU held; see the debug port sections).

Flash geometry as the CPU sees it (two 16-bit SST39VF1601 side by side): sector 8 KB, block 128 KB, 4 MB in total.

- Boot copy: after power-on and every board reset the FPGA copies the first 64 KB of the chosen flash block (block 0 = `$FFC0_0000`, unless BOOT_SEL chose another) into SRAM `$0000_0000` - `$0000_FFFF`, holding the CPU in reset until the copy is done. Since build 83 it reads each longword from the flash into an FPGA register and then writes it to the SRAM (the flash and the SRAM share the address bus, so a block other than 0 cannot be copied with one address); 64 KB take about 4.2 ms. The CPU then starts from the reset vectors in SRAM. Only these 64 KB are copied; the boot code must load anything else itself.
- The programmer uses the JEDEC unlock sequence (`$AA` to `$5555`, `$55` to `$2AAA`) on both 16-bit halves at the same time. The debug port erases an 8 KB sector, a 128 KB block or the whole chip.
- Multi-boot (build 83): BOOT_SEL (`$FEC0_0040`) divides the flash into 1 x 4 MB, 2 x 2 MB, 4 x 1 MB or 8 x 512 KB blocks and chooses the block for the next boot (see the table and the GAVIN control registers). An image stored in block N runs at that block's address and must be linked for it. If the chosen block is blank (first longword `$FFFF_FFFF`) or does not exist in the layout, the copy falls back to block 0; BOOT_INFO (`$FEC0_0044`) bit 6 shows it. The choice survives only the reboot it requested: power-on, the reset button (or the GAVIN soft reset), the debug port's system reset and the 68060 RESET instruction (an operating system reboot) all clear it, so the board boots block 0 again.
- MCP 166: BOOTSEL shows what booted (BOOT_INFO); BOOTSEL <block> <512K|1M|2M|4M> reboots into a block. MCP 166 no longer chooses the boot device with the DIP switches: by default it boots to the command line; F1 / F2 on the splash screen still boot from the SD card / the hard drive.
- Boot loader: block 0 holds a boot loader that reads /sd2/boot.cfg and boots the chosen block through BOOT_SEL; see the next section.
- Programming a block from a PC: the protocol v2 host tool a2560me_dbg.py --flash FILE --block N (about 6 s for 512 KB).

> The flash parts are Microchip SST39VF1601-70 (16-bit, 70 ns). The 4 MB size comes from the board description and the size of the address window.

Boot block layouts (BOOT_SEL bits 5-4)

LAYOUT	BLOCKS	BLOCK SIZE (BOOT_SIZE)	BLOCK ADDRESSES (BOOT_BASE)
0	1	4 MB ($0040_0000)	Block 0 = $FFC0_0000
1	2	2 MB ($0020_0000)	$FFC0_0000 , $FFE0_0000
2	4	1 MB ($0010_0000)	$FFC0_0000 , $FFD0_0000 , $FFE0_0000 , $FFF0_0000
3	8	512 KB ($0008_0000)	$FFC0_0000 + block x $0008_0000 ($FFC0_0000 - $FFF8_0000)

Boot loader and /sd2/boot.cfg $FFC0_0000

Since 2026-10-14 the boards ship with a boot loader in flash block 0 (the LVGL boot loader, lvboot.bin, 512 KB). Every cold boot starts it. It shows the entries of /sd2/boot.cfg (the internal micro SD card), counts down and starts the last choice: it writes the hand-over record into SRAM and then BOOT_SEL, and the FPGA reboots the board into that block. A key or a mouse movement stops the countdown. Boards without the loader run the MCP from block 0 (foenixmcp_gen2.bin); with the loader the MCP lives in block 1 at $FFC8_0000 (foenixmcp_gen2_b1.bin).

boot.cfg is plain text; # starts a comment. An entry is <number 1-9> = <flash address> <512K | 1M | 2M | 4M> "<name>". The address must be a multiple of the size and must not be block 0 ($FFC0_0000 , the loader itself). An entry may have options (<entry>.<1-9> = "<name>"), shown in a pop-up; they all start the same block and only change the option number in the hand-over record. The loader writes DEFAULT (and <entry>.DEFAULT) back when the choice changes.

If BOOT_INFO shows a fall-back ((BOOT_INFO & $C0) = $C0 : the chosen block was blank or does not exist), the loader says that the last choice could not be started and waits instead of counting down, so a missing image cannot cause a reboot loop. With no card or no file it lists the non-blank blocks and waits. A broken block 0 is rewritten from the PC through the debug port (a2560me_dbg.py --flash lvboot.bin --block 0).

- SRAM keeps its contents through the BOOT_SEL reboot (the boot copy rewrites only $0000_0000 - $0000_FFFF), so the started program finds the record. Use it only if the magic and the check both match, read it early, and clear the magic afterwards. Without a valid record (for example after MCP BOOTSEL), use option 1.
- The started program comes up on freshly reset hardware, as after power-on (caches and MMU off, the Super I/O held in reset until GAVIN_CTRL bit 8 is set). SRAM above $0001_0000 and DDR3 keep the loader's leftovers. BOOT_INFO bit 7 says the boot came from a BOOT_SEL choice; BOOT_BASE / BOOT_SIZE give the block.
- An image for block N is linked to run at that block's address; its first 8 bytes are the reset stack pointer and PC (the PC inside the block), and its first longword must not be $FFFF_FFFF (blank). Details, a C example for the record, and the MCP's own layout are in the developer notes "A2560Me boot selection" and "A2560Me boot loader: what EmuTOS and VioletBSD need".

Example /sd2/boot.cfg

LINE	MEANING
`TIMEOUT = 6`	Countdown in seconds (0 = wait for a key). Monitors need 2-4 s to show the picture.
`DEFAULT = 1`	The last choice; written back by the loader.
`1 = $FFC80000 512K "Foenix/MCP"`	Entry 1: block 1 of the 512 KB layout.
`2 = $FFD00000 512K "EmuTOS"`	Entry 2: block 2 of the 512 KB layout.
`3 = $FFE00000 2M "VioletBSD"`	Entry 3: a 2 MB image at $FFE0_0000 = block 1 of the 2 MB layout (it covers 512 KB blocks 4-7). The image must be linked for $FFE0_0000 .
`3.1 = "VioletBSD"`	Option 1 of entry 3 (the pop-up).
`3.2 = "VioletBSD (Single User)"`	Option 2 of entry 3.
`3.DEFAULT = 1`	The last option of entry 3; written back by the loader.

Boot hand-over record at `$007F_BF00` (big-endian)

OFFSET	SIZE	FIELD
`$00`	4	Magic `$4C56_4254` ("LVBT")
`$04`	4	Entry number, 1-9
`$08`	4	Option number, 1-9; 0 = the entry has no options
`$0C`	4	Flash address of the started block
`$10`	40	Entry name, NUL-terminated
`$38`	40	Option name, NUL-terminated (empty if no option)
`$60`	4	Check = magic XOR entry XOR option XOR flash address

DDR3 and the L2 cache `$0080_0000`

The DDR3 is 1 GB made of two MT41K256M16 parts, 32 bits wide, running as DDR3-800 (400 MHz memory clock). The Xilinx memory controller (MIG) runs in 2:1 mode with a 200 MHz user interface that is 128 bits wide. The smallest unit the controller moves is a 32-byte packet (one BL8 burst = one command and two 128-bit beats).

The CPU sees DDR3 at `$0080_0000` – `$3FFF_FFFF` through a 64 KB unified L2 cache in the FPGA. Every CPU access to that window goes through the L2; every other access type except CPU space (interrupt acknowledge) is decoded too, including cache pushes and MMU table walks.

The L2 is write-through: DDR3 always receives every CPU write, so DDR3 is always up to date except for writes still waiting in the L2's small posted write buffer. The buffer drains on its own within a few microseconds, and a read miss always waits for it to drain first, so the CPU never reads data older than its own writes.

- Byte lanes: big-endian, as on the SRAM, while GAVIN register 0 bit 6 (endian inversion, at `$FEC0_0000`) is 0. That is the reset value since build 23, and software should keep it.
- With endian inversion set, the L2 swaps lanes on CPU byte and word accesses to DDR3 only: a byte access is mirrored inside its longword (offset 0 <-> 3, 1 <-> 2) and a word access swaps the two halves of the longword. Longword and line accesses are not changed. Video engines read DDR3 in memory order, so this mode makes CPU data and engine data disagree.
- MOVE16 to and from DDR3 works: it is handled as a line burst. With DDR3 bursts switched off (bit 2 of `$FEC0_0038`) it falls back to the older single-cycle path.
- Builds 53 and later: the L2 looks up its tag and data RAMs with an address held in a flip-flop. Up to build 52 the lookup address came straight from the CPU address pins on every clock; while an FPGA bus master owned the bus those pins changed at any time, and a block RAM whose address changes inside its setup window can lose stored bits. That was the rare single-bit error in DDR3 data read through the L2 (DDR3 itself correct) seen with MDMA and the display fetch running. The gamma tables and the GUI palette had the same hazard and were fixed the same way. Timing seen by software is unchanged.
- Build 59: CPU write data and byte enables are taken into flip-flops first, and the L2 RAM write and the write-buffer push happen one clock later from those flip-flops. Up to build 58 a single write took them live from the CPU pins, and a burst write its 4th longword live on the last TA clock, straight into the block RAMs (0.58 ns of calculated margin on build 57). With the 68060 caches on, bit D3 of a 4th longword was sometimes written wrong into DDR3 and NetBSD stopped at boot (builds 57 / 58). TA timing is unchanged, and a following read always sees the written data. MCP TEST CACHE (MCP 145) runs the caches as NetBSD does and is part of the board regression.
- Build 74, line burst reads: during a cache-line burst read from DDR3 the L2 now moves its output to the next longword on the clock edge that ends each beat, right after the CPU has sampled it, so the next longword has two full bus clocks before the next TA. Up to build 73 the output moved on the same edge as the next TA: that left one clock for a path that needs two (the slowest data bits arrive after about 13 ns), while the timing constraints assumed two clocks, so Vivado reported the path as fine. TEST CACHE failed on build 73 (bit 26 or bit 28 set in the third longword of a 16-byte line read from DDR3); builds 59-72 passed only because their placement happened to be fast enough on those bits. With the 68060 caches on, rare unexplained crashes or corrupted data on builds 59-73 may have this cause.

L2 cache organisation

ITEM	VALUE
Size	64 KB, unified (code and data)
Organisation	2-way set associative, 1024 sets, true LRU
Line size	32 bytes = one DDR3 packet (one miss = one DDR3 read). A 68060 cache line (16 bytes) is half an L2 line.
Write policy	Write-through, no write-allocate. A write hit also updates the L2 line.
Write buffer	4 entries of 16 bytes with byte enables. A write gets TA right away unless the buffer is full.
After reset	All lines are invalidated by a sweep of 1 set per clock (1024 clocks, about 20 µs), long before the CPU leaves reset.
Flush	Bit 1 of `$FEC0_0038` invalidates every line (same 20 µs sweep). A DDR3 access that arrives during the sweep waits until it ends.
Range invalidate (build 26)	The MDMA and VDMA invalidate the sets they wrote (both ways), one set per idle L2 clock, CPU accesses first. A request names the first set and a count. The PCIe DMA path (build 74, sets written by a card) and the debug port (build 81, sets written by a DDR3 write frame) are further requesters. The four requesters (0 MDMA, 1 VDMA, 2 PCIe DMA, 3 debug port DDR3 writes) are served in turn (round robin).
Write buffer empty (build 26)	Signal to the DMAs: no CPU write is waiting for DDR3. They wait for it before they read DDR3 (the PCIe DMA path too, before a card reads DDR3, build 74; and the debug port before a DDR3 read, build 81).

Measured CPU timing (simulation, bus clocks from TS to TA at the cache, plus about 2.5 clocks in the TA logic)

ACCESS	BUS CLOCKS
Single read, L2 hit	7
Sequential reads from a cold cache (1 miss + 7 hits per 32-byte line)	8.4 on average
16-byte line burst read, L2 hit	13 (whole line, including TS)
16-byte line burst read, L2 miss	about 18.5 (whole line)
16-byte line burst write	12 (whole line)
Single write	posted: TA right away unless the write buffer is full

Memory-system control register (GAVIN register 14) `$FEC0_0038`

This GAVIN register controls the L2 cache and the line bursts, and holds local-bus settings: the LAN9221 read strobe length, the LAN9221 read filter, the local-bus clock output (since build 34), the local-bus clock phase (bit 14, build 38) the IDE write strobe and recovery (bits 23-16, build 39) and, read only, the board type (bit 15, build 40). All bits are 0 after reset, which is normal operation. The register was added to help find memory faults; only the flush bit (bit 1) is needed in normal software.

Like every GAVIN control register, it takes only 32-bit writes; byte and word writes are ignored. A write sets every bit, so to flush keep the other bits: read the register, mask it with `$0000_3F1D`, OR in `$0000_0002` and write it back. Bits 13-8 read back as written; writing 0 to them gives the normal behaviour.

> Flush the L2 after turning it back on (bit 0 from 1 to 0) if another master may have written DDR3 in the meantime: lines that were valid before the L2 was turned off are not invalidated by the off/on switch itself.

$38 MEMSYS_CTRL RW · reset $0000_0000

L2 cache and line-burst control, plus local-bus diagnostics (bits 13-8). Longword accesses only. Reads back with the signature `$4C` in bits 31:24 (a read after reset gives `$4C00_0000`).



BITS	FIELD	ACCESS	MEANING
31:24	**SIGNATURE**	R	Always reads `$4C`. Lets software check that the register exists.
23:20	**IDE_REC_EXTRA**	RW	Build 39+: IDE write recovery + 2n bus clocks: the write data is held, and DIOW- stays high, that much longer after each data-register write (default 100 ns). Diagnostic / margin; 0 after reset.
19:16	**IDE_WR_EXTRA**	RW	Build 39+: IDE write strobe (DIOW-) + 2n bus clocks (default 340 ns; 2 = 420 ns). Diagnostic / margin; 0 after reset.
15	**WHITE_BOARD**	R	Build 40+: 1 = the FPGA latched white-board mode from DIP switch 7 at the last reset (F14 = IDE DMACK- held high, D14 = IDE DMARQ not driven); 0 = black board.
14	**LBCLK_PHASE**	RW	0 = local-bus clock edges half a BCLK after the BCLK edges (normal), 1 = on the BCLK edges (the build 37 phase; about half of the 10M02 writes are lost). 0 after reset.
13	**LBCLK_OFF**	RW	1 = the local-bus clock output (LOCAL_BUS_CLK) is held low. Diagnostic; 0 after reset (since build 34).
12	**LAN_FILTER_OFF**	RW	1 = the LAN9221 read majority filter is off. Diagnostic; 0 after reset = filter on (since build 34).
11:8	**LAN_RD_EXTRA**	RW	LAN9221 read strobe + n bus clocks. Diagnostic; 0 after reset (since build 34).
7:5	**-**	R	Read as 0.
4	**SLOW_BEATS**	RW	1 = line bursts use 3 bus clocks per beat instead of 2. Debug only.
3	**SRAM_BURST_OFF**	RW	1 = no line bursts to SRAM; line transfers end with TBI and the CPU splits them into single cycles.
2	**DDR3_BURST_OFF**	RW	1 = no line bursts to DDR3; line transfers end with TBI, and MOVE16 uses the older single-cycle path.
1	**L2_FLUSH**	W	Write 1 to invalidate the whole L2 (a 1-clock pulse, about 20 µs to complete). Always reads 0.
0	**L2_OFF**	RW	1 = L2 disabled: every CPU read goes to DDR3 and nothing new is allocated. Write hits on lines that are still valid keep updating them.

Coherency and the 68060 caches

Two levels of cache sit between the CPU and memory: the 68060's own instruction and data caches (for SRAM, DDR3 and flash) and, for DDR3 only, the FPGA's L2. The FPGA never snoops the 68060 caches. Since build 26 the DMA engines that write DDR3 (MDMA, VDMA) keep the L2 coherent in hardware (figure 3), since build 74 the PCIe DMA path does the same for card DMA, and since build 81 the debug port for its DDR3 writes; the 68060 caches are software's job, with the rules below.

The FPGA asserts TCI (cache inhibit) on every access to the register blocks, so registers are never cached whatever the MMU or transparent-translation settings are. SRAM, DDR3 and flash are cacheable; the choice of cache mode is left to the MMU / TTR setup.

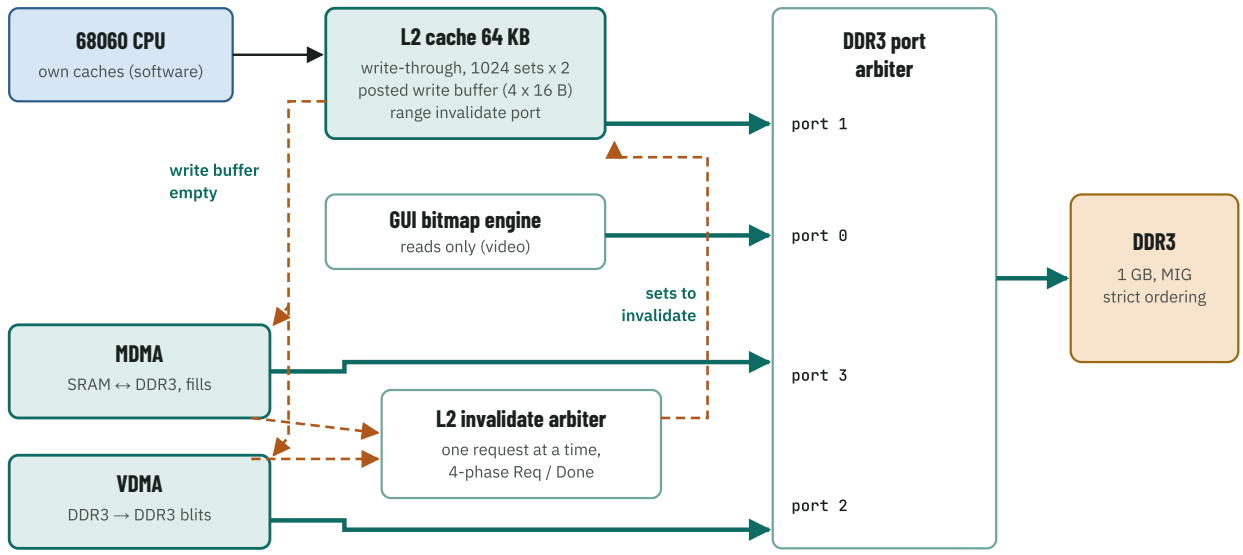


Figure 3. DDR3 coherency with the DMA engines (builds 26-28). Teal: data paths to DDR3. Orange: coherency signals. After each DDR3 write the MDMA and VDMA ask the L2 to drop the sets they touched, and they report DONE only when that is finished; before reading DDR3 they wait until the CPU's posted writes are out of the L2 write buffer.

- CPU writes, other masters read DDR3 (for example the GUI bitmap engine reading a frame buffer): the L2 is write-through, so data that has left the 68060 reaches DDR3 within a few microseconds. Data that is still only in the 68060 data cache in copyback mode is invisible. Map frame buffers and DMA buffers as write-through or cache-inhibited, or push the lines (CPUSHL / CPUSHA) before the other master reads them.
- Another master writes DDR3 and the CPU then reads it: the MDMA and VDMA invalidate the L2 sets they wrote before they report DONE (or raise their interrupt), so no L2 flush is needed after them; invalidate the affected 68060 cache lines (CINV) before reading. For a master without that hardware, flush the L2 (bit 1 of `$FEC0_0038`) as before.
- CPU writes, MDMA or VDMA reads DDR3: the engines wait until the L2 write buffer is empty before they read DDR3, so CPU writes that have left the 68060 are seen. Data still in the 68060 data cache in copyback mode must be pushed first (CPUSHL / CPUSHA).
- PCIe card DMA (build 74): when a card writes DDR3 the FPGA invalidates the matching L2 lines before the write completes, and before a card reads DDR3 it waits until the CPU's posted writes are in DDR3. The 68060 caches are not snooped: push the lines a card will read (CPUSHL / CPUSHP) or map descriptor rings write-through or cache-inhibited, and invalidate the lines a card wrote (CINVL, or CPUSHL when they may be dirty) before the CPU reads them. See the PCI Express chapter.
- Debug port DDR3 access (protocol v2, build 81): debug-port DDR3 reads wait until the CPU's posted writes are in DDR3, and after a write frame the debug port invalidates the L2 lines it wrote. The 68060's own caches are not snooped (as for every DMA): code or data written this way while the CPU runs must be invalidated in the 68060 caches (CINV) before the CPU uses it.
- SDMA (SRAM to SRAM): the L2 is not involved. Push the 68060 data cache lines of the source before the DMA, and invalidate the destination lines after it (or use cache-inhibited buffers).
- Code loaded into memory by DMA or by the debug port: invalidate the 68060 instruction cache before running it (and flush the L2 for DDR3 written by a master that does not invalidate it itself).
- Copyback mode is supported for private data in SRAM and DDR3: cache pushes and line bursts work on both. MCP runs with the 68060 caches off; NetBSD runs with them on.

DDR3 port arbiter

Every DDR3 client talks to one arbiter in front of the memory controller. It has 7 ports (port 5, PCIe DMA, since build 74; port 6, the debug port, since build 80). A port gives a command (read or write, 32-byte aligned byte address, length 1 to 256 packets of 32 bytes); the arbiter grants one port and one direction at a time and only switches between reads and writes between grants.

Software does not program the arbiter; there are no registers. This section explains who shares DDR3 and in which order.

- Packet: 32 bytes = one DDR3 BL8 burst = one controller command and two 128-bit beats (low 16 bytes first). Write data carries a 16-bit byte mask (1 = byte not written).

- Priority: port 0 whenever it has packets to issue, then the fast port (1), then ports 2–6 in round robin. A round-robin port with a longer command continues in a later grant.
- URGENT: when the video engine that owns port 0 (GUI engine or GGE) is close to its deadline it raises URGENT. Another port's grant then stops at the next packet boundary (never between the two beats of a write packet) and continues later.
- Read data returns in order. Each read command carries a tag (port, last packet), so each beat goes back to the right port.
- The controller uses strict ordering and the BANK_ROW_COLUMN address map.
- Bandwidth: the controller's raw rate is 3.2 GB/s (128 bits at 200 MHz). In simulation, with a memory model that accepts commands only 75 % of the time, the arbiter sustained 2.34 GB/s with all ports busy and the video port never late.

> The bandwidth figure comes from simulation, not from a hardware measurement.

Ports

PORT	CLIENT	KIND	GRANT SIZE	STATE IN BUILD 87
0	Video fetch: the GUI bitmap engine, or the GGE line engine while GGE_ENABLE is set (build 60). One owner at a time; the owner changes only when no read command is in flight	Pipelined (up to 15 read commands in flight), absolute priority, URGENT	Its whole command	Connected, reads only
1	CPU through the L2 cache	Fast port: served right after port 0, ahead of the round robin	Its whole command (1 packet = one L2 line, or one write-buffer entry)	Connected
2	VDMA (DDR3 block mover / blitter)	Round robin	Up to 64 packets (2 KB) per grant	Connected (build 27)
3	MDMA, SRAM <-> DDR3	Round robin	Up to 64 packets per grant	Connected (build 26)
4	GGE DAC player (build 66)	Round robin	One 128-byte block (4 packets) per request	Connected, reads only
5	PCIe DMA: memory reads and writes of the PCI Express cards (build 74), for DDR3 `$0080_0000` - `$3FFF_FFFF`	Round robin	Up to 64 packets per grant	Connected (build 74)
6	Debug port: protocol v2 memory reads and writes of DDR3 `$0080_0000` - `$3FFF_FFFF` (port since build 80; DDR3 access from the host since build 81), while the CPU runs. Reads wait for the CPU's posted writes; after a write frame the L2 lines written are invalidated	Round robin	Up to 64 packets per grant	Connected (build 80)

SDMA: system DMA (SRAM to SRAM) `$FEC0_C000`

The SDMA copies or fills blocks of SRAM, in one dimension (a linear block) or two (a rectangle with source and destination strides), in 8, 16 or 32-bit units. It is a real bus master on the CPU/SRAM bus: it asks the bus arbiter for the bus, runs its SRAM cycles itself and gives the bus back.

It moves at most 32 units per bus grant (or one 2D line, whichever ends first), then stays off the bus for 8 clocks before it asks again, so the CPU, MemText and GemTOS are never locked out for long. SRAM cycles run on the SRAM-side clock, which is the 66.67 MHz bus clock: a read takes 6 clocks (90 ns) and a write 5 clocks (75 ns, 2 clocks of write strobe).

The SDMA reaches SRAM only. It cannot read or write DDR3, flash or registers, so the L2 cache is never involved.

The registers are 32-bit, write only as longwords (byte and word writes are ignored), and read back as written. Register `$00` also returns the busy flag in bit 31. Only address bits 4..2 are decoded, so the 8 registers repeat every 32 bytes through `$FEC0_C000` - `$FEC0_C0FF` (since build 26; `$FEC0_C100` is the MDMA).

- How to run a job: write SDMA_CTRL = 0; write the source, destination, sizes, strides and fill value; write SDMA_CTRL with ENABLE and the mode bits; write it again with START (bit 7) added; wait until BUSY (bit 31) is 0 or for the interrupt; write SDMA_CTRL = 0.
- Example, 4 KB copy in 32-bit units with an interrupt: SRC, DST, SIZE1D = 4096, CTRL = `$0000_0029`, then CTRL = `$0000_00A9`.
- 8 and 16-bit copies may use different source and destination alignments; the engine moves the bytes to the right lanes.
- Interrupt: at the end of a job with IRQ_EN set, the SDMA sends a 160 ns pulse to the GAVIN interrupt controller on interrupt `$17` (DMA group, the bit MCP calls INT17_DMA_CH7, mask `$0080`). See the interrupt chapter for how to enable it.
- The 68060 caches do not see SDMA writes: see Coherency and the 68060 caches.

> The SDMA does not check addresses. Keep the source and destination inside the SRAM (`$0000_0000`–`$007F_FFFF`); higher address bits do not select another memory.

Speed (simulated with a 100 MHz SRAM-side clock, with bus hand-overs and a busy CPU and MemText; the SDMA counts in clocks, so at 66.67 MHz expect two thirds of these figures)

JOB	SPEED
32-bit copy	about 32 MB/s
8-bit copy	about 8 MB/s
32-bit fill	about 60–80 MB/s

$00 SDMA_CTRL

RW · reset $0000_0000

Control and status. A job starts on a 0-to-1 change of START while ENABLE is 1. Bits 23:0 read back as written; bits 31:24 are status.



BITS	FIELD	ACCESS	MEANING
31	**BUSY**	R	1 while a job runs (synchronised, so it can lag the engine by 2 bus clocks).
30:24	**-**	R	Read as 0.
23:12	**-**	RW	No function. Write 0.
11:8	**BYTE_MASK**	RW	Byte lanes not to write, 1 = lane not written. Bit 11 = byte offset 0 (D31..D24), bit 8 = byte offset 3 (D7..D0). Applies to writes only.
7	**START**	RW	A rising edge (write with 0, then with 1) starts a job when ENABLE = 1.
6	**-**	RW	No function. Write 0.
5	**UNIT32**	RW	1 = 32-bit units. Wins over bit 4. Address bits 1:0 are ignored.
4	**UNIT16**	RW	1 = 16-bit units (when bit 5 = 0). Address bit 0 is ignored. With bits 5:4 = 00 the unit is 8 bits.
3	**IRQ_EN**	RW	1 = send an interrupt at the end of the job.
2	**FILL**	RW	1 = fill the destination with SDMA_FILL; 0 = copy from the source.
1	**MODE_2D**	RW	1 = 2D job (uses SDMA_SIZE2D and SDMA_STRIDE); 0 = 1D job (uses SDMA_SIZE1D).
0	**ENABLE**	RW	Must be 1 to start a job. Clearing it during a job aborts the job at the next unit.

$04 SDMA_SRC

RW · reset $0000_0000

Source byte address in SRAM (`$0000_0000` – `$007F_FFFF`). Not used by a fill.

$08 SDMA_DST

RW · reset $0000_0000

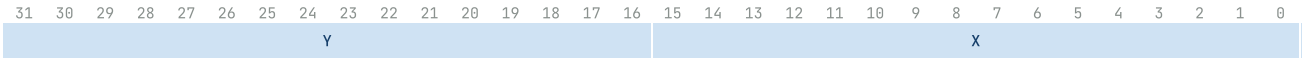
Destination byte address in SRAM (`$0000_0000` – `$007F_FFFF`).

$0C SDMA_SIZE1D RW · reset $0000_0000

1D job length in bytes. The engine moves size / unit-width units; a remainder smaller than one unit is not moved. 0 moves nothing.

$10 SDMA_SIZE2D RW · reset $0000_0000

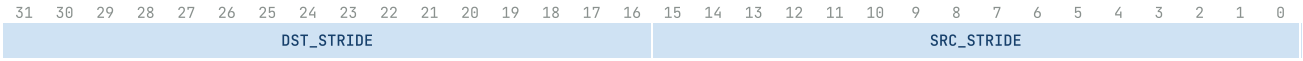
2D job size.



BITS	FIELD	ACCESS	MEANING
31:16	**Y**	RW	Number of lines.
15:0	**X**	RW	Bytes per line. Each line moves X / unit-width units; a remainder smaller than one unit is not moved.

$14 SDMA_STRIDE RW · reset $0000_0000

2D strides: bytes from the start of one line to the start of the next.



BITS	FIELD	ACCESS	MEANING
31:16	**DST_STRIDE**	RW	Destination stride in bytes.
15:0	**SRC_STRIDE**	RW	Source stride in bytes.

$18 SDMA_FILL RW · reset $0000_0000

Fill value. 8-bit fills use bits 31:24, 16-bit fills use bits 31:16, 32-bit fills use bits 31:0.

$1C - RW · reset $0000_0000

Reserved. Reads back as written; no function.

MDMA: memory DMA (SRAM <-> DDR3) $FEC0_C100

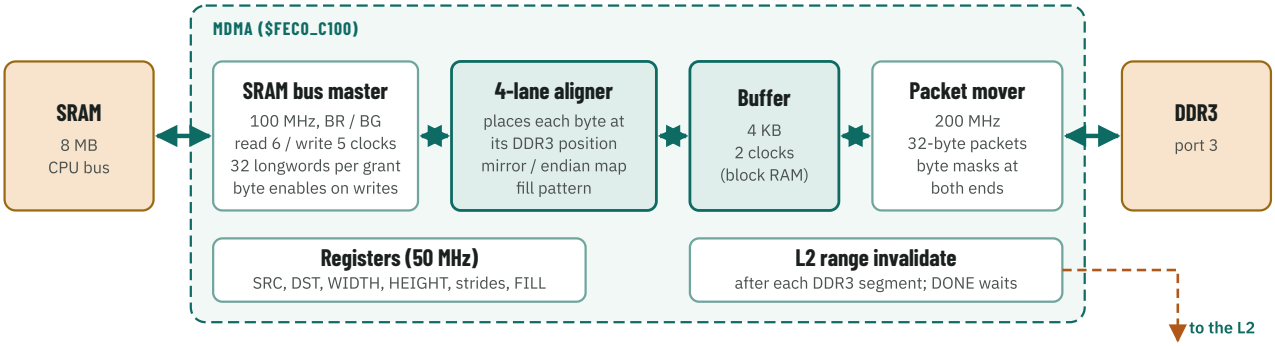
The MDMA (build 26) moves data between the SRAM and DDR3 in both directions, and fills DDR3 or SRAM with a pattern. It is byte exact: the source and the destination may start at any byte (8-bit elements) and the length may be any number of bytes; destination bytes outside the range are never written (DDR3 byte masks, SRAM byte enables).

Shapes: 1D (WIDTH bytes) or 2D (HEIGHT lines of WIDTH bytes, with a signed stride on each side; a negative stride walks that side bottom-up = flip). MIRROR reverses the order of the elements inside each line. Element size 8, 16 or 32 bits; ENDIAN reverses the bytes inside each 16 or 32-bit element. 16 and 32-bit elements need element-aligned addresses, width and strides.

On the SRAM side the MDMA is a bus master (arbiter code 11, after the SDMA) with the SDMA's proven cycles: 6 clocks per read and 5 per write on the 66.67 MHz bus clock, 32 longwords per bus grant, then 8 clocks off the bus. On the DDR3 side it uses arbiter port 3. A line is moved in segments of up to 2 KB through a 4 KB buffer (figure 4).

Coherency: after each DDR3 write the MDMA invalidates the L2 sets it touched, and DONE (and the interrupt) only come after that; before reading DDR3 it waits until the L2 write buffer is empty. The 68060's own caches must be handled by software as for any DMA.

Speed (hardware, MCP TEST MDMABENCH, 8 MB per operation, timed with the frame counter, measured on builds 33 / 34 at 66.67 MHz): 32 MB/s SRAM -> DDR3, 28 MB/s DDR3 -> SRAM, 27 MB/s SRAM fill, 160 MB/s DDR3 fill. The SRAM cycle is the limit.



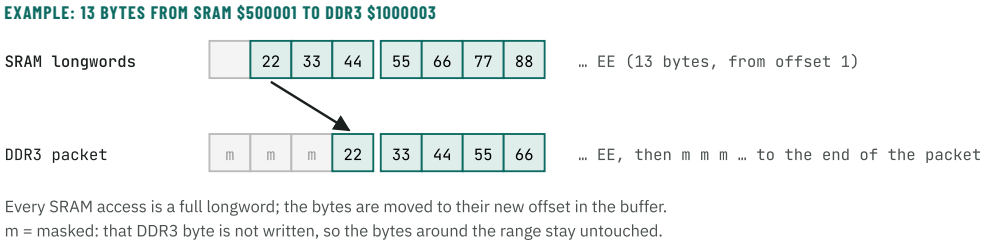


Figure 4. MDMA data path (build 26): SRAM ↔ DDR3 in both directions. In the DDR3 → SRAM direction the mover fills the buffer first and the bus master gathers each destination longword from it.

- How to run a job: write SRC, DST, WIDTH (HEIGHT and the strides for 2D, FILL for a fill), then CTRL with the options and bit 0 (START). Poll STATUS until DONE (bit 1), or enable the interrupt. Check ERROR (bit 2) and the code.
- Example: 13 bytes from SRAM `$0050_0001` to DDR3 `$0100_0003`: SRC = `$0050_0001`, DST = `$0100_0003`, WIDTH = 13, CTRL = `$0000_0001`.
- Interrupt: source `$11` (DMA group, level 4), a 4-clock pulse at the end of a job with CTRL bit 9.
- MCP 123: TEST MDMA [jobs] [seed] (random jobs checked by the CPU) and TEST MDMABENCH.

Registers are 32-bit and accept longword writes only. START is ignored while BUSY is set. Addresses are checked segment by segment (up to 2 KB of a line): a job that leaves the SRAM (`$0000_0000-$007F_FFFF`) or the DDR3 window (`$0080_0000-$3FFF_FFFF`) stops with error code 2 at the first bad segment, after the segments before it have been moved.

$00 MDMA_CTRL

RW (bits 1-0 W) · reset $0000_0000

Options and start. A write with bit 0 starts a job with the current registers; the options are kept.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-																					IRQ…	END…	MIR…	2D	ELEMENT			OP		ABO…	STA…	

BITS	FIELD	ACCESS	MEANING
31-10	-	R	Read 0.
9	IRQ_EN	RW	1 = interrupt `$11` at the end of the job.
8	ENDIAN	RW	1 = reverse the bytes inside each 16 or 32-bit element. No effect on 8-bit elements.
7	MIRROR	RW	1 = elements of each line in reverse order.
6	2D	RW	1 = HEIGHT lines with the two strides; 0 = one run of WIDTH bytes.
5-4	ELEMENT	RW	0 = 8, 1 = 16, 2 = 32 bits.
3-2	OP	RW	0 = SRAM -> DDR3, 1 = DDR3 -> SRAM, 2 = fill DDR3, 3 = fill SRAM.
1	ABORT	W	1 = stop at the next segment or bus release (error code 7). Reads 0.
0	START	W	1 = start a job. Reads 0.

$04 MDMA_STATUS R / W1C · reset $0000_0000

State of the last job.



BITS	FIELD	ACCESS	MEANING
31-8	-	R	Read 0.
7-4	CODE	R	Error code: 1 = alignment (16 / 32-bit element), 2 = address outside SRAM / DDR3, 7 = aborted.
3	-	R	Read 0.
2	ERROR	W1C	1 = the job ended with an error. Write 1 to clear (START also clears it).
1	DONE	W1C	1 = the job has ended (and the L2 invalidates are finished). Write 1 to clear (START also clears it).
0	BUSY	R	1 = a job is running.

$08 MDMA_SRC RW · reset $0000_0000

Source byte address (SRAM or DDR3, by OP).

$0C MDMA_DST RW · reset $0000_0000

Destination byte address.

$10 MDMA_WIDTH RW · reset $0000_0000

Bytes: the whole length (1D) or the bytes per line (2D), bits 23-0 (1 to 16 M).

$14 MDMA_HEIGHT RW · reset $0000_0001

Lines for a 2D job, bits 15-0 (1 to 65535).

$18 MDMA_SRC_STRIDE RW · reset $0000_0000

Signed bytes from one source line start to the next (2D). Negative = bottom-up.

$1C MDMA_DST_STRIDE RW · reset $0000_0000

Signed bytes from one destination line start to the next (2D).

$20 MDMA_FILL RW · reset $0000_0000

Fill pattern, big-endian per element: 8-bit uses bits 7-0, 16-bit bits 15-0, 32-bit bits 31-0.

$24 MDMA_BYTES R · reset $0000_0000

Bytes moved so far by the current or last job.

VDMA: DDR3 block mover / blitter $FEC0_C200

The VDMA (build 27) works inside DDR3. Its main job is to move a rectangle (or a 1D run) from one DDR3 place to another; it can also fill, and combine up to three sources A, B, C into the destination D with a logic function, a colour key and constants. Every channel has its own address and signed stride; any pixel-aligned start address works on every channel, and nothing outside the rectangle is written.

A blit is described by a 64-byte descriptor (16 longwords). START ONE runs the descriptor held in the registers at `$40` - `$7C`; START LIST runs descriptors one after the other from LIST_ADDR in DDR3, up to the first one with the LAST bit.

Pixels are 8, 16 or 32 bits (the same for all channels). For each pixel: D = LF[a, b, c] bit by bit, where LF is the 8-bit logic function and a, b, c are the bits of the three sources (A is the high index bit). A source that is off reads its constant. With KEY on, A is not fetched: a is all ones where the B pixel differs from the KEY colour and all zeros where it equals it.

MIRROR is set per source (a sprite and its mask can be mirrored over a normal background). A negative stride walks a channel bottom-up. Lines are processed in segments of up to 2 KB of D; all sources of a segment are read before its D bytes are written, and DESCENDING processes the segments right to left, so overlapping moves inside one buffer are safe (with negative strides when the destination is lower in memory).

The VDMA runs on the DDR3 clock (200 MHz) and uses arbiter port 2; it never touches the SRAM (figure 5). Coherency as for the MDMA: it waits for the L2 write buffer to be empty before the first descriptor, and invalidates the L2 sets of every D segment; DONE and the interrupts come after that.

Speed (hardware, MCP TEST VDMABENCH, 8 MB per operation, timed with the frame counter, builds 33 / 34): move 32 bpp 960 MB/s, move 8 bpp with odd offsets 768 MB/s, fill 1280 MB/s, colour-keyed sprite 548 MB/s. The VDMA runs on the DDR3 clock, so it does not depend on the CPU bus clock; build 28 measured about 800-960 MB/s for a move, 1.2 GB/s for a fill and 550-640 MB/s for a sprite.

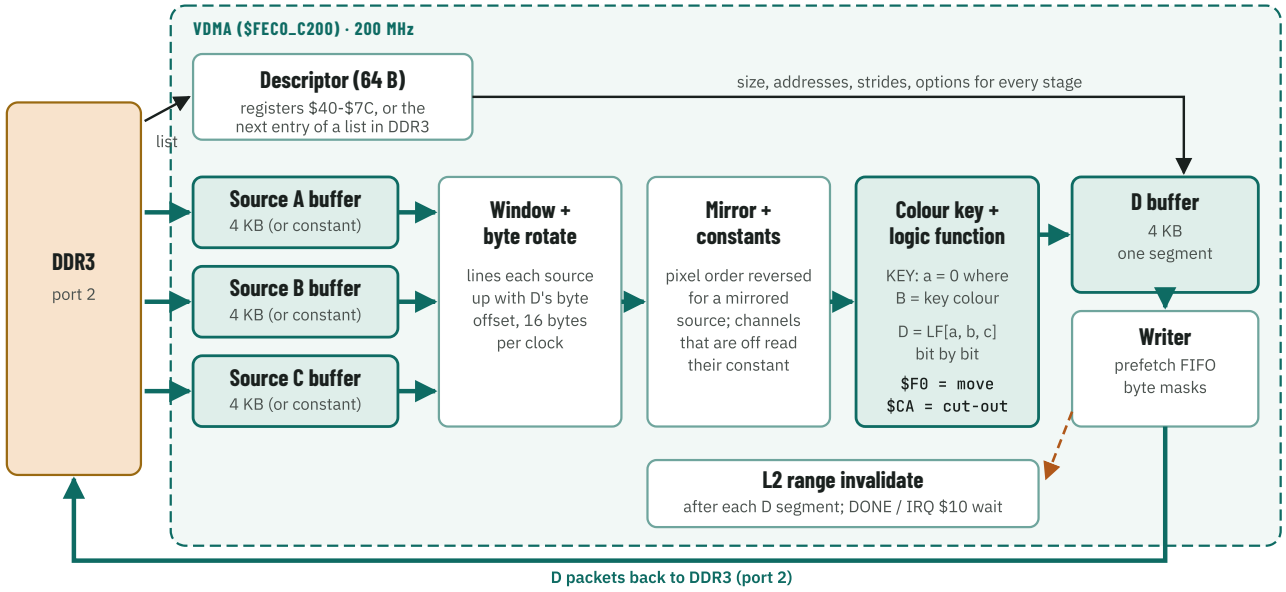


Figure 5. VDMA (build 27). A line is processed in segments of up to 2 KB of D: the sources that are on are read first, then D is built and written. A plain move uses source A only (LF $F0) and costs one DDR3 read and one write per byte.

- Plain move: CTRL = `$0000_08F0` + pixel size x `$100` (A on, LF `$F0`); B and C are not fetched, so a move costs one DDR3 read and one write per byte.
- Fill: CTRL = `$0000_00F0` + pixel size x `$100` (A off), CONST_A = colour.
- Transparent sprite over the screen (8 bpp, key colour 0): CTRL = `$0000_3CCA` (KEY, A, B and C on, LF `$CA`), B = sprite, C = D = screen position, KEY = 0.
- MCP 123 helpers: vdma_move(src, dst, width, height, src_stride, dst_stride, bpp_code) and vdma_fill(dst, width, height, stride, bpp_code, colour). Tests: TEST VDMA [jobs] [seed], TEST VDMABENCH, TEST VDMADEMO (after TEST GUIPAT 5: a block move and 64 colour-keyed sprites from a list).
- Interrupt: source `$10` (DMA group, level 4), one 4-clock pulse per event: after a descriptor with CTRL bit 18, and at the end of a list when CONTROL bit 3 is set.

Registers are 32-bit and accept longword writes only. Error codes stop a list at the failing descriptor (CURRENT shows its address): 1 = an address or stride not pixel-aligned, 2 = a line outside DDR3 (`$0080_0000-$3FFF_FFFF`; checked for the first and last line of every channel before anything moves), 3 = width or height 0 or pixel size 3, 7 = aborted (at the next segment).

Descriptor (in the registers at $40-$7C, or 64 bytes in DDR3, 64-byte aligned)

LONG	OFFSET	FIELD
0	$00	CTRL: [7:0] LF (logic function); [9:8] pixel 0 = 8, 1 = 16, 2 = 32 bpp; [10] KEY; [11] A on; [12] B on; [13] C on; [14] mirror A; [15] mirror B; [16] mirror C; [17] descending; [18] interrupt after this blit; [19] last descriptor of a list
1	$04	SIZE: [15:0] width in pixels (1-65535), [31:16] height in lines (1-65535)
2, 3	$08, $0C	A address, A stride (signed bytes)
4, 5	$10, $14	B address, B stride
6, 7	$18, $1C	C address, C stride
8, 9	$20, $24	D address, D stride
10, 11, 12	$28 - $30	CONST_A, CONST_B, CONST_C: the pixel value of a source that is off (8 bpp uses bits 7-0, 16 bpp bits 15-0)
13	$34	KEY colour (same format)
14, 15	$38, $3C	Free for software (not used by the VDMA)

Useful logic functions

LF	RESULT	USE
$F0	D = A	Move (A on) or fill (A off, CONST_A)
$CA	D = A ? B : C	Cut-out: B where the mask A is set, else C. With KEY on: transparent sprites
$5A	D = A xor C	XOR drawing (cursor, rubber band); twice erases
$CC / $AA	D = B / D = C	Copy B / keep C
$0F	D = not A	Invert
$00 / $FF	D = 0 / all ones	Clear / set
$C0 / $FC	D = A and B / A or B	Apply / combine masks

$00 VDMA_CONTROL W (bit 3 RW) · reset $0000_0000

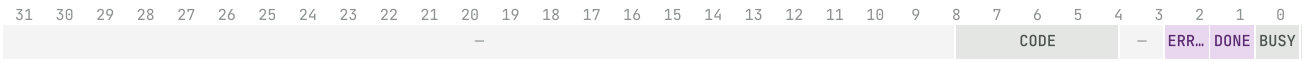
Start, abort and the end-of-list interrupt.



BITS	FIELD	ACCESS	MEANING
31-4	-	R	Read 0.
3	**IRQ_END**	RW	1 = interrupt $10 at the end of a list (or of a START ONE blit).
2	**ABORT**	W	1 = stop at the next segment (error code 7).
1	**START_ONE**	W	1 = run the descriptor in the registers at $40 - $7C.
0	**START_LIST**	W	1 = run the list at LIST_ADDR (wins over bit 1).

$04 VDMA_STATUS R / W1C · reset $0000_0000

State of the last start.



BITS	FIELD	ACCESS	MEANING
7-4	**CODE**	R	Error code: 1 alignment, 2 outside DDR3, 3 size or pixel size, 7 aborted.
2	**ERROR**	W1C	1 = stopped with an error. Write 1 to clear (a start also clears it).
1	**DONE**	W1C	1 = finished, L2 invalidates included. Write 1 to clear.
0	**BUSY**	R	1 = running.

$08 VDMA_LIST_ADDR RW · reset $0000_0000

DDR3 address of the first descriptor of a list (64-byte aligned; bits 5-0 ignored).

$0C VDMA_CURRENT R · reset $0000_0000

DDR3 address of the descriptor being run, or of the one that failed.

$10 VDMA_COUNT R · reset $0000_0000

Blits finished since the last start.

$40-$7C VDMA_DESC RW · reset $0000_0000

The 16 longwords of a descriptor (layout in the table above), used by START ONE.

Debug port: link and frames

The debug port lets a host PC read and write memory, program the flash, pause and step the CPU, set breakpoints, stop the CPU in a monitor and reset the board. It is a serial line on the FPGA pins DBG_RxD / DBG_TxD (FT4232H channel B; on the PC a COM port or /dev/ttyUSBn): 8 000 000 bit/s, 8 data bits, no parity, 1 stop bit, no flow control (RTS / CTS are not used). At 10 bits per byte the link carries at most 800 KB/s. The port runs inside the FPGA on the bus clock (66.67 MHz) and has no CPU registers.

Since build 86 the port speaks only protocol v2. The old Foenix protocol (`$55` requests, `$AA` replies, 6 Mbit/s; FoenixMgr and the GenX Uploader) has been removed: a `$55` frame gets no reply. Use the A2560Me host tool a2560me_dbg.py, which speaks v2.

A request is `$5A`, SEQ, CMD, ATTR, a 32-bit address, a 16-bit SIZE, the payload (write commands only) and a CRC-16. The reply is a data reply (`$A5`, with a payload) or a status reply (`$B5`); both carry SEQ and the 16-bit STATUS. The port handles one frame at a time: bytes before a `$5A` are ignored, a frame that stops for 10 ms between two bytes is dropped without a reply, and a command takes effect only after its CRC has been checked (a write payload is stored first and written only after a good CRC).

This manual gives an overview. Every frame, every command with its ATTR values, worked examples and the host tool are in the separate document "A2560Me Debug Protocol v2" (docs\debug_protocol\A2560Me_Debug_Protocol_v2.pdf, revision 1.3).

- CRC: CRC-16/CCITT-FALSE (polynomial `$1021`, initial value `$FFFF`), sent most significant byte first, over all bytes before it (sync byte included) in requests and replies.
- History: protocol v2 since build 75 (next to the old protocol), DDR3 access build 81, flash ID / erase / program build 82, live SRAM / flash access, pause, breakpoints and the monitor stop build 85 (debug block revision `$14`), v2 only build 86 (revision `$15`). Build 78 ran v2 at 6 Mbit/s; every other build runs it at 8 Mbit/s.

Request frame

BYTE	CONTENT
0	$5A (sync)
1	SEQ (sequence number, returned in the reply)
2	CMD (command code)
3	ATTR (command attribute, for example the erase size or hold / release)
4–7	ADDR[31:24], ADDR[23:16], ADDR[15:8], ADDR[7:0] (32-bit address)
8–9	SIZE[15:8], SIZE[7:0]
10 … 9+SIZE	Payload: SIZE bytes, write commands only
last 2	CRC[15:8], CRC[7:0]

Reply frames

REPLY	BYTES
Data reply	$A5, SEQ, STATUS (2 bytes), payload, CRC (2 bytes)
Status reply	$B5, SEQ, STATUS (2 bytes), CRC (2 bytes)

Debug port: commands and status

Memory commands ($00 / $01) work in every CPU state since build 85. With the CPU held (command $30 ATTR 0) the debug port drives the SRAM / flash bus directly, as before. Otherwise it asks the bus arbiter for the bus as a bus master with the highest priority, and every 256 bus clocks (3.8 us) it gives the bus back between two bytes and asks again, so the CPU, the video fetch and the DMA engines keep running. DDR3 goes through DDR3 arbiter port 6 in every state (build 81+). Only the flash commands ($10 - $12) still need the CPU held.

Every request with a good CRC gets a reply with the STATUS word; a request with a bad CRC gets a status reply with CRC_BAD and nothing is done.

- Speed: about 420 KB/s for SRAM reads and writes with the CPU running (16 KB frames, measured on build 85); the link (800 KB/s) and the bus sharing are the limits. With the CPU held, the SRAM side takes 4 bus clocks (60 ns) per byte read and 6 (90 ns) per byte written.
- Caches: the 68060 does not snoop the debug port. A host read does not see data still dirty in a copyback data cache, and data or code the host writes while the CPU runs must be pushed / invalidated in the 68060 caches before the CPU uses it (as for any DMA). The MCP runs with the caches off.
- Commands $02 / $03 (FPGA register read / write by the debug port) were planned but are not part of the protocol (build 86: they answer UNKNOWN). Registers are read and written through the monitor stub's peek / poke instead, which uses the CPU's own bus cycles.
- The flash programmer runs on a 25 MHz clock and drives both 16-bit flash halves (JEDEC unlock sequence on both at the same time).
- Build 77: while the debug port owns the bus with the CPU held, no FPGA bus master (display fetch, DMA) is granted, and a debug job waits for a running master tenure to end. Up to build 76 a video or DMA master could take the SRAM bus in debug mode.

Commands

CMD	NAME	ATTR / ADDR / SIZE	ACTION
$00	Read memory	ADDR, SIZE = 1 to 16384	Returns SIZE bytes in a data reply. SRAM $0000_0000 - $007F_FFFF, boot flash $FFC0_0000 - $FFFF_FFFF and DDR3 $0080_0000 - $3FFF_FFFF, in every CPU state. Other addresses: RANGE.
$01	Write memory	ADDR, SIZE = 1 to 16384 payload bytes	Writes SRAM or DDR3, in every CPU state. After a DDR3 write the L2 lines written are invalidated. The flash is not writable this way (RANGE).
$10	Flash ID	-	CPU held. Answers BF BF 4B 4B for the SST39VF1601 pair.

CMD	NAME	ATTR / ADDR / SIZE	ACTION
$11	Flash program	ADDR, SIZE = 4 to 16384 (a multiple of 4), payload	CPU held. The range must be erased. The payload is first staged in SRAM $007F_C000 - $007F_FFFF, which is overwritten.
$12	Flash erase	ATTR 0 = 8 KB sector, 1 = 128 KB block, 2 = whole chip; ADDR inside the area	CPU held. Takes up to 25 ms (sector, block) or 50 ms (chip).
$20	Breakpoint set / clear	ATTR = slot 0-3, ADDR = address, SIZE = flags (table below)	Build 85. Flags 0 clear the slot.
$21	Debug report	ATTR 0 state, 1 hit address, 2 CPU bus cycles seen, 4-7 slot address, 8-11 slot flags	Build 85. A data reply with 4 bytes.
$30	CPU control	ATTR 0 hold, 1 release, 2 pause, 3 continue	Hold = CPU in reset, the debug port owns the bus; release lets it boot. Pause / continue (build 85): see the next section. A hold ends a pause.
$31	Bus step	-	Build 85. Only while paused (else STATE): the CPU gets the bus for one tenure, then is paused again.
$32	Monitor stop	-	Build 85. Raises the level-7 interrupt (NMI) for the monitor stub. Refused (STATE) while the CPU is held.
$3F	System reset	-	Sends the reply, then starts the full board reset (about 1.2 s, then a normal boot with the flash -> SRAM copy). The FPGA is not reloaded.
$7E	Revision	-	A data reply: debug block revision ($15 in builds 86-87, $14 in build 85), protocol version (2), the FPGA build number (low byte), 0.
$7F	Status	ATTR 0 status, 1 extended status, 2 / 3 RxD line analyser (read / clear, diagnostic)	Extended status: bit 0 CPU held, 1 debug bus mode, 2 paused, 3 NMI pending; bits 15-8 CRC errors, 23-16 frame time-outs, 31-24 layout (1).
other	-	-	Not implemented: STATUS bit 2 (UNKNOWN). This includes $02 / $03 (see below).

STATUS (16 bits, in every reply)

BIT	NAME	MEANING
0	CRC_OK	The request CRC was correct
1	CRC_BAD	The request CRC was wrong: nothing was done
2	UNKNOWN	Command or ATTR not known
3	BAD_SIZE	SIZE out of range for this command (0, over 16384, or not a multiple of 4 for $11)
4	RANGE	The address range is not allowed, or the CPU is not held for a flash command
6	TIMEOUT	Pause, continue or step not confirmed within 10 ms (a pause stays requested)
7	STATE	Not allowed in this CPU state (pause or monitor stop while held, step while not paused)
8	CPU_HELD	The CPU is held in reset by the debug port
9	CPU_PAUSED	The CPU is paused (off the bus, not reset)
10	BP_HIT	A breakpoint stopped the CPU (until continue or hold)
5, 11-15	-	Reserved, read 0

Typical sequences

JOB	COMMANDS
Load and run a program in SRAM	`$30` ATTR 0 (hold), `$01` blocks of up to 16 KB, `$30` ATTR 1 (release: the CPU boots from the SRAM contents)
Look at memory while the program runs	`$00` blocks of up to 16 KB (SRAM, flash, DDR3); no hold needed (build 85+)
Pause, look around, continue	`$30` ATTR 2, `$00` / `$01` as needed, `$31` for single bus steps, `$30` ATTR 3
Stop when an address is written	`$20` with flags enable + data write, wait for BP_HIT, `$21` ATTR 0 / 1 (who, where), `$30` ATTR 3
Read the CPU registers	`$32`, wait for the stub's STATE = 1 in the mailbox, read the mailbox with `$00`
Write a flash block	a2560me_dbg.py --flash FILE --block N (hold, erase, program, verify, reset; about 6 s for 512 KB)
Reboot the board	`$3F`

Debug port: pause, breakpoints and the monitor stop

Pause (command `$30` ATTR 2, build 85): the bus arbiter takes the bus from the CPU with the normal safe hand-over and keeps it. The CPU is not reset: it simply waits for the bus, so its registers, caches and program stay as they were. The video fetch and the DMA engines are still granted, so the screen stays alive. Continue (`$30` ATTR 3) gives the bus back. A bus step (`$31`, only while paused) lets the CPU make one bus tenure and then pauses it again. Pause, continue and step reply once the arbiter confirms, or with TIMEOUT after 10 ms.

A pause stops the CPU at a bus access, not at an instruction: with its caches on, the 68060 can go on running from them until it needs the bus. Interrupts that arrive during a pause are taken after continue (the acknowledge is a bus cycle).

Breakpoints (build 85): 4 slots. Each compares the start of every CPU bus cycle (TS while the CPU owns the bus) with its address, ignoring the low MASK address bits (so a slot can cover an aligned range), and with the kinds of cycle it watches. Only normal and MOVE16 cycles count; interrupt acknowledges and alternate-space cycles never match; a cache push counts as a data write. The first match sets BP_HIT (STATUS bit 10), records the slot, the address and the cycle kind, and pauses the CPU (or raises the monitor stop, if the slot has flag 4). Later matches are ignored until continue or hold, which also arm the slots again, so a step out of a breakpoint does not stop on it again.

Breakpoints see only the CPU's bus cycles. An access that hits the 68060's caches is invisible, the CPU stops at its next bus access rather than exactly on the instruction, and a code breakpoint can also match an instruction prefetch. With the caches off (the MCP's normal state) every access is seen.

Monitor stop (command `$32`, build 85): the FPGA raises interrupt level 7, the 68060's non-maskable interrupt, answered with the autovector (vector 31, address `$0000_007C`). It stays pending until the CPU's level-7 acknowledge has ended (extended status bit 3); raised during a pause, it is taken after continue. Level 7 is used for nothing else. A program on the CPU must have a handler on vector 31: the MCP (172 and later) contains a small monitor stub. The stub saves all CPU registers into a mailbox in SRAM at `$007F_BE00` and waits; through the debug port's live SRAM access the host can then read and change the registers, run one instruction (trace), continue, and peek / poke any address through the CPU, FPGA registers included.

- Software without a stub (another operating system, or an MCP older than 172) should install a handler on vector 31 that only does RTE. Then a stray monitor stop does no harm. The host must check MAGIC at `$007F_BE00` before it raises the monitor stop, and should check that STATE changes afterwards (MAGIC survives a reset until the next MCP start-up).
- While the CPU is stopped in the stub, interrupts are masked (level 7): timers, the keyboard and the console stop until continue; video, DMA and audio hardware keep running.
- Host tool (a2560me_dbg.py): pause, continue, step, report, nmi, regs, mstep, mcont, --bp SLOT:ADDR[:KINDS[:MASK]], --bp-clear SLOT, --peek ADDR[:SIZE], --poke ADDR:VALUE[:SIZE]. MCP 172+: DBGMON shows the mailbox state, DBGMON REGS the saved registers.
- Tested on the board (build 85, MCP 172 / 173): live SRAM at 420 KB/s with the CPU running, pause, an NMI raised while paused and taken on continue, bus steps, single steps in the monitor, peek of BUILD_ID through the stub, data write, data read (4-byte range) and code breakpoints.

Breakpoint flags (SIZE of command $20)

BIT	NAME	MEANING
0	ENABLE	The slot is active
1	CODE	Match instruction fetches (user or supervisor program)
2	READ	Match data reads
3	WRITE	Match data writes (cache pushes included)
4	NMI	On a hit raise the monitor stop instead of a pause
12-8	MASK	Number of low address bits ignored (0-31)
others	-	0

Debug report (command $21, ATTR 0: state)

BITS	MEANING
31-24	Layout version (1)
21-16	Cycle kind of the hit: R/W, TT1, TT0, TM2-TM0
11-8	Enabled slots (one bit per slot)
7	BP_HIT
3	CPU paused
2	CPU held
1-0	The slot that hit

Monitor stub mailbox at $007F_BE00 (MCP 172+, big-endian longwords)

OFFSET	NAME	MEANING
+$00	MAGIC	`$444D_4F4E` ('DMON') once the stub is installed
+$04	VERSION	1
+$08	STATE	0 running, 1 stopped in the stub
+$0C	REASON	1 NMI (debug port or breakpoint), 2 trace (one instruction done)
+$10	COUNT	Number of stops since start-up
+$14	CMD	Written by the host last: 1 continue, 2 step, 3 peek, 4 poke. The stub writes 0 when done
+$18 / +$1C / +$20 / +$24	ADDR / SIZE / DATA / RESULT	Peek / poke address, size (1, 2, 4), value, result (0 OK, `$FFFF_FFFE` bus error, `$FFFF_FFFF` bad command)
+$40-$7F	D0-D7, A0-A7	Saved registers (A7 = supervisor stack pointer, read only)
+$80 / +$84 / +$88 / +$8C	USP / SR / PC / FMTVEC	User stack pointer, status register, PC to resume at, the exception frame's format / vector word

5 BEATRIX: sound

BEATRIX is the sound block of the A2560Me FPGA. It holds the FPGA sound chips (two SN76489 PSGs, two SID 6581 cores, one YM2151 OPM and one YM2612 OPN2), the mixer that sends them, together with the first output of the GGE DAC player, to the CODEC DAC (WM8776) at 48 kHz, the serial control port of the CODEC, the serial output to a second DAC (AK4432, 44.1 kHz) and, since build 70, the SPI link to the VS1053b MP3 / audio decoder chip. Since build 68 software sets a gain per source, a master attenuation, mute and a stereo blend. Since build 69 the AK4432 plays the second output of the GGE DAC player and nothing else; it is not part of the mix (the CPU-fed PCM FIFO that used it before is retired).

BEATRIX answers at `$FEC2_0000` - `$FEC3_FFFF` (128 KB, data accesses in user or supervisor mode, never cached). Inside that range only bits 16..4 of the address are decoded, and every sub-block decode needs A16 = 0, so `$FEC3_0000` - `$FEC3_FFFF` holds no device. Like every register block, BEATRIX completes all cycles in its range, holes included (no bus error).

Sound-chip registers accept byte writes only. Writes go into a command FIFO and are applied a little later in the 14.318 MHz sound clock domain; since build 68 the FPGA paces OPM and OPN2 writes to the rate the chips can take. Reading the OPM or OPN2 window returns a status byte (busy, FIFO full, timer flags); the PSG and SID windows return no chip data (see each section).

Not in this release: no OPL3 core, no YM2149 core, no 48 kHz CPU PCM FIFO, no ADC (recording) path, no MIDI port inside BEATRIX (the SAM2695 MIDI synth is driven by the Super I/O, see SAM2695 MIDI synthesiser), no interrupts from the sound chips (the only BEATRIX interrupt is the VS1053 link's, source `$29`).

Address map `$FEC2_0000`

Offsets are relative to `$FEC2_0000`. A window is mirrored over its whole size when its device does not decode all the address bits.

Read data: the BEATRIX read multiplexer only knows the configuration, OPL3, OPN2, OPM, CODEC, SID left, SID right, PCM 48 kHz, PCM 44 kHz and VS1053 (build 70) windows. Every other read (PSG windows, SID stereo window, YM2149 windows, holes) returns `$0000_DEAD`.

> The MCP header sound_a2560k.h also lists an external PSG at $FEC2_0100, external OPN2 at $FEC2_0400, external OPM at $FEC2_0600 and external SIDs at $FEC2_0800 / $FEC2_0900. None of these is decoded by the FPGA (no external sound chips). The header puts the stereo ('neutral') SID at $FEC4_1200; the FPGA has it at $FEC2_1400.

BEATRIX windows

ADDRESS	SIZE	BLOCK	STATUS IN THIS RELEASE
$FEC2_0000-$FEC2_00FF	256 B	BEATRIX control registers (2 x 32 bit)	Present: mixer controls (build 68)
$FEC2_0110-$FEC2_011F	16 B	PSG left (SN76489)	Present
$FEC2_0120-$FEC2_012F	16 B	PSG right (SN76489)	Present
$FEC2_0130-$FEC2_013F	16 B	PSG stereo (writes both PSGs)	Present
$FEC2_0200-$FEC2_03FF	512 B	OPL3	Decoded only: no chip, writes ignored, reads $0000_0000
$FEC2_0A00-$FEC2_0BFF	512 B	OPN2 (YM2612)	Present; reads the status byte (build 68)
$FEC2_0C00-$FEC2_0DFF	512 B	OPM (YM2151)	Present; reads the status byte (build 68)

ADDRESS	SIZE	BLOCK	STATUS IN THIS RELEASE
$FEC2_0E00-$FEC2_0FFF	512 B	CODEC control port	Present
$FEC2_1000-$FEC2_11FF	512 B	SID left (6581)	Present
$FEC2_1200-$FEC2_13FF	512 B	SID right (6581)	Present
$FEC2_1400-$FEC2_15FF	512 B	SID stereo (writes both SIDs)	Present
$FEC2_2000-$FEC2_20FF	256 B	PCM FIFO 48 kHz	Decoded only: no logic, reads 0
$FEC2_2100-$FEC2_21FF	256 B	PCM FIFO to the AK4432 (retired in build 69)	Decoded only: reads 0, writes ignored
$FEC2_2200-$FEC2_22FF	256 B	VS1053b link (MP3 / audio decoder)	Present (build 70)
$FEC2_3000-$FEC2_31FF	512 B	YM2149 left	Decoded only: no chip, writes have no effect
$FEC2_3200-$FEC2_33FF	512 B	YM2149 right	Decoded only: no chip, writes have no effect
$FEC2_3400-$FEC2_35FF	512 B	YM2149 stereo	Decoded only: no chip, writes have no effect

BEATRIX control registers $FEC2_0000

Two 32-bit read/write registers that control the mixer (build 68; up to build 67 they were scratch registers with no function). Address bit 2 selects the register; the pair is mirrored every 8 bytes over $FEC2_0000 - $FEC2_00FF.

Only 32-bit (long) writes are taken; byte and word writes are ignored. Any read size works, and all 32 bits read back as written. The values reach the 24.576 MHz mixer through a stable capture (the whole word is taken only once it has been the same for two clocks), so a change is applied from the next 48 kHz sample.

The mixer is described under Mixer and CODEC audio path. It acts on the WM8776 codec output only; the AK4432 output has its own master volume (MASTER_VOL2 in the GGE DAC player).

$00 BTX_CTRL0 RW · reset $0000_0000

Mixer output control: master attenuation, mute and stereo blend. The reset value plays everything at full level, in full stereo.



BITS	FIELD	ACCESS	MEANING
31:7	-	RW	Stored and read back, no function.
6:5	BLEND	RW	Stereo blend: each side gets part of the other side's sum. 0 = none (full stereo), 1 = 7/8 own side + 1/8 other side, 2 = 3/4 + 1/4, 3 = mono (1/2 + 1/2).
4	MUTE	RW	1 = the mixer output is 0 on both sides (silence).
3:0	MASTER_ATT	RW	Master attenuation: the mixed result is shifted right by this many bits, 6.02 dB per step. 0 = none, 15 = -90 dB.

$04 BTX_CTRL1 RW · reset $0008_8888

Gain per mixer source, 4 bits each: the source is multiplied by gain / 8. 0 = off, 8 = unity (reset value), 15 = x 1.875 (+5.5 dB).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-												GAIN_GGE				GAIN_OPM				GAIN_OPN2				GAIN_SID				GAIN_PSG			

BITS	FIELD	ACCESS	MEANING
31:20	-	RW	Stored and read back, no function.
19:16	**GAIN_GGE**	RW	GGE DAC player (its WM8776 output: the voices with V_CTRL bit 8 = 0).
15:12	**GAIN_OPM**	RW	OPM (YM2151).
11:8	**GAIN_OPN2**	RW	OPN2 (YM2612).
7:4	**GAIN_SID**	RW	SID (both SIDs).
3:0	**GAIN_PSG**	RW	PSG (both PSGs).

Sound chip write path

All FPGA sound chips share one write path. A byte write to a PSG, SID, OPM, OPN2 or YM2149 window is put in a command FIFO (1024 entries, written on the CPU clock) with the low 12 address bits, the data byte and the window that was hit. Only byte writes are accepted; word and long writes to these windows are ignored. The CPU write itself completes at once.

A sequencer in the 14.318 MHz sound clock domain empties the FIFO in order. A PSG or SID command takes 9 sound clocks (about 0.63 us). An OPM or OPN2 command is sent to the chip as an address write followed by a data write (12 sound clocks), and since build 68 only when the chip can take it: the sequencer waits until a settle timer has run out and the target core no longer reports busy. The timer is loaded after every OPM or OPN2 write: 300 sound clocks (21 us) after an OPM write, 400 (28 us) after an OPN2 write. The cores apply an operator or channel register only when their slot loop comes round and hold only one pending update, as the real chips do; up to build 67 writes were passed on at full speed and register dumps lost writes.

The FIFO is shared and in order: while an OPM or OPN2 write waits, the PSG and SID writes queued behind it wait too. A dump of 200 OPM registers takes about 4.3 ms. The FIFO holds 1024 writes, so a whole register dump can be written at full CPU speed; check FULL only when more can be queued than that.

Status: reading the OPM or the OPN2 window returns a status byte (see OPM_STATUS / OPN2_STATUS): bit 7 BUSY = a write is still in the FIFO or being applied (to any chip) or the core is busy, bit 6 FULL = the FIFO is full and a write now would be lost, bits 1:0 = the core's timer flags. Up to build 67 these windows read a constant `$4E`. There is no status for the PSG and SID windows.

> BUSY is registered in the sound clock and crossed into the bus clock through two flip-flops, and the FIFO's empty flag also needs a few sound clocks to cross: a status read immediately after a write can still show BUSY = 0 (for well under 1 us). BUSY = 0 does not mean that the settle time of the last OPM / OPN2 write is over; a following OPM / OPN2 write still waits for it in the FIFO.

PSG: SN76489 (x2) `$FEC2_0110`

Two SN76489-compatible PSG cores (jt89): one feeds the left channel, the other the right channel. Each has 3 square-wave tone voices and 1 noise voice, registers as the original chip.

Windows: `$FEC2_0110` = left PSG, `$FEC2_0120` = right PSG, `$FEC2_0130` = both PSGs at once. Write one byte to any address inside the window (the address bits inside the window are ignored). Reads return `$0000_DEAD`.

Byte protocol (as the original chip): a byte with bit 7 = 1 is a latch byte, %1 RR T DDDD, where RR = voice 0-3, T = 0 for tone/noise data and 1 for attenuation, DDDD = low 4 bits of the tone period, the noise control or the attenuation. A byte with bit 7 = 0 (%0 x DDDDDD) gives the upper 6 bits of the 10-bit tone period of the last latched voice. Attenuation 0 = loudest, 15 = off. Noise control (voice 3): bit 2 = white (1) / periodic (0) noise, bits 1:0 = rate (3 = use tone 2 period).

Chip clock: 14.31818 MHz / 4 = 3.5795 MHz (clock enable; NTSC SN76489 clock). Tone frequency = 3.5795 MHz / (32 x period). At reset all voices are at attenuation 15 (silent).

Output: 11-bit signed sample, scaled by 8 into the 16-bit mixer, after a fixed IIR low-pass filter and DC blocker.

> Up to build 67 the divider gave 2.864 MHz, so the notes of the MCP driver (psg.c, which computes the period for a 3.579545 MHz chip) sounded a major third low. Build 68 corrects the divider and the polarity of the PSG write enable. MCP 157 TEST BEATRIX plays a concert A (440 Hz) on the PSG for a check with a tuner.

SID: 6581 (x2) `$FEC2_1000`

Two SID cores (FPGA SID, 6581 model, 3 voices each, registers as the original chip). The left SID feeds the left channel and the right SID the right channel.

Windows: `$FEC2_1000` = left SID, `$FEC2_1200` = right SID, `$FEC2_1400` = both SIDs at once. Registers sit at window offsets `$00` - `$18` in the standard 6581 layout: `$00` - `$06` voice 1 (frequency lo/hi, pulse width lo/hi, control, attack/decay, sustain/release), `$07` - `$0D` voice 2, `$0E` - `$14` voice 3, `$15` - `$16` filter cutoff lo/hi, `$17` resonance/filter routing, `$18` filter mode/volume. Since build 68 only writes to window offsets `$00` - `$1C` reach the core; writes to the rest of the 512-byte window are ignored. (Up to build 67 the low 8 address bits went to the core unfiltered, and the core's own map has further voices above `$1C`, so a write at an offset above `$1C` changed other parts of the core.)

Write-only, byte writes. Reads do not return SID data: any read of the left SID window returns `$4C4C_4C4C` ('L'), of the right SID window `$5252_5252` ('R'), and of the stereo window `$0000_DEAD`. POTX, POTY, OSC3 and ENV3 (`$19` - `$1C`) cannot be read.

Clock: the SID core runs on 14.31818 MHz with a step enable every 14 clocks, i.e. a 1.0227 MHz SID clock, as in an NTSC C64 (up to build 67: every 15 clocks, 0.9545 MHz). External filter emulation is on.

Output: 18-bit signed sample, divided by 4 and clamped to 16 bits into the mixer.

OPM: YM2151 `$FEC2_0C00`

One YM2151-compatible FM core (jt51): 8 channels, 4 operators, stereo output (left/right enable bits per channel as the original chip). Registers as the original chip.

Addressing: write the data byte to `$FEC2_0C00` + register number (`$00` - `$FF`). The FPGA performs the address write and the data write to the chip itself, so one CPU byte write sets one register. Address bit 8 is ignored (`$FEC2_0D00` - `$FEC2_0DFF` mirrors `$FEC2_0C00` - `$FEC2_0CFF`).

Reads of any address in the window return the status byte OPM_STATUS on all four byte lanes (build 68; up to build 67 they returned `$4E4E_4E4E`). Writes are queued and paced (see Sound chip write path). The chip IRQ output and CT1/CT2 outputs are not connected (the GAVIN OPM interrupt input is tied inactive); poll the timer flags instead.

Clock: 14.31818 MHz with clock enables of 14.31818 / 4 = 3.5795 MHz (cen, the YM2151 clock) and every second cen = 1.790 MHz (cen_p1).

Output: full-resolution 16-bit left and right samples into the mixer.

> Up to build 67 the dividers gave 2.864 MHz and 1.591 MHz (cen_p1 not half of cen), so pitch, envelope and timer rates differed from a 3.579545 MHz YM2151. Build 68 runs the core at the YM2151 clock.

$00 OPM_STATUS R · reset $00

Status byte (build 68), read at any address of `$FEC2_0C00` - `$FEC2_0DFF`, any read size (the byte is repeated on all four lanes).

7	6	5	4	3	2	1	0
BUSY	FULL	-				TIMER_B	TIMER_A

BITS	FIELD	ACCESS	MEANING
7	**BUSY**	R	1 = a sound chip write is still waiting in the command FIFO or being applied, or the OPM core reports busy. 0 = every queued write has been applied.
6	**FULL**	R	1 = the command FIFO is full: a write now is lost.
5:2	**-**	R	Read 0.
1	**TIMER_B**	R	Timer B flag of the OPM core.
0	**TIMER_A**	R	Timer A flag of the OPM core.

OPN2: YM2612 $FEC2_0A00

One YM2612-compatible FM core (jt12): 6 channels, 4 operators, channel 6 DAC mode, stereo output (L/R bits per channel as the original chip). Registers as the original chip.

Addressing: write the data byte to `$FEC2_0A00` + register number for part I (registers `$00` - `$FF`, channels 1-3 and globals) and to `$FEC2_0B00` + register number for part II (channels 4-6). Address bit 8 selects the part; the FPGA performs the address and data writes to the chip itself.

Reads of any address in the window return the status byte OPN2_STATUS on all four byte lanes (build 68; up to build 67 they returned `$4E4E_4E4E`). Writes are queued and paced (see Sound chip write path). The chip IRQ output is not connected (the GAVIN OPN2 interrupt input is tied inactive); poll the timer flags instead. High-fidelity PCM interpolation is off.

Clock: 14.31818 MHz with a clock enable every 2 clocks = 7.159 MHz.

Output: 16-bit left and right samples into the mixer.

> Up to build 67 the divider gave 4.773 MHz. A Mega Drive YM2612 runs at 7.67 MHz, so on build 68 and later pitches and timers are still about 7 % (1.2 semitones) lower than on that machine unless software compensates.

$00 OPN2_STATUS R · reset $00

Status byte (build 68), read at any address of `$FEC2_0A00` - `$FEC2_0BFF`, any read size (the byte is repeated on all four lanes). The same BUSY and FULL as OPM_STATUS (one shared FIFO), with the OPN2 core's busy and timer flags.

7	6	5	4	3	2	1	0
BUSY	FULL	-				TIMER_B	TIMER_A

BITS	FIELD	ACCESS	MEANING
7	**BUSY**	R	1 = a sound chip write is still waiting in the command FIFO or being applied, or the OPN2 core reports busy. 0 = every queued write has been applied.
6	**FULL**	R	1 = the command FIFO is full: a write now is lost.
5:2	**-**	R	Read 0.
1	**TIMER_B**	R	Timer B flag of the OPN2 core.
0	**TIMER_A**	R	Timer A flag of the OPN2 core.

Mixer and CODEC audio path

Each sound chip sample crosses from the 14.318 MHz sound clock to the 24.576 MHz audio clock through a stable-capture synchronizer.

Per channel the mixer takes five 16-bit sources: PSG (through a fixed IIR low-pass filter and a DC blocker), SID, OPN2, OPM and the WM8776 output of the GGE DAC player (the voices with V_CTRL bit 8 = 0, see the GGE chapter). The left channel gets the left PSG, left SID, the left outputs of OPM and OPN2 and the left DAC player output; the right channel the right ones.

Once per 48 kHz sample each source is multiplied by its gain from BTX_CTRL1, (x * gain) >> 3, and the five terms are added in 20 bits, which cannot overflow. Then come the stereo blend (BTX_CTRL0 bits 6:5; the other side's part is taken from its sum, halved and limited to 16 bits), the master attenuation (an arithmetic shift right by BTX_CTRL0 bits 3:0, 6 dB per step) or MUTE (output 0), and only then one clamp to 16 bits. A mix that is too loud can therefore be brought back by the master attenuation without clipping. The gains, attenuation, mute and blend are new in build 68; before, the sum was always clamped at unity gain.

The AK4432 is not part of this mix: it plays only the second output of the GGE DAC player (see AK4432 DAC output).

The result goes to the CODEC DAC as a 16-bit I2S-style serial stream at 48 kHz: CODEC MCLK = 24.576 MHz (512 x fs), BCLK = 1.536 MHz (32 x fs, 16 bits per channel), LRCK = 48 kHz. Since build 68 MCLK leaves the FPGA through an ODDR output register, so the pin carries a clean copy of the 24.576 MHz clock.

The CODEC ADC pins (BCLK, LRCK, data) reach BEATRIX but are not used: there is no audio input or recording path in this release.

- MCP 157+: TEST BEATRIX [MIX src 0-15 | MASTER 0-15 | MUTE | UNMUTE | RESET | STATUS]. Without arguments it checks the mixer registers (read-back) and that the OPM / OPN2 status is idle, then plays a listening sequence: a concert A (440 Hz) on the PSG, an A4 triangle on the left SID, a concert A on the OPM, a tone on the OPN2, then the OPM tone with the master attenuation stepping down 6 dB at a time and back up. MIX (src = PSG, SID, OPN, OPM or GGE), MASTER, MUTE, UNMUTE and RESET set BTX_CTRL0 / BTX_CTRL1; STATUS prints them.
- MCP 157+: TEST OPM and TEST OPN play a tone on the internal YM2151 / YM2612. TEST OPM EXT / TEST OPN EXT run the older tests for external chips, which this board does not decode.
- TEST GGEAUD [AK | RATE] (MCP 156, AK since MCP 158, RATE since MCP 159) tests the GGE DAC player; AK plays it on the AK4432 output, RATE measures the sample rates of both outputs. See the GGE chapter.

Audio clocks

CLOCK	FREQUENCY	USED FOR
`Sound chip clock`	14.31818 MHz	PSG, SID, OPM, OPN2 cores and the command FIFO read side
`CODEC clock`	24.576 MHz	Mixer, CODEC MCLK (through an ODDR), 48 kHz serial stream, the 48 kHz tick of the GGE DAC player; divided by 2 for the VS1053 (build 70)
`AK4432 clock`	22.5792 MHz	AK4432 MCLK (through an ODDR) and its 44.1 kHz serial stream, the 44.1 kHz tick of the GGE DAC player
`VS1053 clock (CLKI)`	12.288 MHz	CODEC clock / 2, sent out through an ODDR to the VS1053b (build 70); the chip multiplies it by 3.5 (SCI_CLOCKF)
`CPU bus clock`	66.67 MHz	Register interface, CODEC control port, VS1053 link (SPI engine)

CODEC control port `$FEC2_0E00`

A 3-wire serial port (CE, CL, DI) that writes 16-bit control words to the CODEC. The MCP driver treats the CODEC as a Wolfson WM8776: each word is a 7-bit register address in bits 15:9 and 9 bits of data in bits 8:0 (for example `$2E00` = reset).

Write the 16-bit word with a word (16-bit) write anywhere in `$FEC2_0E00` - `$FEC2_0FFF`; byte and long writes are ignored. The word is sent MSB first, 16 CL pulses of about 280 ns each, then a CE pulse latches it; a whole transfer takes about 5 us.

Wait until BUSY is 0 before the next write: a write while busy is ignored. The port is write-only towards the CODEC (no read-back of CODEC registers).

$00 **CODEC_CTRL** RW · reset $0000

Write: 16-bit control word to send to the CODEC. Read: transfer status. A long read returns the same 16 bits in both halves.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BUSY								-							

BITS	FIELD	ACCESS	MEANING
15	**BUSY**	R	1 = a word is being sent; 0 = ready for the next word.
14:0	**-**	R	Read as 0.
15:0	**WORD**	W	Control word, sent MSB first (WM8776: bits 15:9 register, bits 8:0 data).

AK4432 DAC output

Since build 69 the AK4432 stereo DAC plays the second output of the GGE DAC player (see the GGE chapter: voices with V_CTRL bit 8 OUTPUT = 1, master volume MASTER_VOL2). It is not part of the sound chip mix, which goes to the WM8776 codec at 48 kHz, and the BEATRIX mixer controls do not act on it. Its analogue output goes to the AUX input of the WM8776 (see WM8776 analogue inputs).

Serial stream: I2S, 16 bits per channel, 32 bit clocks per frame. MCLK = 22.5792 MHz from its own oscillator, sent out through an ODDR output register (build 68); BICK = MCLK / 16 = 1.4112 MHz; LRCK = MCLK / 512 = exactly 44.1 kHz, locked to MCLK (512 fs).

Board strapping (schematic): the AK4432 runs in parallel mode (P/S pulled high), so it has no control port to set up; ACKS is pulled high (automatic clock detection: MCLK 22.5792 MHz is taken as 512 fs at 44.1 kHz); DIF is pulled high (I2S input format). Played correctly on hardware with build 69 (MCP TEST GGEAUD AK).

The CPU-fed PCM FIFO that drove the AK4432 up to build 68 is retired. Its window `$FEC2_2100` - `$FEC2_21FF` still decodes (no bus error) and reads 0; writes are ignored. Its interrupt input `$2E` (46, BTX_DAC44_FIFO) is driven inactive, like the 48 kHz FIFO input `$2F` (47). Software that streamed PCM there uses a DAC player voice in STREAM mode with OUTPUT = 1 instead: 44.1 kHz material plays at V_STEP `$0001_0000`.

VS1053b MP3 / audio decoder link `$FEC2_2200`

Build 70 adds a link to the VLSI VS1053b decoder chip. The FPGA only runs the chip's SPI bus; software decides what to play and feeds the data (there is no player in the FPGA). The chip decodes MP3, WMA, AAC, Ogg Vorbis, WAV and MIDI files by itself. Its analogue output goes to input AIN4 of the WM8776 (see WM8776 analogue inputs).

Bus: the link answers at `$FEC2_2200` - `$FEC2_22FF`. Address bits 4:2 select the register, so the block repeats every 32 bytes; offsets `$14` - `$1C` read 0. VS_CTRL, VS_STATUS (clearing) and VS_SCI take only 32-bit (long) writes; byte and word writes to them are ignored. VS_DATA takes byte, word and long writes. Read the registers as longwords. Each access acts once.

SPI: one bus in mode 0 (SCLK idle low, MSB first; the VS1053 samples MOSI on the rising edge of SCLK, the FPGA samples MISO just before the falling edge) with the chip's two selects. XCS carries SCI register commands: a write is `$02`, the register number, the 16-bit value; a read is `$03`, the register number, then 16 bits in. XDCS carries the SDI data stream, in bursts of up to 32 bytes, each started only while DREQ is high (DREQ high = room for at least 32 bytes); XDCS goes high between bursts. After every SCI write the link waits 5 us, then for DREQ high (at most 50 ms), before the next SPI operation. A queued SCI command is always started before the next data burst.

Data FIFO: 4 KB, 1024 words of 1 to 4 bytes. Each VS_DATA write fills one word: a byte write 1 byte, a word write 2, a long write 4. Bytes go out in 68000 memory order: a long's bits 31:24 first, a word's bits 15:8 first. VS_ROOM counts free words, so stream with long writes (a byte write uses a whole word for one byte). A write while the FIFO is full is lost and sets OVF.

SCI commands: VS_SCI writes go into an 8-entry queue and are sent in order. The result of an SCI read lands in VS_SCI (read) when it is done: wait until SCI_BUSY is 0, then read VS_SCI. A VS_SCI write while SCI_FULL is 1 is lost.

RUN = 0 holds the link idle (XCS and XDCS high, SCLK low, READY 0); queued data and SCI commands are kept and go out after the next start-up. Setting RUN (0 -> 1) runs the whole start-up sequence below, a software reset of the chip included. READY then goes to 1. After a system reset RUN is 0.

Interrupt: the link drives GAVIN interrupt source `$29` (group 5, CPU level 1, vector `$61`). It is active (low) while LOW and LOW_IRQ_EN are both 1: a level, so clear LOW (write `$10` to VS_STATUS) in the handler after refilling. LOW is set when the number of bytes in the FIFO falls below LOW_MARK x 16 as the link sends them.

Volume: SCI register `$B` (SCI_VOL), left in bits 15:8 and right in bits 7:0, 0.5 dB per step down from `$00` (the loudest). The start-up sets `$1010` (-8 dB each side).

- Playing a file: write VS_CTRL = `$0000_0001` (RUN) and wait for READY (VS_STATUS bit 0, normally a few ms). Write the file into VS_DATA with long writes while VS_ROOM shows space; for interrupt-driven refills set LOW_MARK and LOW_IRQ_EN as well (for example `$0000_8005`: refill when fewer than 2 KB are queued).
- SCI read: write VS_SCI = `$0100_0000` + register x `$1_0000`, wait until SCI_BUSY (VS_STATUS bit 2) is 0, read VS_SCI bits 15:0. SCI write: wait until SCI_FULL (bit 3) is 0, write VS_SCI = register x `$1_0000` + value.
- Checks: SCI_STATUS (register 1) bits 7:4 read 4 on a VS1053; after the start-up SCI_MODE reads `$4800` and SCI_CLOCKF `$8800`.
- MCP 160: TEST VS1053 starts the link (RUN 0 then 1, READY within 0.5 s), reads SCI_STATUS, SCI_MODE and SCI_CLOCKF, then plays the chip's built-in 689 Hz sine test for 1.5 s (SM_TESTS, sent as SDI data bytes) at volume `$0000` and restores MODE `$4800` and VOL `$1010`. Board-tested on build 70: version 4, MODE `$4800`, CLOCKF `$8800`, sine heard.

> VS_CTRL takes only long writes and every write sets all its fields, so write FLUSH together with the current RUN, LOW_IRQ_EN and LOW_MARK (writing `$0000_0002` alone also clears RUN and stops the link). FLUSH drops the queued bytes at once; a byte being sent finishes. To stop a file cleanly, use the cancel procedure of the VS1053b datasheet (SM_CANCEL) before flushing. The link has an XRESET output (low while RUN = 0 and for 2 ms after RUN is set), but on this board the chip's XRESET pin is wired to the 10M02 (pin K2), so the output is not connected: the start-up uses the software reset SM_RESET instead. If DREQ never rises, each wait of the start-up ends after 50 ms and READY still comes up, so check SCI_STATUS to see whether a chip answers.

Start-up sequence (after RUN 0 -> 1)

STEP	ACTION
1	Wait 2 ms (the XRESET output is released at the end; not connected on this board).
2	Wait for DREQ high (at most 50 ms).
3	SCI_MODE = `$4804`: SM_SDINEW (new mode, separate XCS / XDCS) + SM_RESET (software reset). Then 5 us and DREQ, as after every SCI write.
4	SCI_MODE = `$4800`.
5	SCI_CLOCKF = `$8800`: CLKI = 3.5 x 12.288 MHz = 43.0 MHz.
6	Switch to the fast SPI write rate, then SCI_VOL = `$1010` (-8 dB each side).
7	READY = 1. Queued SCI commands, then data bursts, are sent.

SPI clock rates (66.67 MHz bus clock)

TRANSFER	BEFORE SCI_CLOCKF	AFTER SCI_CLOCKF	VS1053 LIMIT
`SCI write`	3.03 MHz (bus clock / 22)	5.56 MHz (bus clock / 12)	CLKI / 4: 3.07 MHz, then 10.75 MHz
`SCI read`	1.67 MHz (bus clock / 40)	1.67 MHz	CLKI / 7: 1.76 MHz, then 6.1 MHz
`SDI data`	-	5.56 MHz: about 690 KB/s, far more than any MP3 needs	CLKI / 4

Signals (board XDC)

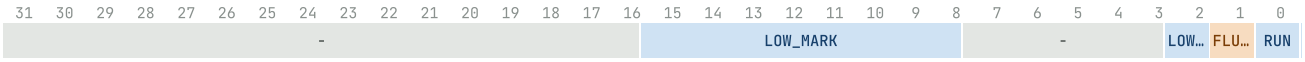
SIGNAL	FPGA PIN	DIRECTION
`CLKI (12.288 MHz)`	D19	out (ODDR)
`XCS`	G20	out
`XDCS`	F20	out

SIGNAL	FPGA PIN	DIRECTION
SCLK	J20	out
MOSI (SI)	K20	out
MISO (SO)	L20	in
DREQ	E20	in
XRESET	- (10M02 pin K2)	not driven by the Kintex

$00 VS_CTRL

RW · reset $0000_0000

Link control. Long writes only; every write sets RUN, LOW_IRQ_EN and LOW_MARK.



BITS	FIELD	ACCESS	MEANING
31:16	-	R	Read 0.
15:8	LOW_MARK	RW	LOW is set when the bytes in the data FIFO fall below LOW_MARK x 16. 0 = never.
7:3	-	R	Read 0.
2	LOW_IRQ_EN	RW	1 = interrupt $29 is active while LOW is set.
1	FLUSH	W	Write 1: drop all queued data bytes. Reads 0.
0	RUN	RW	1 = start up and run (0 -> 1 runs the start-up sequence). 0 = hold the link idle; the FIFO and the SCI queue are kept.

$04 VS_STATUS

R / W1C · reset $0000_0004 (+ DREQ)

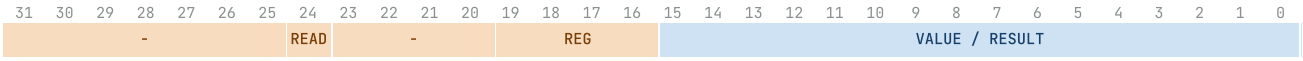
Link status. A long write of 1 clears LOW (bit 4) or OVF (bit 5); other bits ignore writes.



BITS	FIELD	ACCESS	MEANING
31:29	-	R	Read 0.
28:16	BYTES	R	Bytes in the data FIFO (0-4096).
15:6	-	R	Read 0.
5	OVF	W1C	Sticky: a VS_DATA write found the FIFO full and was lost.
4	LOW	W1C	Sticky: the FIFO fell below LOW_MARK x 16 bytes.
3	SCI_FULL	R	1 = the 8-entry SCI command queue is full: a VS_SCI write now is lost.
2	SCI_BUSY	R	1 = SCI commands queued or one running (including the 5 us and DREQ wait after a write), or the link is not READY (RUN = 0 or the start-up). 0 = the result of the last SCI read is in VS_SCI.
1	DREQ	R	The chip's DREQ pin (synchronised): 1 = room for at least 32 data bytes.
0	READY	R	1 = the start-up sequence is done.

$08 VS_SCI RW · reset $0000_0000

Write (long): queue one SCI command. Read: the result of the last SCI read (bits 31:16 read 0).



BITS	FIELD	ACCESS	MEANING
31:25	-	W	Ignored.
24	**READ**	W	1 = an SCI read (VALUE is ignored), 0 = an SCI write.
23:20	-	W	Ignored.
19:16	**REG**	W	SCI register number (0 = MODE, 1 = STATUS, 3 = CLOCKF, `$B` = VOL, ...).
15:0	**VALUE / RESULT**	RW	Write: the 16-bit value for an SCI write. Read: the 16-bit result of the last SCI read.

$0C VS_ROOM R · reset $0000_0400

Free words in the data FIFO.



BITS	FIELD	ACCESS	MEANING
31:11	-	R	Read 0.
10:0	**ROOM**	R	Free words, 1024 = empty. Every VS_DATA write takes one word, whatever its size.

$10 VS_DATA W · reset -

Data bytes for the decoder (SDI). A byte, word or long write adds 1, 2 or 4 bytes, sent in 68000 memory order (a long's bits 31:24 first). Reads 0.

SAM2695 MIDI synthesiser $FEC0_2330

The Dream SAM2695 is a General MIDI synthesiser (128 instruments, drums on channel 10). It is not part of BEATRIX: it listens to the MIDI output of the Super I/O's MPU-401 port on the LPC bus (see the GAVIN chapter). MCP puts the MPU-401 (Super I/O logical device `$0B`) at LPC I/O `$330` with serial IRQ 5, so the data port is at `$FEC0_2330` and the status / command port at `$FEC0_2331`; the MPU-401 interrupt is source `$0E`. The FPGA only gives the SAM2695 its 12 MHz clock (see Clocks in the system chapter). Its analogue output goes to input AIN1 of the WM8776.

Set-up: write `$FF` (reset) to the command port `$FEC0_2331`, then `$3F` (UART mode). The MPU-401 answers each with `$FE` in the data port (read it while status bit 7 is 0). Then write MIDI bytes to the data port `$FEC0_2330`, each only while status bit 6 is 0.

Interrupt source `$28` (SAM2695 MIDI receive FIFO) is tied inactive: there is no FPGA path from the SAM2695.

- MCP 160: TEST SAM2695 [SOLO] resets the MPU-401 and sets UART mode (reports the `$FE` acknowledge), sends a GM reset, sets everything to full level, then plays 4 piano notes, a chord and 8 drum hits. SOLO selects only WM8776 input AIN1 during the test (CODEC word `$2A01`) and restores `$2A1F` after it. Heard on build 70.
- MCP 160 also fixes the MIDI device and MIDITX (the status bits were swapped in midi_reg.h; the device did not open, 'bad device number').

> Up to MCP 159 the MCP header midi_reg.h had the two status bits swapped. Code that copied those defines must swap them: bit 6 = 1 means not ready for a byte, bit 7 = 1 means nothing to read.

$00 MPU_DATA RW · reset -

Write: one MIDI byte to send. Read: one byte from the MPU-401 (the `$FE` acknowledge after a command).

$01 MPU_STAT / MPU_CMD RW · reset -

Read: MPU-401 status. Write: command ($FF reset, $3F UART mode).

7	6	5	4	3	2	1	0
DSR	DRR	-					

BITS	FIELD	ACCESS	MEANING
7	**DSR**	R	1 = nothing to read.
6	**DRR**	R	1 = not ready for a byte (wait before writing).
5:0	**-**	R	Not used.

WM8776 analogue inputs

Besides the 48 kHz digital stream from the BEATRIX mixer, the WM8776 codec takes analogue inputs. On the A2560Me they carry the chips that make their own sound; every source ends on the WM8776 line and headphone outputs.

AIN1-AIN5 are selected in WM8776 register 21 (ADC input mux, bits 4:0 = AIN5..AIN1), written through the CODEC control port: MCP writes $2A1F (all five inputs); $2A01 selects AIN1 (the SAM2695) only.

- MCP 160: TEST CODEC <word> writes one WM8776 control word (register in bits 15:9, data in bits 8:0, hex), for experiments: TEST CODEC 2A01 selects AIN1 only, 2A1F all five inputs.

> Known board point (build 70): the SAM2695 and VS1053 are heard, but at a low level. Their outputs reach the WM8776 through a resistor network on the board; a board change is planned. Nothing in the FPGA or the MCP changes it.

WM8776 analogue inputs (board)

INPUT	SOURCE
AIN1	SAM2695 MIDI synthesiser
AIN2	Not connected
AIN3	Not connected
AIN4	VS1053b decoder
AIN5	Line-in
AUX	AK4432 (DAC2) output

Other blocks

The top level of the design instantiates: the CPU interface, the DDR3 memory system, GAVIN, BEATRIX, VICKY II (legacy video), the video timing generator and pixel mixer, the GUI bitmap engine, the Game Graphics Engine (with the DAC player), the debug port, the I2C master (build 84) and, since build 74, the PCI Express root port (its own chapter). The general-purpose logic in MISC_LOGIC (buzzer, RGB status LEDs, LFSR random number generator), the UARTs, the EmuTOS bitmap and MemText all sit inside GAVIN or the video blocks and are described in those chapters.

The I2C master is a top-level module, decoded inside the GAVIN window at $FEC0_0E00 - $FEC0_0EFF. Since build 84 it is a new block (A2560Me_I2C) that the CPU can use for the whole system I2C bus; it still programs the SiI9022 HDMI transmitter by itself after reset and on every video mode change. Its registers, the devices on the bus and an example (reading the MAC address) are in the GAVIN chapter, section I2C master.

- PCIe (PCIE_ON) is built in since build 74: a root port with configuration and memory windows, card DMA into DDR3 and interrupt source $23 (see the PCI Express chapter).
- The old VICKY IV game engine (GGE_ON) is not in this release. The Game Graphics Engine of builds 60-70 replaces it; its DAC player feeds the BEATRIX mixer and, since build 69, the AK4432 (see the GGE chapter).

6 Game Graphics Engine (GGE)

The Game Graphics Engine (GGE, builds 60-70) is a second video system, made for games. It shows 640 x 480 at 60 Hz or 640 x 400 at 70 Hz from a 25.000 MHz dot clock. A picture is built from 16 layers: 4 bitmap layers, 4 tile layers and 8 sprite layers. All graphics are 8 bits per pixel, indexed, and are read from DDR3. Every layer has its own palette block (4 tables of 256 ARGB colours), and all 16 layers are alpha-blended in a programmable order.

When GGE_ENABLE (`$FD00_0000` bit 0) is set, the whole video output switches to the GGE: the dot clock, the HDMI pixel path and the SiI9022 set-up. The 108 MHz video system (block text, MemText, TOSGRAPH, the GUI bitmap engine, the mouse pointer) keeps running but is not shown. A sprite can be used as the mouse pointer.

The GGE also holds a 32-voice DAC player. It reads its samples from DDR3 by itself and plays them on two outputs, chosen per voice (build 69): the 48 kHz codec output of BEATRIX, where they are mixed with the sound chips, and the AK4432 DAC at 44.1 kHz, on their own.

The GGE answers at `$FD00_0000` - `$FD0F_FFFF` (chip select CS_GGE, 1 MB, data accesses only, never cached). It uses the internal register timing: TA after 2 wait states, 4 bus clocks per access. Use longword accesses only: byte and word writes are ignored. Unused addresses in the block read 0 and ignore writes; they do not give a bus error.

Overview and address map `$FD00_0000`

The GGE is a new design (no code from the earlier VICKY IV game engine, which is no longer in the build). Its registers live on the CPU bus clock (66.67 MHz). The line engine runs on the DDR3 user clock (200 MHz) and builds every display line one line ahead of the screen. The timing generator and the output run on the 25.000 MHz dot clock (figure 6).

Layers are numbered as 16 sources. The same numbering is used by LAYER_ENABLE, LAYER_ORDER, OPACITY0-3, COLL_ENABLE, COLL_ANY and COLL_WITH0-15, and it selects the palette block of the layer.

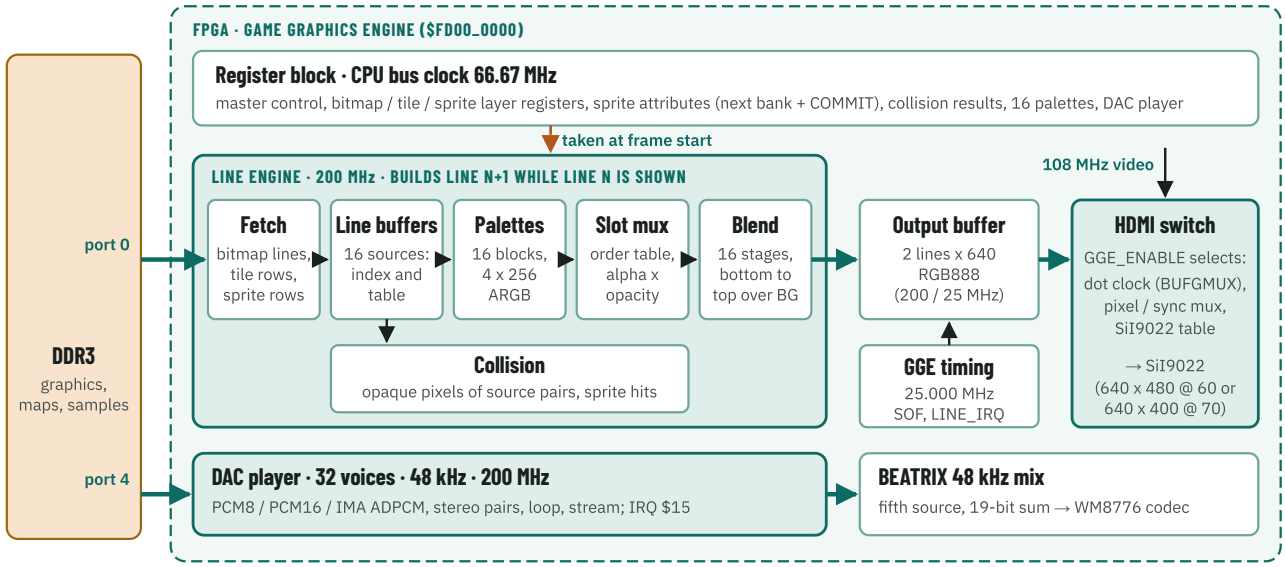


Figure 6. GGE data path (builds 60-66). The line engine reads each line from DDR3 one line ahead and blends 16 layers; GGE_ENABLE switches the dot clock, the HDMI pixel path and the SiI9022 set-up. The DAC player reads its samples from DDR3 on port 4.

- All layers are 8 bpp indexed. Index 0 is always transparent, whatever its palette entry holds.
- DDR3 addresses in GGE registers are DDR3 byte addresses (bits 29:0). They equal CPU addresses in `$0080_0000` - `$3FFF_FFFF`. The GGE can also read DDR3 below `$0080_0000`, which the CPU cannot reach (fill it with the MDMA or VDMA).
- Graphics are read in memory order (big-endian, as the 68060 writes them). Keep GAVIN register 0 bit 6 (endian inversion) at 0.
- The L2 cache is write-through, so CPU writes reach DDR3. With the 68060 data cache in copyback mode, push the lines of graphics, maps and samples (CPUSHL / CPUSHA) before the GGE reads them, or map them write-through.

GGE address map

ADDRESS	SIZE	BLOCK
$FD00_0000-$FD00_004F	80 B	Master control: GGE_CTRL, layer enable and order, background colour, status, line interrupt, LATE, SP_STAT, opacity
$FD00_1000-$FD00_107F	4 x $20	Bitmap layers BM0-BM3 (layer n at $FD00_1000 + n x $20)
$FD00_2000-$FD00_20FF	4 x $40	Tile layers TL0-TL3 (layer n at $FD00_2000 + n x $40)
$FD00_3000-$FD00_307F	128 B	Collision: COLL_ENABLE, COLL_ANY, COLL_WITH0-15
$FD00_4000-$FD00_401F	32 B	DAC player global registers
$FD00_4400-$FD00_47FF	32 x $20	DAC player voices 0-31 (voice v at $FD00_4400 + v x $20)
$FD02_0000-$FD02_0FFF	8 x $200	Sprite layers SP0-SP7 (layer n at $FD02_0000 + n x $200): control, image base, hit bits, 32 sprite attributes
$FD04_0000-$FD04_FFFF	16 x 4 KB	Palettes: source n at $FD04_0000 + n x $1000
rest of $FD00_0000-$FD0F_FFFF	-	Reserved: reads 0, writes ignored, no bus error

Sources (layers)

SOURCE	LAYER	REGISTERS	PALETTE BLOCK
0-3	Bitmap layers BM0-BM3	$FD00_1000 + n x $20	$FD04_0000 + n x $1000
4-7	Tile layers TL0-TL3	$FD00_2000 + n x $40	$FD04_4000 + n x $1000
8-15	Sprite layers SP0-SP7	$FD02_0000 + n x $200	$FD04_8000 + n x $1000

Display timing and the line engine

The GGE timing generator runs on the 25.000 MHz dot clock at all times, also while GGE_ENABLE is 0. A line is 800 dot clocks (32.0 us). Each line starts with the 640 visible pixels; each frame starts with the visible lines (line 0 is the first visible line). MODE_400 is taken at the end of a frame, so a mode change never gives a broken frame.

The line engine works one line ahead. At the start of line N the timing asks for line N+1; the engine fetches the source data of every enabled layer from DDR3 (arbiter port 0), composes 640 pixels (one per 200 MHz clock) and writes them into the free half of a 2-line output buffer, which the 25 MHz side reads while showing line N+1. A line has 6400 clocks of 200 MHz.

Frame start: the request for line 0 comes at the start of the last line of the previous frame (line 524 in 640 x 480, line 448 in 640 x 400). At that request the engine takes the whole layer configuration (GGE_CTRL, LAYER_ENABLE, LAYER_ORDER, BG_COLOR, OPACITY, COLL_ENABLE, all bitmap and tile layer registers, sprite CHUNKY and IMAGE_BASE) into its own copy, takes the sprite attributes of every layer whose COMMIT is set, and publishes the collision results of the frame that has just been composed. Register writes therefore never tear a frame: a value written before line 524 (448) starts is shown from the next frame on.

CHUNKY (per layer): the layer works in 320 x 240 (320 x 200) space. It fetches 320 pixels per line, shows each pixel twice, and its line counter advances every second display line (display line L shows layer line L / 2). Scroll values and sprite positions of a CHUNKY layer count in its own pixels and lines.

- DDR3 port 0 is shared with the GUI bitmap engine (GGE_Port0_Share): only one of them owns it. When GGE_ENABLE changes, the current owner gets no new command accepted, and the port changes owner as soon as no read command is in flight, so read data never reaches the wrong engine.
- Port 0 has the highest priority in the DDR3 arbiter and up to 15 reads in flight. When a line fetch has taken more than 3000 clocks (15 us), the line engine raises URGENT: the other ports are then stopped at their next packet boundary.
- LATE ($FD00_0020) counts lines whose request came while the previous line was still being built (only while the GGE is on and not blanked). It should stay unchanged during normal operation.
- When GGE_ENABLE is cleared, the engine starts no new fetch and drops a command that was not yet accepted (build 63).

GGE video modes (GGE_Timing)

PARAMETER	640 X 480 (MODE_400 = 0)	640 X 400 (MODE_400 = 1)
Dot clock	25.000 MHz	25.000 MHz
Pixels per line: visible / front porch / sync / back porch / total	640 / 16 / 96 / 48 / 800	640 / 16 / 96 / 48 / 800
Line frequency	31.25 kHz (32.0 us)	31.25 kHz (32.0 us)
Lines: visible / front porch / sync / back porch / total	480 / 10 / 2 / 33 / 525	400 / 12 / 2 / 35 / 449
Frame rate	59.52 Hz	69.60 Hz
Sync polarity	H- V-	H- V+
Line numbers (LINE register)	0-479 visible, 480-524 blanking	0-399 visible, 400-448 blanking

Mode switch, HDMI and interrupts

Setting GGE_ENABLE (GGE_CTRL bit 0) switches the whole video output, with no VICKY II bit involved: (1) the dot-clock BUFGMUX selects 25.000 MHz instead of 108 MHz (glitch-free); (2) the HDMI output registers take the GGE pixels, DE and syncs (the select is synchronised to the pixel clock through two flip-flops, build 62); (3) the I2C master re-programs the SiI9022 with the 640 x 480 or 640 x 400 table (a change of MODE_400 re-programs it too); (4) DDR3 port 0 passes to the GGE. Clearing GGE_ENABLE reverses all of it and the 108 MHz display returns.

BLANK (GGE_CTRL bit 2) keeps the GGE mode and timing but stops the line engine and outputs black pixels.

Interrupts (GAVIN interrupt controller, group 0 = level 6, autovector): while GGE_ENABLE is set, source `$00` (SOF) comes from the GGE: a pulse at the start of line 0 of every GGE frame (60 or 70 per second). Timer 3 (the frame counter) counts the same pulses. While GGE_ENABLE is 0, SOF comes from the 108 MHz video timing as before. Source `$01` (SOL) is the GGE line interrupt: a pulse at the start of the line programmed in LINE_IRQ, when LINE_IRQ bit 31 is set. Source `$15` (group 2, level 4, vector `$4D`) is the DAC player interrupt (see the DAC player).

- The 25 MHz timing runs at all times, so LINE_IRQ can raise SOL even while GGE_ENABLE is 0. Leave LINE_IRQ bit 31 at 0 when the GGE is not used.
- The SOF pulse of the GGE comes at the start of line 0, after the frame's configuration was taken (at line 524 / 448). Registers written in the SOF handler are shown one frame later. To change a frame tear-free with no delay, write in the vertical blanking: STATUS bit 16 = 1, or a LINE_IRQ at line 480 (400).

SiI9022 table entries for the GGE modes (the rest as in the 1280 x 960 table of the video chapter)

TPI REG	640 X 480	640 X 400	MEANING
$00 / $01	$D5 / $09	$D5 / $09	Pixel clock / 10 kHz = $09D5 = 2517 (25.17 MHz; the real dot clock is 25.000 MHz)
$02	$3C	$46	Vertical frequency 60 / 70 Hz
$04 / $05	$20 / $03	$20 / $03	Total pixels per line $0320 = 800
$06 / $07	$0D / $02	$C1 / $01	Total lines 525 / 449
$61	$03	$01	Sync polarity (read only register)
$62 / $63	$90 / $70	$90 / $50	DE generator on, DE delay $090 = 144 (sync + back porch), sync polarity bits
$64	$23	$25	DE top = 35 / 37 lines
$66 / $67	$80 / $02	$80 / $02	DE width 640
$68 / $69	$E0 / $01	$90 / $01	DE height 480 / 400

Master control registers `$FD00_0000`

Longword accesses only. Offsets `$28`-`$3C` and `$50`-`$FFC` read 0. All registers are cleared by the bus reset except where a reset value is given.

`$00` GGE_CTRL `RW · reset $0000_0000`

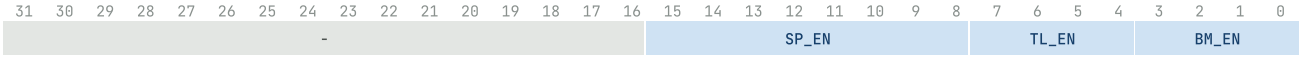
GGE on / off and mode. Bits 31:3 read 0.



BITS	FIELD	ACCESS	MEANING
31:3	**-**	R	Read 0.
2	**BLANK**	RW	1 = the line engine stops and the GGE outputs black (the GGE mode and timing stay).
1	**MODE_400**	RW	0 = 640 x 480 at 60 Hz, 1 = 640 x 400 at 70 Hz. Taken at the end of a frame; the SiI9022 is re-programmed.
0	**GGE_ENABLE**	RW	1 = the whole video output (dot clock, HDMI path, SiI9022 table, DDR3 port 0) switches to the GGE. 0 = the 108 MHz video system is shown.

`$04` LAYER_ENABLE `RW · reset $0000_0000`

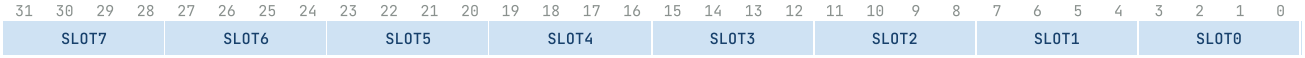
One bit per source. A layer that is off is not fetched, is transparent and takes no part in collisions. Taken at frame start.



BITS	FIELD	ACCESS	MEANING
31:16	**-**	R	Read 0.
15:8	**SP_EN**	RW	Sprite layers SP7-SP0 (bit 8 = SP0).
7:4	**TL_EN**	RW	Tile layers TL3-TL0 (bit 4 = TL0).
3:0	**BM_EN**	RW	Bitmap layers BM3-BM0 (bit 0 = BM0).

`$08` LAYER_ORDER_LO `RW · reset $15BA_0498`

Slots 0-7 of the 16-slot layer order. Each 4-bit field names the source shown in that slot; slot 0 is the top. The reset value gives slots 0-7 = SP0, SP1, TL0, BM0, SP2, SP3, TL1, BM1. Taken at frame start.

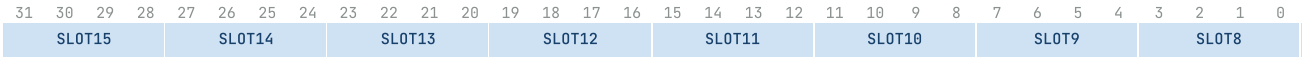


BITS	FIELD	ACCESS	MEANING
31:28	**SLOT7**	RW	Source of slot 7 (reset 1 = BM1).
27:24	**SLOT6**	RW	Source of slot 6 (reset 5 = TL1).
23:20	**SLOT5**	RW	Source of slot 5 (reset `$B` = SP3).
19:16	**SLOT4**	RW	Source of slot 4 (reset `$A` = SP2).
15:12	**SLOT3**	RW	Source of slot 3 (reset 0 = BM0).
11:8	**SLOT2**	RW	Source of slot 2 (reset 4 = TL0).
7:4	**SLOT1**	RW	Source of slot 1 (reset 9 = SP1).
3:0	**SLOT0**	RW	Source of slot 0, the top (reset 8 = SP0).

$0C LAYER_ORDER_HI

RW · reset $37FE_26DC

Slots 8-15 (slot 15 is the bottom, just above BG_COLOR). The reset value gives slots 8-15 = SP4, SP5, TL2, BM2, SP6, SP7, TL3, BM3.

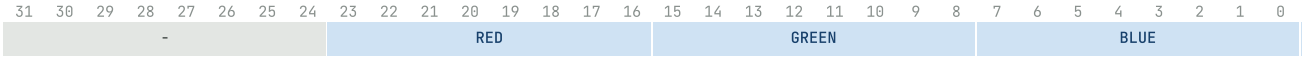


BITS	FIELD	ACCESS	MEANING
31:28	**SLOT15**	RW	Source of slot 15, the bottom (reset 3 = BM3).
27:24	**SLOT14**	RW	Source of slot 14 (reset 7 = TL3).
23:20	**SLOT13**	RW	Source of slot 13 (reset `$F` = SP7).
19:16	**SLOT12**	RW	Source of slot 12 (reset `$E` = SP6).
15:12	**SLOT11**	RW	Source of slot 11 (reset 2 = BM2).
11:8	**SLOT10**	RW	Source of slot 10 (reset 6 = TL2).
7:4	**SLOT9**	RW	Source of slot 9 (reset `$D` = SP5).
3:0	**SLOT8**	RW	Source of slot 8 (reset `$C` = SP4).

$10 BG_COLOR

RW · reset $0000_0000

Colour behind all layers (where every slot is transparent). Taken at frame start.



BITS	FIELD	ACCESS	MEANING
31:24	**-**	R	Read 0.
23:16	**RED**	RW	Red.
15:8	**GREEN**	RW	Green.
7:0	**BLUE**	RW	Blue.

$14 STATUS

R · reset -

Frame counter and vertical blank, from the 25 MHz timing (which always runs). Writes are ignored.



BITS	FIELD	ACCESS	MEANING
31:17	**-**	R	Read 0.
16	**VBLANK**	R	1 while the timing is in the vertical blanking (line 480-524, or 400-448).
15:0	**FRAME**	R	Frame counter: +1 at the start of line 0 of every frame (wraps). Counts while GGE_ENABLE is 0 too.

$18 LINE R · reset -

Current line of the GGE timing (0 = first visible line; blanking lines included).



BITS	FIELD	ACCESS	MEANING
31:10	-	R	Read 0.
9:0	**LINE**	R	0-524 (640 x 480) or 0-448 (640 x 400).

$1C LINE_IRQ RW · reset $0000_0000

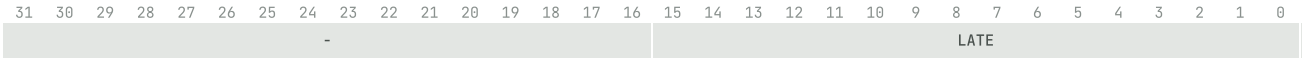
Line interrupt: a pulse on interrupt source `$01` (SOL) at the start of the line given in bits 9:0. Bits 30:10 are stored and read back but have no function.



BITS	FIELD	ACCESS	MEANING
31	**ENABLE**	RW	1 = the line interrupt is on. The timing always runs: leave it 0 when the GGE is not in use.
30:10	-	RW	Stored, no function.
9:0	**LINE**	RW	Line number, counted like the LINE register (blanking lines allowed).

$20 LATE R · reset -

Lines not built in time: a new line request came while the previous line was still being fetched or composed (only counted while the GGE is on and not blanked). Since power-up, wraps.



BITS	FIELD	ACCESS	MEANING
31:16	-	R	Read 0.
15:0	**LATE**	R	Count of late lines.

$24 SP_STAT R · reset -

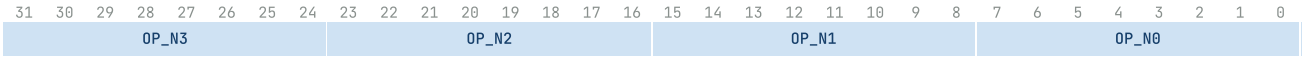
Sprite statistics (stage 4).



BITS	FIELD	ACCESS	MEANING
31:24	-	R	Read 0.
23:16	**SP_MAX**	R	Most sprites wanted on one line in the last frame (saturates at 255). Published at frame start.
15:0	**SP_DROPPED**	R	Lines that wanted more than 64 sprites (the extra sprites were not drawn). Since power-up, wraps.

$40 OPACITY0-3 RW · reset $FFFF_FFFF

Opacity per source, 8 bits each, 4 sources per register: OPACITY0 at `$40` holds sources 0-3 (source 0 in bits 7:0), OPACITY1 at `$44` sources 4-7, OPACITY2 at `$48` sources 8-11, OPACITY3 at `$4C` sources 12-15. The effective alpha of a pixel is palette alpha x (OPACITY + 1) / 256: 255 = as the palette says, 0 = invisible (the layer still takes part in collisions). Taken at frame start.



BITS	FIELD	ACCESS	MEANING
31:24	**OP_N3**	RW	Opacity of source 4n + 3.
23:16	**OP_N2**	RW	Opacity of source 4n + 2.
15:8	**OP_N1**	RW	Opacity of source 4n + 1.
7:0	**OP_N0**	RW	Opacity of source 4n (n = register number 0-3).

Layer order, palettes lookup and blending

For every pixel the compositor reads the 16 source line buffers. For each source: if the layer is on and its index is not 0, the index and the palette table give an ARGB entry of the layer's palette block; otherwise the source is transparent (alpha 0). The alpha is then scaled by the layer's opacity: a = A x (OPACITY + 1) / 256.

The slot multiplexer puts source LAYER_ORDER[s] into slot s. The blend chain starts with BG_COLOR and blends slot 15 (bottom) up to slot 0 (top). For each slot and each colour channel: c = c_below + (a' x (c_slot - c_below)) / 256, with a' = a + 1 when a >= 128, else a. So alpha 0 leaves the colour below unchanged and alpha 255 replaces it exactly.

Every slot can name any source. A source named in two slots is blended twice; a source that no slot names is not shown (it is still fetched if enabled and still takes part in collisions).

> The palette alpha is used as is: a palette entry with A = 0 is invisible even when its index is not 0, but it still counts as opaque for collisions (collision only looks at the index).

Reset layer order (top to bottom)

SLOTS	SOURCES
0-3	SP0, SP1, TL0, BM0
4-7	SP2, SP3, TL1, BM1
8-11	SP4, SP5, TL2, BM2
12-15	SP6, SP7, TL3, BM3 (bottom)

Bitmap layers `$FD00_1000`

Four bitmap layers (sources 0-3). Layer n has its registers at `$FD00_1000` + n x `$20` (offsets below are for layer 0). Offsets `$10` - `$1C` of each layer read 0. All registers are taken at frame start; the layer is switched on with LAYER_ENABLE.

A bitmap is one byte per pixel, left to right, in DDR3. For display line L the layer shows its line Y = L + SCROLL_Y (CHUNKY: L / 2 + SCROLL_Y). The engine reads 640 bytes (CHUNKY: 320) starting at ADDR + Y x STRIDE + SCROLL_X, one DDR3 command per line. There is no wrap-around: scrolling moves the window over the bitmap in memory, so the bitmap must be at least as large as the visible window plus the scroll range.

Any byte address works for ADDR, STRIDE and SCROLL_X: the engine starts the read at the 32-byte packet that holds the first byte and skips the bytes before it. A 32-byte aligned line start saves one packet per line.

$00 BM_CTRL RW · reset $0000_0000

Layer control. Bits 31:3 are stored and read back but have no function.



BITS	FIELD	ACCESS	MEANING
31:3	-	RW	Stored, no function.
2	**CHUNKY**	RW	1 = 320-pixel layer: each pixel and each line shown twice.
1:0	**TABLE**	RW	Palette table (0-3) of the layer's palette block.

$04 BM_ADDR RW · reset $0000_0000

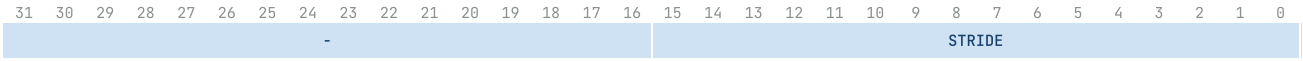
DDR3 byte address of the bitmap's line 0, pixel 0. Bits 29:0 are used; bits 31:30 are stored and ignored.



BITS	FIELD	ACCESS	MEANING
31:30	-	RW	Ignored.
29:0	**ADDR**	RW	DDR3 byte address.

$08 BM_STRIDE RW · reset $0000_0000

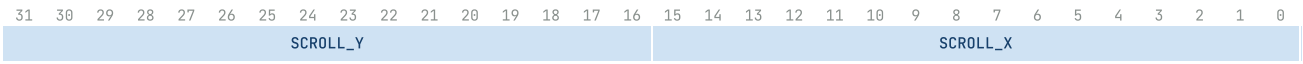
Bytes from one bitmap line to the next. Bits 15:0 are used.



BITS	FIELD	ACCESS	MEANING
31:16	-	RW	Ignored.
15:0	**STRIDE**	RW	Line pitch in bytes. 0 = natural: 640, or 320 in CHUNKY.

$0C BM_SCROLL RW · reset $0000_0000

Scroll offsets in layer pixels and lines (unsigned, added to the line and to the byte address).



BITS	FIELD	ACCESS	MEANING
31:16	**SCROLL_Y**	RW	First bitmap line shown.
15:0	**SCROLL_X**	RW	First pixel shown in each line.

Tile layers $FD00_2000

Four tile layers (sources 4-7). Layer n has its registers at `$FD00_2000` + n x `$40` (offsets below are for layer 0); offsets `$1C` - `$3C` read 0. All registers are taken at frame start. A tile layer is a map of 16-bit entries that point into a set of tiles of 16 x 16 or 32 x 32 pixels, 8 bpp, all in DDR3.

Map: the map is MAP_W x MAP_H entries (32, 64, 128 or 256 each way), stored row by row: entry (column, row) is at MAP_ADDR + (row x MAP_W + column) x 2. Each entry is a big-endian 16-bit word (table below). The map wraps in both directions: scrolling past its right or bottom edge continues at the other side.

Tiles: tile t is stored whole, row by row: at TILES_ADDR + t x 256 for 16 x 16 tiles (16 rows of 16 bytes), or TILES_ADDR + t x 1024 for 32 x 32 tiles (32 rows of 32 bytes). Up to 1024 tiles per layer.

Per line the engine reads (LSCROLL only) one packet with the line's scroll value, then the whole map row that covers the line (MAP_W entries: 2 to 16 packets), then one packet per visible tile (41 for 16 x 16 tiles at 640 pixels), with up to 12 tile reads in flight. MIRROR reverses the pixels of the tile row, INVERT reads row (size - 1 - r) instead of row r.

Line scroll (LSCROLL = 1): a table of signed 16-bit big-endian X offsets, one per layer line, at LSCROLL_ADDR + line x 2 (line = display line, or display line / 2 in CHUNKY). The value is added to SCROLL_X for that line only (raster effects).

Map entry (16 bits, big-endian)

BITS	FIELD	MEANING
15:14	-	Reserved, write 0.
13:12	TABLE	Palette table (0-3) of the layer's palette block for this tile.
11	INVERT	1 = flip the tile vertically.
10	MIRROR	1 = flip the tile horizontally.
9:0	TILE	Tile number 0-1023.

$00 TL_CTRL

RW · reset $0000_0000

Layer control. Bits 31:4 are stored and read back but have no function.

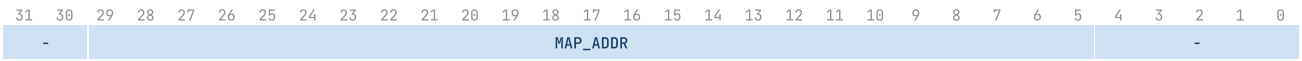


BITS	FIELD	ACCESS	MEANING
31:4	-	RW	Stored, no function.
3	**COLLIDE_ONLY**	RW	1 = the layer is fetched and takes part in collisions but is not shown (transparent). Its LAYER_ENABLE bit must be set.
2	**LSCROLL**	RW	1 = use the line scroll table at TL_LSCROLL_ADDR.
1	**CHUNKY**	RW	1 = 320-pixel layer: each pixel and each line shown twice.
0	**TILE32**	RW	0 = 16 x 16 tiles, 1 = 32 x 32 tiles.

$04 TL_MAP_ADDR

RW · reset $0000_0000

DDR3 byte address of the map. Bits 29:5 are used: the map must be 32-byte aligned.



BITS	FIELD	ACCESS	MEANING
31:30	-	RW	Ignored.
29:5	**MAP_ADDR**	RW	Map address in 32-byte units.
4:0	-	RW	Ignored (treated as 0).

$08 TL_MAP_SIZE

RW · reset $0000_0000

Map size: 32 << code entries in each direction. Other bits are stored and ignored.



BITS	FIELD	ACCESS	MEANING
31:6	-	RW	Ignored.
5:4	**HEIGHT**	RW	Map height: 0 = 32, 1 = 64, 2 = 128, 3 = 256 entries.
3:2	-	RW	Ignored.
1:0	**WIDTH**	RW	Map width: 0 = 32, 1 = 64, 2 = 128, 3 = 256 entries.

$0C TL_TILES_ADDR

RW · reset $0000_0000

DDR3 byte address of tile 0. Bits 29:5 are used: 32-byte aligned.

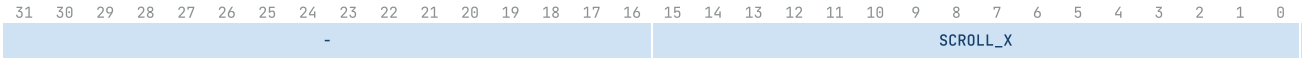


BITS	FIELD	ACCESS	MEANING
31:30	-	RW	Ignored.
29:5	**TILES_ADDR**	RW	Tile set address in 32-byte units.
4:0	-	RW	Ignored (treated as 0).

$10 TL_SCROLL_X

RW · reset $0000_0000

Horizontal scroll in layer pixels. Wraps at the map width (MAP_W x tile size).

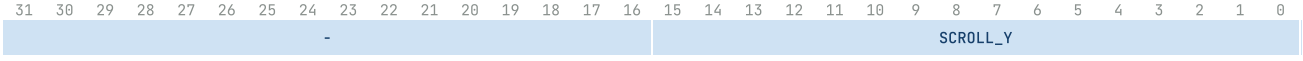


BITS	FIELD	ACCESS	MEANING
31:16	-	RW	Ignored.
15:0	**SCROLL_X**	RW	Map pixel shown at the left edge.

$14 TL_SCROLL_Y

RW · reset $0000_0000

Vertical scroll in layer lines. Wraps at the map height.



BITS	FIELD	ACCESS	MEANING
31:16	-	RW	Ignored.
15:0	**SCROLL_Y**	RW	Map line shown at the top.

$18 TL_LSCROLL_ADDR RW · reset $0000_0000

DDR3 byte address of the line scroll table (used when LSCROLL = 1). Bits 29:1 are used: 2-byte aligned.



BITS	FIELD	ACCESS	MEANING
31:30	-	RW	Ignored.
29:1	**LSCROLL_ADDR**	RW	Table address in 16-bit units.
0	-	RW	Ignored.

Sprite layers $FD02_0000

Eight sprite layers (sources 8-15) of 32 sprites each, 256 sprites in all. Layer n has its registers at $FD02_0000 + n x $200 (offsets below are for layer 0). Offsets $00C - $0FC read 0. A sprite is 16 x 16 pixels, 8 bpp; image i of a layer is 256 bytes (16 rows of 16 bytes) at IMAGE_BASE + i x 256, up to 1024 images per layer. Layers may share one IMAGE_BASE.

Double buffer: the CPU writes the attributes into the 'next' bank ($100 - $1FF ; a read returns what was written). The engine draws from its own 'active' bank. Writing SP_CTRL with COMMIT = 1 asks for the next bank of that layer to be copied into the active bank at the next frame start (the request for line 0); COMMIT then reads 0. Do not write the layer's attributes while COMMIT reads 1. A COMMIT written while one is pending is ignored. Commits are taken at the GGE frame start even while GGE_ENABLE is 0, because the GGE timing always runs.

Per line the engine scans all 256 active attributes (SP0 sprite 0 first, SP7 sprite 31 last). A sprite is on the line when its layer is on, its ENABLE bit is set, 0 <= line - Y < 16 (CHUNKY: line / 2 - Y) and -15 <= X <= 639 (CHUNKY: 319). The first 64 sprites found are drawn; the others are dropped and SP_STAT counts the line, so the back sprites of the last layers go first. Each drawn sprite costs one DDR3 packet.

Drawing order: inside a layer, sprite 0 is on top of sprite 31 (the engine draws from the last sprite found back to the first). Between layers, LAYER_ORDER decides. Pixels with index 0 are transparent; parts outside the line are clipped.

$000 SP_CTRL RW · reset $0000_0000

Layer control and commit. Bits 31:2 read 0.



BITS	FIELD	ACCESS	MEANING
31:2	-	R	Read 0.
1	**COMMIT**	RW	Write 1 = take the next attribute bank at the next frame start. Reads 1 until it has been taken. Write 0 has no effect.
0	**CHUNKY**	RW	1 = the layer works in 320 x 240 (320 x 200) space: X and Y count in double pixels and lines. Taken at frame start.

$004 SP_IMAGE_BASE RW · reset $0000_0000

DDR3 byte address of image 0. Bits 29:5 are used: 32-byte aligned. Taken at frame start.



BITS	FIELD	ACCESS	MEANING
31:30	-	RW	Ignored.
29:5	IMAGE_BASE	RW	Image base in 32-byte units.
4:0	-	RW	Ignored (treated as 0).

$008 SP_HIT R · reset $0000_0000

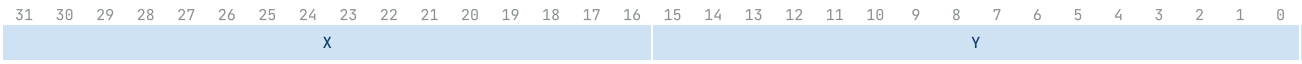
Per-sprite hit bits of the last complete frame (see Collision detection).



BITS	FIELD	ACCESS	MEANING
31:0	HIT	R	Bit s = 1: sprite s of this layer had an opaque pixel at the same place as an opaque pixel of another source.

$100 SP_ATTR_XY RW · reset -

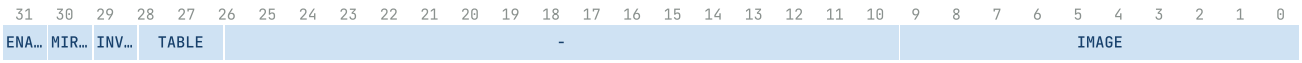
Sprite s word 0, at `$100` + s x 8 (next bank). Not cleared by reset.



BITS	FIELD	ACCESS	MEANING
31:16	X	RW	Left pixel, signed (-15 to 639 is at least partly visible).
15:0	Y	RW	Top line, signed.

$104 SP_ATTR_CTRL RW · reset -

Sprite s word 1, at `$104` + s x 8 (next bank). Not cleared by reset.



BITS	FIELD	ACCESS	MEANING
31	ENABLE	RW	1 = the sprite is shown.
30	MIRROR	RW	1 = flip horizontally.
29	INVERT	RW	1 = flip vertically.
28:27	TABLE	RW	Palette table (0-3) of the layer's palette block.
26:10	-	RW	Reserved, write 0.
9:0	IMAGE	RW	Image number 0-1023.

Collision detection $FD00_3000

The compositor checks every displayed pixel. A source is opaque at a pixel when its layer is on (LAYER_ENABLE at frame start, GGE on and not blanked), it takes part (COLL_ENABLE) and its index there is not 0. Palette alpha and opacity do not matter. Every pair of

opaque sources at the same pixel sets bit j of COLL_WITH i and bit i of COLL_WITH j. An opaque sprite pixel that meets any other opaque source sets the hit bit of that sprite in its layer's SP_HIT register.

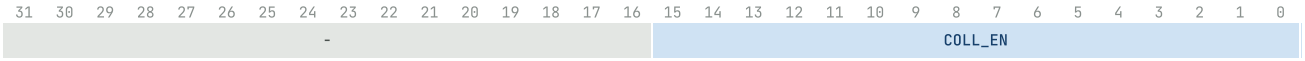
The results are collected over a whole frame and published at the next frame start (the request for line 0, at line 524 / 448); they then stay stable for one frame. Read them after SOF. A COLLIDE_ONLY tile layer takes part like a visible layer.

Two sprites of the same layer that overlap are not detected (they are one source, and the line buffer keeps only the top sprite's pixel). Put sprites that must detect each other in different layers. Only the visible area is checked.

$00 COLL_ENABLE

RW · reset $0000_0000

Sources that take part in collision detection (same numbering as LAYER_ENABLE). Taken at frame start.



BITS	FIELD	ACCESS	MEANING
31:16	-	R	Read 0.
15:0	COLL_EN	RW	Bit i = 1: source i takes part.

$14 COLL_ANY

R · reset $0000_0000

Last complete frame: bit i = source i touched any other source (the OR of COLL_WITH i). Offsets $04 - $10 and $18 - $3C read 0.



BITS	FIELD	ACCESS	MEANING
31:16	-	R	Read 0.
15:0	ANY	R	One bit per source.

$40 COLL_WITH0-15

R · reset $0000_0000

COLL_WITH i at $40 + 4 x i (i = 0-15). Last complete frame: bit j = an opaque pixel of source i was at the same place as an opaque pixel of source j. Symmetric; bit i of COLL_WITH i is always 0.



BITS	FIELD	ACCESS	MEANING
31:16	-	R	Read 0.
15:0	WITH	R	One bit per other source.

Palettes $FD04_0000

Each source has its own palette block of 4 KB: 4 tables of 256 entries of 32 bits. Block of source n at $FD04_0000 + n x $1000; table t at + t x $400; entry e at + e x 4. Longword accesses.

The palettes are dual-port block RAMs: the CPU can read and write them at any time, and a change shows from the next pixel that uses the entry (palettes are not latched per frame). Entry 0 of every table is never shown: index 0 is transparent. The palettes are 0 (transparent black) after the FPGA is configured; a reset does not clear them.

Palette blocks

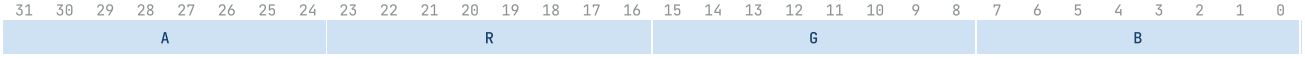
ADDRESS	SOURCE
$FD04_0000 / 1000 / 2000 / 3000	BM0 / BM1 / BM2 / BM3
$FD04_4000 / 5000 / 6000 / 7000	TL0 / TL1 / TL2 / TL3

ADDRESS	SOURCE
$FD04_8000 / 9000 / A000 / B000	SP0 / SP1 / SP2 / SP3
$FD04_C000 / D000 / E000 / F000	SP4 / SP5 / SP6 / SP7

$0000 PALETTE entry

RW · reset $0000_0000 (at configuration)

ARGB8888 entry. Address = $FD04_0000 + source x $1000 + table x $400 + index x 4.



BITS	FIELD	ACCESS	MEANING
31:24	**A**	RW	Alpha: 0 = transparent, 255 = opaque (scaled by the layer's opacity).
23:16	**R**	RW	Red.
15:8	**G**	RW	Green.
7:0	**B**	RW	Blue.

DAC player: global registers $FD00_4000

The DAC player (build 66) plays up to 32 voices from DDR3. It has two outputs (build 69): the WM8776 codec at 48 kHz and the AK4432 DAC at 44.1 kHz; each voice plays on one of them (V_CTRL bit 8 OUTPUT). Every sample of an output (a tick of its clock / 512: 24.576 MHz for the WM8776, 22.5792 MHz for the AK4432) starts a pass over the 32 voices on the DDR3 clock (200 MHz, about 4100 clocks per 48 kHz sample; a pass takes about 1000-2500). A pass builds one output: the voices of the other output are skipped, and their PLAY / STOP / seek wait for their own output's pass. A tick that arrives while the other output's pass runs waits for it to end.

Output: the voices are summed in 32-bit accumulators, the test tone is added, the sum is multiplied by MASTER_VOL / 128 (WM8776) or MASTER_VOL2 / 128 (AK4432) and clamped to 16 bits (CLIP). The WM8776 result is the fifth source of BEATRIX's 48 kHz mix (with the PSG, SID, OPN2 and OPM). The AK4432 result goes straight to the AK4432 (I2S, 16 bits, BCLK 1.4112 MHz, LRCK 44.1 kHz, MCLK 22.5792 MHz = 512 fs); it replaced the CPU-fed PCM FIFO in build 69.

Longword accesses. Offsets $01C - $3FC read 0. The voice registers ($400 - $7FF) are block RAM and are not cleared by reset (0 after configuration).

$00 AUD_CTRL

RW · reset $0000_0000

Player control. Bits 31:2 read 0.



BITS	FIELD	ACCESS	MEANING
31:2	**-**	R	Read 0.
1	**TEST_TONE**	RW	1 = add a sine at -12 dBFS (amplitude 8192) to both channels of both outputs, also while ENABLE = 0: 1 kHz on the WM8776, 919 Hz on the AK4432 (the same 48-step table).
0	**ENABLE**	RW	1 = the voices are processed. 0 = no voice is processed (they keep their state and position, and a PLAY stays pending); the output is silence or the test tone.

$04 AUD_STATUS R/W1C · reset $0000_0000

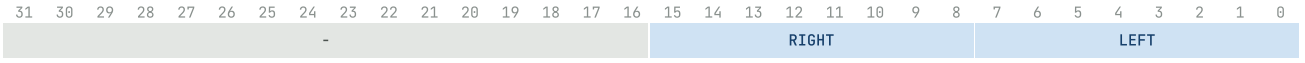
Sticky error flags; write 1 to clear.



BITS	FIELD	ACCESS	MEANING
31:2	-	R	Read 0.
1	**STARVE**	W1C	A voice needed a frame that was not in its prefetch buffer: it played silence for that sample and did not advance.
0	**CLIP**	W1C	The output was clamped to 16 bits.

$08 MASTER_VOL RW · reset $0000_8080

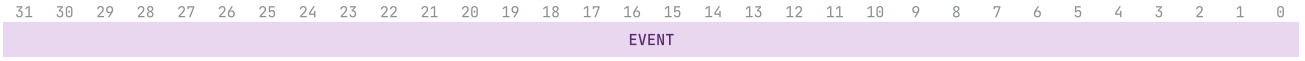
Master volume of the WM8776 output, `$80` = unity (x 1.0), `$FF` = x 1.99.



BITS	FIELD	ACCESS	MEANING
31:16	-	R	Read 0.
15:8	**RIGHT**	RW	Right channel.
7:0	**LEFT**	RW	Left channel.

$0C EVENT R/W1C · reset $0000_0000

Voice events, sticky; write 1 to clear. Bit v is set when voice v ends (end of a one-shot, end of the STOP ramp, LEN = 0), loops (wraps to its loop start, or to frame 0 of a STREAM ring; at most one event per sample) or, in STREAM mode, crosses the middle of its ring.



BITS	FIELD	ACCESS	MEANING
31:0	**EVENT**	W1C	One bit per voice.

$10 EVENT_IRQ_EN RW · reset $0000_0000

Interrupt enable per voice. Interrupt source `$15` is held active while (EVENT AND EVENT_IRQ_EN) is not 0.



BITS	FIELD	ACCESS	MEANING
31:0	**IRQ_EN**	RW	One bit per voice.

$14 PLAYING R · reset $0000_0000

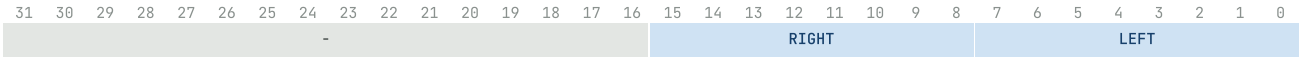
Voices that are playing, or whose PLAY has been written but not yet taken.



BITS	FIELD	ACCESS	MEANING
31:0	**PLAYING**	R	One bit per voice.

$18 MASTER_VOL2 RW · reset $0000_8080

Master volume of the AK4432 output (build 69), `$80` = unity (x 1.0), `$FF` = x 1.99.



BITS	FIELD	ACCESS	MEANING
31:16	**-**	R	Read 0.
15:8	**RIGHT**	RW	Right channel.
7:0	**LEFT**	RW	Left channel.

DAC player: voices $FD00_4400

Voice v has its registers at `$FD00_4400` + v x `$20` (offsets below are for voice 0). A frame is one sample of a mono voice, or one left + right pair of a stereo voice. Positions, lengths and loop points count in frames.

Formats: PCM8 = signed 8-bit; PCM16 = signed 16-bit, little-endian (low byte first in memory, as in WAV files; the 68060 writes words big-endian, so swap the bytes); IMA ADPCM = 4 bits per sample, low nibble first, mono only, a raw stream with no headers that starts with predictor 0 and step index 0. STEREO (even voice, PCM only): frames hold the left sample, then the right sample; the next odd voice is not played and its registers are ignored. A mono voice sends the same sample to both sides, scaled by its left and right volume (this is also the pan).

Pitch: the position advances by STEP (16.16 fixed point, frames per output sample) every sample of the voice's output: 1.0 plays at the output's rate (48 kHz on the WM8776, 44.1 kHz on the AK4432); on the WM8776, 0.91875 (`$0000_EB33`) plays 44.1 kHz material. Between frames n and n + 1 the sample is interpolated linearly with the 16-bit fraction (NEAREST turns this off). The result is multiplied by the voice's current volume / 128.

Volume ramp: the current volume moves 1 step per sample toward V_VOL, on each side (128 samples from 0 to `$80`: about 2.7 ms on the WM8776, 2.9 ms on the AK4432). PLAY starts at volume 0; STOP ramps to 0 and then ends the voice. So starts, stops and volume changes do not click.

Commands: PLAY, STOP and a seek (a write to V_POS) are taken at the voice's turn in the next sample. After PLAY the voice is silent in that sample and the next two (its first blocks are read) and plays frame 0 from the third. A seek is the same (silent for 3 samples, then on from the new frame, volume unchanged); an ADPCM voice restarts its decoder at the new frame with predictor 0 and index 0. A second PLAY, STOP or seek written before the first one is taken is ignored. A PLAY also cancels a pending STOP or seek.

End and loop: when the position reaches LEN, a one-shot voice ends (V_POS = LEN, EVENT bit set). With LOOP (and LOOP < LEN), the position wraps to LOOP plus the overshoot and the EVENT bit is set. An ADPCM voice keeps the decoder state it had before the loop start's nibble and restores it at each wrap.

Prefetch: each voice has 4 slots of 128 bytes. The player keeps the block of the current frame and the next block present, and near the end the block of the loop start. Blocks are read from DDR3 through arbiter port 4 (one 4-packet read per block, reads only). A frame that is missing when it is needed plays silence, does not advance and sets STARVE; this only happens at a loop wrap that jumps past the loop start's block (very high STEP with a short loop) or when DDR3 is overloaded.

$00 V_CTRL RW · reset $0000_0000

Commands and format. Bits 8:2 read back as written; bits 31:9 and bit 1 read 0. Change FORMAT, STEREO and OUTPUT only while the voice is stopped.



BITS	FIELD	ACCESS	MEANING
31:9	**-**	R	Read 0.
8	**OUTPUT**	RW	0 = WM8776 codec (48 kHz), 1 = AK4432 DAC (44.1 kHz, build 69). V_STEP 1.0 is the sample rate of this output. A stereo pair plays on the output of its even voice; its odd voice is not played on either output.
7	**STREAM**	RW	1 = endless ring of LEN frames starting at frame 0 (LOOP is ignored), with an event at each half (see Streaming).
6	**NEAREST**	RW	1 = no interpolation (plays the sample of frame n).
5	**LOOP**	RW	1 = at LEN, continue at frame V_LOOP.
4	**STEREO**	RW	1 = interleaved left / right frames (even voice and PCM only; the odd voice above it is not played).
3:2	**FORMAT**	RW	0 = PCM8, 1 = PCM16 little-endian, 2 = IMA ADPCM (mono), 3 = reserved.
1	**STOP**	W	Write 1 = ramp the volume down and end. Reads 0.
0	**PLAY**	RW	Write 1 = start at frame 0 (also restarts a playing voice). Reads 1 while the voice plays or its PLAY is pending.

$04 V_ADDR RW · reset -

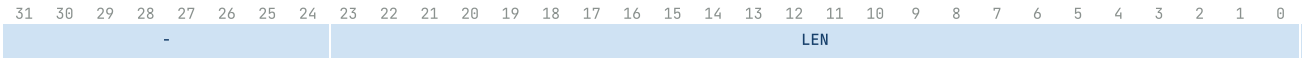
DDR3 byte address of frame 0, bits 29:0. Align it to the frame size (1, 2 or 4 bytes): a frame must not cross a 16-byte boundary.



BITS	FIELD	ACCESS	MEANING
31:0	**ADDR**	RW	Sample data address (bits 31:30 ignored).

$08 V_LEN RW · reset -

Length in frames (STREAM: the ring size). 0 = the voice ends at once (with an event).



BITS	FIELD	ACCESS	MEANING
31:24	**-**	RW	Ignored.
23:0	**LEN**	RW	Frames.

$0C V_LOOP RW · reset -

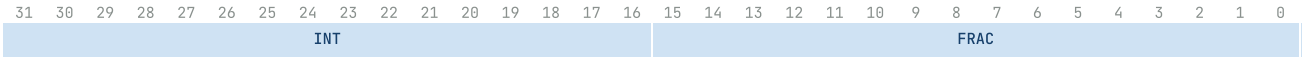
Loop start frame (used with LOOP when it is below LEN; ignored with STREAM).



BITS	FIELD	ACCESS	MEANING
31:24	-	RW	Ignored.
23:0	LOOP	RW	Frame number.

$10 V_STEP RW · reset -

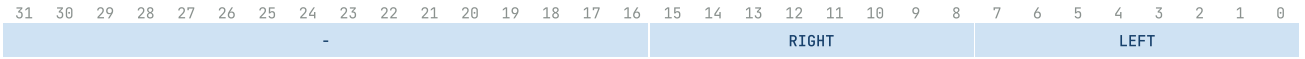
Pitch: frames per output sample, 16.16 fixed point (`$0001_0000` = 1.0 = the voice's output rate: 48 kHz on the WM8776, 44.1 kHz on the AK4432). Values above 16.0 play as 16.0; for ADPCM above 8.0 as 8.0, so that 32 ADPCM voices always fit in one sample period.



BITS	FIELD	ACCESS	MEANING
31:16	INT	RW	Integer part.
15:0	FRAC	RW	Fraction.

$14 V_VOL RW · reset -

Target volume per side, `$80` = unity, `$FF` = x 1.99. The current volume ramps to it at 1 step per sample.



BITS	FIELD	ACCESS	MEANING
31:16	-	RW	Ignored.
15:8	RIGHT	RW	Right volume.
7:0	LEFT	RW	Left volume.

$18 V_POS RW · reset -

Read: the current frame (bits 31:24 read 0). Write: seek to that frame (taken like a PLAY, see above).



BITS	FIELD	ACCESS	MEANING
31:24	-	R	Read 0.
23:0	POS	RW	Frame number.

$1C - R · reset -

Reserved: reads 0. (A write pointer register was planned; it is not needed, see Streaming.)

Programming notes

Access: the GGE block takes longword accesses only; byte and word writes are ignored. Registers live on the CPU bus clock and are copied into the 200 MHz engine with stable-for-2-clocks captures, so a value is never seen half-written.

Tear-free updates: the line engine takes its configuration once per frame, at the start of line 524 (640 x 480) or 448 (640 x 400). Write scroll values, addresses, the order, opacities and enables during the vertical blanking (STATUS bit 16 = 1, lines 480-523 / 400-447), or from a LINE_IRQ at line 480 / 400, and they are shown in the next frame. Palette entries are not latched: change them in the blanking if a change in mid-picture must be avoided.

Sprite commit protocol, per layer: (1) wait until SP_CTRL bit 1 (COMMIT) reads 0; (2) write the attributes of the next frame into the next bank (`$100` - `$1FF`); (3) write SP_CTRL = COMMIT | CHUNKY. The new attributes are taken at the next frame start, all 32 sprites of the layer together.

Streaming protocol (DAC player, STREAM = 1): put a ring of LEN frames at V_ADDR (LEN even). Fill the whole ring, set EVENT_IRQ_EN for the voice, then PLAY. The voice raises its EVENT bit when it crosses the middle of the ring (the first half has been played) and when it wraps to frame 0 (the second half has been played). At each event, clear the EVENT bit and refill the half that was just played. V_POS shows which half is playing. The player reads about one 128-byte block ahead of the frame it plays, and every block of the ring is read again on each lap (build 67; build 66 kept an old copy of the ring's first block from the third lap on). Sound of any length can be streamed or decoded this way.

Interrupt `$15` is a level held while (EVENT AND EVENT_IRQ_EN) is not 0. In the controller's reset (edge) mode, clear the EVENT bits in the handler before clearing the pending bit, so that the next event gives a new edge.

- Set-up order: put the graphics, maps and sprite images into DDR3 and push the 68060 data cache; write the palettes; write the layer registers, LAYER_ORDER (or keep the reset order), OPACITY, BG_COLOR and LAYER_ENABLE; write the sprite attributes and set COMMIT in each sprite layer; then write GGE_CTRL = 1 (640 x 480) or 3 (640 x 400). The monitor resynchronises. GGE_CTRL = 0 returns to the 108 MHz display.
- Check LATE and SP_STAT: LATE should not count during normal operation, SP_STAT bits 15:0 count lines that wanted more than 64 sprites.
- Mouse pointer: the hardware pointer of VICKY II is not shown in GGE mode. Use a sprite (for example SP0 sprite 0, on top by default).
- MCP tests (FPGA builds 60-70): TEST GGEPAT [OFF | 400 | QUAD | TILE | SPRITE | BALLS [s] | SCENE [s] | PLAY [s] | COLL] (MCP 146-155) shows the GGE test picture on BM0 (400 = 640 x 400), the same picture built from 4 bitmap layers (QUAD), from a tile layer (TILE) or with a sprite band (SPRITE), a sprite demo (BALLS), a 4-layer parallax tile scene loaded at `$10_0000` (SCENE), the scene with 64 bouncing sprites (PLAY), and a collision check with known results (COLL). The pictures decode bit-exactly from an HDMI capture. TEST GGEAUD (MCP 156) checks the DAC player (register read-back, a PCM16 one-shot, a looped PCM8 voice, STOP) and plays a demo with an IMA ADPCM voice. TEST GGEAUD AK (MCP 158, build 69) runs the same on the AK4432 output, with the steps scaled by 48000 / 44100 so the notes keep their pitch, and also checks MASTER_VOL2 and the V_CTRL bit 8 read-back. TEST GGEAUD RATE (MCP 159) measures the sample rates of both outputs (AK4432 / WM8776 = 0.91875). The old GGE_BM / GGE_TM tests were removed in MCP 149.

7 PCI Express root port

Since build 74 the FPGA holds a PCI Express root port: the AMD/Xilinx "AXI Bridge for PCI Express" (axi_pcie 2.9, documented in PG055) in root-port mode, configured for Gen2 (5 GT/s) x4. The CPU reaches the cards through two windows, one for configuration space (ECAM) and one for memory space; the cards reach DDR3 by bus-master DMA through DDR3 arbiter port 5; the core's interrupt output drives GAVIN interrupt source `$23`.

It was tested through a PLX PEX8747 switch with a Renesas uPD720201 USB 3 card and a JMicron JMB585 SATA card; an mSATA drive on the JMB585 was read and written through the MCP's FatFs (volume /sata0). The MCP 164 sources are a complete working example: `dev/PCIe_A2560Me.c` (windows, ECAM, enumeration) and `dev/ahci.c` (an AHCI driver with DMA).

This chapter describes FPGA build 87 and MCP 176; the root port was brought up on build 74 and MCP 164. For everything about the core itself (all registers, the MSI address registers and the MSI FIFO), see PG055.

Overview

What is in the FPGA

ITEM	BUILD 87
Core	AXI Bridge for PCI Express (axi_pcie 2.9, PG055), root-port mode
Link	Gen2 (5 GT/s), x4. On the test setup the link trains x2 (see Known limits).
Root port ID	10EE:7118 (bus 0, device 0, function 0)
Configuration window	`$FA00_0000` - `$FA7F_FFFF`: buses 0-7 and the core's own registers; `$FA80_0000` - `$FAFF_FFFF` mirrors it
Memory window	`$4000_0000` - `$7FFF_FFFF`: PCIe memory space at the same addresses
DMA	Card memory reads and writes reach DDR3 `$0080_0000` - `$3FFF_FFFF` at the CPU address, through DDR3 arbiter port 5, with the L2 cache kept coherent
Interrupt	GAVIN source `$23`: group 4, bit 3, CPU level 2, vector `$5B`; active low, a level while the core's interrupt is pending
Byte order	Address invariant on both windows and on the DMA path: a byte keeps its address
I/O space	None: the core cannot send PCI I/O transactions

CPU windows

The CPU sees two windows (they replace the window modules of the earlier builds). The FPGA asserts TCI on every access to them, so the 68060 never caches them, whatever the MMU says.

- Use byte, word and long accesses: a 32-bit access must be 4-byte aligned, a 16-bit access 2-byte aligned. No MOVE16 into the windows.
- Memory accesses from the CPU are sent only after the root port's bridge enable (core register `$148` bit 0) is set; set it after enumeration.
- Card BARs are placed in `$4000_0000` - `$7FFF_FFFF`. There is no PCI I/O space (see Configuration space).
- To find out whether the root port is there, read the vendor ID at bus 0, device 0: it reads `$10EE`. A build without PCIe returns a fixed pattern in this window.

Windows

WINDOW	CPU ADDRESSES	WHAT IT REACHES
Configuration (ECAM)	$FA00_0000 - $FA7F_FFFF	Configuration space of buses 0-7, plus the core's own registers (in the root port's configuration space). $FA80_0000 - $FAFF_FFFF mirrors it.
Memory	$4000_0000 - $7FFF_FFFF	PCIe memory space, same addresses (CPU $4010_0000 = PCIe $4010_0000)

Byte order

Both windows and the DMA path are address invariant: a byte keeps its address. The 68060 is big-endian and PCIe is little-endian, so a byte access needs nothing, but the 16- and 32-bit registers and in-memory structures of a card are little-endian and software swaps them.

This applies to configuration registers, card registers (BARs) and descriptors and rings in DDR3 that a card reads or writes. Data buffers need no swap: a disk sector or a network frame lands byte for byte at its address.

In NetBSD terms the bus_space tags for both windows are little-endian with byte-swapping 16/32-bit accessors (the le stream variants are plain copies). Example from the MCP: pcie_cfg_read32() reads the longword at the ECAM address and swaps it.

Configuration space (ECAM) $FA00_0000

Address of a configuration register: $FA00_0000 + (bus << 20) + (device << 15) + (function << 12) + register.

Bus 0, device 0 is the root port itself: a type 1 (PCI-to-PCI bridge) header. Program it like any bridge: primary 0, secondary 1, subordinate, memory base / limit, command = memory + bus master. The core's own registers are in the root port's configuration space, after the header; they are listed in PG055, "Register Space". The ones the MCP uses are below, at their bus 0, device 0 configuration offsets (CPU address $FA00_0000 + offset).

- Byte, word and long accesses all work.
- Only buses 0-7 fit in the window. Assign bus numbers from 1 upwards and keep subordinate bus numbers <= 7. The PEX8747 test setup uses buses 1-6.
- No answer: an access that the core does not complete within about 2 ms ends normally and returns $FFFF_FFFF. There is no bus error and no hang. This is also what an empty device slot reads.
- No PCI I/O space: the core cannot send I/O transactions. I/O BARs can be sized, but the card cannot be reached through them. Cards that only work through I/O BARs (the CH382 serial card that was tried) cannot be used. The JMB585 has 5 I/O BARs, but its AHCI registers are in memory BAR5, so it works.

> Bit numbers below are those of the little-endian register value, that is after the software byte swap (as pcie_cfg_read32() returns it). Reset values not given here are in PG055.

$130 PCIE_BRIDGE_INFO

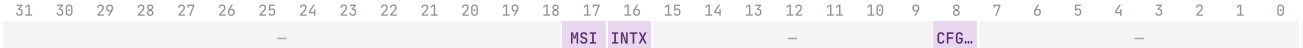
R · reset see PG055

Bridge Info register of the core. The MCP reads it; its fields are listed in PG055.

$138 PCIE_INT_DECODE

RW1C · reset see PG055

Interrupt Decode: the core's pending interrupt causes. Write 1 to a bit to clear it. The error bits not listed here are as in PG055.



BITS	FIELD	ACCESS	MEANING
17	**MSI**	W1C	1 = an MSI write was received.
16	**INTX**	W1C	1 = an INTx message was received. After handling the card, write 1 here.
8	**CFG_TIMEOUT**	W1C	1 = a configuration access timed out.

$13C PCIE_INT_MASK RW · reset $0000_0000

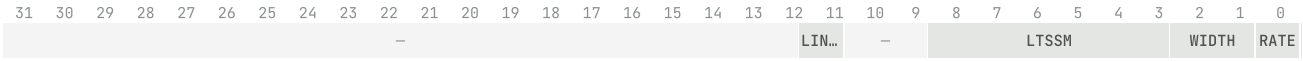
Interrupt Mask: chooses which Interrupt Decode causes drive the core's interrupt output (GAVIN source `$23`). It is 0 after reset, so nothing reaches source `$23` until bits are set here. Other bits: see PG055.



BITS	FIELD	ACCESS	MEANING
16	**INTX**	RW	1 = INTx messages reach the interrupt output (the setting for INTx).

$144 PCIE_PHY_STATUS R · reset -

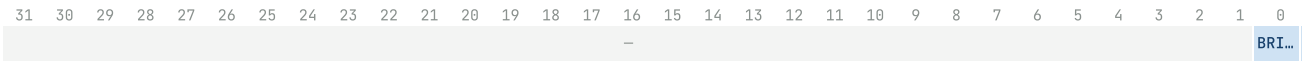
PHY Status / Control. MCP TEST PCIE on the test setup reads `$0000_08B3`: link up, x2, 5 GT/s, LTSSM state `$16`.



BITS	FIELD	ACCESS	MEANING
11	**LINK_UP**	R	1 = the link is up.
8:3	**LTSSM**	R	Link training state (LTSSM).
2:1	**WIDTH**	R	Link width: 0 = x1, 1 = x2, 2 = x4, 3 = x8.
0	**RATE**	R	1 = 5 GT/s (Gen2).

$148 PCIE_RP_CTRL RW · reset see PG055

Root Port Status / Control.



BITS	FIELD	ACCESS	MEANING
0	**BRIDGE_EN**	RW	Bridge enable. Set it after enumeration, or memory accesses from the CPU are not sent.

Enumeration

The MCP's pcie_enumerate() (in `dev/PCIe_A2560Me.c`) works depth first, like a PC BIOS. NetBSD's own PCI configuration code can do the same job instead. The constraints are: buses <= 7, memory in `$4000_0000` - `$7FFF_FFFF`, no I/O. It needs a configuration access method for the ECAM window, plus the bridge enable bit.

What the MCP does

STEP	ACTION
1	Check that bus 0, device 0 reads vendor `$10EE`. A build without PCIe returns a fixed pattern in this window.
2	Read `$144` (PHY status): the link must be up.
3	Treat the root port as a bridge to bus 1. For every bridge found: set primary / secondary bus numbers, and subordinate = 7 while scanning below it; scan its secondary bus (all 32 devices; 8 functions when the header type has bit 7 set); set subordinate to the last bus used; set the memory window to the range its children got (1 MB granularity); close the I/O and prefetchable windows (base > limit); enable memory + bus master.
4	Size every endpoint BAR by writing `$FFFF_FFFF`, reading back and restoring. Then place it, naturally aligned, from `$4000_0000` upwards. 64-bit BARs get upper half 0. Enable memory + bus master.
5	Set bridge enable (`$148` bit 0).

Result on the test setup (MCP TEST PCIE, build 74: PHY status `$0000_08B3`, link up, x2 at 5.0 GT/s, LTSSM `$16`; 7 functions found, memory used `$4000_0000`-`$4030_0000`, bridge enabled)

BUS:DEV.FN	ID	FUNCTION
00:00.0	10EE:7118	Root port
01:00.0	10B5:8747	PCI-to-PCI bridge (PEX8747 upstream port), BAR0 `$4000_0000`, 256 KB
02:08.0	10B5:8747	Bridge -> bus 3 (empty slot)
02:09.0	10B5:8747	Bridge -> bus 4
04:00.0	1912:0014	Class 0C0330, uPD720201 USB 3 xHCI, BAR0 64-bit `$4010_0000` (8 KB)
02:10.0	10B5:8747	Bridge -> bus 5
05:00.0	197B:0585	Class 010601, JMB585 AHCI, BAR0-4 I/O (unreachable), BAR5 `$4020_0000` (8 KB)
02:11.0	10B5:8747	Bridge -> bus 6 (empty slot)

DMA, caches and ordering

The cards' memory reads and writes go to DDR3 through DDR3 arbiter port 5 (round robin with the VDMA, the MDMA, the GGE DAC player and the debug port). Inside the core, PCIe addresses `$0000_0000` - `$3FFF_FFFF` map 1:1 to AXI addresses.

- Addresses: give the card CPU physical addresses in DDR3 (`$0080_0000` - `$3FFF_FFFF`), upper 32 bits 0. No IOMMU, no translation; bus_dma can be a 1:1 map limited to that range. The first 8 MB of the CPU map is the fast SRAM, which cards cannot reach.
- A DMA outside that range is refused: writes are dropped, and reads end with an error response from the core.
- The FPGA's L2 cache is coherent with card DMA: when a card writes DDR3, the FPGA invalidates the matching L2 lines before the write completes on the AXI side (the PCIe DMA path is requester 2 of the four requesters of the L2 invalidate arbiter); before a card reads DDR3, the FPGA waits until the CPU's posted writes in the L2 write buffer are in DDR3.
- The 68060's own caches are not snooped (the DMA does not run on the CPU bus), so the driver must handle them. Before a card reads a buffer the CPU wrote (PREWRITE): push the lines (CPUSHL / CPUSHP), or use write-through or non-cacheable mappings for descriptor rings. After a card wrote a buffer (POSTREAD): invalidate the lines (CINVL, or CPUSHL when the lines may be dirty) before the CPU reads it. The MCP runs with the caches off, so its code does none of this.
- Write order: card writes are done one transaction at a time, in order. A completion entry written after its data is never visible before that data.
- Speed: not measured yet for the DMA itself. The MCP AHCI driver does one sector per command, polled, and gets about 1 MB/s, so the driver is the limit there.

> Not guaranteed: that the CPU's read of a card register (or the interrupt) comes after the card's earlier DMA writes have reached DDR3. The core's master side (card DMA) and slave side (CPU accesses) are independent paths. Prefer completion information in memory (written last by the card), or re-check memory, over trusting a status register read alone. The window is short (microseconds) and was never hit in the MCP tests.

Interrupts

The core turns the cards' INTx messages and MSI writes into its interrupt output, filtered by its Interrupt Mask register (`$13C`) and reported in Interrupt Decode (`$138`). See PG055 for the bits, the MSI address registers and the MSI FIFO. The output drives GAVIN interrupt source `$23`: group 4, bit 3, CPU level 2, vector `$5B`. It is active low, a level while the core has an unmasked interrupt pending.

- Set bits in the core's Interrupt Mask (`$13C`; 0 after reset), for example bit 16 for INTx.
- In GAVIN, source `$23` needs its mask bit cleared and polarity active low; preferably use level mode (the line stays low while the core has an unmasked interrupt pending).
- INTx in root-port mode does not say which card or pin asserted it. Check the drivers' status registers (shared interrupt), clear the cause in the card, then write 1 to `$138` bit 16.

> Untested on the board so far: the MCP polls.

Known limits

- Link width: both the PEX8747 and an Intel Arc A-series card trained x2 on the x4 core (Gen2 rate). Board lanes 2-3 are suspect (to be checked on the board). x2 Gen2 still gives about 1 GB/s.
- Only 8 buses: one switch with a few cards fits; nested switches may run out of bus numbers.
- SATA Gen3 on the test setup (JMB585 + an mSATA adapter): the link comes up but has CRC errors (SERR `$00BA_0100`). At Gen2 it is clean. The MCP driver limits the speed through PxSCTL.SPD after a COMRESET and falls back Gen3 -> Gen2 -> Gen1 by itself. This looks like the adapter, not the FPGA, but a NetBSD AHCI driver may need the same.
- Intel Arc GPU: the link trains, but configuration reads behind it fail. Parked; it needs auxiliary power and more work.
- USB (Renesas uPD720201, 1912:0014): the test card loads its firmware (2.0.26) from its own ROM. MCP 175 TEST USB (xHCI step 1) starts the controller, resets the root ports and lists the devices on them; devices behind a hub are not enumerated yet. Cards without a firmware ROM need the firmware loaded by the driver.
- The CPU windows are uncached: the FPGA asserts TCI on every PCIe window access. Use byte / word / long accesses; no MOVE16 into the windows.
- No PCI I/O space (see Configuration space).

MCP commands (MCP 161-175)

The PCIe and AHCI tests need build 74 or later. The driver sources to look at are `dev/PCIe_A2560Me.c/.h` (windows, ECAM, enumeration) and `dev/ahci.c/.h`: one command slot, polled; the work area in DDR3 at `$0330_0000` (command list, received FIS, command table with one PRD, a 4 KB buffer); everything little-endian, with the swaps shown.

MCP 165 mounts a SATA disk behind PCIe as /sata0 at boot, read only. The SATA command handles that volume.

> Fixed in MCP 163, important if you use the MCP FatFs code: `fatfs/c256_diskio.c` was wrong for transfers of more than one sector. disk_read() and disk_write() did not advance the buffer between sectors, and disk_write() advanced the sector number by 512 instead of 1. Multi-sector FatFs transfers were wrong on every drive (SD cards and IDE too). Single-sector transfers, which most MCP commands use, were fine.

Commands

COMMAND	WHAT IT DOES
`TEST PCIE`	Enumerates everything behind the root port (bus numbers, bridge windows, memory BARs), enables the bridge, prints the tree.
`TEST PCIE DUMP b d f`	The first 256 bytes of a function's configuration space.
`TEST PCIE RD / WR / RD8 / WR8 addr [value]`	One access in the memory window (32-bit little-endian, or a byte).
`TEST AHCI [GEN1 | GEN2 | GEN3]`	Finds the AHCI controller (class 010601, BAR5) and the first disk, COMRESET with a speed limit, IDENTIFY, sector 0 by DMA. Read only.
`TEST AHCI MOUNT`	Installs the disk as block device 5 and mounts FatFs volume /sata0, read only (DIR, TYPE, COPY from it...).
`TEST AHCI READ file`	Reads a whole file through FatFs and prints a checksum and the speed.
`TEST AHCI WRITE [lba]`	Raw write test on 8 sectors near the end of the disk: saves them, writes and checks 16 patterns, restores them. It writes to the disk.
`TEST AHCI FILE [KB]`	Creates /sata0/A2560T.BIN, reads it back, compares, deletes it. It writes to the disk.
`SATA [MOUNT | RW | RO]`	MCP 165: the SATA disk behind PCIe (/sata0). No argument: status. MOUNT: mount it. RW / RO: allow / forbid writes (read only after boot).
`TEST USB`	MCP 175: the xHCI (USB 3) card behind PCIe, step 1: starts the controller, powers and resets the root ports, addresses every device found and lists it (vendor / product, class, speed, names). Devices behind a hub are not enumerated yet.

A Appendix: register index

REGISTER	ADDRESS	BLOCK
-	$FEC0_0228	Timers
-	$FEC0_022C	Timers
-	$FEC0_0B06	USB-C UARTs COM3 and COM4
-	$FEC0_C01C	SDMA: system DMA (SRAM to SRAM)
-	$FD00_441C	DAC player: voices
ADDR	$FC00_0004	GUI bitmap engine
AUD_CTRL	$FD00_4000	DAC player: global registers
AUD_STATUS	$FD00_4004	DAC player: global registers
BACKGROUND_COLOR	$FEC8_000C	VICKY II control registers $04 - $3C
BG_COLOR	$FD00_0010	Master control registers
BM_ADDR	$FD00_1004	Bitmap layers
BM_CTRL	$FD00_1000	Bitmap layers
BM_SCROLL	$FD00_100C	Bitmap layers
BM_STRIDE	$FD00_1008	Bitmap layers
BOOT_BASE	$FEC0_0048	GAVIN control registers
BOOT_INFO	$FEC0_0044	GAVIN control registers
BOOT_SEL	$FEC0_0040	GAVIN control registers
BOOT_SIZE	$FEC0_004C	GAVIN control registers
BORDER_COLOR	$FEC8_0008	VICKY II control registers $04 - $3C
BORDER_CTRL	$FEC8_0004	VICKY II control registers $04 - $3C
BTX_CTRL0	$FEC2_0000	BEATRIX control registers
BTX_CTRL1	$FEC2_0004	BEATRIX control registers
BUILD_ID	$FEC0_003C	GAVIN control registers
BUZZER_DIV	$FEC0_0034	GAVIN control registers
CHIP_ID	$FEC8_003C	VICKY II control registers $04 - $3C
CODEC_CTRL	$FEC2_0E00	CODEC control port
COLL_ANY	$FD00_3014	Collision detection
COLL_ENABLE	$FD00_3000	Collision detection
COLL_WITH0-15	$FD00_3040	Collision detection
CTRL	$FC00_0000	GUI bitmap engine
CURSOR_CTRL	$FEC8_0010	VICKY II control registers $04 - $3C
CURSOR_POS	$FEC8_0014	VICKY II control registers $04 - $3C
EDGE_0	$FEC0_0110	Interrupt controller
EDGE_1	$FEC0_0112	Interrupt controller
EDGE_2	$FEC0_0114	Interrupt controller
EVENT	$FD00_400C	DAC player: global registers
EVENT_IRQ_EN	$FD00_4010	DAC player: global registers
FONT_COUNT	$FEC8_0024	VICKY II control registers $04 - $3C
FONT_SIZE	$FEC8_0020	VICKY II control registers $04 - $3C
FPGA_DATE	$FEC0_0014	GAVIN control registers

REGISTER	ADDRESS	BLOCK
FPU_CTRL	$FEC0_4000	Floating Math Block (FPU)
FPU_ID	$FEC0_401C	Floating Math Block (FPU)
FPU_IN0	$FEC0_4008	Floating Math Block (FPU)
FPU_IN1	$FEC0_400C	Floating Math Block (FPU)
FPU_RESULT	$FEC0_4010	Floating Math Block (FPU)
FPU_RESULT_FIXED	$FEC0_4014	Floating Math Block (FPU)
FPU_RESULT_INT	$FEC0_4018	Floating Math Block (FPU)
FPU_STATUS	$FEC0_4004	Floating Math Block (FPU)
GAVIN_CHIP	$FEC0_0010	Board identity and DIP switches
GAVIN_CTRL	$FEC0_0000	GAVIN control registers
GAVIN_CTRL0 (status part)	$FEC0_0000	Board identity and DIP switches
GAVIN_HW_REV	$FEC0_0018	Board identity and DIP switches
GAVIN_ID	$FEC0_000C	Board identity and DIP switches
GGE_CTRL	$FD00_0000	Master control registers
HW_REVISION	$FEC0_0018	GAVIN control registers
I2C_ADDR	$FEC0_0E07	I2C master
I2C_CMD	$FEC0_0E02	I2C master
I2C_COUNT	$FEC0_0E12	I2C master
I2C_CTRL	$FEC0_0E00	I2C master
I2C_DATA	$FEC0_0E03	I2C master
I2C_DIVIDER	$FEC0_0E04	I2C master
I2C_FIFO	$FEC0_0E18	I2C master
I2C_HDMI	$FEC0_0E08	I2C master
I2C_ID	$FEC0_0E0C	I2C master
I2C_LEVEL	$FEC0_0E1A	I2C master
I2C_PAGE	$FEC0_0E16	I2C master
I2C_REG	$FEC0_0E14	I2C master
I2C_STATUS	$FEC0_0E01	I2C master
I2C_TIMEOUT	$FEC0_0E06	I2C master
I2C_XCTRL	$FEC0_0E10	I2C master
I2C_XFER	$FEC0_0E1C	I2C master
I2C_XSTATUS	$FEC0_0E11	I2C master
ID_CHIP	$FEC0_0010	GAVIN control registers
ID_CPU	$FEC0_000C	GAVIN control registers
LATE	$FD00_0020	Master control registers
LAYER_ENABLE	$FD00_0004	Master control registers
LAYER_ORDER_HI	$FD00_000C	Master control registers
LAYER_ORDER_LO	$FD00_0008	Master control registers
LFSR_CTRL	$FEC0_0004	GAVIN control registers
LFSR_DATA	$FEC0_0008	GAVIN control registers
LINE	$FD00_0018	Master control registers
LINE_INT_01	$FEC8_0018	VICKY II control registers $04 - $3C
LINE_INT_23	$FEC8_001C	VICKY II control registers $04 - $3C

REGISTER	ADDRESS	BLOCK
LINE_IRQ	$FD00_001C	Master control registers
MASK_0	$FEC0_0118	Interrupt controller
MASK_1	$FEC0_011A	Interrupt controller
MASK_2	$FEC0_011C	Interrupt controller
MASTER_VOL	$FD00_4008	DAC player: global registers
MASTER_VOL2	$FD00_4018	DAC player: global registers
MDMA_BYTES	$FEC0_C124	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_CTRL	$FEC0_C100	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_DST	$FEC0_C10C	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_DST_STRIDE	$FEC0_C11C	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_FILL	$FEC0_C120	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_HEIGHT	$FEC0_C114	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_SRC	$FEC0_C108	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_SRC_STRIDE	$FEC0_C118	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_STATUS	$FEC0_C104	MDMA: memory DMA (SRAM <-> DDR3)
MDMA_WIDTH	$FEC0_C110	MDMA: memory DMA (SRAM <-> DDR3)
MEMSYS_CTRL	$FEC0_0038	GAVIN control registers
MEMSYS_CTRL	$FEC0_0070	Memory-system control register (GAVIN register 14)
MEMSYS_CTRL (GAVIN control register 14)	$FEC0_0038	Line (burst) transfers
MONO	$FC00_0008	GUI bitmap engine
MONO_BG	$FC00_0010	GUI bitmap engine
MOUSE_CTRL	$FEC8_0C00	Mouse pointer
MOUSE_POS	$FEC8_0C04	Mouse pointer
MOUSE_X_SET	$FEC8_0C02	Mouse pointer
MPU_DATA	$FEC0_2330	SAM2695 MIDI synthesiser
MPU_STAT / MPU_CMD	$FEC0_2331	SAM2695 MIDI synthesiser
MSTR_CTRL	$FEC8_0000	VICKY II master control register
MT_COLOR_ADDR	$FEC9_800C	MemText: memory-backed text mode
MT_CTRL	$FEC9_8000	MemText: memory-backed text mode
MT_CURSOR_COLOR	$FEC9_8010	MemText: memory-backed text mode
MT_CURSOR_POS	$FEC9_8004	MemText: memory-backed text mode
MT_CURSOR_SHAPE0	$FEC9_8020	MemText: memory-backed text mode
MT_CURSOR_SHAPE1	$FEC9_8024	MemText: memory-backed text mode
MT_CURSOR_SHAPE2	$FEC9_8028	MemText: memory-backed text mode
MT_CURSOR_SHAPE3	$FEC9_802C	MemText: memory-backed text mode
MT_TEXT_ADDR	$FEC9_8008	MemText: memory-backed text mode
MT_WR_MASK	$FEC9_8014	MemText: memory-backed text mode
OPACITY0-3	$FD00_0040	Master control registers
OPM_STATUS	$FEC2_0C00	OPM: YM2151
OPN2_STATUS	$FEC2_0A00	OPN2: YM2612
PALETTE	$FC00_0400	GUI bitmap engine
PALETTE entry	$FD04_0000	Palettes
PCB_DATE	$FEC0_001C	GAVIN control registers

REGISTER	ADDRESS	BLOCK
PCIE_BRIDGE_INFO	$FA00_0130	Configuration space (ECAM)
PCIE_INT_DECODE	$FA00_0138	Configuration space (ECAM)
PCIE_INT_MASK	$FA00_013C	Configuration space (ECAM)
PCIE_PHY_STATUS	$FA00_0144	Configuration space (ECAM)
PCIE_RP_CTRL	$FA00_0148	Configuration space (ECAM)
PENDING_0	$FEC0_0100	Interrupt controller
PENDING_1	$FEC0_0102	Interrupt controller
PENDING_2	$FEC0_0104	Interrupt controller
PLAYING	$FD00_4014	DAC player: global registers
POLARITY_0	$FEC0_0108	Interrupt controller
POLARITY_1	$FEC0_010A	Interrupt controller
POLARITY_2	$FEC0_010C	Interrupt controller
PS2_BYTE0	$FEC8_0C0A	Mouse pointer
PS2_BYTE1	$FEC8_0C0C	Mouse pointer
PS2_BYTE2	$FEC8_0C0E	Mouse pointer
RGB_MSATA	$FEC0_002C	GAVIN control registers
RGB_NETWORK	$FEC0_0030	GAVIN control registers
RGB_POWER	$FEC0_0020	GAVIN control registers
RGB_SD0	$FEC0_0024	GAVIN control registers
RGB_SD1	$FEC0_0028	GAVIN control registers
RX_COUNT	$FEC0_0B02	USB-C UARTs COM3 and COM4
RX_COUNT	$FEC0_0802	WiFi UART (WizFi360)
SCRATCH_0	$FEC0_0106	Interrupt controller
SCRATCH_1	$FEC0_010E	Interrupt controller
SCRATCH_2	$FEC0_0116	Interrupt controller
SCRATCH_3	$FEC0_011E	Interrupt controller
SDIV_DIVIDEND	$FEC0_3034	Fixed-point maths unit
SDIV_DIVISOR	$FEC0_3030	Fixed-point maths unit
SDIV_QUOTIENT	$FEC0_3038	Fixed-point maths unit
SDIV_REMAINDER	$FEC0_303C	Fixed-point maths unit
SDMA_CTRL	$FEC0_C000	SDMA: system DMA (SRAM to SRAM)
SDMA_DST	$FEC0_C008	SDMA: system DMA (SRAM to SRAM)
SDMA_FILL	$FEC0_C018	SDMA: system DMA (SRAM to SRAM)
SDMA_SIZE1D	$FEC0_C00C	SDMA: system DMA (SRAM to SRAM)
SDMA_SIZE2D	$FEC0_C010	SDMA: system DMA (SRAM to SRAM)
SDMA_SRC	$FEC0_C004	SDMA: system DMA (SRAM to SRAM)
SDMA_STRIDE	$FEC0_C014	SDMA: system DMA (SRAM to SRAM)
SD_CTRL	$FEC0_0300	SD card controllers 0 and 1 (front and back slots)
SD_DATA	$FEC0_0301	SD card controllers 0 and 1 (front and back slots)
SD_SLOW_DIV	$FEC0_0302	SD card controllers 0 and 1 (front and back slots)
SMUL_A	$FEC0_3010	Fixed-point maths unit
SMUL_B	$FEC0_3014	Fixed-point maths unit
SMUL_HI	$FEC0_301C	Fixed-point maths unit

REGISTER	ADDRESS	BLOCK
SMUL_LO	$FEC0_3018	Fixed-point maths unit
SPARE0	$FEC8_0028	VICKY II control registers $04 - $3C
SP_ATTR_CTRL	$FD02_0104	Sprite layers
SP_ATTR_XY	$FD02_0100	Sprite layers
SP_CTRL	$FD02_0000	Sprite layers
SP_HIT	$FD02_0008	Sprite layers
SP_IMAGE_BASE	$FD02_0004	Sprite layers
SP_STAT	$FD00_0024	Master control registers
STATUS	$FC00_0014	GUI bitmap engine
STATUS	$FD00_0014	Master control registers
STRIDE	$FC00_000C	GUI bitmap engine
SYSMON_ID	$FEC0_0D00	FPGA temperature monitor (System Monitor)
SYSMON_LIMIT	$FEC0_0D14	FPGA temperature monitor (System Monitor)
SYSMON_MAX	$FEC0_0D0C	FPGA temperature monitor (System Monitor)
SYSMON_MIN	$FEC0_0D10	FPGA temperature monitor (System Monitor)
SYSMON_RAW	$FEC0_0D08	FPGA temperature monitor (System Monitor)
SYSMON_STATUS	$FEC0_0D18	FPGA temperature monitor (System Monitor)
SYSMON_TEMP	$FEC0_0D04	FPGA temperature monitor (System Monitor)
TIMER0_COMPARE	$FEC0_020C	Timers
TIMER0_VALUE	$FEC0_0208	Timers
TIMER1_COMPARE	$FEC0_0214	Timers
TIMER1_VALUE	$FEC0_0210	Timers
TIMER2_COMPARE	$FEC0_021C	Timers
TIMER2_VALUE	$FEC0_0218	Timers
TIMER3_COMPARE	$FEC0_0224	Timers
TIMER3_VALUE	$FEC0_0220	Timers
TIMER_CTRL0	$FEC0_0200	Timers
TIMER_CTRL1	$FEC0_0204	Timers
TL_CTRL	$FD00_2000	Tile layers
TL_LSCROLL_ADDR	$FD00_2018	Tile layers
TL_MAP_ADDR	$FD00_2004	Tile layers
TL_MAP_SIZE	$FD00_2008	Tile layers
TL_SCROLL_X	$FD00_2010	Tile layers
TL_SCROLL_Y	$FD00_2014	Tile layers
TL_TILES_ADDR	$FD00_200C	Tile layers
TOS_ADDR	$FEC9_7004	TOSGRAPH: EmuTOS bitmap
TOS_CTRL	$FEC9_7000	TOSGRAPH: EmuTOS bitmap
TOS_PAL0	$FEC9_7010	TOSGRAPH: EmuTOS bitmap
TOS_SPARE	$FEC9_7008	TOSGRAPH: EmuTOS bitmap
TX_COUNT	$FEC0_0B04	USB-C UARTs COM3 and COM4
TX_COUNT	$FEC0_0804	WiFi UART (WizFi360)
UART_CTRL	$FEC0_0B00	USB-C UARTs COM3 and COM4
UART_DATA	$FEC0_0B01	USB-C UARTs COM3 and COM4

REGISTER	ADDRESS	BLOCK
UART_STATUS	$FEC0_0B00	USB-C UARTs COM3 and COM4
UDIV_DIVIDEND	$FEC0_3024	Fixed-point maths unit
UDIV_DIVISOR	$FEC0_3020	Fixed-point maths unit
UDIV_QUOTIENT	$FEC0_3028	Fixed-point maths unit
UDIV_REMAINDER	$FEC0_302C	Fixed-point maths unit
UMUL_A	$FEC0_3000	Fixed-point maths unit
UMUL_B	$FEC0_3004	Fixed-point maths unit
UMUL_HI	$FEC0_300C	Fixed-point maths unit
UMUL_LO	$FEC0_3008	Fixed-point maths unit
VDMA_CONTROL	$FEC0_C200	VDMA: DDR3 block mover / blitter
VDMA_COUNT	$FEC0_C210	VDMA: DDR3 block mover / blitter
VDMA_CURRENT	$FEC0_C20C	VDMA: DDR3 block mover / blitter
VDMA_DESC	$FEC0_C200 $40 - $7C	VDMA: DDR3 block mover / blitter
VDMA_LIST_ADDR	$FEC0_C208	VDMA: DDR3 block mover / blitter
VDMA_STATUS	$FEC0_C204	VDMA: DDR3 block mover / blitter
VERSION	$FEC8_0038	VICKY II control registers $04 - $3C
VS_CTRL	$FEC2_2200	VS1053b MP3 / audio decoder link
VS_DATA	$FEC2_2210	VS1053b MP3 / audio decoder link
VS_ROOM	$FEC2_220C	VS1053b MP3 / audio decoder link
VS_SCI	$FEC2_2208	VS1053b MP3 / audio decoder link
VS_STATUS	$FEC2_2204	VS1053b MP3 / audio decoder link
V_ADDR	$FD00_4404	DAC player: voices
V_CTRL	$FD00_4400	DAC player: voices
V_LEN	$FD00_4408	DAC player: voices
V_LOOP	$FD00_440C	DAC player: voices
V_POS	$FD00_4418	DAC player: voices
V_STEP	$FD00_4410	DAC player: voices
V_VOL	$FD00_4414	DAC player: voices
WIFI_CTRL	$FEC0_0800	WiFi UART (WizFi360)
WIFI_DATA	$FEC0_0801	WiFi UART (WizFi360)
WIFI_STATUS	$FEC0_0800	WiFi UART (WizFi360)

B Appendix: notes, limits, revisions

Things to know

Behaviours of this release worth keeping in mind when writing software.

WHERE	NOTE
Bus clock	Build 87 runs BCLK (and the 68LC060) at 66.67 MHz, the only released clock since build 59; CPU_SPEED (GAVIN_ID bits 7-4) reads 6. SRAM reads have no wait state and writes 1; the flash has 5; timers 0-2 count 15 ns steps; the local-bus clock is 33.33 MHz. Use MCP 176 or later: MCP 145 adds TEST CACHE, MCP 146-155 TEST GGEPAT, MCP 156 TEST GGEAUD, MCP 157 TEST BEATRIX, MCP 158 / 159 TEST GGEAUD AK / RATE, MCP 160 TEST VS1053, WIZFI, SAM2695 and CODEC, MCP 161-164 TEST PCIE and TEST AHCI, MCP 165 SATA (/sata0 mounted at boot), MCP 166 BOOTSEL, MCP 171 I2C, MCP 172 / 173 DBGMON (the debug monitor stub), MCP 174 FAN (EMC2101 PWM fan), MCP 175 TEST USB (xHCI step 1), MCP 176 TEMP and FPU [TEST].
Access sizes	GAVIN control registers (`$FEC0_0000` - `$FEC0_007F` , boot selection included) take 32-bit writes only (a 16-bit write is ignored). Since build 25 the VICKY II control registers (`$FEC8_0000` - `$FEC8_003F`) also accept byte and word writes, which change only the bytes written. The interrupt controller takes 16-bit writes. The BEATRIX sound chips take byte writes only. The GGE (`$FD00_0000`) takes 32-bit accesses only. The I2C master (`$FEC0_0E00` , build 84), the temperature monitor (`$FEC0_0D00`) and the Floating Math Block (`$FEC0_4000` , both build 87) take any access size.
Endian inversion	GAVIN control bit 6 resets to 0 from build 23. When set, byte and word accesses to DDR3 are stored in swapped lanes, so the video engine and the DMA see the data scrambled. Leave it off.
Register 14	`$FEC0_0038` is the memory-system control register (L2 cache, line bursts, L2 flush, and local-bus bits 23-16 and 14-8: 13-8 since build 34, 14 since build 38, 23-16 since build 39; bit 15 = board type, read only), not a scratch register. Modify it with a read-mask-write; leave bits 23-16 and 14-8 at 0.
DDR3 window	The CPU sees DDR3 at `$0080_0000` - `$3FFF_FFFF` : the address is the DDR3 byte address, so the first 8 MB of DDR3 lie behind the SRAM and cannot be reached by the CPU.
Coherency	The L2 cache is write-through, so CPU writes reach DDR3. With the 68060 data cache in copyback mode, push the lines of a frame buffer, of GGE graphics and maps, and of DAC player samples (CPUSHL / CPUSHA), or map them write-through or cache-inhibited, before the GUI engine or the GGE reads them.
Bus errors	An access to an unmapped address ends with TEA (bus error). Since build 60 that includes `$FC00_1000` - `$FC0F_FFFF` (it used to repeat the GUI engine registers). The legacy window `$FEC4_0000` - `$FEC7_FFFF` always acknowledges. Unused GAVIN addresses read `$B00B_FEED` and unused GGE addresses read 0, without a bus error.
Flash	The boot flash window (`$FFC0_0000`) is read only for the CPU: do not write to it. The flash is programmed through the debug port (protocol v2 host tool: a2560me_dbg.py --flash FILE --block N, about 6 s for 512 KB).
Boot block (build 83)	BOOT_SEL (`$FEC0_0040` , key `$B0` in bits 31-24) reboots into flash block N of a 1 x 4 MB, 2 x 2 MB, 4 x 1 MB or 8 x 512 KB layout; BOOT_INFO (`$FEC0_0044`) shows what booted. The choice lasts for that one reboot only: power-on, the reset button (or the GAVIN soft reset), the debug port's reset and the RESET instruction boot block 0 again. A blank or non-existent block falls back to block 0. The boot DIP switches (GAVIN_CTRL bits 23-20) are no longer used for booting: free for software, OFF reads 1. Block 0 holds the boot loader: it reads /sd2/boot.cfg and leaves a hand-over record at SRAM `$007F_BF00` for the started program.

WHERE	NOTE
I2C bus (build 84)	New I2C master at $FEC0_0E00 (I2C_ID $4932_4D01): transactions of up to 256 bytes through a FIFO, or byte-level commands; 100 kHz (default), 400 kHz or 1 MHz. The RP2040 supervisor is a second master on the bus: on ERR 3 (bus lost) just try again. The board's MAC address is in the 24AA02E48T EEPROM (device $50) at $FA - $FF. The SiI9022 set-up still runs by itself. No interrupt line (builds 84-87): poll XSTATUS. MCP 171: I2C SCAN / MAC / READ / WRITE / HDMI / SPEED.
Floating Math Block (build 87)	Single-precision FPU at $FEC0_4000 (FPU_ID $4650_5531). Write CTRL, IN0, IN1, poll STATUS bit 0 (BUSY, 5-36 clocks), read RESULT; the CPU never waits on the bus. SIN / COS take -pi .. +pi only (STATUS bit 26) and give about 6-7 digits. MCP 176: FPU TEST.
FPGA temperature (build 87)	$FEC0_0D04 reads the die temperature in 1/16 °C (signed), valid when STATUS ($FEC0_0D18) bit 0 is 1, which needs the DDR3 controller running (the value comes from its XADC). MAX, MIN and a limit (85 °C) with a sticky flag. MCP 176: TEMP.
Video modes	1280 x 960 and 1280 x 1024 are both verified on hardware (since build 24). The HiRes DIP switch reads 0 in this build, so the system starts at 1280 x 960; software switches with master control bit 8.
GUI engine	ADDR, STRIDE and the mode are taken at the start of each frame (tear-free page flips). ADDR must be 32-byte aligned. STATUS bits 31:16 count underruns and should stay 0.
Write-only tables	The block text colour tables and the mouse pointer images read $DEAD_BEEF; the MemText colour tables read $AAAA_5555. Keep a copy in software.
Sound chips	PSG, SID, OPM and OPN2 take byte writes through a shared 1024-entry command FIFO. Since build 68 the FPGA paces OPM / OPN2 writes (about 21 / 28 us each) and their windows read a status byte: bit 7 BUSY, bit 6 FIFO FULL, bits 1:0 timer flags. The mixer (gain per source, master attenuation, mute, stereo blend) is set in BTX_CTRL0 / BTX_CTRL1 at $FEC2_0000 / $FEC2_0004.
VS1053 / WizFi360 / SAM2695	Links only, no players in the FPGA. VS1053b at $FEC2_2200 (build 70): set RUN, wait for READY, stream the file into VS_DATA with long writes (68000 byte order) while VS_ROOM has space; SCI commands through VS_SCI; interrupt $29 when the FIFO runs low. WizFi360 UART at $FEC0_0800: COM3 / COM4 layout since build 70 (data byte at +1, counts at +2 / +4), control bit 4 resets the module, 115 200 baud by default. SAM2695: the Super I/O MPU-401 at $FEC0_2330 / $FEC0_2331 ($FF, $3F, then MIDI bytes while status bit 6 is 0). WM8776 analogue inputs: AIN1 SAM2695, AIN4 VS1053, AIN5 line-in, AUX AK4432.
Debug port	Protocol v2 only since build 86 ($5A frames, 8 Mbit/s, CRC-16, 32-bit addresses, up to 16 KB per frame); the old $55 protocol (FoenixMgr, GenX Uploader) is gone: use a2560me_dbg.py. Since build 85 SRAM, flash and DDR3 can be read while the CPU runs, the CPU can be paused and stepped without a reset, 4 breakpoints stop it, and command $32 stops it in the MCP's monitor stub (level 7). Flash commands still need the CPU held. See the separate document "A2560Me Debug Protocol v2" (rev 1.3).
Level 7 (build 85)	Interrupt level 7 (the 68060 NMI, autovector 31 at $0000_007C) is the debug port's monitor stop. Install an RTE handler on vector 31 if your software has no monitor stub (the MCP 172+ has one).
Top of SRAM	$007F_BE00 - $007F_BEFF monitor stub mailbox (MCP 172+), $007F_BF00 boot hand-over record (boot loader), $007F_C000 - $007F_FFFF debug-port flash staging area (overwritten by every flash program frame). Keep your own data out of these if you use them.

WHERE	NOTE
PCI Express (build 74)	Root port, Gen2 x4. Configuration (ECAM) window `$FA00_0000` - `$FA7F_FFFF` (buses 0-7 only), memory window `$4000_0000` - `$7FFF_FFFF` (same addresses on PCIe). Both are uncached and address invariant: 16- and 32-bit card values are little-endian, swap them. No PCI I/O space. Set bridge enable (core register `$148` bit 0) after enumeration, and the core's Interrupt Mask (`$13C`) before using source `$23`. Card DMA reaches DDR3 `$0080_0000` - `$3FFF_FFFF` at the CPU address and keeps the L2 coherent; the 68060 caches are not snooped.
Build number	`$FEC0_003C` reads the FPGA build number (`$0000_0057` = build 87). Use it to check which bitstream is running.
DMA engines	The MDMA (`$FEC0_C100` , SRAM <-> DDR3) the VDMA (`$FEC0_C200` , DDR3 blitter) and, since build 74, PCIe card DMA keep the L2 cache coherent by themselves. The 68060's own caches are still software's job, as for any DMA.
Ethernet	Since build 25 the LAN9221 takes 16 and 32-bit accesses; since build 29 longword reads are FIFO-safe. A longword gives the bytes in memory order, so numeric register values need a 32-bit byte swap (BYTE_TEST reads `$2143_6587`). Byte writes are ignored. Since build 34 the FPGA filters every LAN read (3-sample majority) against a D6 glitch on the prototype boards. Open item: TEST LAN loopback frames longer than about 136 bytes are cut short.
HDMI clock	GAVIN register 0 bit 9 inverts the HDMI pixel clock (build 28). Leave it 0; both settings were clean on the test board.
mSATA / ATA	The prototype boards have no IORDY and no INTRQ connection: polled PIO only. Data-register reads give sector bytes in memory order; IDENTIFY numeric words read byte-reversed.
SD card 2 and the 10M02	The 10M02 (SD card 2, joysticks) runs on the 33.33 MHz local-bus clock (it needs 33 MHz or less: SD card 2 does not work on builds 34 and 36). Older 10M02 images do not decode A1, so `$FEC0_0382` is SD2_CTRL there; do not write the SD2 divider (MCP 128 and later leave it at its reset value).
Local bus (build 39 and later)	Every local-bus output is registered in the FPGA I/O cells, the device select is held for the whole CPU cycle, the data pins are SLOW slew and write data is on the bus before the write strobe. Reason: duplicated 16-bit words in mSATA writes under NetBSD on build 35 (a write strobe counted twice). Read strobes are one clock longer.
White or black board: DIP switch 7	Set DIP switch 7 (as marked on the part, 1-8) ON on a white board (JM20330 directly on the local bus) and OFF on a black board (IDE translators). A white board with switch 7 OFF, or with builds up to 39, has the IDE DMACK- asserted: the JM20330 then drives the shared data bus during LAN9221, RTC and joystick reads after any disk read, and IDE writes can be counted twice. MEMSYS_CTRL (`$FEC0_0038`) bit 15 = 1 shows white mode.
L2 cache (build 59)	The L2 registers CPU write data and byte enables before it writes its RAM. This fixed the NetBSD boot failure of builds 57 / 58 with the 68060 caches on. Run MCP TEST CACHE (caches on, as NetBSD) on every new FPGA load.
L2 burst reads (build 74)	The L2 now moves to the next longword of a line burst read on the clock edge that ends each beat, giving the data two bus clocks before the next TA. Up to build 73 TEST CACHE could fail on the slowest data bits; rare crashes or corrupted data with the caches on, on builds 59-73, may have this cause.
Game Graphics Engine	GGE_ENABLE (`$FD00_0000` bit 0) switches the whole HDMI output to the GGE (640 x 480 at 60 Hz or 640 x 400 at 70 Hz, 25 MHz): text, MemText, the mouse pointer and the GUI engine are not shown, and DDR3 port 0 belongs to the GGE. Layer registers are taken once per frame (at line 524 / 448); sprite attributes only through COMMIT. While the GGE is on, SOF (`$00`) is its frame start; SOL (`$01`) is its LINE_IRQ, which works even with the GGE off.

WHERE	NOTE
GGE DAC player	32 voices played from DDR3 (port 4) to two outputs, chosen per voice by V_CTRL bit 8 (build 69): the WM8776 codec at 48 kHz (mixed with the sound chips in BEATRIX; MASTER_VOL) and the AK4432 DAC at 44.1 kHz (on its own; MASTER_VOL2). V_STEP 1.0 is the voice's output rate. PCM16 samples are little-endian. A PLAY is heard 3 samples after it is taken. Interrupt `$15` is a level: clear the EVENT bits before the pending bit.

Revisions

REVISION	DATE	FPGA BUILD	CHANGES
0.1	2026-10-07	#23	First edition: the system and CPU bus, GAVIN, the video and the GUI bitmap engine, the memory system and DMA, the sound, the debug port.
0.2	2026-10-08	#28	Builds 24-28: 1280 x 1024 HDMI set-up; SD slow clock; 8 KB font window; VICKY II byte / word writes; build-number register; faster LAN9221 interface with 32-bit accesses; SDMA window split; new MDMA (SRAM <-> DDR3) and VDMA (DDR3 blitter); L2 range invalidate shared by the DMAs; HDMI output registers in the I/O pads and an ODDR pixel clock. New block diagrams: the HDMI output, DDR3 coherency, the MDMA and the VDMA.
0.3	2026-10-09	#34 / #35	Builds 29-35: BCLK at 66.67 MHz (build 34) or 50 MHz (build 35), with the SRAM side on the bus clock at 66.67 MHz and BCLK from a phase-advanced ODDR; SRAM and flash wait states per build and the part numbers; build-number and CPU-speed values; timers 0-2 count the bus clock; SD0 / SD1 slow divider derived from the engine clock and fast-clock figures; SD2 clock and divider (`$FEC0_0382`); LAN9221 FIFO-safe longword reads (build 29), byte order, read majority filter and bus timing; ATA 400 ns read strobe, majority filter, no IORDY / INTRQ; MEMSYS_CTRL bits 13-8; MemText / EmuTOS SRAM stream rate; MDMA and VDMA throughput on both clocks; MCP 124 / 125 delay scaling and test commands; TEST LAN open item.
0.4	2026-10-09	#38 / #35	Builds 36-38: zero-wait SRAM reads at 66.67 MHz (CE held low, A3:A2 straight through; line read 2-2-2-2); SD0 / SD1 fast clock 22.2 MHz at 66.67 MHz; local-bus clock 33.33 MHz on the 66.67 MHz build with a half-BCLK phase shift for the 10M02 (MEMSYS_CTRL bit 14); build ID `$26` ; SD card 2 divider and the 10M02 A1 decode; MCP 128 (no SD2 divider write) and TEST SRAM; TEST MEM SYS warning; TEST LAN maxlen advice; LAN D6 wording.
0.5	2026-10-09	#39 / #38	Build 39 (50 MHz, current): local-bus hardening (one device select per CPU cycle, I/O-cell registers on every local-bus output, SLOW data slew, write data before the write strobe, IDE write recovery 100 ns, MEMSYS_CTRL bits 23-16 for the IDE write strobe / recovery); new strobe and LAN timing figures; build ID `$27` ; build 38 marked as the parked 66.67 MHz build; MCP 129 TEST ATAWRITE.
0.6	2026-10-10	#40 / #41	Builds 40 (50 MHz) and 41 (66.67 MHz) as the same released design; white and black board types, DIP switch 7 board select (F14 = DMACK- / translator DIR, D14 = DMARQ / translator OEn), the white-board DMACK fault and its effect on LAN9221 reads and IDE writes; MEMSYS_CTRL bit 15; build IDs `$28` / `$29` ; 66.67 MHz local-bus figures with the build 39 hardening.
0.7	2026-10-10	#45 / #46	Builds 42-46: SRAM write data held one clock after WE for FPGA masters (MDMA corruption); TOSGRAPH fetched one line ahead into a two-line buffer with on-the-fly conversion at 108 MHz (no stale lines under DMA, 4 bpp at 1280 on both clocks); TIP pull-up and CPU chip selects gated by the arbiter during FPGA tenures, GAVIN register 2 bits 31-24 diagnostic; block text and MemText font RAM with byte / word / long access, block text font bank in CURSOR_CTRL bit 3, MemText / EmuTOS read-back path fixed; build IDs `$2D` / `$2E` ; MCP 133 TEST FONT, MCP 134 TEST LAN fix.
0.8	2026-10-10	#50 / #49	Builds 47-50: MemText 4-byte cells {FG, BG, attribute, character} in one SRAM array, one 640-byte fetch per row; MemText cell write mask (MT_WR_MASK, `$FEC9_8014`) applied to CPU writes in the cell 64 KB block; 66.67 MHz SRAM display stream at one longword per clock with falling-edge capture (MemText / EmuTOS fetch 2x); L2 write recorder diagnostic (MEMSYS_CTRL 27:24, GAVIN register 2 31:24); 66 MHz A3:A2 pass-through bound 7.15 ns; build IDs `$32` / `$31` ; MCP 135-140 tests (MEMTEXTROWS cells, MTMASK, DDR3, L2WH).

REVISION	DATE	FPGA BUILD	CHANGES
0.9	2026-10-11	#58 / #57	Builds 51-58: L2 lookup address from a flip-flop (fixes the rare single-bit L2 error with MDMA + display fetch; floating CPU address pins on the block RAM address); gamma and GUI palette RAM addresses the same; CPU writes to the gamma tables fixed; 50 MHz SRAM display stream at one longword per 2 clocks of 100 MHz, sampled 25 ns after the address; L2 recorder = anomaly detector (refills excluded, one clock later); build IDs `$3A` / `$39`; MCP 141-144 (TEST MDMA recorder, LUTRB, TOSPAT border).
1.0	2026-10-12	#67	Builds 59-66, 66.67 MHz only: L2 write data and byte enables registered (build 59; fixes the NetBSD crash of builds 57 / 58 with the caches on); new chip select CS_GGE (`$FD00_0000`-`$FD0F_FFFF`), CS_VICKY_IV narrowed to CS_GUI (`$FC00_0000`-`$FC00_0FFF`), the rest of `$FC0`x a bus error (build 60); new chapter: the Game Graphics Engine (builds 60-67: 640 x 480 / 640 x 400 at 25 MHz, 4 bitmap, 4 tile and 8 sprite layers, 16 palettes, alpha blending with layer order and opacity, pixel collision, 32-voice DAC player) with figure 6; DDR3 port 0 shared by the GUI engine and the GGE, port 4 = DAC player; SOF / SOL from the GGE, IRQ `$15` = DAC player; BEATRIX mix with a fifth source and a 19-bit sum; SiI9022 tables for the GGE modes; build ID `$42`; MCP 145-156 (TEST CACHE, GGEPAT, GGEAUD). Build 67: DAC player STREAM voices read every block of the ring again on each lap (build 66 played an old copy of the ring's first block from the third lap on).
1.1	2026-10-12	#69	Builds 68-69, MCP 157-159. Build 68 (BEATRIX review): mixer controls in BTX_CTRL0 (master attenuation in 6 dB steps, mute, stereo blend) and BTX_CTRL1 (a 4-bit gain per source, 8 = unity), one clamp after the attenuation; OPM / OPN2 writes queued and paced by the hardware (settle timer and busy flag) and a status byte {BUSY, FULL, timer flags} in their windows (was `$4E`); chip clocks corrected (PSG and OPM 3.5795 MHz, OPM cen_p1 1.79 MHz, SID 1.0227 MHz, OPN2 7.159 MHz; the PSG was a major third flat); SID writes limited to registers `$00`-`$1C`; PSG write enable fixed; CODEC and AK4432 MCLK through ODDRs. Build 69: the GGE DAC player has a second output on the AK4432 (44.1 kHz I2S, per-voice V_CTRL bit 8 OUTPUT, MASTER_VOL2 at `$FD00_4018`); the CPU-fed 44.1 kHz PCM FIFO at `$FEC2_2100` and its interrupt `$2E` are retired; AK4432 board strapping (parallel mode, automatic clock detection, I2S); build ID `$45`. MCP 157 TEST BEATRIX and TEST OPM / OPN on the internal cores, MCP 158 TEST GGEAUD AK, MCP 159 TEST GGEAUD RATE.
1.2	2026-10-12	#70	Build 70, MCP 160. New in BEATRIX: the VS1053b link at `$FEC2_2200` (VS_CTRL, VS_STATUS, VS_SCI, VS_ROOM, VS_DATA; SPI mode 0 with XCS / XDCS, 8-entry SCI queue, 4 KB data FIFO sent in 32-byte bursts on DREQ, automatic start-up with a software reset, MODE `$4800`, CLOCKF `$8800`, VOL `$1010`; the chip's 12.288 MHz CLKI = 24.576 MHz / 2 through an ODDR); its interrupt drives source `$29` (data FIFO low). New sections: SAM2695 MIDI synthesiser (Super I/O MPU-401 at `$FEC0_2330`, set-up, status bits 6 / 7) and WM8776 analogue inputs (AIN1 SAM2695, AIN4 VS1053, AIN5 line-in, AUX AK4432). WizFi360 UART (`$FEC0_0800`) rewritten: COM3 / COM4 layout, receive head-byte prefetch (a data read returned the previous byte), counts at +2 / +4, FAST really 921 600 baud (the transmitter ran at about 828 000), control bit 4 drives the module's RST pin (B16). Build ID `$46`. MCP 160: TEST VS1053, TEST WIZFI [NORESET | LOOP | RAW], TEST SAM2695 [SOLO], TEST CODEC; MPU-401 status bits fixed in midi_reg.h.
1.3	2026-10-13	#74	Build 74, MCP 164. PCI Express root port built in (new chapter): AXI Bridge for PCI Express (axi_pcie 2.9, PG055), Gen2 x4, root port ID 10EE:7118; configuration (ECAM) window `$FA00_0000`-`$FA7F_FFFF` for buses 0-7 (mirrored to `$FAFF_FFFF`), memory window `$4000_0000`-`$7FFF_FFFF` at the same PCIe addresses; address invariant byte order; core registers `$130`, `$138`, `$13C`, `$144`, `$148`; card DMA into DDR3 `$0080_0000`-`$3FFF_FFFF` at the CPU address through new DDR3 arbiter port 5 (6 ports), with L2 coherency (the L2 invalidate arbiter now serves 3 requesters: MDMA, VDMA, PCIe DMA, round robin); interrupt source `$23` active (level, active low). L2 burst read fix: the output moves to the next longword on the clock edge that ends each beat (up to build 73 on the same edge as the next TA; TEST CACHE could fail on the slowest data bits). Build ID `$4A`. MCP 161-164: TEST PCIE [DUMP | RD | WR | RD8 | WR8], TEST AHCI [GEN1-3 | MOUNT | READ | WRITE | FILE]; MCP 163 fixes multi-sector transfers in the FatFs diskio.

REVISION	DATE	FPGA BUILD	CHANGES
1.4	2026-10-14	#83	Builds 75-83, MCP 165-166. Boot block selection (build 83): GAVIN control chip select widened to `$FEC0_0000` - `$FEC0_007F` with BOOT_SEL (`$40` , key `$B0` , layout 1 x 4 MB to 8 x 512 KB, block; a keyed write reboots into that block), BOOT_INFO (`$44`), BOOT_BASE (`$48`) and BOOT_SIZE (`$4C`); the choice lasts for one reboot only; fall-back to block 0 for a blank or non-existent block; the boot copy reads each longword into a register and then writes the SRAM (about 4.2 ms for 64 KB); flash geometry (8 KB sector, 128 KB block); DIP_BOOT (GAVIN_CTRL bits 23-20) free for software. Debug port protocol v2 next to the old `$55` protocol: `$5A` frames at 8 Mbit/s with SEQ, ATTR, 32-bit address and CRC-16/CCITT-FALSE, `$A5` / `$B5` replies, STATUS bits, commands `$00` / `$01` (up to 16 KB; DDR3 while the CPU runs, build 81), `$10` / `$11` / `$12` flash ID / program / erase (build 82), `$30` , `$3F` , `$7E` , `$7F` ; pointer to the separate Debug Protocol v2 document. Build 77: no FPGA bus master is granted while the debug port owns the bus. DDR3 arbiter port 6 = debug port (7 ports, build 80); the L2 invalidate arbiter serves 4 requesters (MDMA, VDMA, PCIe DMA, debug port), round robin. Build 81: the interrupt-acknowledge decode is qualified by TIP; timing bounds on the arbiter's SRAM control and CPU address / data paths. Build ID `$53` . MCP 165: /sata0 mounted at boot, SATA [MOUNT | RW | RO]; MCP 166: BOOTSEL, no DIP-switch boot device selection (F1 / F2 on the splash still boot SD card / hard drive).
1.5	2026-10-15	#84	Build 84, MCP 171. New I2C master (build 84): register map, devices, MAC address. A2560Me_I2C at `$FEC0_0E00` - `$FEC0_0EFF` (64-byte register set mirrored 4 times, any access size, big-endian): CTRL, STATUS, CMD, DATA, DIVIDER, TIMEOUT, ADDR, HDMI, ID `$4932_4D01` , XCTRL, XSTATUS, COUNT, REG, PAGE, the 256-byte FIFO, LEVEL, XFER; it replaces the old block (I2C_CTRL / SLAVE / REG / DATA / COUNT), which the CPU could not write on builds 38-83. Devices: SiI9022 `$39` (PCA9306 level shifter, 1.8 V FPGA bank), EMC2101 `$4C` , 24AA02E48T `$50` with the MAC at `$FA` - `$FF` , Si5351A `$60` , the RP2040 as a second master; multi-master rules; MAC read example. The SiI9022 set-up is unchanged and can be run again through I2C_HDMI bit 1. Build ID `$54` . MCP 171: I2C [SCAN | MAC | READ | WRITE | HDMI | SPEED].
1.6	2026-10-15	#86	Builds 85-86, MCP 172-173. Debug port build 85 (revision `$14`): SRAM / flash reads and SRAM writes while the CPU runs or is paused (the debug port is the arbiter's top-priority bus master and gives the bus back every 256 clocks), pause / continue (`$30` ATTR 2 / 3) and bus step (`$31`) without a reset, 4 breakpoints (`$20` , address + MASK, code / read / write, NMI flag) and the debug report (`$21`), the monitor stop (`$32`) on interrupt level 7 (autovector 31) for the MCP 172 monitor stub with its mailbox at `$007F_BE00` ; new STATUS bits TIMEOUT, STATE, CPU_PAUSED, BP_HIT. Build 86 (revision `$15`): protocol v2 only, the old `$55` protocol (FoenixMgr, GenX Uploader) removed; `$02` / `$03` retired. Debug port sections rewritten (overview; frame details in the Debug Protocol v2 document rev 1.3). Interrupt tables: level 7. Bus arbitration: the debug master and the pause. New: SRAM areas used by convention, and a section on the block-0 boot loader (/sd2/boot.cfg, hand-over record at `$007F_BF00`). Build ID `$56` .
1.7	2026-10-16	#87	Build 87, MCP 174-176. New Floating Math Block (A2560Me_FPU) at `$FEC0_4000` , replacing the placeholder window: single-precision MUL, DIV, ADD / SUB with a selectable adder input, SQRT, SIN, COS, float32 / 20.12 / int32 formats, BUSY bit and flags, BUSY times, example, limits. New FPGA temperature monitor (A2560Me_SysMon) at `$FEC0_0D00` : TEMP, RAW, MAX, MIN, LIMIT, STATUS. Address maps updated. I2C: EMC2101 fan set-up (MCP 174 FAN). PCIe: the USB card is a uPD720201, MCP 175 TEST USB. Build ID `$57` .
1.8	2026-10-16	#87	Build 87, corrections after an outside review of rev 1.6: Figure 1 updated (66.67 MHz bus, 7 DDR3 ports, debug port protocol v2 at 8 Mbit/s, build 87, no PCM FIFO; it still said 50 MHz / build 28); TRINITY and RTC strobe times in the device sections now match the Local-bus cycles table (they were 50 MHz figures); fixed-point math wait 180 ns (was 240); access-size table: block text and MemText fonts take any size at any offset since builds 44 / 45 (the table still showed the old 8-bit / A1 = 1 limits); TEST LAN note: the truncation with a cable is a limit of the loopback test only.